

Chapter 1 — pandas fundamentals

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

40 questions across the pandas surface area that every data role interview walks through first — Series vs DataFrame, indexing, dtypes, column ops, and the 2.x gotchas that trip up candidates who learned pandas on 1.x tutorials. Each question follows the same format: tags · question · 30-second answer · step-by-step thought process · code · edge cases · what NOT to say · common follow-up · stand-out line.

Assume `import pandas as pd` and `import numpy as np` in every code block.

Table of Sections

- Section A — Series + DataFrame Basics (Q1 to Q10)
 - Section B — Indexing & Selection (Q11 to Q20)
 - Section C — Column Operations (Q21 to Q30)
 - Section D — Common Gotchas & 2.x Patterns (Q31 to Q40)
-

Section A — Series + DataFrame Basics

This is the first ten minutes of any data interview — TCS analyst pool, Razorpay data screen, Walmart Labs GCC panel, Swiggy growth team. Most candidates get the rough idea right but fumble on what an index actually does, why `df.info()` and `df.describe()` answer different questions, or how labels behave when two Series with different indexes meet. Master these ten and you walk into Section B with the interviewer already nodding.

Q1 · Series vs DataFrame — what's the actual difference

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Accenture · Topic: pandas basics

The question (interviewer's verbatim phrasing):

What is the difference between a Series and a DataFrame in pandas? Can you build one from the other?

The 30-second answer: A Series is a 1-D labelled array — one column with an index. A DataFrame is a 2-D labelled table — many Series sharing one index. Selecting a single column out of a DataFrame returns a Series; concatenating Series along axis 1 gives you back a DataFrame.

Step-by-step thought process: 1. Start with dimensionality — Series is 1-D, DataFrame is 2-D. Index is shared in both cases; the DataFrame just adds a column axis. 2. Mention that every column in a DataFrame is internally a Series — `df['col']` returns one. That mental model alone clears up half the confusion candidates carry. 3. Note that a Series has a single `dtype`; a DataFrame has one `dtype` per column. This matters when you ask `df.dtypes` vs `s.dtype`. 4. Anticipate the follow-up about Panel — Panel was removed in pandas 0.25, replaced by MultiIndex DataFrames. Don't bring it up unless asked.

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'], name='amount')
df = pd.DataFrame({'amount': [10, 20, 30], 'status': ['paid', 'paid', 'failed']},
                  index=['a', 'b', 'c'])

type(df['amount']) # <class 'pandas.core.series.Series'>
df['amount'].equals(s) # True
```

Edge cases to mention: - A DataFrame with one column is still a DataFrame, not a Series — `df[['col']]` is 2-D, `df['col']` is 1-D. The double-bracket trick matters. - A Series can have a `name` attribute that becomes the column name when promoted to a DataFrame via `.to_frame()`. - Both share the same Index machinery — anything you can do to a DataFrame's index works on a Series's index.

What NOT to say: - "A Series is a list with extra features" — it's an ndarray with a label axis, not a list. The Index distinction is the whole point. - "DataFrame is just a dict of lists" — it's a dict of Series, and the Series have a shared index. The shared index is what makes alignment work.

Common follow-up: "If I select one column with single brackets vs double brackets, what's the difference?" — Single bracket returns a Series (1-D, has `.dtype`); double bracket returns a one-column DataFrame (2-D, has `.dtypes`). Downstream code that expects a DataFrame breaks on a Series.

Stand-out line: "A DataFrame is a dict of Series with a shared index — once you internalize that, alignment, broadcasting, and `groupby` all stop feeling magical."

Q2 · The Index — why pandas isn't just numpy with column names

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, Meesho, PhonePe · Topic: pandas basics

The question (interviewer's verbatim phrasing):

What does the Index do in a pandas object? Why can't we just use the default 0-N integer position like numpy?

The 30-second answer: The Index is the row label axis. It enables label-based selection (`.loc`), automatic alignment when two objects are combined, fast set operations (union, intersection), and `groupby` semantics. Without it, pandas would just be numpy with a column dict — no alignment, no `.loc` , no time-series resampling.

Step-by-step thought process: 1. State the four jobs the Index does: label-based selection, alignment, set ops, group keys. Each one is something numpy cannot do natively. 2. Give the alignment example out loud — add two Series with overlapping but unequal indexes and pandas matches on label, fills missing with NaN. That's the killer feature. 3. Mention the default RangeIndex — when you don't specify, pandas gives you a `RangeIndex(0, n)` , which is memory-cheap and behaves like positions. People mistake it for "the index is the row number" — it's actually a labelled axis that happens to use 0..n-1 labels. 4. Set the stage for the next question — `.loc` vs `.iloc` is a direct consequence of the Index being labels, not positions.

```
a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
b = pd.Series([10, 20, 30], index=['y', 'z', 'w'])
a + b
# w      NaN
# x      NaN
# y     12.0
# z     23.0
```

Edge cases to mention: - An Index can hold duplicates — `pd.Index([1, 1, 2])` is valid. Set ops still work, but selection on a duplicate label returns multiple rows. - A DataFrame has both a row Index and a column Index — `df.columns` is also an Index object with all the same methods. - `RangeIndex` is stored as start/stop/step, not as a materialized array — much cheaper than a 1M-element `Int64Index`.

What NOT to say: - "The index is the row number" — it's the row label. The default happens to be 0..n-1, but you can replace it with any unique-or-not values. - "You can ignore the index in pandas" — you cannot. Alignment will silently produce NaN if you do, and you'll spend an afternoon debugging.

Common follow-up: "What happens if I concat two DataFrames whose indexes overlap?" — By default, you get duplicate labels in the result. `ignore_index=True` resets to a fresh `RangeIndex`; `verify_integrity=True` raises if there are dupes.

Stand-out line: "The Index is what turns pandas from a fancy 2-D array into an aligned data model — every gotcha eventually traces back to the index, which is why senior practitioners read the index first and the values second."

Q3 · `df.info()` vs `df.describe()` — when to call which

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture, HCL · Topic: pandas basics

The question (interviewer's verbatim phrasing):

When you first load a CSV into pandas, what's the difference between calling `df.info()` and `df.describe()` ? Which one do you call first?

The 30-second answer: `df.info()` tells you the shape, dtype per column, non-null count, and memory usage — structural metadata. `df.describe()` tells you count, mean, std, min, quartiles, max — statistical summary of numeric columns. Call `info()` first to know what you're looking at; call `describe()` second to know how it's distributed.

Step-by-step thought process: 1. Frame it as structure vs distribution. `info()` answers "what is in this DataFrame"; `describe()` answers "what does it look like." 2. Walk through the order in a real EDA flow: read CSV, `.shape`, `.head()`, `.info()`, `.describe()`, then start digging. `info()` reveals the dtypes you need to fix; `describe()` reveals the outliers and ranges. 3. Mention that `describe()` skips non-numeric by default. Pass `include='all'` or `include='object'` to see categorical summaries (unique, top, freq). 4. Note that in pandas 2.x, `describe()` includes datetime columns with min/max/mean by default — the older `datetime_is_numeric` keyword (introduced in 1.1, defaulting to False) is now gone, since that behavior is the default.

```
df = pd.read_csv('orders.csv')
df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 50000 entries, 0 to 49999
# Data columns (total 5 columns):
#  #   Column      Non-Null Count  Dtype
# ---  -
#  0   order_id    50000 non-null   int64
#  1   amount      49920 non-null   float64
#  2   status      50000 non-null   object
#  3   paid_at     49850 non-null   object    # ← should be datetime, fix this
#  4   customer_id 50000 non-null   int64
# memory usage: 1.9+ MB

df.describe(include='all')
```

Edge cases to mention: - `info()` shows `memory_usage='deep'` only if you pass it — the default underestimates for object columns because Python strings live elsewhere on the heap. - `describe()` on an all-object DataFrame returns count, unique, top, freq — useful for categorical EDA without writing extra code. - `describe(percentiles=[0.01, 0.5, 0.99])` gives you tail percentiles for outlier hunting.

What NOT to say: - "They show the same thing" — they answer different questions. `info()` is structural, `describe()` is statistical. - "I always start with `.head()`" — `.head()` shows three rows; it doesn't tell you that the date column is actually parsed as object. `info()` does.

Common follow-up: "Your `df.info()` shows `paid_at` as object but you need it as datetime — what do you do?" — `df['paid_at'] = pd.to_datetime(df['paid_at'], errors='coerce')`. The `errors='coerce'` turns unparseable strings into NaT instead of raising. Then call `.info()` again to confirm.

Stand-out line: "In any real EDA flow, `info()` tells me the schema is broken, `describe()` tells me the data is messy — I fix the schema first because every other operation depends on it."

Q4 · `df.head()`, `df.tail()`, `df.sample()` — three peeks, three reasons

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, HCL · Topic: pandas basics

The question (interviewer's verbatim phrasing):

What's the difference between `df.head()`, `df.tail()` and `df.sample()`, and when would you use each?

The 30-second answer: `head()` returns the first N rows, `tail()` the last N, `sample()` a random subset. Head and tail are for inspection of sorted or time-ordered data — first orders, last orders. Sample is for sanity-checking a dataset that has structure you don't want to inspect in order — sorted-by-id files where rows 0..5 all look identical.

Step-by-step thought process: 1. State the three behaviours one-liner each. All three default to N=5 except `sample()` which defaults to n=1. 2. Real-world example: a 10M-row payments CSV sorted by date. `head()` and `tail()` both show one day's worth of similar data; `sample(20)` shows you 20 random rows across the whole time range — much more representative. 3. Mention `sample(frac=0.01)` for percentage-based subsampling — taking 1% of a huge file before EDA. 4. Note that `sample()` accepts `random_state=42` for reproducibility — always set it when sharing analysis in a notebook.

```
df.head()           # first 5
df.tail(10)         # last 10
df.sample(n=20, random_state=42) # 20 random, reproducible
df.sample(frac=0.01, random_state=42) # 1% random
df.sample(n=10, weights='amount') # weighted by amount column
```

Edge cases to mention: - `sample()` with `replace=True` allows the same row to appear multiple times — useful for bootstrap analysis. - Pass `weights='colname'` to sample probabilistically — high-amount rows are more likely to be picked. - `head(-5)` returns all rows except the last 5; `tail(-5)` returns all except the first 5. Negative N is a real and useful pattern.

What NOT to say: - "I always use `head()` to inspect data" — that misses outliers and tail-heavy distributions. Sample shows you the messy middle. - "Sample is slow on big data" — it's O(n) on a single DataFrame; the bottleneck is reading the file, not sampling it.

Common follow-up: "How would you take a representative 1000-row sample from a 100GB CSV that won't fit in memory?" — Use `pd.read_csv(..., chunksize=100_000)` and `sample()` per chunk with `frac=` such that the total adds up to 1000, then `pd.concat` the chunks. Or use `polars` / `duckdb` for the sampling step and load only the sample into pandas.

Stand-out line: "In production EDA I almost always call `sample(20, random_state=42)` before `head()` — sorted CSVs lie to you, random samples don't."

Q5 · dtypes — the seven you actually see and what they mean

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Target, PhonePe · Topic: dtypes

The question (interviewer's verbatim phrasing):

What dtypes does pandas commonly assign when you read a CSV, and which one is the "I have a problem" dtype?

The 30-second answer: The common ones are `int64`, `float64`, `object`, `bool`, `datetime64[ns]`, `category`, and the nullable `Int64` / `Float64` / `boolean` / `string`. The "I have a problem" dtype is `object` — it means pandas couldn't infer anything and is storing Python objects. Either you actually have mixed types (often you don't) or your numeric column has a stray string that broke inference.

Step-by-step thought process: 1. List the seven you actually see day to day. Memorize this list — it's what `df.dtypes` returns and what the interviewer expects you to read. 2. Explain `object` honestly: in modern pandas, strings should be `string` dtype (pyarrow-backed in 2.x). If your text column shows `object`, you're either on pandas 1.x defaults or your `read_csv` didn't auto-promote. 3. Mention

`int64` cannot hold NaN — that's why a numeric column with one missing value silently becomes `float64`. The nullable `Int64` (capital I) was added to fix this — it backs nulls with `pd.NA`. 4. Note `category` for low-cardinality string columns — memory dropped 10x on a 50M-row status column at one client.

```
df = pd.DataFrame({
    'id': pd.array([1, 2, 3], dtype='Int64'),
    'amount': [99.5, 199.0, np.nan],          # float64 (NaN forces it)
    'status': pd.Categorical(['paid', 'paid', 'failed']),
    'paid_at': pd.to_datetime(['2026-05-29', '2026-05-30', '2026-05-30']),
    'is_premium': pd.array([True, False, None], dtype='boolean'),
    'note': pd.array(['hi', 'hi', None], dtype='string'),
})
df.dtypes
# id                Int64
# amount            float64
# status            category
# paid_at           datetime64[ns]
# is_premium        boolean
# note              string
```

Edge cases to mention: - `int64` and `Int64` are NOT the same. Lower-case is numpy-backed and cannot hold NaN; upper-case is pandas nullable and uses `pd.NA`. - `object` columns can hold mixed types — a column with both strings and integers is `object` and slow for everything. - The `category` dtype has a fixed set of allowed values; assigning a new value not in the categories raises unless you `add_categories` first.

What NOT to say: - "Object means string" — it means Python object, which is often string but sometimes int, sometimes mixed. - "I'll cast everything to object to be safe" — object is the slowest dtype. Casting numeric data to object kills vectorization.

Common follow-up: "Your `amount` column came in as object after `read_csv` — what's likely wrong and how do you debug?" — Likely a stray non-numeric value (a `-`, an `N/A`, a comma-in-numbers like `1,200`). Debug with `df[~df['amount'].str.match(r'^[\d.]+$')]` to find the offending rows. Fix with `pd.to_numeric(df['amount'], errors='coerce')` to push them to NaN.

Stand-out line: "Every dtype is a contract — `Int64` says 'I can hold nulls,' `category` says 'I have a closed vocabulary,' `object` says 'pandas gave up.' I read dtypes before I trust the data."

Q6 · Label-based vs position-based indexing — the mental model

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Meesho, PhonePe · Topic: indexing

The question (interviewer's verbatim phrasing):

What's the difference between label-based and position-based indexing in pandas, and why does pandas need both?

The 30-second answer: Label-based uses the Index values — `df.loc['order_123']` looks up by the row label. Position-based uses the 0..n-1 integer position — `df.iloc[5]` returns the sixth row regardless of label. Pandas needs both because the Index can hold non-integer labels (strings, dates, MultiIndex tuples), but you sometimes still want "give me the third row from the top."

Step-by-step thought process: 1. Anchor it in the Index — labels are the Index values, positions are the implicit 0..n-1 ordering of the rows in memory. 2. Walk through the four-cell matrix: rows × cols can each be label or position. `.loc` is label/label, `.iloc` is position/position. Mixed access is supported via `.loc` + index translation. 3. Explain why beginners get confused: when the Index IS a RangeIndex(0, n), label and position look identical. The difference only bites once you set a non-trivial index. 4. Hint at the gotcha for Q11: `.loc` slicing is INCLUSIVE on both ends; `.iloc` is exclusive on the end like Python. Saves a future debugging session.

```
df = pd.DataFrame({'amount': [100, 200, 300, 400]},
                  index=['a', 'b', 'c', 'd'])

df.loc['b']          # row labelled 'b' → amount = 200
df.iloc[1]           # second row by position → amount = 200 (same here, by coincidence)

df.loc['b':'d']       # labels b, c, d (3 rows – INCLUSIVE)
df.iloc[1:3]          # positions 1, 2 (2 rows – exclusive end)
```

Edge cases to mention: - `df.loc[3]` is label lookup with label = 3 (integer label). `df.iloc[3]` is position 3. If your Index is `[1, 3, 5, 7]`, they're completely different rows. - `.ix` was deprecated and removed — older code uses it; modern code should not. - Negative indices work in `.iloc` (Python-style) but raise in `.loc` unless the label is literally negative.

What NOT to say: - "I just always use `.loc`" — fine until you need "the last row" and don't know the label; `.iloc[-1]` is the answer. - "Both work the same on a default DataFrame" — they do, until they don't. The first day your DataFrame has a non-trivial index, you'll wish you had practised the distinction.

Common follow-up: "If my Index is `[10, 20, 30]`, what does `df.loc[10]` return vs `df.iloc[10]`?" — `df.loc[10]` returns the row with label 10 (the first row). `df.iloc[10]` raises `IndexError` because there are only 3 rows. This is the canonical interview trap.

Stand-out line: "`.loc` reads from the Index, `.iloc` reads from memory order — once you say it that way, every confusing slicing bug becomes obvious."

Q7 · single-bracket vs double-bracket selection

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Walmart Labs · Topic: indexing

The question (interviewer's verbatim phrasing):

What's the difference between `df['col']` and `df[['col']]` ?

The 30-second answer: `df['col']` returns a 1-D Series. `df[['col']]` returns a 2-D DataFrame with one column. The double-bracket version preserves DataFrame-ness, which matters when your next operation expects a DataFrame (e.g., `pd.merge`, `df.to_csv`, anything calling `.columns`).

Step-by-step thought process: 1. State the shape difference plainly. Single bracket = Series (1 dim). Double bracket = DataFrame (2 dims). 2. Show why it matters: `df['col'].to_csv()` writes a single column without the header by default; `df[['col']].to_csv()` writes a proper 1-column CSV. 3. Mention the multi-column case: `df[['a', 'b', 'c']]` selects three columns, returns a DataFrame. There's no Series equivalent — Series is 1-D. 4. Anticipate the dtype follow-up: `df['col'].dtype` exists, `df[['col']].dtypes` (note the s) exists; calling the wrong one raises.

```
df = pd.DataFrame({'amount': [100, 200], 'status': ['paid', 'failed']})

type(df['amount'])      # Series
type(df[['amount']])    # DataFrame
df['amount'].shape      # (2,)
df[['amount']].shape    # (2, 1)
```

Edge cases to mention: - `df[['amount']]` works even if you only want one column for downstream code that expects a DataFrame interface. - Calling `.dtype` on a DataFrame raises `AttributeError`. Calling `.dtypes` on a Series also raises. Mismatch is a common test trap. - For column ordering: `df[['b', 'a']]` returns a DataFrame in that order — useful for reordering columns.

What NOT to say: - "They return the same thing" — they don't; one is 1-D and one is 2-D. - "Double bracket is the wrong syntax" — it's the syntax that returns a DataFrame, and it's correct when that's what you want.

Common follow-up: "You call `.merge()` on the result of `df['key']` and it errors. Why?" — `.merge()` is a DataFrame method, not a Series method. Use `df[['key']]` to keep it as a DataFrame, or convert with `df['key'].to_frame()`.

Stand-out line: "Single bracket is a column extraction, double bracket is a column projection — the second one keeps the table shape, which is what every chained operation downstream actually wants."

Q8 · `.values` vs `.to_numpy()` vs `.array` — getting the underlying data

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman, Target · Topic: pandas basics

The question (interviewer's verbatim phrasing):

If I want the underlying numpy array out of a Series, what's the difference between `.values`, `.to_numpy()` and `.array`?

The 30-second answer: `.values` is the legacy accessor — it returns a numpy array for numpy-backed dtypes and something else (an `ExtensionArray`) for nullable / pyarrow dtypes, which is confusing. `.to_numpy()` always returns a true numpy array, with an optional `dtype` argument. `.array` returns the underlying pandas `ExtensionArray` (preserves `pd.NA`, etc.). Use `.to_numpy()` in new code.

Step-by-step thought process: 1. Lead with the recommendation: `.to_numpy()` in pandas 1.0+. The other two have legacy or specialized uses. 2. Explain why `.values` is tricky: it returns a numpy array sometimes and a pandas `Categorical` / `IntegerArray` other times — same call, different type. That's why `to_numpy()` was added — it always returns numpy. 3. Mention `.array` for the rare case you want the actual pandas extension array (which preserves `pd.NA`). For most analysts, you don't. 4. Note `.to_numpy(dtype=float)` casts on extraction — useful when you have nullable `Int64` and want to push it to `float64` with `np.nan` for compatibility with scikit-learn.

```
s = pd.Series([1, 2, pd.NA], dtype='Int64')

s.values          # IntegerArray (not pure numpy)
s.to_numpy()      # array([1, 2, <NA>], dtype=object) — preserves NA
s.to_numpy(dtype='float64', na_value=np.nan) # array([1., 2., nan])
s.array           # IntegerArray (same as .values here)
```

Edge cases to mention: - `df.values` on a mixed-dtype DataFrame returns an object array — every column upcast to object. Almost never what you want. - `df.to_numpy()` does the same upcast but at least the behaviour is explicit and documented. - For 2.x with `dtype_backend='pyarrow'`, `.values` returns a pyarrow chunked array via the extension interface — yet another shape.

What NOT to say: - "I always use `.values` for performance" — `.to_numpy()` is the modern recommendation and has the same performance for numpy-backed dtypes. - "They all return the same thing" — they don't; the type returned depends on the dtype, which is the whole point of having three accessors.

Common follow-up: "You pass `df['Int64_col'].values` to scikit-learn and it crashes. Why?" — scikit-learn expects a numpy array of a real numeric dtype; `.values` on a nullable `Int64` returns an

`IntegerArray` containing `pd.NA`. Use `.to_numpy(dtype='float64', na_value=np.nan)` to get a numeric array sklearn accepts.

Stand-out line: "`.to_numpy()` is the only accessor I trust to give me a real numpy array — `.values` is the bug magnet for nullable dtypes."

Q9 · Creating DataFrames — the four idioms you actually use

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, HCL, Accenture · Topic: pandas basics

The question (interviewer's verbatim phrasing):

What are the common ways to create a DataFrame in pandas? When would you use each one?

The 30-second answer: Four idioms — from dict of lists / dict of Series, from list of dicts, from list of tuples / list of lists, and from another data source via `pd.read_csv` / `pd.read_parquet` / `pd.read_sql`. Dict-of-lists is the cleanest for hand-written examples; list-of-dicts is the natural fit when you're consuming JSON-like records.

Step-by-step thought process: 1. Walk through dict-of-lists first — the keys become column names, the lists become column values. Most readable for fixed-schema test data. 2. List-of-dicts second — each dict is a row. Useful when you're iterating an API response, each item is a dict. The keys union becomes columns; missing keys fill with NaN. 3. Mention list-of-tuples / 2-D numpy array — you must pass `columns=` separately. Less common in practice because the schema isn't carried with the data. 4. End with file readers — `read_csv`, `read_parquet`, `read_sql`, `read_excel`. The reality of most jobs.

```
# Dict of lists
df1 = pd.DataFrame({'id': [1, 2, 3], 'amount': [100, 200, 300]})

# List of dicts (common from API responses)
records = [{'id': 1, 'amount': 100}, {'id': 2, 'amount': 200}]
df2 = pd.DataFrame.from_records(records)

# List of tuples (need columns separately)
df3 = pd.DataFrame([(1, 100), (2, 200)], columns=['id', 'amount'])

# From file
df4 = pd.read_parquet('orders.parquet')
```

Edge cases to mention: - Dict-of-lists requires equal-length lists or it raises. Dict of Series allows different indexes — pandas will align and fill missing with NaN. - `from_records` accepts numpy structured arrays and named tuples. Handy when you have a numpy buffer with a dtype. - Pass

`index=` to set the row Index at construction time — more efficient than constructing then calling `.set_index()`.

What NOT to say: - "You should always use `pd.DataFrame()`" — `from_records` and the read functions exist for a reason; use them when the input shape matches. - "DataFrames are slow to construct" — construction is $O(n)$; the slowness shows up when you grow them row by row in a loop. Build a list of dicts and construct once.

Common follow-up: "You have a million-row CSV — what's the fastest way to read it into pandas?" — `pd.read_csv(path, engine='pyarrow')` or convert to parquet and use `pd.read_parquet`. Parquet reads 5-20x faster than CSV because it's columnar and compressed.

Stand-out line: "Constructing DataFrames row by row in a loop is the single most-common pandas perf bug — build a list once, construct the DataFrame once, you'll see a 100x speedup."

Q10 · `df.shape`, `df.columns`, `df.index`, `len(df)` — the four metadata accessors

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: pandas basics

The question (interviewer's verbatim phrasing):

How do you find out the number of rows, columns, and the column names of a DataFrame?

The 30-second answer: `df.shape` returns a `(n_rows, n_cols)` tuple — the most useful. `len(df)` returns just rows. `df.columns` returns the column Index; `df.index` returns the row Index. `df.size` returns `n_rows * n_cols`, less commonly needed.

Step-by-step thought process: 1. State the four and what each returns. `shape` is the workhorse — used in every assertion / sanity check. 2. Explain `len()` — runs `__len__`, returns rows only. Convenient inside conditionals: `if len(df) == 0: ...`. 3. Note `df.columns.tolist()` to get a plain list — useful for serialization or comparison. `list(df.columns)` works too. 4. Mention `df.index` is a real Index object — has `.name`, `.dtype`, `.is_unique`, `.is_monotonic_increasing`. Far richer than people assume.

```
df.shape      # (50000, 5)
len(df)       # 50000
df.columns    # Index(['id', 'amount', 'status', 'paid_at', 'customer_id'], dtype='object')
df.columns.tolist() # ['id', 'amount', 'status', 'paid_at', 'customer_id']
df.index      # RangeIndex(start=0, stop=50000, step=1)
df.size       # 250000
```

Edge cases to mention: - `df.shape` is a tuple, not a function — `df.shape()` raises.
`df.shape[0]` for rows, `df.shape[1]` for cols. - `df.empty` returns True if no rows OR no columns — useful guard before downstream calls. - `df.index.is_unique` is the cheap way to confirm you have no duplicate row labels; many merges and joins assume this silently.

What NOT to say: - "I count rows with `df.count()`" — `count()` returns non-null counts PER COLUMN, not row count. Different question. - "I do `len(df.columns)` for column count" — `df.shape[1]` is faster and clearer.

Common follow-up: "How do you check that two DataFrames have the same columns in the same order?" — `df1.columns.equals(df2.columns)` is the right comparison. Don't use `==` because that does element-wise comparison and returns an array.

Stand-out line: "`df.shape` is the first thing I assert on after every transformation — if the row count changed unexpectedly, my merge or filter went wrong and I want to know in line 3, not line 30."

Section B — Indexing & Selection

This is where most candidates lose the round. `.loc` vs `.iloc` everyone has read about; what trips people up is boolean indexing with chained conditions, `.query()` performance, MultiIndex slicing, and the `SettingWithCopyWarning`. Ten questions, all from real interview transcripts.

Q11 · `.loc` vs `.iloc` — the slicing gotcha

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs, JP Morgan · Topic: indexing

The question (interviewer's verbatim phrasing):

If I do `df.loc[2:5]` and `df.iloc[2:5]` on a default-index DataFrame, do I get the same rows? Why or why not?

The 30-second answer: On a default RangeIndex DataFrame they look similar but they aren't the same. `.loc[2:5]` is label-based and INCLUSIVE on both ends — returns labels 2, 3, 4, 5 (four rows). `.iloc[2:5]` is position-based and EXCLUSIVE on the end like Python slicing — returns positions 2, 3, 4 (three rows).

Step-by-step thought process: 1. State the inclusive vs exclusive rule first — that's the entire answer. `.loc` is inclusive both ends because labels don't have a natural "exclusive end" concept; `.iloc` is half-open like Python lists. 2. Show the count mismatch — three rows from `iloc`, four from `loc`. Most candidates have read this rule and forgotten it; saying the count out loud cements it. 3. Bring up the case

when the Index is non-trivial: `df.loc['a':'c']` returns labels a, b, c (3 rows). Same inclusive rule. 4. Mention that pandas raises on `df.iloc[2:5]` if the DataFrame has fewer than 5 rows — out-of-bounds slicing is silently clipped, like Python lists.

```
df = pd.DataFrame({'amount': [10, 20, 30, 40, 50, 60]})

df.loc[2:5]      # rows with labels 2, 3, 4, 5 → 4 rows
df.iloc[2:5]     # positions 2, 3, 4 → 3 rows
```

Edge cases to mention: - For a non-monotonic index (`['a', 'c', 'b', 'd']`), `.loc['a':'c']` returns ALL rows up to the FIRST 'c' — be careful with unordered labels. - `.iloc[-1]` is the last row, like Python. `.loc[-1]` looks up label -1 — raises if no such label exists. - Mixed slicing: `.loc[2:5, ['amount']]` slices rows by label and columns by label. `.iloc[2:5, [0]]` slices rows by position and columns by position. Don't mix the two in one call.

What NOT to say: - "They return the same rows" — wrong; the count differs by one due to inclusive vs exclusive end. - "`.loc` is just slower" — performance is irrelevant here; the semantic difference matters far more than nanoseconds.

Common follow-up: "Why did the pandas team make `.loc` inclusive on both ends?" — Because labels can be arbitrary (strings, dates, MultiIndex tuples) and there's no natural "next label" to exclude. The inclusive convention is consistent across all label types; the exclusive end of `.iloc` mirrors Python list slicing.

Stand-out line: "`.loc` is inclusive because labels have no natural successor; `.iloc` is exclusive because positions do — the rule isn't arbitrary, it's a consequence of what each one indexes into."

Q12 · Boolean indexing — and / or / not, parentheses, and the bitwise traps

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Meesho, PhonePe, Swiggy · Topic: indexing

The question (interviewer's verbatim phrasing):

How do you filter a DataFrame for rows where amount is over 1000 AND status is 'paid'? Walk me through the syntax.

The 30-second answer: `df[(df['amount'] > 1000) & (df['status'] == 'paid')]`. Bitwise `&` for AND, `|` for OR, `~` for NOT — NOT the Python keywords `and / or / not`. Parentheses around each condition because `&` has lower precedence than `>`. Forgetting the parentheses produces a misleading error.

Step-by-step thought process: 1. Lead with the bitwise operator rule — `&`, `|`, `~`. Then explain why: `and` / `or` / `not` short-circuit on a single truthy value, and a Series of booleans is ambiguous — pandas raises "The truth value of a Series is ambiguous." That error means "you used `and` instead of `&`." 2. Stress the parentheses. Operator precedence in Python: `&` is below `>`. Without parens, `df['amount'] > 1000 & df['status'] == 'paid'` evaluates as `df['amount'] > (1000 & df['status']) == 'paid'` — gibberish. 3. Mention the alternative APIs: `df.query("amount > 1000 and status == 'paid'")` uses Python-style operators because it's parsed as an expression string. Cleaner syntax for complex filters. 4. Note `df[mask]` returns a new DataFrame; if you want to assign back, `df.loc[mask, 'col'] = value`.

```
mask = (df['amount'] > 1000) & (df['status'] == 'paid')
df[mask]

# Alternatives
df.query("amount > 1000 and status == 'paid'")
df[df['amount'].gt(1000) & df['status'].eq('paid')]
```

Edge cases to mention: - `df['col'].isin([1, 2, 3])` returns a boolean mask for "is any of these values" — cleaner than chained ORs. - `~df['col'].isin([1, 2, 3])` is the NOT-IN. The tilde is the bitwise NOT. - Chained boolean conditions with many columns become unreadable fast — `.query()` or `.pipe(lambda df: df[df['amount'] > 1000])` are alternatives.

What NOT to say: - "I'll use `and` here" — raises immediately on multi-row Series. Always `&`. - "I don't need parentheses" — operator precedence will silently produce the wrong mask or a confusing error.

Common follow-up: "Your filter returns zero rows even though you can see matching rows in `df.head()`. What might be wrong?" — Whitespace or case mismatch in string columns (`' paid'` vs `'paid'`, `'Paid'` vs `'paid'`), float comparison precision (`amount == 99.9` failing due to binary representation), or NaN values that don't equal anything. Check with `df['status'].str.strip().str.lower().value_counts()`.

Stand-out line: "Every pandas filter bug I've debugged was either a missing parenthesis or an `and` where `&` belonged — those two together account for 80% of 'pandas is being weird' tickets."

Q13 · `.query()` vs boolean indexing — when to switch

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Cred, Target, Goldman · Topic: indexing

The question (interviewer's verbatim phrasing):

What does `df.query()` do? Why would you use it over a boolean mask?

The 30-second answer: `df.query()` parses a string expression and applies it as a filter — same result as a boolean mask but more readable for complex conditions. It uses Python-style `and / or / not`, accepts variables via `@varname`, and on numeric data with `numexpr` installed can be slightly faster than the equivalent boolean mask on large DataFrames.

Step-by-step thought process: 1. State what it does — string expression in, filtered DataFrame out. Equivalent to boolean indexing for most queries. 2. Highlight the readability win: complex conditions with three or four ANDs and ORs become legible. Especially good for parameterized queries from config files or notebooks. 3. Mention the `@variable` syntax — `df.query("amount > @threshold")` injects a Python variable. Useful for parameterized filters in scripts. 4. Be honest about limits: column names with spaces or starting with digits need backticks (``paid at``), some operations (`.str.contains`) require fallback to boolean masks.

```
threshold = 1000
df.query("amount > @threshold and status == 'paid'")
df.query("`paid at` > '2026-01-01'")      # column name with space
df.query("status in ['paid', 'refunded']") # in operator works
```

Edge cases to mention: - Column names with reserved words (`class`, `if`) or special chars need backticks. - `df.query()` returns a new DataFrame; pass `inplace=True` to modify the caller. - With pandas 2.x and the pyarrow string backend, some query paths fall back to pure Python parsing — performance can be different from numpy-backed boolean masks.

What NOT to say: - "Query is always faster" — it isn't; the parsing overhead can dominate on small DataFrames. The win is readability. - "Query is just sugar for boolean masks" — for the common cases yes, but it has its own parser, accepts string expressions, and supports `@var` references.

Common follow-up: "Your query string has a column name with a hyphen — how do you reference it?" — Backticks around the column name: `df.query("`paid-at` > '2026-01-01'")`. Same trick for column names with spaces, dots, or reserved words.

Stand-out line: "`.query()` shines on five-condition filters and config-driven dashboards — for two-line ad-hoc filters in a notebook, boolean masks are fine."

Q14 · `.at` and `.iat` — scalar access

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Goldman, Target, Cred · Topic: indexing

The question (interviewer's verbatim phrasing):

What are `df.at` and `df.iat`? Why would you use them instead of `.loc` and `.iloc`?

The 30-second answer: `df.at[label_row, label_col]` and `df.iat[pos_row, pos_col]` are scalar-only accessors. They skip the slicing logic that `.loc` and `.iloc` carry, so reading or writing a single cell is faster. Use them in loops that touch one cell at a time; for slices or vectors, stay with `.loc` / `.iloc`.

Step-by-step thought process: 1. Frame it as a perf optimization for scalar access. `.loc` and `.iloc` accept slices, lists, masks; `.at` and `.iat` accept only single labels/positions and return a single value. 2. The speed-up is real — for setting one cell in a tight loop, `.at` can be 5-10x faster than `.loc`. In practice, you should rarely write tight pandas loops, so this matters less than people think. 3. Show the symmetry: `.at` mirrors `.loc` (labels); `.iat` mirrors `.iloc` (positions). Same naming convention. 4. Note `.iat` is the only way to set a cell at a known position without thinking about the index.

```
df = pd.DataFrame({'amount': [100, 200, 300]}, index=['a', 'b', 'c'])

df.at['b', 'amount']          # 200, fast scalar read
df.iat[1, 0]                  # 200, position-based scalar read
df.at['b', 'amount'] = 999    # fast scalar write
```

Edge cases to mention: - Pass a slice to `.at` and it raises — scalar only. - `.at` requires the label to exist; if the row label is missing, it raises. (Unlike `.loc` with a new label, which can ADD a row.) - `.iat` won't enlarge the DataFrame either; out-of-bounds raises.

What NOT to say: - "I always use `.at` for speed" — premature optimization. Most pandas code is vectorized; scalar accessors matter only inside genuine cell-by-cell loops. - "`.at` and `.loc` are the same" — semantically yes for single cells, but `.at` has a faster code path and rejects slices.

Common follow-up: "You're computing a running balance row by row — should you use `.at` in a loop, or rewrite vectorized?" — Rewrite vectorized with `df['balance'] = df['delta'].cumsum()`. The `.at` loop will still be 50-100x slower than the vectorized version on any non-trivial DataFrame.

Stand-out line: "`.at` and `.iat` are the right answer to a wrong question — if you're reaching for them, you probably need to rewrite the loop as a vectorized operation instead."

Q15 · `SettingWithCopyWarning` — what it actually means

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Walmart Labs, Goldman, JP Morgan · Topic: indexing

The question (interviewer's verbatim phrasing):

What is the `SettingWithCopyWarning` in pandas, why does it happen, and how do you fix it?

The 30-second answer: It's a warning that pandas thinks you're modifying a slice that may or may not be a view of the original DataFrame, and your assignment might not stick. It happens on chained indexing — `df[df['x'] > 0]['y'] = 5`. The fix is to combine the access into one `.loc` call: `df.loc[df['x'] > 0, 'y'] = 5`. In pandas 2.x with copy-on-write enabled, the warning is gone because the behaviour is now deterministic.

Step-by-step thought process: 1. State why it appears — chained indexing produces a temporary object, and pandas can't always tell whether assigning to it modifies the original or a copy. It warns because the result is implementation-defined. 2. Demonstrate the fix: replace `df[mask]['col'] = x` with `df.loc[mask, 'col'] = x`. The single `.loc` call sees both the row and column selection together and assigns to the original. 3. Bring up copy-on-write — pandas 2.0 introduced an opt-in mode (`pd.set_option('mode.copy_on_write', True)`) where the warning is no longer needed because the semantics are deterministic: every operation returns a new object. This becomes the default in pandas 3.0. 4. Mention the secondary path — sometimes the warning fires because you sliced a DataFrame, assigned it to a new variable, then modified.

`subset = df[df['x'] > 0]; subset['y'] = 5` — fix with `subset = df[df['x'] > 0].copy()`.

```
# The bug – chained indexing
df[df['amount'] > 1000]['status'] = 'high' # WARNING, may not stick

# Fix 1 – combine in a single .loc
df.loc[df['amount'] > 1000, 'status'] = 'high'

# Fix 2 – explicit copy if you needed the subset
subset = df[df['amount'] > 1000].copy()
subset['status'] = 'high'

# Fix 3 – turn on copy-on-write (pandas 2.x)
pd.set_option('mode.copy_on_write', True)
```

Edge cases to mention: - The warning is sometimes a false positive — pandas errs on the side of warning, even when the behaviour would be correct. - Adding `.copy()` is the universal sledgehammer fix but allocates memory; for big DataFrames, prefer the single-`.loc` form. - In a Jupyter notebook the warning prints once per cell — you may have many of them lurking that you've been ignoring.

What NOT to say: - "I just suppress it with `pd.options.mode.chained_assignment = None`" — that hides a real correctness bug; your assignment may be silently dropped. - "It only happens to beginners" — even pandas core developers wrote this bug; that's why CoW was introduced.

Common follow-up: *"What is copy-on-write in pandas 2.x and why was it introduced?"* — A mode where every indexing operation conceptually returns a new object. No more "is this a view or a copy" ambiguity; assignments always behave predictably. Trades a tiny perf cost for huge correctness wins. Opt in via `pd.set_option('mode.copy_on_write', True)` ; becomes default in pandas 3.0.

Stand-out line: *"`SettingWithCopyWarning` is pandas saying 'I don't know if your assignment worked' — in 2.x, enable copy-on-write and the entire warning category goes away because the behaviour becomes deterministic."*

Q16 · MultiIndex slicing — `.loc[(level0, level1), :]` and `pd.IndexSlice`

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay, Target · Topic: indexing

The question (interviewer's verbatim phrasing):

If a DataFrame has a 2-level MultiIndex on rows — say (region, store_id) — how do you select all rows for region = 'south' and store_id between 100 and 200?

The 30-second answer: Use `df.loc[('south', slice(100, 200)), :]` or the cleaner `df.loc[pd.IndexSlice['south', 100:200], :]`. The `slice` object (or `pd.IndexSlice`) lets you specify a range on the inner level. Without it, you can only specify single tuple values, not ranges.

Step-by-step thought process: 1. Start with single-value slicing: `df.loc['south']` returns all stores in south. That's the easy case. 2. For ranges on an inner level, you need either Python `slice(start, stop)` or `pd.IndexSlice[...]`. The `IndexSlice` helper exists specifically because the slice syntax inside a tuple looks awkward. 3. Mention the sortedness requirement — MultiIndex slicing on an inner level requires the index to be lexicographically sorted. If you didn't sort, pandas raises `UnsortedIndexError`. Fix with `df = df.sort_index()`. 4. Note `df.xs('south', level='region')` as the cleaner cross-section accessor when you want all rows at a specific level value.

```
df = pd.DataFrame({'sales': [100, 200, 300, 400]},
                  index=pd.MultiIndex.from_tuples(
                      [('south', 101), ('south', 150), ('north', 100), ('south', 250)],
                      names=['region', 'store_id']))
df = df.sort_index()

# Range on inner level
df.loc[('south', slice(100, 200)), :]
# Cleaner with IndexSlice
df.loc[pd.IndexSlice['south', 100:200], :]
# Cross-section
df.xs('south', level='region')
```

Edge cases to mention: - `df.loc['south']` drops the 'region' level from the result by default — pass `drop_level=False` to keep it. - Cross-section with `.xs(key, level=N)` works on any level, not just the outermost — useful when you want all rows where `store_id = 150` regardless of region. - `pd.IndexSlice` is just a slice-builder; `idx = pd.IndexSlice; df.loc[idx[:, 100:200], :]` works.

What NOT to say: - "Just reset the index and filter normally" — works but defeats the purpose of having a MultiIndex; `groupby` and `unstack` rely on the index structure. - "MultiIndex is too confusing, I avoid it" — it's the right tool for time-series with multiple grouping keys; learn the slicing once.

Common follow-up: "Your `.loc[('south', 100:200)]` raises `UnsortedIndexError` — what do you do?" — `df.sort_index()` to sort lexicographically across both levels, then re-run. Sorting is $O(n \log n)$ but lets pandas use the sorted structure for fast range lookups.

Stand-out line: "MultiIndex slicing fights you until you sort the index — sorted MultiIndex with `pd.IndexSlice` is one of the most expressive selection APIs in any data tool."

Q17 · `.isin()`, `.between()`, `.str.contains()` — convenience filters

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Meesho · Topic: indexing

The question (interviewer's verbatim phrasing):

How do you filter rows where the status is one of ['paid', 'refunded', 'authorized']? And where amount is between 100 and 1000?

The 30-second answer: `df[df['status'].isin(['paid', 'refunded', 'authorized'])]` for membership filtering. `df[df['amount'].between(100, 1000)]` for range filtering — inclusive by default. For substring search use `df[df['note'].str.contains('refund', na=False)]` — pass `na=False` to treat NaN as not-matching.

Step-by-step thought process: 1. `.isin()` is the cleanest membership filter — beats chained `==` ORs. Negate with `~df['status'].isin(...)`. 2. `.between(low, high)` defaults to inclusive on both ends — pass `inclusive='neither'`, `'left'`, or `'right'` for half-open ranges. Equivalent to `(df['amount'] >= low) & (df['amount'] <= high)` but more readable. 3. `.str.contains()` requires `na=False` when the column has nulls — otherwise the result has NaN values that turn into errors in boolean indexing. Common bug. 4. Mention `.str.startswith()` and `.str.endswith()` for prefix/suffix filtering — same `na=False` trick applies.

```
df[df['status'].isin(['paid', 'refunded', 'authorized'])]
df[df['amount'].between(100, 1000)]                # inclusive both
df[df['amount'].between(100, 1000, inclusive='left')] # >=100, <1000
df[df['note'].str.contains('refund', na=False, case=False)]
df[~df['status'].isin(['failed'])]                  # NOT IN
```

Edge cases to mention: - `.isin()` on a Series of strings vs a Series of integers is dtype-strict — `isin([1, 2])` won't match string '1'. - `.str.contains(regex=True)` is the default — use `regex=False` for literal substring search, faster and avoids regex-injection bugs from user input. - `.between()` on datetime columns works — `df[df['paid_at'].between('2026-01-01', '2026-12-31')]`.

What NOT to say: - "I do

`df[(df['status'] == 'paid') | (df['status'] == 'refunded') | ...]` — works but unmaintainable for any list longer than three. Use `.isin()`. - "`.str.contains()` is slow" — it's the same speed as a regex would be; the right comparison is "vs the alternatives in pandas," which it wins on readability.

Common follow-up: "You filter with `.str.contains('refund')` and get a boolean-mask error. What's the cause?" — Your column has NaN values. `.str.contains` returns NaN for NaN inputs, and boolean indexing can't handle NaN in the mask. Add `na=False` (treat NaN as not-matching) or `na=True` (treat as matching).

Stand-out line: "`.isin()` and `.between()` aren't just sugar — they read like the SQL `IN` and `BETWEEN` they descend from, which makes mixed SQL/pandas teams a lot happier."

Q18 · `df.reindex()` vs `df.reset_index()` vs `df.set_index()`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Cred, Swiggy, Target · Topic: indexing

The question (interviewer's verbatim phrasing):

Walk me through the difference between `reindex`, `reset_index` and `set_index` on a DataFrame.

The 30-second answer: `set_index('col')` promotes a column to be the row Index. `reset_index()` does the reverse — demotes the Index back to a column and replaces with a default RangeIndex. `reindex([...])` keeps the Index name but rearranges / adds / removes rows to match the new label list, filling new labels with NaN by default.

Step-by-step thought process: 1. Walk through each one with one sentence and an example. Three operations, three completely different purposes — but their names confusingly all contain "index". 2. `set_index` and `reset_index` are inverses for round-tripping. Use `set_index` after `read_csv` if your CSV has a natural key; use `reset_index` after a `groupby` if you want the group key back as a column. 3. `reindex` is the alignment tool — give it a target Index, get back a DataFrame aligned to that Index. Missing labels get NaN; existing labels are preserved; extra labels are dropped. 4. Mention `set_index('col', drop=False)` to keep the column as both Index and regular column — sometimes handy.

```
df = pd.DataFrame({'id': ['a', 'b', 'c'], 'amount': [10, 20, 30]})

df1 = df.set_index('id')      # 'id' becomes the row index
df1.reset_index()            # back to default 0..n RangeIndex, 'id' is a column again

df2 = df1.reindex(['a', 'b', 'c', 'd']) # 'd' is new → NaN row added
df3 = df1.reindex(['c', 'a'])           # only those two, reordered
```

Edge cases to mention: - `reindex` accepts `fill_value=0` to fill new labels with 0 instead of NaN. - `reset_index(drop=True)` discards the old index entirely instead of moving it to a column — common after filtering. - `set_index([...])` with a list creates a MultiIndex from multiple columns.

What NOT to say: - "They all do the same thing" — three completely different behaviours, three different use cases. - "I use `reindex` to reset the index" — `reset_index` is the right call; `reindex` rearranges to a target.

Common follow-up: "You did a `groupby('region').sum()` and now want region back as a regular column — what do you call?" — `.reset_index()`. The groupby promoted 'region' to the Index; `reset_index()` demotes it back. Alternatively pass `as_index=False` to `groupby` so it never becomes the Index in the first place.

Stand-out line: "Three methods with 'index' in the name and three completely different jobs — `set_index` builds the index, `reset_index` undoes that, `reindex` aligns to a target. Read the verb."

Q19 · `.loc` assignment — adding rows and creating new columns**Tags:** Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Meesho, Walmart Labs · Topic: indexing**The question (interviewer's verbatim phrasing):**

Can you use `.loc` to add a new row to a DataFrame? And to create a new column? Show me both.

The 30-second answer: Yes to both, with caveats. `df.loc[new_label] = [...]` adds a row if `new_label` doesn't exist. `df.loc[:, 'new_col'] = ...` (or just `df['new_col'] = ...`) creates a column. For appending many rows, `pd.concat` is far faster than repeated `.loc` calls.

Step-by-step thought process: 1. Show the syntax for both. `.loc[new_label] = [...]` enlarges the DataFrame by one row. The length of the right-hand side must match the column count. 2. New column via `.loc[:, 'col'] = value` or the shorter `df['col'] = value`. Both work; the shorter form is preferred in new code. 3. Warn against row-by-row append in a loop — it's $O(n^2)$ because pandas reallocates the underlying array every time. Build a list of dicts and `pd.concat` at the end. 4. Mention `df.assign(new_col=...)` as the functional alternative that returns a new DataFrame and chains cleanly.

```
df = pd.DataFrame({'amount': [100, 200]}, index=['a', 'b'])

# Add a row
df.loc['c'] = [300]                # appends row labelled 'c'

# Add a column
df['status'] = ['paid', 'paid', 'failed']    # short form, preferred
df.loc[:, 'status'] = ['paid', 'paid', 'failed'] # equivalent

# Don't do this in a loop — O(n²)
# for record in records:
#     df.loc[len(df)] = record    # SLOW

# Do this instead
df2 = pd.concat([df, pd.DataFrame(records)], ignore_index=True)
```

Edge cases to mention: - `.loc[new_label]` with a label that already exists **OVERWRITES** the row, no warning. Pandas can't tell if you meant insert or update. - `df.append()` was removed in pandas 2.0 — old tutorials still teach it; use `pd.concat` instead. - `df.assign(col=lambda d: d['a'] + d['b'])` lets the new column reference other columns of the current state of the DataFrame — useful for chaining.

What NOT to say: - "I use `.append` in a loop" — `.append` was deprecated in pandas 1.4 and removed in 2.0. Use `pd.concat` once at the end. - "Adding rows is fast" — it's $O(n)$ per row in pandas; for many rows, that's quadratic. Batch the construction.

Common follow-up: "You're collecting rows in a loop from an API — what's the fastest pandas pattern?" — Append to a Python list of dicts inside the loop; outside the loop, `pd.DataFrame(records)` constructs once in $O(n)$. For huge volumes, write rows to a CSV / Parquet file and read back in bulk.

Stand-out line: "Row-by-row `.loc` append in a loop is the canonical pandas anti-pattern — build a list, construct once, you'll see 100x throughput."

Q20 · `.where()` and `.mask()` — conditional updates

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay, Target · Topic: indexing

The question (interviewer's verbatim phrasing):

What do `df.where()` and `df.mask()` do? How are they different from boolean indexing?

The 30-second answer: `df.where(cond)` returns a DataFrame with values KEPT where `cond` is True and replaced with NaN where False. `df.mask(cond)` does the opposite — KEEPS where False, REPLACES where True. Both preserve shape (same number of rows); boolean indexing `df[cond]` returns FEWER rows.

Step-by-step thought process: 1. Lead with the shape distinction. Boolean indexing filters out rows; `.where` and `.mask` keep all rows and replace values. Different use cases. 2. `.where` is "keep where condition holds" — easy to remember from numpy's `np.where(cond, x, y)`. 3. `.mask` is the inverse — "replace where condition holds." Often the more natural framing for "clip outliers" or "censor values." 4. Both accept a replacement value as a second argument: `df.where(cond, other=0)` puts 0 instead of NaN where the condition is False.

```
df = pd.DataFrame({'amount': [100, 200, 300, 400]})

df.where(df['amount'] > 200)           # NaN, NaN, 300, 400
df.where(df['amount'] > 200, other=0)  # 0, 0, 300, 400
df.mask(df['amount'] > 200)           # 100, 200, NaN, NaN
df.mask(df['amount'] > 200, other=999) # 100, 200, 999, 999

# vs boolean indexing — returns only 2 rows
df[df['amount'] > 200]
```

Edge cases to mention: - `.where()` is the broadcasting version of `np.where` — the condition can be a scalar, Series, or DataFrame. - Both work on Series too — `s.where(s > 0)` returns positive values, NaN elsewhere. - `.where(cond, inplace=True)` modifies the caller; default returns a new object.

What NOT to say: - "`.where` and boolean indexing are the same" — they're not; one preserves shape, the other filters. - "I'd just write `.loc[cond] = ...`" — works for assignment but you lose the functional / chainable style.

Common follow-up: "You want to cap all amount values over 10000 to exactly 10000 — what's the cleanest pandas call?" — `df['amount'] = df['amount'].clip(upper=10000)`. Or `df['amount'].mask(df['amount'] > 10000, 10000)`. `clip` is the idiomatic answer for capping.

Stand-out line: "`.where` and `.mask` are the shape-preserving cousins of boolean indexing — when you need to censor or transform values without dropping rows, they're the right tool."

Section C — Column Operations

The middle ten — add a column, drop one, rename, chain ops, vectorize a slow apply. Less glamorous than indexing but every interview has at least two questions from this section. The `apply` perf cliff alone has been asked at Razorpay, Swiggy, and Walmart Labs in the same week.

Q21 · Adding columns — `df['x'] = ...` vs `df.assign(x=...)`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Walmart Labs · Topic: column ops

The question (interviewer's verbatim phrasing):

What's the difference between `df['new_col'] = ...` and `df.assign(new_col=...)` ?
Why does pandas have both?

The 30-second answer: `df['new_col'] = value` modifies the existing DataFrame in place. `df.assign(new_col=value)` returns a NEW DataFrame with the column added — the original is untouched. The functional `.assign` is meant for method chaining; the bracket form is meant for ad-hoc analysis.

Step-by-step thought process: 1. Mutation vs functional return — that's the fundamental split. `df['x'] = ...` is the imperative style; `.assign()` is the functional style. 2. Method chaining is where `.assign()` shines — `(df.query(...).assign(profit=lambda d: d['rev'] -`

`df['cost']).groupby(...).sum())` reads top-down with no intermediate variables. 3. The `lambda d: ...` form inside `.assign()` is key: it sees the DataFrame in its CURRENT state, including columns added earlier in the same chain. Critical for multi-step derivations. 4. Mention that in pandas 2.x with copy-on-write enabled, the perf difference between in-place and functional is essentially gone — pick the one that reads cleaner.

```
# In-place
df['profit'] = df['revenue'] - df['cost']
df['margin'] = df['profit'] / df['revenue']

# Functional / chainable
result = (df
    .query("status == 'paid'")
    .assign(
        profit=lambda d: d['revenue'] - d['cost'],
        margin=lambda d: d['profit'] / d['revenue'], # sees 'profit' just added
    )
    .groupby('region')
    .agg({'profit': 'sum', 'margin': 'mean'}))
```

Edge cases to mention: - `.assign(**{'new col with space': value})` works for column names that aren't valid Python identifiers. - `.assign(a=1, b=2)` adds both at once; order matters because each step sees the previously added columns. - Bracket assignment with a Series respects index alignment — values get matched on label, missing labels fill with NaN.

What NOT to say: - "They do exactly the same thing" — one mutates, one returns new. Different defaults, different idioms. - ".assign is slower" — in 2.x with CoW it's effectively free; readability of chained code outweighs micro-perf in 99% of cases.

Common follow-up: "You're building a chain that derives column B from a newly-added column A in the same step — can you do that with `.assign()`?" — Yes, with the lambda form: `df.assign(A=..., B=lambda d: d['A'] * 2)`. The dict order is preserved (Python 3.7+) and each lambda sees the DataFrame with all earlier additions applied.

Stand-out line: "`.assign()` turns pandas into a pipeline language — once you write a multi-step ETL with `.assign(lambda d: ...)` chains, you stop wanting to read intermediate-variable code ever again."

Q22 · Dropping columns — `.drop()`, `del`, `pop`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: column ops

The question (interviewer's verbatim phrasing):

What are the three ways to remove a column from a DataFrame? When would you use each?

The 30-second answer: `df.drop(columns=['col'])` returns a new DataFrame without that column — functional, doesn't mutate. `del df['col']` is the Python built-in delete — mutates in place. `df.pop('col')` removes the column AND returns it as a Series — useful when you want to extract and remove in one step.

Step-by-step thought process: 1. Lead with `.drop()` — the most common because it composes in chains and accepts a list of columns. 2. `del df['col']` is the imperative one-liner — works because DataFrames support `__delitem__`. Mutates, no return value. 3. `pop` is the dual of dict-pop — gives you the column AND removes it. Use it when you want to move a column out of the frame to use it standalone. 4. Mention `axis=1` legacy — `df.drop(['col'], axis=1)` predates the `columns=` kwarg. Both still work; `columns=` is the readable modern form.

```
df = pd.DataFrame({'a': [1, 2], 'b': [3, 4], 'c': [5, 6]})

df2 = df.drop(columns=['a', 'b'])    # functional, returns new
del df['c']                          # in-place, no return
extracted = df.pop('a')              # removes 'a', returns it as Series

# Legacy axis=1 form, still works
df.drop(['a'], axis=1)
```

Edge cases to mention: - `.drop()` accepts both `columns=` and `index=` to drop rows or columns. The unified API. - `.drop(columns='col')` (string, not list) works for a single column — pandas accepts both. - `errors='ignore'` makes drop silent when the column doesn't exist — useful in cleanup pipelines.

What NOT to say: - "I always use `del`" — fine for one column, ugly for many.

`.drop(columns=[...])` accepts a list and chains. - "Drop deletes the column from the original" — `.drop()` is NON-mutating by default. Pass `inplace=True` to mutate (and even then, the trend is away from inplace).

Common follow-up: "How do you drop a column only if it exists?" —

`df.drop(columns=['maybe_col'], errors='ignore')`. Without `errors='ignore'`, a missing column raises `KeyError`.

Stand-out line: "`.drop()` for new code, `pop()` when you also want the dropped Series — `del` is fine but not chainable, so it doesn't fit in pipeline-style pandas."

Q23 · Renaming columns — `.rename()` vs assignment to `.columns`**Tags:** Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Cognizant · Topic: column ops**The question (interviewer's verbatim phrasing):**

What's the cleanest way to rename one column in a DataFrame? And what if you want to rename all columns at once?

The 30-second answer: For one column: `df.rename(columns={'old_name': 'new_name'})`. For all columns: assign a new list to `df.columns = [...]`. The dict form is non-mutating and chainable; the list assignment mutates in place and requires the right length.

Step-by-step thought process: 1. `.rename(columns={...})` is the targeted, dict-based rename. Returns a new DataFrame; pass `inplace=True` to mutate. Accepts a function too: `.rename(columns=str.lower)` lowercases all column names. 2. `df.columns = ['a', 'b', 'c']` is the bulk rename. Must match the existing column count or it raises. Used when you want a clean slate. 3. Mention `.rename(index={...})` — same method renames row labels. Same `.rename(columns=str.upper)` trick works. 4. Note `.rename` accepts a `mapper=...` arg as positional shortcut and `axis=0` or `1` to pick rows or columns. Less common; the explicit `columns=` is clearer.

```
df.rename(columns={'order id': 'order_id'})      # one column
df.rename(columns=str.lower)                    # all lowercase
df.rename(columns=lambda c: c.replace(' ', '_')) # all spaces to underscore

# Bulk rename, mutates
df.columns = ['id', 'amount', 'status']
```

Edge cases to mention: - `.rename` is forgiving — passing an old name that doesn't exist silently does nothing (unless you pass `errors='raise'`). - Direct assignment to `df.columns` requires the exact length; otherwise it raises `ValueError`. - `df.columns.str.replace(' ', '_')` returns a new Index — assign back to `df.columns` for a vectorized rename of all columns.

What NOT to say: - "I rename by loop iterating over columns" — `.rename(columns=function)` does this in one line, vectorized. - "I always use `inplace=True`" — modern pandas style returns new objects; the trend is away from `inplace`. Especially under copy-on-write semantics.

Common follow-up: "Your CSV has column names with leading/trailing whitespace — how do you clean all of them at once?" — `df.columns = df.columns.str.strip()`. The columns Index has full `.str` accessor support.

Stand-out line: "`.rename(columns=function)` is the rename idiom for any dataset that came from a non-engineer — one line and every column is normalized."

Q24 · `df.astype()` vs `pd.to_numeric()` vs `convert_dtypes()`**Tags:** Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Cred, Target, Goldman · Topic: column ops**The question (interviewer's verbatim phrasing):**

If a column came in as object but should be numeric, how do you convert it? What's the difference between `astype`, `to_numeric`, and `convert_dtypes`?

The 30-second answer: `df['col'].astype('int64')` casts blindly — raises if any value can't be converted. `pd.to_numeric(df['col'], errors='coerce')` casts with safety — unparseable values become NaN instead of raising. `df.convert_dtypes()` is the bulk converter that picks the best modern dtype per column (pandas nullable dtypes).

Step-by-step thought process: 1. `astype()` is the all-or-nothing cast. Use when you're confident the data is clean. Raises on any failure. 2. `pd.to_numeric` with `errors='coerce'` is the cleaning workhorse — bad rows become NaN, you investigate them, fix them, move on. Same pattern for `pd.to_datetime`. 3. `convert_dtypes()` is the bulk upgrade — call after `read_csv` to push columns to nullable dtypes (`Int64`, `string`, `boolean`) which handle missing values better than the numpy-backed defaults. 4. Mention `df.astype({'a': 'int64', 'b': 'string'})` for multi-column `astype` in one call.

```
df['amount'].astype('float64')           # strict
pd.to_numeric(df['amount'], errors='coerce') # safe, NaN on failure
pd.to_datetime(df['paid_at'], errors='coerce') # safe datetime
df.convert_dtypes()                       # bulk to modern dtypes
df.astype({'amount': 'float64', 'status': 'category'}) # multi-column
```

Edge cases to mention: - `astype('int64')` on a column with NaN raises because numpy int64 can't hold NaN — cast to nullable `'Int64'` instead. - `pd.to_numeric(downcast='integer')` finds the smallest int type that fits — useful for memory optimization. - `convert_dtypes(dtype_backend='pyarrow')` (2.x) pushes columns to pyarrow-backed dtypes — faster for string-heavy data.

What NOT to say: - "I always use `astype`" — fine when data is clean, dangerous when it isn't. `to_numeric` is the defensive choice. - "`convert_dtypes` is just `astype` for many columns" — it's smarter; it picks dtypes based on inspection of the data, not blind casting.

Common follow-up: "Your `astype('int64')` raises because there's one NaN — what do you do?" — Cast to nullable `'Int64'` (capital I) which supports `pd.NA`, OR fillna with a sentinel like 0 first, OR use `pd.to_numeric(errors='coerce')` which produces float64 with NaN. Pick based on whether the NaN means "unknown" or "0".

Stand-out line: " `to_numeric(errors='coerce')` is my default loader — it surfaces the dirty rows as NaN where I can find them, instead of crashing the pipeline at row 412,000."

Q25 · The `.str` accessor — vectorized string operations

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Meesho, Walmart Labs · Topic: column ops

The question (interviewer's verbatim phrasing):

How do you apply a string method like `lower()` or `strip()` to every value in a Series of strings?

The 30-second answer: Through the `.str` accessor — `df['name'].str.lower()`, `df['name'].str.strip()`, `df['name'].str.replace(' ', '_')`. It's the vectorized bridge between pandas and Python string methods. NaN values are passed through unchanged; you don't need to handle them manually.

Step-by-step thought process: 1. State the accessor pattern — `.str.X()` where X is any Python string method. Vectorized, runs in C-level loop over the column. 2. Mention the chaining: `.str.strip().str.lower().str.replace(' ', '_')` is a typical normalization chain. 3. Note NaN handling — `.str` methods return NaN for NaN input. Important for the `.str.contains(na=False)` trick (see Q17). 4. Regex support — `.str.replace('regex', 'repl', regex=True)`, `.str.extract(r'(\d+)')`, `.str.findall(r'\w+')`. Full regex toolbox.

```
df['name'].str.lower()
df['name'].str.strip()
df['name'].str.replace(r'\s+', '_', regex=True)
df['email'].str.extract(r'@(.+)\.$')[0]           # extract domain
df['phone'].str.match(r'^\+?\d{10,15}$')         # validate format
df['notes'].str.contains('refund', case=False, na=False)
```

Edge cases to mention: - `.str.split(',', expand=True)` splits each value and expands to multiple columns — common for parsing CSV-in-a-cell. - `.str.cat(sep=',')` concatenates all values into one string (the inverse). - On pandas 2.x with `string[pyarrow]` dtype, `.str` operations are 2-5x faster than on object-dtype columns.

What NOT to say: - "I'd do `df['col'].apply(str.lower)`" — works but slow; `.str.lower()` is vectorized in C. Don't use `apply` when an accessor exists. - "`.str` doesn't handle NaN" — it does; NaN is passed through. The gotcha is in boolean indexing downstream, not in the `str` method itself.

Common follow-up: "You have a column of phone numbers in formats like `+91-98765 43210`, `9876543210`, `(091) 9876543210` — how do you normalize all to `+919876543210`?" — Chain `.str.replace(r'[\s\-\(\)]', '', regex=True)` then handle the `+91` prefix with `.str.replace(r'^0?', '+91', regex=True)`. Two regex passes, vectorized.

Stand-out line: "The `.str` accessor is what makes pandas competitive with awk for log cleaning — chain four `.str` calls and you've normalized a million dirty values in under a second."

Q26 · The `.dt` accessor — datetime operations

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, JP Morgan, Cred · Topic: column ops

The question (interviewer's verbatim phrasing):

How do you extract the month or weekday from a datetime column? And how do you check if a date is the end of the month?

The 30-second answer: The `.dt` accessor. `df['paid_at'].dt.month`, `df['paid_at'].dt.day_name()`, `df['paid_at'].dt.is_month_end`. Vectorized like `.str`, requires the column dtype to be datetime — call `pd.to_datetime` first if it's still object.

Step-by-step thought process: 1. State the prerequisite — column must be datetime dtype. `df['paid_at'] = pd.to_datetime(df['paid_at'])` first. 2. Walk the common extractors: `.dt.year`, `.dt.month`, `.dt.day`, `.dt.hour`, `.dt.dayofweek` (Monday=0), `.dt.day_name()` (string name). 3. Mention boundary helpers: `.dt.is_month_end`, `.dt.is_quarter_start`, `.dt.is_year_end`. Common in finance reporting. 4. Time arithmetic: `df['paid_at'] - pd.Timedelta(days=7)` gives last week's same instant; `(df['shipped_at'] - df['paid_at']).dt.days` gives integer days between two columns.

```
df['paid_at'] = pd.to_datetime(df['paid_at'])
df['paid_at'].dt.year
df['paid_at'].dt.month
df['paid_at'].dt.day_name()           # 'Friday', 'Saturday', ...
df['paid_at'].dt.dayofweek           # 0=Mon, 6=Sun
df['paid_at'].dt.is_month_end
df['paid_at'].dt.tz_localize('Asia/Kolkata')
df['paid_at'].dt.tz_convert('UTC')

# Time delta arithmetic
ship_lag = (df['shipped_at'] - df['paid_at']).dt.days
```

Edge cases to mention: - `.dt.dayofweek` returns 0 for Monday — common confusion with SQL-style 1=Sunday. - Timezone-naive vs timezone-aware columns can't be compared without one side being converted — pandas raises. - `.dt.normalize()` zeroes out the time portion — handy for "group by date" without a separate column.

What NOT to say: - "I'd `apply(lambda x: x.month)`" — slow; `.dt.month` is vectorized. - "Pandas handles timezones automatically" — it doesn't; you must `.dt.tz_localize` to attach a timezone or `.dt.tz_convert` to change one.

Common follow-up: "Your timestamps are stored in UTC but reports need IST — what's the cleanest conversion?" — `df['paid_at'].dt.tz_convert('Asia/Kolkata')`. If the column is timezone-naive but you know the values are in UTC, first `dt.tz_localize('UTC')`, then `dt.tz_convert('Asia/Kolkata')`.

Stand-out line: "`.dt` is the date-arithmetic version of `.str` — once your timestamps are properly typed, every date question becomes a one-liner."

Q27 · Vectorization vs `.apply()` — the perf cliff

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Walmart Labs, JP Morgan, Goldman, Cred · Topic: column ops

The question (interviewer's verbatim phrasing):

You wrote `df['margin'] = df.apply(lambda row: row['revenue'] - row['cost'], axis=1)` and your colleague says it's slow. Why, and what's the fix?

The 30-second answer: `apply(axis=1)` iterates row by row in Python — pandas converts each row to a Series and calls the lambda once per row. That's 100-1000x slower than `df['margin'] = df['revenue'] - df['cost']`, which operates on the whole numpy array in C. The fix is to write the expression directly; pandas auto-broadcasts over the column.

Step-by-step thought process: 1. State why apply is slow — Python loop over rows, Series allocation per row, function call overhead per row. None of which the optimizer can eliminate. 2. Show the vectorized form. Most arithmetic, comparison, and string operations can be expressed in vectorized form directly. The `apply` is almost never the right answer. 3. When `apply` IS okay: when the function genuinely needs the entire row context AND can't be expressed in column ops (rare — usually composition of `.str`, `.dt`, `np.where`, `.where`, `.mask` covers it). 4. Mention `np.where(cond, x, y)` for if-else logic, `np.select` for multi-way, `np.vectorize` if you're stuck with a scalar Python function (still slow but cleaner than apply).

```
# Slow – apply over rows
df['margin'] = df.apply(lambda row: row['revenue'] - row['cost'], axis=1)

# Fast – vectorized
df['margin'] = df['revenue'] - df['cost']

# Conditional, also vectorized
df['tier'] = np.where(df['amount'] > 10000, 'high',
                    np.where(df['amount'] > 1000, 'medium', 'low'))

# Or np.select for multi-way
df['tier'] = np.select(
    [df['amount'] > 10000, df['amount'] > 1000],
    ['high', 'medium'],
    default='low'
)
```

Edge cases to mention: - `apply(axis=0)` over columns (the default) is much faster than `axis=1` over rows — fewer iterations. - On pandas 2.x, some `apply` paths got a `engine='numba'` option that JITs the function — only helps for pure-numeric scalar operations. - For string ops, `.str.X()` is the vectorized accessor — almost never use `apply` on a string column.

What NOT to say: - "Apply is fine, the CPU is fast enough" — at 50k rows you don't feel it; at 50M rows the apply takes 10 minutes and the vectorized form takes 200ms. - "I'll just rewrite in Cython" — overkill; 99% of `apply` calls have a vectorized equivalent in plain pandas/numpy.

Common follow-up: "Your function calls an external API per row — `apply` is unavoidable. What's the next best step?" — Batch it. Collect unique inputs into a set, call the API once per unique input, build a dict, then `df['result'] = df['input'].map(dict)`. The map call is vectorized; the API calls are deduped.

Stand-out line: "`df.apply(axis=1)` is a Python loop wearing a pandas costume — every time I see it in production, the fix is a one-line vectorized expression that's 200x faster."

Q28 · `.map()` vs `.apply()` vs `.applymap()` (and `.map` on DataFrame in 2.x)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman, Target · Topic: column ops

The question (interviewer's verbatim phrasing):

What's the difference between `.map()`, `.apply()` and `.applymap()` on a DataFrame?

The 30-second answer: On a Series — `.map()` substitutes values via a dict or function, element-wise; `.apply()` applies a function over the values (similar to map for scalars). On a DataFrame —

`.apply()` applies a function over rows or columns; `.applymap()` applied element-wise to every cell, but was renamed to `.map()` in pandas 2.1. So `df.map(fn)` is the modern element-wise call.

Step-by-step thought process: 1. Split by object type — Series vs DataFrame. The methods have overloaded names but different semantics. 2. On a Series, `.map()` is the substitution tool: `df['country'].map({'IN': 'India', 'US': 'United States'})`. Common for code-to-label mapping. 3. On a DataFrame, `.apply(fn, axis=0)` runs `fn` per column; `axis=1` runs per row (slow, see Q27). `.applymap(fn)` runs on every cell — renamed to `.map(fn)` in pandas 2.1. 4. Mention the deprecation — `applymap` still works in 2.x but emits a `FutureWarning`. New code should use `df.map(fn)`.

```
# Series.map — substitution / function
df['country_label'] = df['country'].map({'IN': 'India', 'US': 'United States'})
df['amount_log'] = df['amount'].map(np.log)

# DataFrame.apply
df.apply(lambda col: col.max() - col.min()) # range per column
df.apply(lambda row: row.sum(), axis=1)     # row sum (slow)

# DataFrame.map (replaces applymap as of 2.1)
df.map(lambda x: f"{x:.2f}")                # format every cell as 2-decimal string
```

Edge cases to mention: - `.map()` on a Series with a dict that doesn't cover all values returns NaN for unmapped keys. Pass `na_action='ignore'` to keep originals. - `.apply` on a DataFrame returning a Series per call vs returning a scalar per call produces different shaped output — read the result `.shape` to confirm. - `df.map(fn)` (2.1+) replaces `df.applymap(fn)` — same behaviour, less confusing name (since Series already has `.map`).

What NOT to say: - "They're all the same thing" — three different shapes, three different jobs. - "`.applymap` is the right call for element-wise" — deprecated in 2.1 in favor of `.map`. Use the new name.

Common follow-up: "`df.map(fn)` raises in older code — why?" — Before pandas 2.1, DataFrames don't have `.map`; only `.applymap`. Either upgrade pandas or use the older method name. Check `pd.__version__`.

Stand-out line: "Pandas 2.1 unified the element-wise transform under `.map()` on both Series and DataFrame — one less name to memorize, one fewer deprecation to follow next year."

Q29 · Conditional column derivation — `np.where`, `np.select`, `.where`, `.mask`

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman, Target · Topic: column ops

The question (interviewer's verbatim phrasing):

You need to create a tier column: 'gold' if amount > 10000, 'silver' if amount > 5000, 'bronze' otherwise. What's the pandas way?

The 30-second answer: `np.select` is the cleanest for multi-way conditions: pass a list of conditions, a list of corresponding values, and a default. For binary if-else, `np.where(cond, x, y)` is simpler. Avoid `apply(lambda row: ...)` — same logic, 100x slower.

Step-by-step thought process: 1. Recognize the shape — multi-way conditional. `np.select` is the dedicated tool. 2. `np.select` evaluates conditions in order; first match wins. The list of values must match the list of conditions in length. 3. For two-way, `np.where(cond, x, y)` is shorter — equivalent to a vectorized ternary. 4. Mention the alternative — `pd.cut` for binning numeric ranges into named labels; cleaner than `np.select` for pure-range cases.

```
# np.select for multi-way
df['tier'] = np.select(
    condlist=[df['amount'] > 10000, df['amount'] > 5000],
    choicelist=['gold', 'silver'],
    default='bronze'
)

# np.where for binary
df['risk'] = np.where(df['amount'] > 10000, 'high', 'normal')

# pd.cut for pure numeric ranges
df['tier'] = pd.cut(df['amount'],
                    bins=[0, 5000, 10000, np.inf],
                    labels=['bronze', 'silver', 'gold'])
```

Edge cases to mention: - `np.select` returns object dtype by default — cast back if you need a category or string. - `pd.cut` defaults to right-inclusive bins — pass `right=False` for left-inclusive (which matches the `>` style in `np.select`). - Stack `np.where` calls for nested conditions but stop at three levels — past that, `np.select` reads better.

What NOT to say: - "I'd use `apply(lambda row: 'gold' if row['amount'] > 10000 else 'silver' if ...)`" — works but 100x slower. The interviewer is testing whether you know the vectorized form. - "`pd.cut` doesn't work for strings" — `pd.cut` is for numeric ranges; conditions involving multiple columns need `np.select`.

Common follow-up: "Conditions are on TWO columns — `amount > 10000` AND `country == 'IN'` qualify for 'gold'. How does that change your approach?" — `np.select` accepts compound conditions: `condlist=[(df['amount'] > 10000) & (df['country'] == 'IN'), df['amount'] > 5000]`. Same structure, just more complex per-condition.

Stand-out line: "`np.select` is the SQL `CASE WHEN` of pandas — once you reach for it instead of nested `np.where`, your transformation pipelines become readable again."

Q30 · Memory profiling — `df.memory_usage(deep=True)`

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay, Target · Topic: column ops

The question (interviewer's verbatim phrasing):

Your DataFrame is using 8GB of RAM and you need to bring it under 2GB. Walk me through the diagnostic and the fixes.

The 30-second answer: Start with `df.memory_usage(deep=True)` to see per-column memory; `deep=True` is critical for object columns (otherwise it underreports). Common fixes — convert low-cardinality string columns to `category`, downcast `int64` to the smallest fitting int via `pd.to_numeric(downcast='integer')`, switch to pyarrow-backed strings, drop columns you don't need.

Step-by-step thought process: 1. State the diagnostic call first. `df.memory_usage(deep=True)` returns bytes per column. Without `deep=True`, object columns are reported as 8 bytes per cell (pointer size only) — wildly underestimates. 2. Walk through the fix toolkit in order: drop unused columns first (free wins); convert low-cardinality strings (status, region, country) to `category` — often 10-30x reduction; downcast integers — `pd.to_numeric(downcast='integer')` finds the smallest fitting int type. 3. Mention pyarrow string backend (2.x) — `df.convert_dtypes(dtype_backend='pyarrow')` converts strings to pyarrow chunked arrays which are 2-3x smaller than object/Python strings. 4. For the hard cases — read in chunks, process incrementally, or move to `polars` / `duckdb` for the heavy lift. Pandas has a "must fit in RAM" architecture.

```
# Diagnostic
df.memory_usage(deep=True)          # bytes per column, including object overhead
df.memory_usage(deep=True).sum() / 1e9  # total GB

# Fix 1 – categorical for low-cardinality
df['status'] = df['status'].astype('category')

# Fix 2 – downcast ints
df['count'] = pd.to_numeric(df['count'], downcast='integer')

# Fix 3 – pyarrow backend
df = df.convert_dtypes(dtype_backend='pyarrow')

# Verify
df.memory_usage(deep=True).sum() / 1e9
```

Edge cases to mention: - `category` dtype helps only when cardinality is low (under ~50% of the row count). For high-cardinality columns, it actually adds overhead. - `downcast='integer'` checks the actual range — a `count` column with values 0-200 will downcast to `int8`, saving 8x. - pyarrow strings handle missing values via `pd.NA` natively — bonus correctness win.

What NOT to say: - "Just buy more RAM" — true sometimes, but if the next dataset is 80GB, you've kicked the can. Learn the fixes. - "I'd write everything to parquet" — parquet helps on disk; in-memory you still need the dtype fixes.

Common follow-up: "You converted a status column to category and now `groupby('status')` is faster — why?" — Categorical columns store integer codes internally; groupby on integer codes is faster than groupby on strings. Bonus: the codes are dense, so grouping is contiguous-memory friendly.

Stand-out line: " `memory_usage(deep=True)` then category-ize the low-cardinality columns — that combo has saved every memory-bound pandas job I've inherited."

Section D — Common Gotchas & 2.x Patterns

The final ten — the questions interviewers ask to separate candidates who learned pandas from a 2020 tutorial from those who use it daily in 2026. Copy-on-write, `pd.NA` vs `np.nan`, the pyarrow backend, chained-assignment trap, the `inplace=True` debate. These show up in product-co and GCC panels far more than in service-MNC pools.

Q31 · Copy-on-write in pandas 2.x — what changed

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman, Cred · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

What is copy-on-write in pandas 2.x, and why is it being made the default in pandas 3.0?

The 30-second answer: Copy-on-write (CoW) is a semantic change where every operation conceptually returns an independent object — no shared memory under the hood that you can mutate by accident. The user-facing effect is that `SettingWithCopyWarning` disappears because the behaviour is now deterministic: an assignment either succeeds on the object you called it on, or pandas raises if the operation makes no sense.

Step-by-step thought process: 1. State the principle: every indexing / slicing operation returns a logical copy. Pandas optimizes under the hood to avoid actually copying data until you write, but the semantics are "as if copied." 2. The bug it fixes: chained indexing previously could silently no-op assignments. With CoW, you get a clear behaviour — assignments always work on the object you have a reference to. 3. The cost: a small perf hit because pandas tracks reference counts to know when it can avoid an actual copy. Negligible in most workloads. 4. The migration path: enable with `pd.set_option('mode.copy_on_write', True)` today; default in 3.0. Code that relied on view-mutation will need fixes — uncommon in practice.

```
# Opt in today (2.x)
pd.set_option('mode.copy_on_write', True)

# Behaviour under CoW
df = pd.DataFrame({'a': [1, 2, 3]})
view = df['a']          # logical copy under CoW
view.iloc[0] = 999      # modifies view only; df['a'].iloc[0] is still 1

# Without CoW (legacy)
# view.iloc[0] = 999 might or might not propagate to df depending on internals
```

Edge cases to mention: - Code that explicitly relies on chained mutation (`df['a'][0] = 5`) will need to switch to `df.loc[0, 'a'] = 5` . The error you'll see under CoW is more predictable. - Functions that return slices (e.g., `def get_paid(df): return df[df['status']=='paid']`) used to be ambiguous; under CoW, the returned slice is always logically independent. - Performance: CoW is generally faster on average because pandas can skip defensive copies it used to make.

What NOT to say: - "CoW makes pandas slower" — empirically it's neutral to slightly faster. It removes defensive copies pandas used to make. - "It's just the same as `inplace=False` everywhere" — it's a deeper semantic guarantee about view-vs-copy behaviour, enforced at the C level.

Common follow-up: "How do I test my code for CoW compatibility before pandas 3.0?" — Enable `pd.set_option('mode.copy_on_write', 'warn')` in 2.x — pandas emits warnings on operations whose behaviour changes. Run your test suite; fix the warnings.

Stand-out line: "CoW is pandas finally answering the question 'is this a view or a copy?' with 'doesn't matter — write semantics are predictable either way.'"

Q32 · `pd.NA` vs `np.nan` vs `None` — three missing values

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, JP Morgan, Goldman, Cred · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

Pandas has `np.nan`, `None`, and `pd.NA`. What's the difference and when does each one show up?

The 30-second answer: `np.nan` is a numpy float — used in numpy-backed numeric columns. `None` is Python's None — shows up in object columns. `pd.NA` is pandas's new scalar for missing values in nullable dtypes (`Int64`, `boolean`, `string`) — propagates through operations correctly (e.g., `pd.NA == pd.NA` returns `pd.NA`, not False). New code on nullable dtypes uses `pd.NA`.

Step-by-step thought process: 1. Three values, three contexts. Memorize which one appears where. 2. `np.nan` is what `read_csv` produces in numeric columns. It's a float, so any int column with missing values upcasts to float64. The original sin of pandas missing-value handling. 3. `None` is Python's missing — shows up in object columns from CSVs with empty cells where pandas didn't promote to numeric. Often equivalent to NaN but technically distinct. 4. `pd.NA` (introduced 1.0, default in nullable dtypes) — preserves dtype (Int64 stays Int64), propagates correctly through comparisons. Use when you want missing-value support without losing int-ness.

```
s_legacy = pd.Series([1, 2, None])           # float64, the None becomes NaN
s_legacy.dtype                               # float64

s_nullable = pd.Series([1, 2, None], dtype='Int64') # Int64, the None becomes pd.NA
s_nullable.dtype                             # Int64
s_nullable[2]                               # <NA>

# Comparison semantics
np.nan == np.nan        # False (numpy)
pd.NA == pd.NA          # <NA> (propagating)
pd.isna(np.nan)         # True
pd.isna(pd.NA)          # True (universal check)
```

Edge cases to mention: - `pd.isna(x)` is the universal missing-value check — works for all three. Always use it over `x == np.nan` or `x is None`. - Comparing `pd.NA == something` returns `pd.NA`, not True/False. This propagation means boolean indexing with NA values raises — you must filter NAs first. - pyarrow-backed dtypes use `pd.NA` too.

What NOT to say: - "They're all the same" — they're not; the dtype that contains them is different and the comparison semantics differ. - "I check with `x == np.nan`" — that always returns False (NaN != itself). Use `pd.isna(x)`.

Common follow-up: "You filter with `df[df['flag'] == True]` on a nullable boolean column with NAs — why don't the NA rows appear?" — Because `pd.NA == True` returns `pd.NA`, not False, and boolean indexing drops NA-mask values silently. Use `df[(df['flag'] == True).fillna(False)]` or rewrite as `df.loc[df['flag'].eq(True).fillna(False)]`.

Stand-out line: "`pd.NA` is the missing value that finally behaves like SQL NULL — three-valued logic, propagation, dtype preservation. It's why nullable dtypes are worth adopting even before pandas 3.0 forces the issue."

Q33 · Nullable `Int64` vs numpy `int64`

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, Goldman, Target, JP Morgan · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

Why does pandas have both `int64` and `Int64`? When would you use the capital-I version?

The 30-second answer: `int64` is numpy-backed — fast, but cannot represent NaN, so any missing value upcasts the column to `float64`. `Int64` (capital) is pandas's nullable integer — stores `pd.NA` natively, keeps the column integer-typed. Use `Int64` whenever the column conceptually represents counts/ids that might be missing.

Step-by-step thought process: 1. State the contract difference — one supports missing values without upcasting, the other doesn't. 2. The pain point `Int64` solves: `read_csv` on `[1, 2, , 4]` upcasts the column to `float64` because of the missing value. Now downstream code that did `range(n)` or used the column as an index breaks because it's `1.0, 2.0, ...`. 3. Show the conversion: `df['col'] = df['col'].astype('Int64')` — converts and uses `pd.NA` for missing. 4. Trade-off: nullable `Int64` is slightly slower for some operations and not all third-party libraries accept it (scikit-learn pre-1.4 raised). The pandas 2.x ecosystem has mostly caught up.

```
s_legacy = pd.Series([1, 2, None, 4])
s_legacy.dtype           # float64 (upcast)
s_legacy.iloc[0]         # 1.0 (float)

s_nullable = pd.Series([1, 2, None, 4], dtype='Int64')
s_nullable.dtype         # Int64
s_nullable.iloc[0]       # 1 (still int)
s_nullable.iloc[2]       # <NA>

# Bulk conversion
df = df.convert_dtypes()           # smart auto-conversion
df['count'] = df['count'].astype('Int64') # explicit
```

Edge cases to mention: - `Int8` , `Int16` , `Int32` , `Int64` , and `UInt8` .. `UInt64` all exist — all nullable. Pick the smallest that fits for memory wins. - Operations between `Int64` and numpy `int64` are allowed; the result is `Int64` if any operand was nullable. - `to_numpy()` on a nullable `Int64` with NAs returns object dtype unless you specify `na_value=` .

What NOT to say: - "I always use `int64` for speed" — when your data has nulls, you don't have a choice — `int64` can't hold them, so you've forced `float64` upcast. - "Capital-I is just a label" — no, it's a completely different storage class (`IntegerArray` with a mask), with different semantics.

Common follow-up: "Why is `Int64` the convention for ids that might be missing?" — `Float64` representations of large ids lose precision past 2^{53} — a 16-digit transaction id rounds. `Int64` stays exact up to 2^{63} . So for id columns, `Int64` preserves both nullability AND precision.

Stand-out line: " `Int64` is pandas finally treating ints like a first-class type that can have missing values — every legacy pipeline that has `df['count'].fillna(0).astype(int)` is just simulating it badly."

Q34 · The pyarrow backend — `dtype_backend="pyarrow"`

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, Goldman, Cred, JP Morgan · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

What's the pyarrow backend in pandas 2.x and what do you gain by switching to it?

The 30-second answer: Pyarrow-backed dtypes store the column data as Apache Arrow arrays under the hood instead of numpy. You get faster string operations (2-5x), smaller memory for strings, native `pd.NA` everywhere, and free interop with Polars / DuckDB / Spark that also use Arrow. Trade-off: not every operation has a pyarrow code path yet; some fall back to numpy.

Step-by-step thought process: 1. State the storage change — pyarrow arrays instead of numpy. Same DataFrame surface API, different backend. 2. List the wins: faster string ops, smaller memory footprint, consistent missing-value semantics (`pd.NA` always), zero-copy interop with the Arrow ecosystem. 3. List the trade-offs honestly: not 100% feature parity yet — some operations fall back to numpy, occasional perf cliffs. Most warehouse-style workloads are net wins; some scientific workloads are still better on numpy. 4. Show how to enable. There's no global default — pass `dtype_backend='pyarrow'` per-call to `read_csv`, `read_parquet`, `convert_dtypes`, etc. (`pd.read_csv(..., dtype_backend='pyarrow')` at read time; `df.convert_dtypes(dtype_backend='pyarrow')` on an existing frame). The closest thing to a global switch is `pd.set_option('future.infer_string', True)`, which only flips string columns to pyarrow strings.

```
# At read time
df = pd.read_parquet('orders.parquet', dtype_backend='pyarrow')

# Convert existing
df = df.convert_dtypes(dtype_backend='pyarrow')

# Check
df.dtypes
# id          int64[pyarrow]
# amount      double[pyarrow]
# status      string[pyarrow]
# paid_at     timestamp[ns][pyarrow]
```

Edge cases to mention: - `pyarrow` package is a hard dependency for this backend — `pip install pyarrow`. - Not all third-party libraries handle pyarrow-backed columns yet; you may need `.to_numpy()` at the boundary. - For string-heavy DataFrames (logs, text), the speed-up is dramatic — `.str.lower()` runs 3-5x faster than on object-dtype strings.

What NOT to say: - "Pyarrow makes everything faster" — most things, not all. Some numeric ops are similar; some have pyarrow code paths still missing. - "It's experimental" — it's been production-ready since pandas 2.0 (early 2023). Many fintech and logistics shops use it as the default.

Common follow-up: "You convert to pyarrow backend and one operation gets slower — what do you check?" — Whether that operation has a pyarrow implementation or falls back to numpy via conversion. Inspect with the pandas changelog or measure with `%timeit` before and after. If the fallback is costly, keep just that column on numpy.

Stand-out line: "The pyarrow backend is pandas accepting that Arrow won the columnar format war — adopting it gets you compatibility with Polars and DuckDB for free, and your string-heavy pipelines run 3x faster."

Q35 · `inplace=True` — why the pandas team is deprecating it

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Cred, Target, Goldman · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

What's the deal with `inplace=True` in pandas? Is it good practice to use it?

The 30-second answer: `inplace=True` was meant to save memory by modifying the DataFrame without creating a copy — but it usually didn't actually save memory because pandas often allocated under the hood anyway, and it broke method chaining. Modern pandas style is to NOT use it: write `df = df.dropna()` not `df.dropna(inplace=True)`. Under copy-on-write in 2.x, `inplace=True` becomes increasingly meaningless.

Step-by-step thought process: 1. State the historical promise — save memory by modifying the original. State the reality — pandas often allocated a new buffer internally anyway, so the saving was illusory. 2. List the costs: breaks method chaining (no return value), the implementation is inconsistent across methods, and CoW makes the distinction moot. 3. Reference the official guidance: pandas docs now recommend the functional style for almost all methods. Some methods (`update` , `iloc[] =`) genuinely do mutate; everything else is moving toward "return new." 4. Show the equivalent: `df = df.dropna()` is the same outcome with cleaner code, no perf penalty.

```
# Avoid
df.dropna(inplace=True)
df.fillna(0, inplace=True)
df.rename(columns={'x': 'X'}, inplace=True)

# Prefer
df = df.dropna()
df = df.fillna(0)
df = df.rename(columns={'x': 'X'})

# Or chain
df = (df
      .dropna()
      .fillna(0)
      .rename(columns={'x': 'X'}))
```

Edge cases to mention: - `inplace=True` is being deprecated method-by-method; in some versions it raises warnings already. - For huge DataFrames, perf-conscious code can sometimes benefit from `inplace=True`, but the difference is usually <5% — not worth the readability cost. - Method-chaining style (`.pipe` , `.assign`) requires the functional return — `inplace=True` actively breaks it.

What NOT to say: - "`inplace=True` is faster" — not measurably; pandas usually allocates internally regardless. - "Method chaining doesn't work in pandas" — it does and is the modern idiom; that's why functional returns matter.

Common follow-up: "*You have a 5GB DataFrame and want to drop a column without allocating 5GB — does `inplace=True` help?*" — Honestly, probably not by much; pandas's `.drop()` mostly drops the column reference rather than rebuilding the buffer. The bigger win is to not hold the original reference: `df = df.drop(columns=['x'])` — the old DataFrame can be garbage-collected.

Stand-out line: "`inplace=True` was a perf shortcut that turned out to be neither a real perf win nor a real shortcut — modern pandas style returns new objects, and the chain reads top-down."

Q36 · `read_csv` — the kwargs that save your job

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred, Meesho · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

When you read a CSV, what are the parameters you'd reach for to handle a real, messy file?

The 30-second answer: `dtype=` to pin column types up front; `parse_dates=` to parse datetime columns at read time; `na_values=` to recognize project-specific missing markers (`'N/A'` , `'-'` , `'NULL'`); `usecols=` to read only columns you need; `chunksize=` for files that don't fit in memory; `engine='pyarrow'` for big files for 3-5x speed.

Step-by-step thought process: 1. Walk through each kwarg with a one-line reason. `dtype=` prevents wrong inference (large ids becoming float). `parse_dates=` saves a separate `pd.to_datetime` step. 2. `na_values=['N/A', '-', 'NULL']` is the cleaning workhorse — every real CSV has at least one weird missing marker. 3. `usecols=` is the memory win — if the file has 50 columns and you need 5, read only those. 10x less RAM. 4. `engine='pyarrow'` (pandas 1.4+) is the speed win on big files — typically 3-5x faster than the default Python engine.

```
df = pd.read_csv(
    'orders.csv',
    dtype={'id': 'Int64', 'amount': 'float64'},
    parse_dates=['paid_at', 'shipped_at'],
    na_values=['N/A', '-', 'NULL', ''],
    usecols=['id', 'amount', 'status', 'paid_at'],
    engine='pyarrow',                # speed
    dtype_backend='pyarrow',        # pyarrow-backed columns
)

# For huge files
for chunk in pd.read_csv('huge.csv', chunksize=100_000):
    process(chunk)
```

Edge cases to mention: - `parse_dates=` accepts a list of column names OR a list of lists (combine multiple columns into one timestamp). - `dtype=` with `'Int64'` (nullable) is safer than `'int64'` for id columns — handles empty cells without upcasting. - `engine='pyarrow'` requires the `pyarrow` package and ignores some parameters (`chunksize` doesn't work with it as of pandas 2.x).

What NOT to say: - "I just call `pd.read_csv('file.csv')` and let pandas figure it out" — fine for clean files; for real production CSVs you'll be debugging dtype issues for hours. - "I always use `chunksize` for safety" — only when the file truly doesn't fit; chunked processing makes downstream code harder to write.

Common follow-up: "Your CSV reads in 8 minutes — how do you speed it up?" — Try `engine='pyarrow'` first (often 3-5x). If still slow, convert the source to parquet once and use `pd.read_parquet` thereafter — typically another 5-10x on top. The CSV format itself is the bottleneck.

Stand-out line: "Half of pandas perf issues are people not passing the right `read_csv` kwargs — `dtype=`, `usecols=`, `engine='pyarrow'` together turn an 8-minute load into 45 seconds."

Q37 · The chained-assignment trap

Tags: Difficulty: Hard · Asked at: Razorpay, Walmart Labs, Goldman, JP Morgan, Cred · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

Your colleague writes `df[df['status']=='paid']['amount'] += 100` and the value doesn't change in `df`. Why?

The 30-second answer: That's the chained-assignment trap. `df[df['status']=='paid']` returns either a view OR a copy depending on internals; assigning to the result modifies a temporary that may not be the original DataFrame. Pandas raises `SettingWithCopyWarning` because it can't tell. The fix is to combine row and column selection into a single `.loc` call:

```
df.loc[df['status']=='paid', 'amount'] += 100.
```

Step-by-step thought process: 1. State why two indexing calls are the problem — each call returns a new object; only the outer assignment has a chance of writing back, and that chance is implementation-defined. 2. Show the fix — one `.loc` call combines row mask and column selection, and pandas knows to write back to the original. 3. Mention that copy-on-write removes the ambiguity in 2.x — with CoW enabled, the first chained access is a logical copy and the assignment to it doesn't propagate. Predictable, but you still get the wrong result. 4. Note this trap exists for assignment-via-method too: `df[mask].fillna(0, inplace=True)` does nothing useful — the slice is discarded.

```
# Trap – silent or warning, depending on version / CoW state
df[df['status']=='paid']['amount'] += 100    # bad

# Fix – single .loc
df.loc[df['status']=='paid', 'amount'] += 100

# Or split, with explicit copy if you need the subset
subset = df[df['status']=='paid'].copy()
subset['amount'] += 100

# Method version of the same trap
df[df['status']=='paid'].fillna(0, inplace=True)    # does nothing useful
df.loc[df['status']=='paid'] = df.loc[df['status']=='paid'].fillna(0)
```

Edge cases to mention: - Under copy-on-write, the chained assignment writes nothing — the warning becomes a definitive "no effect." - Without CoW, the behaviour was undefined; pandas warned but might have written through or might not. - The trap also applies to `.at` / `.iat` accessed on a chained slice — single-`.loc` is the universal fix.

What NOT to say: - "It's a pandas bug" — it's a deliberate ambiguity in the indexing protocol that CoW finally resolves. - "I just suppress the warning" — suppressing it doesn't fix the bug; your assignment may still silently do nothing.

Common follow-up: "How do you find chained-assignment bugs in legacy code?" — Enable `pd.set_option('mode.copy_on_write', 'warn')` in test runs. Every chained-assignment path emits a warning. Fix them one at a time before turning CoW fully on.

Stand-out line: "Every chained-assignment bug is one `.loc` away from being correct — and under copy-on-write, the bug becomes a warning that finally points at the right line."

Q38 · `df.sort_values()` vs `df.sort_index()`**Tags:** Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Cognizant · Topic: 2.x semantics**The question (interviewer's verbatim phrasing):**What's the difference between `df.sort_values()` and `df.sort_index()` ?

The 30-second answer: `sort_values(by='col')` sorts rows by the values in one or more columns. `sort_index()` sorts rows by the row Index. Different axes — values are in the columns, index is the row label. Both default to ascending; both accept `ascending=False` for descending.

Step-by-step thought process: 1. State the axis they sort on. `sort_values` looks at the data; `sort_index` looks at the row labels. 2. Multi-column sort: `df.sort_values(by=['region', 'amount'], ascending=[True, False])` sorts by region ascending, then amount descending within region. Common reporting pattern. 3. `sort_index()` is useful after `pd.concat` or `set_index` operations where the result is unsorted. Many MultiIndex operations require a sorted index — `sort_index` is the prep. 4. Mention `kind='mergesort'` for stable sort — needed when sort order within ties must be preserved. Default in pandas 2.x is `quicksort` which is not stable.

```
df.sort_values(by='amount')           # ascending
df.sort_values(by='amount', ascending=False)  # descending
df.sort_values(by=['region', 'amount'],
               ascending=[True, False])      # multi-key

df.sort_index()                       # sort by row labels
df.sort_index(axis=1)                 # sort column names

# Stable sort
df.sort_values(by='region', kind='mergesort')
```

Edge cases to mention: - `na_position='first'` or `'last'` controls where NaN values go in the sorted result. Default is `'last'`. - For datetime columns, sort works on the underlying nanosecond representation — correct chronological order. - `sort_index(level=0)` sorts only the first level of a MultiIndex — the rest stays as-is.

What NOT to say: - "They do the same thing" — different axes. - "I'd use `apply` to sort" — sorting is built in; `apply` is for transformations, not ordering.

Common follow-up: "You sort by amount and groupby region — does the sort survive the groupby?" — Within each group, yes, if you pass `sort=False` to groupby AND your sort was on a non-group key. Otherwise groupby may re-sort. Be explicit: `.sort_values('amount').groupby('region', sort=False).head(1)` is the "first-by-amount-within-region" pattern.

Stand-out line: "`sort_values` is what you do for reporting; `sort_index` is what you do before MultIndex slicing — name says which axis."

Q39 · `df.duplicated()` and `df.drop_duplicates()`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred, Cognizant · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

How do you find and remove duplicate rows in a DataFrame? What if "duplicate" means same `customer_id` + same `paid_at` but the other columns differ?

The 30-second answer: `df.duplicated()` returns a boolean Series marking duplicates (default: full-row equality, second-and-later occurrences). `df.drop_duplicates()` drops them. Pass `subset=['customer_id', 'paid_at']` to define duplicate by a subset of columns; `keep='first' / 'last' / False` controls which copy survives.

Step-by-step thought process: 1. Walk through the defaults — full-row equality, mark second-and-later (`keep='first'`). `df.duplicated().sum()` gives a quick dup count. 2. The `subset=` kwarg is the key — defining duplicates on a business key (customer + timestamp) is the common real-world use case. 3. The `keep=` kwarg has three options: `'first'` (default), `'last'`, and `False`. `keep=False` marks ALL duplicate rows including the first — useful when no copy is "the right one" and you want to investigate them all. 4. Show an idiom for keeping the latest record per key: `df.sort_values('paid_at').drop_duplicates(subset='customer_id', keep='last')`.

```
df.duplicated().sum()           # count duplicates
df.drop_duplicates()           # drop, keep first

# Subset-based
df.drop_duplicates(subset=['customer_id', 'paid_at'])

# Keep the latest per customer
df_latest = (df
    .sort_values('paid_at')
    .drop_duplicates(subset='customer_id', keep='last'))

# Find all duplicate rows (including the original)
df[df.duplicated(subset='customer_id', keep=False)]
```

Edge cases to mention: - `duplicated(subset=[...])` ignores NaN-vs-NaN equality the same way `.equals` does — NaN equals NaN for dup detection (unlike `==`). - `drop_duplicates` does

NOT reset the index — pass `ignore_index=True` for a clean RangeIndex. - For large DataFrames, `sort + drop_duplicates` is faster than apply-based dedup; pandas's internals use a hash-based pass.

What NOT to say: - "I'd `groupby().head(1)` for dedup" — works, but it's slower than `drop_duplicates` and harder to read for this purpose. - "Duplicates are always bugs" — sometimes legitimate (multiple attempts at the same payment); the right call depends on business logic.

Common follow-up: "You `drop_duplicates` on `customer_id` and keep the latest by `paid_at` — but two rows have the exact same `paid_at`. Which one survives?" — Whichever appeared LAST in the sort order, which is non-deterministic if the sort is unstable. Pass `kind='mergesort'` to `sort_values` for stability, or add a tiebreaker column to the sort.

Stand-out line: " `drop_duplicates(subset=key, keep='last')` after a sort is the idiomatic 'keep the latest per group' — far cleaner than the apply-based versions you find in old codebases."

Q40 · `pd.concat` vs `pd.merge` vs `df.join` — when to use which

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Cred, JP Morgan · Topic: 2.x semantics

The question (interviewer's verbatim phrasing):

When would you reach for `pd.concat` vs `pd.merge` vs `df.join` ?

The 30-second answer: `pd.concat` stacks DataFrames along an axis (rows or columns) by Index alignment — for combining data of the same shape. `pd.merge` is SQL-style join on a key column or index — for joining two tables with different shapes by a relation. `df.join` is a convenience wrapper around merge for index-on-index joins — same job as merge with index keys.

Step-by-step thought process: 1. Frame the three by the operation. Concat = stack. Merge = SQL join. Join = merge on indexes (the syntactic shortcut). 2. `pd.concat([df1, df2])` stacks rows; `pd.concat([df1, df2], axis=1)` stacks columns. Both align by the OTHER axis's Index. Powerful but easy to get duplicate-index output. 3. `pd.merge(left, right, on='customer_id', how='left')` is the bread and butter. Full SQL semantics — `inner / left / right / outer`. Detailed treatment in Chapter 3. 4. `df1.join(df2)` is the index-on-index version of merge — same semantics, different default. Used most commonly when you've already `set_index` on both sides.

```
# concat — stack
pd.concat([df1, df2], axis=0, ignore_index=True)    # stack rows
pd.concat([df_a, df_b], axis=1)                    # stack cols, align by row index

# merge — SQL-style join
pd.merge(orders, customers, on='customer_id', how='left')
pd.merge(orders, customers, left_on='cust_id', right_on='id', how='inner')

# join — merge on index, shortcut
df1.set_index('id').join(df2.set_index('id'), how='left')
```

Edge cases to mention: - `pd.concat` with overlapping indexes produces duplicate index labels — pass `verify_integrity=True` to raise if you didn't want that. - `pd.merge(validate='one_to_one')` catches join-cardinality bugs at runtime — saves you from a merge that secretly produced 10x rows. Use it in every production pipeline. - `df.join` differs from `pd.merge` in default `how` — `join` defaults to `how='left'`, `merge` defaults to `how='inner'`.

What NOT to say: - "They all do the same thing" — three different shapes / use cases. Mixing them up is the source of half the "my data went weird after the join" bugs. - "I always use `.join` because it's shorter" — when keys aren't the index, you'll set up wrong; reach for `merge` with explicit `on=` instead.

Common follow-up: "You merge two tables and get more rows than the left side started with — what went wrong?" — One-to-many on the join key — the right table has duplicates on the merge column. Check with `right['key'].is_unique` BEFORE merging. Add `validate='many_to_one'` to the merge to raise on the asymmetry up front instead of debugging row counts after.

Stand-out line: " `concat` for shape-equal stacking, `merge` for relational joins, `join` for index-on-index shortcuts — and always pass `validate=` to merge in production; the day your merge silently doubles your row count, you'll thank yourself. "

End of Chapter 1. Chapter 2 picks up groupby + agg patterns — the second-most-asked area after fundamentals, and the one where most candidates' notebooks break first under real volume.

Chapter 2 — groupby + agg patterns (30 Questions)

Data Track Library · Python Interview Prep 2026 Edition · Learn & Excel · File 06

This is the most-asked chapter in the Data Track. Every Indian product company that hires for a Data role — Razorpay, Swiggy, Flipkart, Zerodha, PhonePe, Meesho, Cred — opens the pandas round with some flavour of "give me the top three customers per region" or "show me month-on-month revenue change per category." The mechanics look simple — call `.groupby()`, call `.agg()`, get a result — but interviewers know exactly where candidates trip: they do not understand what `.groupby()` returns lazily, they reach for `.apply` when `.transform` would have been one line and ten times faster, they get a MultiIndex back and do not know how to flatten it, they let NaN keys silently disappear from the output, and they cannot write the canonical top-N-per-group pattern without four false starts.

The thirty questions below walk through the surface area in the order an interviewer probes it. Section A is the split-apply-combine model and what `.groupby()` actually does before you call any aggregation. Section B is the four `.agg()` syntactic forms — string, list, dict, and the modern named-aggregation tuples — plus the `transform` vs `agg` vs `apply` distinction that the Razorpay-tier panels stress-test. Section C is the patterns Indian interviewers ask by name: top-N per group, `pct_change` within group, lag/lead via `shift`, cumulative sums, ranks, and quantiles per group. Section D is the gotchas — NaN keys being dropped, MultiIndex flattening, categorical ordering, and the `.apply` performance cliff. Answers assume pandas 2.x throughout; the Arrow-backend nuances and copy-on-write semantics get called out where they actually change the answer.

Table of Sections

- Section A — groupby basics (Q1 to Q5)
- Section B — agg patterns and the transform vs apply distinction (Q6 to Q15)
- Section C — Indian-interview canonical patterns (Q16 to Q25)
- Section D — groupby gotchas (Q26 to Q30)

Section A — groupby basics

Five questions on what `.groupby()` actually returns, what split-apply-combine means in the pandas vocabulary, single versus multi-key grouping, iterating groups by hand, and the `as_index=True` versus `as_index=False` flag that decides whether your group keys come back as an index or as plain columns. Get these five right and the rest of the chapter is technique on top of a solid mental model.

Q1 · What does `df.groupby('city')` actually return — and why is it called "lazy"

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Freshworks · Topic: groupby basics · Mental Model

The question (interviewer's verbatim phrasing):

When I write `df.groupby('city')` and assign it to a variable, what object do I actually get back? Has any computation happened yet?

The 30-second answer: `df.groupby('city')` returns a `DataFrameGroupBy` object — it is a lazy handle that knows which rows belong to which group but has not run any aggregation yet. No computation, no copies of the data, just an index of group labels to row positions. The actual split-apply-combine work fires only when you call something on the GroupBy object — `.sum()`, `.agg(...)`, `.transform(...)`, `.apply(...)`, or iteration.

Step-by-step thought process: 1. Name the type first — `DataFrameGroupBy`, or `SeriesGroupBy` if the source was a Series. Interviewers want the exact class name because it tells them you have looked at the repr at least once. 2. State what the object actually holds — the grouping keys, the original frame reference, and an internal mapping from each group label to the row positions that belong to it. No aggregated values yet. 3. Explain the laziness consequence: you can build a GroupBy object on a million-row frame in milliseconds because the heavy work is deferred. The cost lands when you call a method that has to walk the groups. 4. Bring up the second consequence — you can call multiple aggregations on the same GroupBy without re-grouping: assign `gb = df.groupby('city')`, then `gb.sum()`, `gb.mean()`, `gb['amount'].agg(['min', 'max'])` all reuse the same internal index.

Edge cases to mention: - `groupby` does sort the group keys by default (`sort=True`), which costs $O(g \log g)$ where g is the number of groups. On a frame with hundreds of thousands of unique groups, pass `sort=False` to skip that cost — the output then comes back in first-appearance order. - The GroupBy object holds a reference to the original frame; if you delete the frame, the GroupBy is still alive but will operate on the now-orphaned data. Memory does not get freed until both references are gone. - Calling `len(gb)` returns the number of groups, not the number of rows. That trips up candidates who expect `len(gb)` to equal `len(df)`.

What NOT to say: - "It returns a dictionary of dataframes" — that is wrong; it returns a single GroupBy object. You can iterate it as `(key, sub_df)` pairs, but the underlying storage is not a dict of frames. - "It splits the frame into smaller frames immediately" — wrong; no split has happened. The lazy index of positions is what makes pandas groupby fast.

Common follow-up: "If I call `.sum()` and then `.mean()` on the same GroupBy object, does pandas group the data twice?" — No. The grouping index is computed once when the GroupBy is constructed; each aggregation reuses it. That is precisely why holding the GroupBy in a variable is faster than calling `df.groupby('city').sum()` and `df.groupby('city').mean()` as two separate statements.


```
import pandas as pd

df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi', 'Bengaluru'],
    'amount': [499, 1299, 799, 250, 1599],
})

gb = df.groupby('city')
print(type(gb))          # <class 'pandas.core.groupby.generic.DataFrameGroupBy'>
print(len(gb))           # 3 (three cities, not five rows)
print(gb.sum(numeric_only=True)) # actual computation runs here
```

Q2 · Explain split-apply-combine in pandas

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Accenture · Topic: groupby basics · Vocabulary

The question (interviewer's verbatim phrasing):

Walk me through the split-apply-combine model. What does each of the three steps mean when you run `df.groupby('region').agg({'sales': 'sum'})` ?

The 30-second answer: Split-apply-combine is the three-phase mental model that pandas borrowed from R's `plyr`. Split: pandas partitions the rows of the frame into groups by the key column. Apply: pandas runs the aggregation function (sum, mean, custom) on each group independently. Combine: pandas stitches the per-group results back together into a single output frame, usually with the group keys as the index.

Step-by-step thought process: 1. Name the three phases in order and what each does. Split is the partitioning; apply is the function call; combine is the assembly. 2. Walk through the example. `df.groupby('region')` does the split — pandas builds the index of row positions per region. `.agg({'sales': 'sum'})` is the apply — sum runs once per region. The return-value assembly is the combine — one row per region, region as index, `sales` column holding the totals. 3. State the operational consequence: the apply step is the only one that the user can change. Split is fixed by the key, combine is fixed by pandas' assembly rules. The aggregation function is your knob. 4. Tie it to the three GroupBy methods. `.agg()` reduces each group to one row. `.transform()` returns one row per input row (broadcast back). `.apply()` lets each group return any shape it likes and pandas figures out the combine. We will hit transform vs agg vs apply in Q11 and Q12.

Edge cases to mention: - The "combine" step decides the output's index. With `as_index=True` (default), group keys become the index. With `as_index=False`, group keys come back as plain columns. Same data, different shape. - If you group by a list of columns, the combine produces a

MultiIndex with one level per grouping key. Many interview answers want a flat-column output — call `.reset_index()` or pass `as_index=False`. - The apply step is parallelisable in theory but not in pandas itself — each group still runs in the main thread. Tools like Dask or modin do the parallel version.

What NOT to say: - "Split-apply-combine is a SQL concept" — it predates SQL framings and originates from Hadley Wickham's R work. SQL's GROUP BY is the same idea but the vocabulary is different. - "Combine just concatenates the groups" — it does more than concat; it indexes by the group keys and aligns columns. Concat alone would not give you a labelled result.

Common follow-up: *"Which of the three phases is usually the slowest on a real dataset?"* — Apply, almost always — because it runs your function once per group. Split is a single pass over the key column, combine is a single pass over the per-group outputs. If your apply is a Python-level function over millions of groups, that is where the time goes.

```
df = pd.DataFrame({
    'region': ['North', 'South', 'North', 'East', 'South'],
    'sales': [120, 80, 60, 200, 90],
})

# Split-apply-combine in one line
out = df.groupby('region').agg({'sales': 'sum'})
print(out)
#      sales
# region
# East      200
# North     180
# South     170
```

Q3 · Single-key vs multi-key groupby — what changes in the output

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Meesho, Postman, Walmart Labs · Topic: groupby basics · Multi-key

The question (interviewer's verbatim phrasing):

What is the difference between `df.groupby('city')` and `df.groupby(['city', 'product'])` in the result you get back?

The 30-second answer: Single-key grouping produces an output indexed by the single key — one Index level. Multi-key grouping produces an output indexed by a MultiIndex with one level per key. The aggregated values are the same arithmetic; the shape changes. The multi-key result has one row per unique combination of the keys, not the cross-product — combinations that do not appear in the data are absent from the output (unless you set `observed=False` on categorical keys).

Step-by-step thought process: 1. Lead with the shape difference. Single key → Index. Multi-key → MultiIndex. That sentence anchors every follow-up. 2. Mention the row-count difference. Single key: one row per unique value in that key. Multi-key: one row per unique combination of all keys, but only combinations that actually appear in the data. 3. Show the path back to a flat frame.

`.reset_index()` turns the MultiIndex into plain columns. `as_index=False` does the same thing in one step at the groupby call. 4. Bring up the `observed` parameter for categorical keys. With `observed=True` (the pandas 2.x default for some paths and forced default in 2.1+), only observed combinations show. With `observed=False`, every Cartesian combination appears, with NaN aggregates for combos that have no rows. That switch surfaces in real interview gotchas — see Q28.

Edge cases to mention: - The order of keys in the list matters for the MultiIndex level ordering, but it does not change the aggregated values. `groupby(['city', 'product'])` and `groupby(['product', 'city'])` give the same numbers in a different shape. - `groupby(level=...)` lets you group by an existing index level instead of a column. Useful when the column you want is already in the index. - You can pass functions and Series of the same length as the frame as group keys, not just column names — `df.groupby(df['date'].dt.month)` groups by month without first materialising the month column.

What NOT to say: - "Multi-key groupby is just two single-key groupbys" — it is not; it walks the rows once and partitions by the combination, not twice. - "The output of multi-key is always a Cartesian product" — only with `observed=False` on categorical keys; default behaviour is observed-only combinations.

Common follow-up: "How do you flatten a multi-key groupby result into a plain frame with the keys as columns?" — Two equivalent ways: pass `as_index=False` in the groupby call, or call `.reset_index()` on the result after aggregation. The second form is more common in interview code because it composes well with `.agg(...)` chains.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Mumbai', 'Delhi', 'Delhi', 'Bengaluru'],
    'product': ['Pro', 'Lite', 'Pro', 'Pro', 'Lite'],
    'amount': [499, 199, 499, 599, 199],
})

# Single key — Index
print(df.groupby('city')['amount'].sum())

# Multi-key — MultiIndex
print(df.groupby(['city', 'product'])['amount'].sum())

# Flat columns
print(df.groupby(['city', 'product'], as_index=False)['amount'].sum())
```

Q4 · Iterating a GroupBy object — when is it the right tool

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, JP Morgan · Topic: groupby basics · Iteration

The question (interviewer's verbatim phrasing):

Can you loop over a GroupBy object directly? If yes, what does each iteration give you, and when would you ever use this over `.agg()` or `.apply()` ?

The 30-second answer: Yes — iterating a GroupBy yields `(group_key, sub_dataframe)` tuples, one per group. The right time to reach for this is when you want to write per-group results to separate destinations — one CSV per region, one PDF per customer, one S3 prefix per merchant. For producing a single aggregated frame you should use `.agg()` or `.transform()` ; the explicit loop is slower and harder to read.

Step-by-step thought process: 1. State the iteration contract first — you get a tuple of `(key, sub_df)` . If you grouped by multiple keys, `key` is itself a tuple. 2. Show the canonical use-case for hand iteration: writing each group out separately. Saving one Excel sheet per region, uploading one file per merchant, emailing one report per account manager. None of these collapse into a single aggregated frame. 3. State explicitly when not to iterate: any time the goal is one aggregated frame. The Python loop pays the per-group overhead twice — once to surface the sub-frame as a Python object, once to do whatever you do with it — which is exactly the cost `.agg()` saves. 4. Tie this back to the apply-perf-trap conversation in Q12: `.apply(lambda g: ...)` is essentially the loop in disguise. If you find yourself writing the loop, ask whether `.agg()` or `.transform()` would do the same job at C speed.

Edge cases to mention: - The order of iteration is the sorted order of group keys (because `sort=True` is the default on groupby). If you need first-appearance order, build the GroupBy with `sort=False` . - Sub-dataframes returned by iteration are views into the original frame in most cases, not copies. Mutating them with chained indexing triggers the SettingWithCopyWarning. If you need to mutate, make an explicit `.copy()` . - Iterating a multi-key GroupBy gives you a tuple key — `for (city, product), sub in df.groupby(['city', 'product'])` is the idiomatic unpack.

What NOT to say: - "You can't iterate a GroupBy" — you can; the `__iter__` is implemented. - "Iterating is always slower than `.apply`" — `.apply()` is implemented as iteration in many cases, so they are similar; both are slower than vectorised `.agg()` . The point is not iteration-vs-apply, it is iteration/apply-vs-vectorised-agg.

Common follow-up: "You have a thousand groups and you want to write one CSV per group. Show me the loop." — `for key, sub in df.groupby('region'): sub.to_csv(f'out/{key}.csv', index=False)` . This is the textbook right use of iteration — the output is per-group, not aggregated.

```
df = pd.DataFrame({
    'region': ['North', 'South', 'North', 'East'],
    'sales': [120, 80, 60, 200],
})

for region, sub in df.groupby('region'):
    print(region, '→', len(sub), 'rows')
# East → 1 rows
# North → 2 rows
# South → 1 rows

# Multi-key – unpack the tuple
for (city, product), sub in df.groupby(['region', 'sales']):
    ...
```

Q5 · `as_index=True` vs `as_index=False` — what flag changes what

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Freshworks · Topic: groupby basics · Output Shape

The question (interviewer's verbatim phrasing):

When I groupby and aggregate, sometimes the group key comes back as the index and sometimes as a column. What is controlling that, and which form should I prefer?

The 30-second answer: The `as_index` parameter on `.groupby()` controls it. With `as_index=True` (the default), the group keys become the row Index of the output. With `as_index=False`, the group keys come back as plain columns and the output gets a fresh default `RangeIndex`. Pick `as_index=False` when you want to chain further dataframe operations or merge the result back into another frame; pick `as_index=True` when you want to slice by group key later.

Step-by-step thought process: 1. Lead with the mechanic. `as_index` is a boolean on the `.groupby()` call itself, not on the aggregation. Set it once when you create the GroupBy. 2. Show the same aggregation under both flags. Same numbers, different shape — `as_index=True` puts city in the index, `as_index=False` keeps city as a column. 3. State the practical heuristic. Going to merge back to another frame? Use `as_index=False` because merge keys want to be columns. Going to slice by `.loc['Mumbai']`? Use `as_index=True` because index labels are the fast lookup path. 4. Note the equivalence with `.reset_index()`. If you forgot the flag at groupby time, calling `.reset_index()` on the aggregated result gets you to the same flat shape. Slightly less efficient because the index materialises and then gets dropped, but functionally equivalent.

Edge cases to mention: - `as_index=False` does not affect the apply phase; the aggregation function runs identically. Only the combine step's output shape changes. - For `.transform()`, `as_index` has no effect because transform returns one row per input row with the original index intact. - Some aggregations historically forced `as_index=True` regardless (older pandas), but pandas 2.x respects the flag consistently across `.agg()`, `.size()`, `.count()`, and similar reducers.

What NOT to say: - "It changes the values" — it does not; only the shape of the index changes. - "Always use `as_index=False`" — overstated; when you need to slice by group key downstream, the index form is faster.

Common follow-up: "What is the equivalent of `as_index=False` for a result that has already been aggregated with the index form?" — `result.reset_index()`. It moves every index level into a column and gives you a clean integer RangeIndex.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
})

# Default — city as index
df.groupby('city')['amount'].sum()
# city
# Delhi      1549
# Mumbai     1298

# Flat — city as column
df.groupby('city', as_index=False)['amount'].sum()
#      city  amount
# 0    Delhi    1549
# 1    Mumbai   1298
```

Section B — agg patterns and the transform vs apply distinction

Ten questions on the four `.agg()` syntactic forms, multiple aggregations, custom aggregation functions, and the three GroupBy methods that beginners conflate — `.agg()`, `.transform()`, `.apply()`. Section B is where the mid-level Razorpay panels live: you can usually get past the basics with intuition, but `.transform()` returning input shape versus `.agg()` reducing to one row per group is the kind of thing the interviewer wants to hear in precise words.

Q6 · `.agg()` with a string — the simplest aggregation form

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: agg patterns · String form

The question (interviewer's verbatim phrasing):

What does `df.groupby('city')['amount'].agg('sum')` do, and how is it different from `df.groupby('city')['amount'].sum()` ?

The 30-second answer: Functionally identical — both return the per-city sum of the amount column as a Series indexed by city. The string form is the most concise version of the four `.agg()` syntaxes. Use it when you want one aggregation on one column; reach for the list, dict, or named-tuple forms (covered in Q7 to Q10) when you need multiple ops, multiple columns, or custom output names.

Step-by-step thought process: 1. State the equivalence — `.agg('sum')` and `.sum()` are interchangeable for a single named aggregation. The interviewer wants to hear you say it explicitly rather than imply it. 2. Mention the string vocabulary. pandas recognises a fixed set of strings as built-in aggregations — `'sum'`, `'mean'`, `'min'`, `'max'`, `'count'`, `'first'`, `'last'`, `'nunique'`, `'std'`, `'var'`, `'median'`. Anything outside this set goes through the function path. 3. Compare readability. `.sum()` is shorter and more idiomatic when you have one aggregation. `.agg('sum')` is useful when you are parameterising the aggregation by name — e.g. reading the op from a config file and passing it through. 4. Note the only practical edge: `.agg('count')` counts non-null values per column, while `GroupBy.count()` counts non-null per column too — but `GroupBy.size()` counts all rows including nulls. These three look similar and are not the same.

Edge cases to mention: - The string is case-sensitive — `'Sum'` will fail. Use lowercase. - Some strings dispatch to numpy under the hood (`'sum'`, `'mean'`) and some to pandas methods (`'first'`, `'last'`). The user does not need to know which; the resolution is internal. - If you pass an unknown string, the error message is informative but late — it fires at the `.agg()` call, not at `.groupby()`.

What NOT to say: - "`.agg('sum')` is slower than `.sum()`" — they go through the same code path; no measurable difference. - "Strings only work for built-in names" — they only work for the recognised string vocabulary; for anything else, pass the function itself.

Common follow-up: "What strings are recognised by `.agg()` ?" — Quote the common ones: `'sum'`, `'mean'`, `'min'`, `'max'`, `'count'`, `'size'`, `'first'`, `'last'`, `'nunique'`, `'std'`, `'var'`, `'median'`. Anything outside that goes through the callable form.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
})

a = df.groupby('city')['amount'].agg('sum')
b = df.groupby('city')['amount'].sum()
print(a.equals(b))  # True — same Series
```

Q7 · `.agg()` with a list — multiple aggregations on one column

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: agg patterns · List form

The question (interviewer's verbatim phrasing):

How do you compute the sum, mean, and count of `amount` per city in one shot?

The 30-second answer: Pass a list to `.agg()` :

`df.groupby('city')['amount'].agg(['sum', 'mean', 'count'])` . The output is a DataFrame with city as the index and one column per aggregation. The column names come from the function names — `sum` , `mean` , `count` — which you can override by passing tuples of `(name, func)` instead of bare strings.

Step-by-step thought process: 1. Show the canonical syntax. List of strings (or callables) inside `.agg()` , one column per element in the output. 2. Walk through the output shape. Series in → DataFrame out, with the aggregation names as columns. This is one of the few places where the input/output container type genuinely changes. 3. Show the `(name, func)` tuple variant. `('total', 'sum')` puts the column under the label `total` instead of `sum` . Useful when the function name is unfriendly (e.g. a lambda is renamed `'<lambda>'` by default). 4. Bring up the dict and named-aggregation forms (Q8, Q9, Q10) as the right move when you need different aggregations on different columns. The list form is for "same aggregations across one column."

Edge cases to mention: - A list of lambdas all get the same column name `<lambda>` — that overwrites. Wrap each in a `(name, lambda)` tuple to disambiguate. - The list form on a DataFrame (not a single column) produces a column MultiIndex — outer level is the column name, inner level is the aggregation name. This is the classic MultiIndex output that catches candidates off-guard. - Mixing strings and callables in the same list works fine: `.agg(['sum', np.median])` .

What NOT to say: - "The output is always a Series" — it becomes a DataFrame when you pass a list, because each list element is its own column. - "You can only use built-in names" — callables work too; the list is heterogeneous.

Common follow-up: "What if I want the column to be called `total` instead of `sum` ?" — Use `(label, func)` tuples: `.agg([('total', 'sum'), ('avg', 'mean')])` . Or for whole-DataFrame multi-column control, use the named-aggregation form covered in Q10.


```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
})

# Three aggregations on one column
df.groupby('city')['amount'].agg(['sum', 'mean', 'count'])
#           sum    mean  count
# city
# Delhi    1549    774.5      2
# Mumbai   1298    649.0      2

# Custom column labels
df.groupby('city')['amount'].agg([('total', 'sum'), ('avg', 'mean')])
```

Q8 · `.agg()` with a dict — different aggregations on different columns

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, Postman, JP Morgan · Topic: agg patterns · Dict form

The question (interviewer's verbatim phrasing):

Per city, I want the total amount and the count of distinct customers. Show me how you'd do both in one `.agg()` call.

The 30-second answer: Pass a dict mapping each column to the aggregation you want:

`df.groupby('city').agg({'amount': 'sum', 'customer_id': 'nunique'})`. The keys are column names, the values are either a single aggregation (string or callable) or a list of aggregations. The output has city as the index and one column per dict entry — though if you pass lists as values, you get a MultiIndex on the columns.

Step-by-step thought process: 1. Show the canonical dict form. One key per column, value picks the aggregation. 2. Mention the list-value variant. `{'amount': ['sum', 'mean'], 'customer_id': 'nunique'}` gives a MultiIndex column output — outer level is the column name, inner level is the aggregation. The MultiIndex column output is the most-asked gotcha in this section; we cover flattening in Q27. 3. State the modern preference. The dict form was the recommended pattern up to pandas 0.20, but starting from 0.20 the named-aggregation form (covered in Q10) is the official replacement when you need custom output column names. Both still work in pandas 2.x; senior candidates volunteer the modern form. 4. Note that the dict form preserves column order — Python 3.7+ guarantees dict insertion order, and pandas respects it. So you can control left-to-right column placement in the output by ordering the dict.

Edge cases to mention: - Columns not present in the dict are dropped from the output. So the dict acts as a column selector too. - If the same column appears with multiple aggregations (`{ 'amount': ['sum', 'mean'] }`), you get a MultiIndex — outer level `amount` , inner level `sum / mean` . To flatten, call `.columns = ['_'.join(c) for c in result.columns]` . - Passing a callable that returns multiple values (like a tuple) breaks the dict form silently — pandas wraps the tuple as a single cell. Use named aggregation for that case.

What NOT to say: - "The dict form is deprecated" — it is not deprecated; the renaming-via-dict form (a sub-pattern where dict values were themselves dicts of `{out_name: agg}`) is the one that was deprecated. The plain `{column: agg}` form is alive and well. - "You can only aggregate one column at a time with a dict" — you can pass as many keys as you want; each gets its own aggregation.

Common follow-up: "How do you give the output columns custom names — say `total_amount` and `unique_customers` instead of `amount` and `customer_id` ?" — Use the named-aggregation form: `df.groupby('city').agg(total_amount=('amount', 'sum'), unique_customers=('customer_id', 'nunique'))` . That is Q10.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
    'customer_id': [1, 2, 3, 2],
})

df.groupby('city').agg({'amount': 'sum', 'customer_id': 'nunique'})
#           amount  customer_id
# city
# Delhi         1549             1
# Mumbai        1298             2
```

Q9 · `.agg()` with a custom function — when strings are not enough

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Target · Topic: agg patterns · Callable

The question (interviewer's verbatim phrasing):

Per city, I want the difference between the maximum and minimum amount. How would you express that as a custom aggregation function passed to `.agg()` ?

The 30-second answer: Define a function that takes a Series (the per-group slice) and returns a single scalar — `def amount_range(s): return s.max() - s.min()` — and pass it to `.agg()` : `df.groupby('city')['amount'].agg(amount_range)` . The function receives the per-group

Series, runs once per group, and pandas combines the scalar return values into the output. A lambda works the same way: `.agg(lambda s: s.max() - s.min())`.

Step-by-step thought process: 1. State the contract: the custom function gets a Series per group and must return a scalar. That is the same contract built-in aggregations honour internally. 2. Show both forms. Named function — cleaner column label, reusable, easier to debug. Lambda — concise for one-off use, but the column name comes back as `<lambda>`. 3. Note the column-naming wart. With a list of lambdas (Q7) you get duplicate `<lambda>` columns and pandas raises. Either use named functions, name the lambda explicitly via `(name, lambda)` tuples, or use the named-aggregation form. 4. Mention the perf reality. A Python-callable aggregation runs at Python speed per group, not at vectorised C speed. For a million groups, a custom function aggregation can be 100× slower than `'sum'` or `'mean'`. We will hit this in Q12 and Q26.

Edge cases to mention: - The function can return a Series instead of a scalar — pandas detects this and treats it as `.apply()` semantics, producing a wider output. Usually not what you want from `.agg()`; if the return shape varies per group, reach for `.apply()` explicitly. - The function receives the group as a Series (when aggregating a single column) or as a DataFrame (when called via `df.groupby(...).agg(func)` without column selection). The shape depends on what you indexed into before `.agg()`. - `numpy.ufunc` objects like `np.sum` work directly and run at C speed — preferable to a Python `lambda s: s.sum()` for performance.

What NOT to say: - "Custom aggregations are always slow" — they are slower than the named ones, but for small group counts the difference is negligible. The bottleneck is per-group Python overhead, which scales with the number of groups. - "You have to use a lambda" — named functions are fine and read better in shared codebases.

Common follow-up: "What is the difference between passing `np.median` and passing `'median'`?" — Functionally none; both compute the median per group. `'median'` dispatches to pandas' internal implementation, `np.median` is the numpy function. On numeric dtypes the result and performance are equivalent.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
})

def amount_range(s):
    return s.max() - s.min()

df.groupby('city')['amount'].agg(amount_range)
# city
# Delhi      1049
# Mumbai      300

# Or as a lambda — note the column label issue if you stack them
df.groupby('city')['amount'].agg(lambda s: s.max() - s.min())
```

Q10 · Named aggregation — the modern pandas form

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Walmart Labs, JP Morgan · Topic: agg patterns · Named tuples · pandas 2.x

The question (interviewer's verbatim phrasing):

Per city, I want one column called `total_revenue` (sum of amount) and one called `unique_customers` (distinct count of customer_id). Use the modern pandas pattern.

The 30-second answer: Use named aggregation: pass keyword arguments where each kwarg name is the desired output column and each value is a `pd.NamedAgg(column='...', aggfunc='...')` or just a plain `(column, aggfunc)` tuple.

`df.groupby('city').agg(total_revenue=('amount', 'sum'), unique_customers=('customer_id', 'nunique'))`. This is the form pandas recommends from 0.25 onward — it gives flat output columns with your chosen names, no MultiIndex, no `.reset_index()` gymnastics.

Step-by-step thought process: 1. State the syntax. Keyword names become output column names. Values are `(source_column, aggregation)` tuples. One tuple per output column. 2. Explain why this is the preferred modern form. The dict form (Q8) with renaming was a deprecated sub-pattern; named aggregation replaces it with cleaner ergonomics. The output is always a flat DataFrame, no MultiIndex when you supply unique kwarg names. 3. Mention `pd.NamedAgg`. It is the explicit named-tuple version of the inline tuple — `pd.NamedAgg(column='amount', aggfunc='sum')` is identical to `('amount', 'sum')`. Most code uses the inline tuple because it is shorter; some style guides prefer `pd.NamedAgg` for clarity. 4. Tie it to the canonical "top three customers per region" pattern (Q16) — you frequently want the aggregation step to produce columns with meaningful names before you sort and head, and named aggregation is the right tool.

Edge cases to mention: - Kwarg names must be valid Python identifiers. `total_revenue` is fine; `'total revenue'` (with a space) is not — use the dict form for arbitrary column-name strings, or rename after the fact. - You can mix named aggregations with custom functions: `total_range=('amount', lambda s: s.max() - s.min())`. The lambda gets the same per-group Series and must return a scalar. - Named aggregation applies to the whole frame, not to a Series. `df.groupby('city')['amount'].agg(...)` does not accept the kwarg form; you must group the whole DataFrame and let the tuples select columns.

What NOT to say: - "Named aggregation is just a wrapper around dict" — under the hood pandas converts both, but the user-visible behaviour around column naming and MultiIndex avoidance is different. Treat it as its own pattern. - "The dict form is the canonical pandas way" — that was true pre-0.25; in 2.x the recommended form is named aggregation when you need custom output names.

Common follow-up: "What is the equivalent of named aggregation when you want different aggregations on the same column?" — Same form, repeat the source column: `agg(min_amount=('amount', 'min'), max_amount=('amount', 'max'))`. Each kwarg is independent.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
    'customer_id': [1, 2, 3, 2],
})

# Modern form — flat columns, custom names
df.groupby('city').agg(
    total_revenue=('amount', 'sum'),
    unique_customers=('customer_id', 'nunique'),
    avg_ticket=('amount', 'mean'),
)
#           total_revenue  unique_customers  avg_ticket
# city
# Delhi                1549                 1         774.5
# Mumbai               1298                 2         649.0
```

Q11 · `.transform()` vs `.agg()` — what shape does each return

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, Postman · Topic: transform · agg · Shape Semantics

The question (interviewer's verbatim phrasing):

What is the difference between `.transform()` and `.agg()` on a GroupBy? When would you reach for transform?

The 30-second answer: `.agg()` reduces each group to one row — output has one row per group. `.transform()` returns a value for every input row, broadcasting the per-group result back so the output preserves the original frame's shape and index. Reach for transform when you want to add a column like "share of the city's total" to every row — you need the per-city sum on every row of that city, not one row per city.

Step-by-step thought process: 1. State the shape rule first. `agg` shrinks. `transform` preserves. That sentence alone covers eighty percent of the answer. 2. Walk through the use-case. Computing percent of total per group, mean-centering each group's values, normalising features per cohort — all need the per-group statistic available on every row. That is transform's job. 3. Show the syntactic similarity. Both take a function (or string). The difference is purely what shape comes back. `df.groupby('city')['amount'].transform('sum')` returns a Series the same length as `df`, with the city's total

broadcast to every row of that city. 4. Mention the transform contract — the function must return either a scalar (broadcast) or a Series of the same length as the group (returned as-is). If it returns something else, pandas raises. That contract is stricter than apply's.

Edge cases to mention: - `.transform()` does not change the index; the output aligns one-to-one with the input. This is why you can do `df['city_total'] = df.groupby('city')['amount'].transform('sum')` and the assignment works without merge gymnastics. - Transform is significantly faster than apply for the same logical operation because it knows the output shape upfront and can use vectorised paths. - Transform on multiple columns at once works: `df.groupby('city')[['amount', 'quantity']].transform('mean')` returns a DataFrame the same shape as `df[['amount', 'quantity']]`.

What NOT to say: - "Transform is just apply with a different name" — they have different output-shape contracts; transform's strict contract enables the faster path. - "Transform reduces the data" — it does not; it preserves shape and broadcasts the per-group statistic to every row.

Common follow-up: "Show me how to add a `pct_of_city` column that gives each row's amount as a fraction of its city total." — `df['pct_of_city'] = df['amount'] / df.groupby('city')['amount'].transform('sum')`. One line, no merge.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
})

# agg — one row per group
df.groupby('city')['amount'].sum()
# Delhi      1549
# Mumbai     1298

# transform — same shape as input
df.groupby('city')['amount'].transform('sum')
# 0      1298
# 1      1549
# 2      1298
# 3      1549

# canonical use — pct of group total
df['pct_of_city'] = df['amount'] / df.groupby('city')['amount'].transform('sum')
```

Q12 · `.apply()` on GroupBy — the performance trap

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, JP Morgan, Target · Topic: apply · Performance

The question (interviewer's verbatim phrasing):

When should you use `.apply()` on a GroupBy instead of `.agg()` or `.transform()` ?
And why do senior pandas users avoid it?

The 30-second answer: Reach for `.apply()` only when the per-group function returns something neither agg nor transform can express — a DataFrame of different shape, a row-count that depends on the group, a return that mixes multiple shapes. For everything else, agg or transform are faster and clearer. The trap is that `.apply()` is the most "obvious" tool, so beginners default to it and pay a 10× to 100× performance penalty over the equivalent vectorised form on real datasets.

Step-by-step thought process: 1. Name the cost up front. `.apply()` runs your Python function once per group with no shape contract, which means pandas cannot use the vectorised path. On a frame with hundreds of thousands of groups, that is a Python-level loop with all the overhead that implies. 2. State when apply is genuinely the right tool. Returning a different-shaped frame per group (e.g. the top three rows by amount per city, which is a 3-row frame per group). Sliding-window calculations that need the whole group at once. Cross-row computations where the function needs both `group.head()` and `group.tail()`. 3. State when apply is the wrong tool — and is the most-common mistake. Computing per-group sums and means (use `.agg()`). Computing per-group rankings and rolling statistics that have built-in methods (use the dedicated method). Anything that has a one-liner agg/transform form. 4. Mention the modern signal. `.apply()` on a GroupBy has historically had inconsistent semantics — sometimes it returns one row per group, sometimes broadcast, depending on what the function returns. pandas 2.x cleaned some of this up but the safer move is to be explicit: use agg for reductions, transform for broadcasts, apply only for the truly heterogeneous case.

Edge cases to mention: - The `include_groups` parameter (pandas 2.2+) controls whether the group keys are passed into the function as a column. The previous default of "yes" sometimes caused `.apply()` to recompute the keys; the new default is False. - `.apply()` can be 10–100× slower than `.agg()` on the same operation. On a frame with a million groups, that is the difference between 200 milliseconds and 20 seconds. - Returning `None` from the function in apply silently drops that group from the output. Easy bug; the result has fewer rows than you expect.

What NOT to say: - "Apply is always slow" — for a few hundred groups it is fine; the trap kicks in at scale. - "Apply is just sugar for a loop" — it is closer to a loop than agg or transform are, but pandas does optimise common return shapes. Still, agg/transform are faster.

Common follow-up: "You see `.apply(lambda g: g['amount'].sum())` in a code review. What is the better form?" — `.agg({'amount': 'sum'})` or simply `['amount'].sum()`. The apply form does the same arithmetic at Python speed per group; the agg form runs the sum in C across all groups.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'] * 10000,
    'amount': list(range(40000)),
})

# Slow – Python loop per group
df.groupby('city').apply(lambda g: g['amount'].sum())

# Fast – vectorised
df.groupby('city')['amount'].sum()

# apply is justified here – returning a 3-row frame per group
df.groupby('city', group_keys=False).apply(lambda g: g.nlargest(3, 'amount'))
```

Q13 · Multiple columns and multiple aggregations together

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Meesho · Topic: agg patterns · Multi-column

The question (interviewer's verbatim phrasing):

Per city, I want sum and mean of `amount`, plus min and max of `quantity`, all in one call. Show me the cleanest pandas pattern.

The 30-second answer: Use named aggregation:

`df.groupby('city').agg(amount_sum=('amount', 'sum'), amount_mean=('amount', 'mean'), qty_min=('quantity', 'min'), qty_max=('quantity', 'max'))` . Four kwargs, four output columns, flat DataFrame, no MultiIndex. The alternative is the dict form `df.groupby('city').agg({'amount': ['sum', 'mean'], 'quantity': ['min', 'max']})` which produces a MultiIndex column output you then have to flatten.

Step-by-step thought process: 1. Lead with the modern form — named aggregation, one kwarg per desired output column. Cleanest, most readable, no post-processing. 2. Show the legacy dict form for comparison. Same numbers, MultiIndex columns out, downstream code has to handle two levels. Tell the interviewer you know both forms exist and you pick the named form for new code. 3. State the underlying trade-off. The dict form is shorter to type when you want every aggregation for every column; the named form is shorter to read because it spells out the output schema. For interview problems where the output is examined, named is the safer choice. 4. Bring up the MultiIndex flattening recipe in case the dict form is what you inherited: `result.columns = ['_'.join(col).strip() for col in result.columns]` . Covered properly in Q27.

Edge cases to mention: - The named form requires unique kwarg names. If you accidentally write `total=('amount', 'sum'), total=('quantity', 'sum')` , Python's parser keeps only the

last — a silent column-loss bug. - You can mix string aggregations and callables in the same call:

`range_=('amount', lambda s: s.max() - s.min())` . - The dict form preserves the column ordering you wrote; the named form preserves the kwarg order. Both let you control the output column layout.

What NOT to say: - "You can only do this with the dict form" — named aggregation works fine and is preferred in pandas 2.x. - "The MultiIndex columns are a feature" — they are an artefact of the dict form's design. Some shops use them deliberately; most flatten them immediately. Either way, name the choice.

Common follow-up: "How do you flatten the MultiIndex columns that come out of the dict form?" — `result.columns = ['_'.join(col).strip('_') for col in result.columns]` . Tuple per column, join with underscore, strip trailing underscores from single-level entries. We do this in detail in Q27.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
    'quantity': [1, 2, 3, 1],
})

# Modern: flat columns, named outputs
df.groupby('city').agg(
    amount_sum=('amount', 'sum'),
    amount_mean=('amount', 'mean'),
    qty_min=('quantity', 'min'),
    qty_max=('quantity', 'max'),
)

# Legacy: MultiIndex columns
df.groupby('city').agg({'amount': ['sum', 'mean'], 'quantity': ['min', 'max']})
```

Q14 · Custom aggregation that needs two columns at once

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, JP Morgan, Goldman · Topic: agg patterns · Multi-column custom

The question (interviewer's verbatim phrasing):

Per city, I want the weighted average price where the weight is the quantity column. `.agg()` only sees one column at a time — how do you do this cleanly?

The 30-second answer: `.agg()` per column is the wrong tool when the aggregation needs two columns at once. The clean form is `.apply()` with a function that takes the per-group DataFrame and

returns the scalar — `df.groupby('city').apply(lambda g: (g['price'] * g['quantity']).sum() / g['quantity'].sum(), include_groups=False)`. The apply path is justified here because the function genuinely needs both columns together. For higher performance, compute a derived `weighted_amount` column first and then use plain `.agg()` — that vectorises the multiplication outside the loop.

Step-by-step thought process: 1. State why `agg` falls short. The string and callable forms in `.agg()` operate on one column at a time — they cannot reach across columns within the same group. 2. Show the apply solution. Lambda receives the per-group DataFrame, multiplies two columns, sums, divides. The `include_groups=False` keyword is the pandas 2.2+ recommendation to avoid the group keys being passed in as a phantom column. 3. Show the vectorised alternative — the senior-tier answer. Add a precomputed `weighted_amount = price * quantity` column to the source frame, then do `agg(wsum=('weighted_amount', 'sum'), qsum=('quantity', 'sum'))` and divide at the end. The multiplication runs once across the whole frame in C, not once per group in Python. 4. Compare the two on perf grounds. For ten cities, apply is fine. For a million customers, the vectorised path can be 100x faster. The choice is a perf vs readability trade-off and you should name it explicitly in the answer.

Edge cases to mention: - Without `include_groups=False`, older pandas would pass the group key columns into the function — causing the lambda's view of `g` to include `city` as a column. The newer keyword removes that confusion. - If the weighted-sum denominator can be zero in some group (no quantity at all), guard against divide-by-zero either with `np.where` or by filtering empty groups before aggregation. - A weighted average is one example; the same shape applies to any aggregation that crosses columns — correlations per group, group-wise dot products, ratio aggregations.

What NOT to say: - "You have to use apply for this" — you do not; the precompute-then-agg pattern is the vectorised escape hatch and is the right answer at scale. - "Apply is fine because it is one line" — for ten groups, yes. For a real workload, the apply path is the bottleneck.

Common follow-up: "Refactor the apply version to use vectorised ops." — `df['wamt'] = df['price'] * df['quantity']; out = df.groupby('city').agg(wsum=('wamt', 'sum'), qsum=('quantity', 'sum')); out['weighted_avg'] = out['wsum'] / out['qsum']`. Three lines, vectorised throughout.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'price': [100, 200, 150, 250],
    'quantity': [3, 1, 2, 4],
})

# Concise but per-group Python
df.groupby('city').apply(
    lambda g: (g['price'] * g['quantity']).sum() / g['quantity'].sum(),
    include_groups=False,
)

# Vectorised – runs in C
df['weighted_amount'] = df['price'] * df['quantity']
out = df.groupby('city').agg(
    wsum=('weighted_amount', 'sum'),
    qsum=('quantity', 'sum'),
)
out['weighted_avg'] = out['wsum'] / out['qsum']
```

Q15 · `.size()` vs `.count()` — the NULL handling difference

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, JP Morgan · Topic: agg patterns · Count semantics

The question (interviewer's verbatim phrasing):

What is the difference between `GroupBy.size()` and `GroupBy.count()`? When would you use one over the other?

The 30-second answer: `size()` counts the number of rows per group including rows where every column is NaN. `count()` counts the number of non-null entries per column per group — so it returns a DataFrame with one column per non-group column, each showing the count of non-null values in that column for that group. Use `size()` when you want "how many rows landed in this group"; use `count()` when you want "how many non-null observations of this column in this group."

Step-by-step thought process: 1. Lead with the headline difference — rows including NaN versus non-null observations. That distinction is the whole question. 2. Show what each returns. `size()` returns a Series (Index = group key, value = row count). `count()` returns a DataFrame (Index = group key, one column per non-group column, each value = non-null count). 3. State the practical implication. If you have a wide frame with optional columns, `count()` gives you a per-column completeness profile per group. `size()` gives you a single number per group. 4. Note the SQL parallel for the candidates who came up via SQL. `size()` is like `COUNT(*)`. `count()` per-column is like `COUNT(col)` for each col. Same NULL-handling distinction the SQL world has.

Edge cases to mention: - Both methods ignore the `as_index` flag for the most part — both return a Series or DataFrame indexed by the group key by default. `count()` does respect `as_index=False`. - For a single-column groupby with a single column of interest, `df.groupby('city')['amount'].count()` and `df.groupby('city').size()` give different numbers if `amount` has any NaN — that is the most common interview gotcha. - `nunique()` is the third sibling — it counts distinct non-null values per group per column. Useful for "distinct customers per region" patterns.

What NOT to say: - "They are the same" — they differ on NaN handling, which is the question. - "count() includes nulls" — it explicitly does not; that is its whole behaviour.

Common follow-up: "You have a frame where `coupon_code` is NaN for half the rows.

`groupby('city').size()` returns 100 for Mumbai. `groupby('city')['coupon_code'].count()` returns 50 for Mumbai. Why?" — Mumbai has 100 rows total; only 50 of them have a non-null coupon. `size()` counts all rows, `count()` counts non-null entries.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Mumbai', 'Mumbai', 'Delhi', 'Delhi'],
    'amount': [499, 799, None, 250, 1299],
    'coupon_code': ['DIWALI', None, None, 'NEW10', None],
})

df.groupby('city').size()
# city
# Delhi      2
# Mumbai     3

df.groupby('city').count()
#          amount  coupon_code
# city
# Delhi         2             1
# Mumbai        2             1
```

Section C — Indian-interview canonical patterns

These are the ten questions that show up by name in panels at Razorpay, Swiggy, Flipkart, Zerodha, PhonePe, Cred, Meesho, and the GCC-tier Goldman / JP Morgan / Walmart Labs / Target rounds. Top-N per group is the most-asked of all — every Indian Data interview has some version of it. Pct change within group, lag/lead via shift, cumulative sums, ranks per group, and quantiles per group are the next five most-asked. Read these ten as a checklist; if you can write each without looking, you have cleared the pandas portion of every Data-role interview in India.

Q16 · The groupby they ask — top three customers per region with named aggregation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Zerodha, PhonePe, Cred, Meesho ·
Topic: Top-N per group · Canonical · Sample Card 03

The question (interviewer's verbatim phrasing):

Per region, give me the top three customers by total revenue. I want the region, the customer id, and their total revenue in the output. Show me the cleanest pandas pattern.

The 30-second answer: Aggregate first, then sort within each group, then keep the top three. The canonical pattern is `df.groupby(['region', 'customer_id'], as_index=False).agg(total_revenue=('amount', 'sum')).sort_values(['region', 'total_revenue'], ascending=[True, False]).groupby('region').head(3)`. Aggregate the per-customer total in one pass; sort the result; the second groupby plus `.head(3)` keeps the top three rows per region without a Python loop.

Step-by-step thought process: 1. Decompose the problem. We need a per-customer total inside each region, then the top three per region. That is two phases — aggregate and rank. 2. Phase one: group by `(region, customer_id)`, name the aggregation `total_revenue`. Use `as_index=False` so the next step has flat columns to work with. 3. Phase two: sort the aggregated frame by region ascending and revenue descending. This puts each region's customers in rank order within the frame. 4. Phase three: `.groupby('region').head(3)` keeps the first three rows per region. Because the frame is already sorted, those first three are the top three by revenue. This is the senior-level move — many candidates reach for `apply(lambda g: g.nlargest(3, 'total_revenue'))` which is correct but 10× slower at scale.

Edge cases to mention: - Tie-breaking: if two customers tie on revenue, `head(3)` keeps the first three by frame order. If you need a deterministic tie-break, add a secondary sort column (e.g. `customer_id`) to `sort_values`. - If a region has fewer than three customers, `head(3)` returns however many exist — no padding with NaN, no error. That is the right behaviour for interview problems; sometimes downstream code expects exactly three rows and you need to handle the small-group case yourself. - An alternative idiom uses `nlargest`: `df.groupby('region').apply(lambda g: g.nlargest(3, 'amount'))`. Reads well, runs slow on large frames. The sort + head pattern is the vectorised form.

What NOT to say: - "Use `.apply(lambda g: g.head(3))`" — that gives the first three rows of each group, not the top three. Sorting first is the whole point. - "There is no vectorised way" — the sort + groupby + head pattern is the vectorised way.

Common follow-up: "How would you do the same with a window function instead of the two-groupby pattern?" — Use `rank : agg_df['r'] = agg_df.groupby('region')['total_revenue'].rank(method='first', ascending=False); agg_df.query('r <=`

3') . Functionally equivalent, slightly different on tie behaviour depending on `method` . Covered in Q21.

```
import pandas as pd

df = pd.DataFrame({
    'region': ['North', 'North', 'North', 'North', 'South', 'South', 'South'],
    'customer_id': [1, 2, 3, 4, 5, 6, 7],
    'amount': [500, 1200, 800, 300, 700, 900, 400],
})

# Phase 1: per-customer totals
agg = df.groupby(['region', 'customer_id'], as_index=False).agg(
    total_revenue=('amount', 'sum'),
)

# Phase 2: sort within region by revenue desc
agg = agg.sort_values(['region', 'total_revenue'], ascending=[True, False])

# Phase 3: top 3 per region
top3 = agg.groupby('region').head(3)
print(top3)
```

#	region	customer_id	total_revenue
# 1	North	2	1200
# 2	North	3	800
# 0	North	1	500
# 5	South	6	900
# 4	South	5	700
# 6	South	7	400

Q17 · Top-N per group with `nlargest` — the apply variant

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, Postman · Topic: Top-N per group · Alternative

The question (interviewer's verbatim phrasing):

What is `.apply(lambda g: g.nlargest(N, col))` doing, and when is that variant of top-N preferable to the sort + head approach?

The 30-second answer: `apply(lambda g: g.nlargest(3, 'amount'))` runs `nlargest` independently on each group's frame, returning the top three rows by `amount` per group. It is more readable than the sort + head pattern and handles tie-breaking via `nlargest`'s keep semantics. It is also significantly slower at scale because it falls into the apply performance trap from Q12. Prefer it when

group count is low (< 1000 groups) or when readability matters more than raw speed; reach for sort + head otherwise.

Step-by-step thought process: 1. Describe what `nlargest` does on a frame — returns the N rows with the largest values in the named column, sorted descending. It is a vectorised method, fast on a single frame. 2. State why wrapping it in `.apply` is slower than sort + head. apply runs the lambda once per group in Python; sort + head runs once over the whole sorted frame in C. 3. Show when to prefer apply + nlargest. The code reads as exactly the English sentence "for each group, take the top three by amount." That readability is worth something in shared codebases and in interview situations where speed is not the constraint. 4. Note the tie-handling difference. `nlargest(3, 'amount', keep='first')` (default) returns three rows even in ties, picking by frame order. `keep='all'` returns all rows tied for the smallest of the top three — useful for percentile-style problems.

Edge cases to mention: - The `include_groups=False` parameter (pandas 2.2+) on `.apply` avoids the group keys being added as columns — important when the lambda inspects column names. - `nlargest` on a column of mixed dtypes (strings and numbers) raises; ensure the column is numeric. - For a single group's top-N from a non-grouped frame, just call `df.nlargest(3, 'amount')` — no groupby needed.

What NOT to say: - "It is always the cleaner answer" — readability is real but at scale it is a 10–100× perf cost. The senior answer names the trade-off. - "You can't use this in production" — you can; it works fine for small group counts. The bottleneck is per-group Python overhead.

Common follow-up: "Which version is faster: sort + head, or apply + nlargest?" — Sort + head, by 5–20× on a million-row frame with tens of thousands of groups. The single sort is C-speed; the apply is per-group Python.

```
df = pd.DataFrame({
    'region': ['North'] * 4 + ['South'] * 3,
    'customer_id': [1, 2, 3, 4, 5, 6, 7],
    'amount': [500, 1200, 800, 300, 700, 900, 400],
})

# Reads as the English statement
top3 = df.groupby('region', group_keys=False).apply(
    lambda g: g.nlargest(3, 'amount'),
    include_groups=False,
)
print(top3)
```

Q18 · `pct_change` within group — month-on-month growth per category

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, Cred · Topic: `pct_change` · Time series per group

The question (interviewer's verbatim phrasing):

I have a frame with `category`, `month`, `revenue` — one row per category-month. Give me the month-on-month percent change in revenue for each category.

The 30-second answer: Sort by `(category, month)` so the rows are in time order within each category, then use `groupby('category')['revenue'].pct_change()` — pandas computes the percent change row-by-row but resets the calculation at every group boundary, so the first row of each category gets NaN and subsequent rows show the percent change versus the prior month within the same category. Assign the result to a new column and you have month-on-month growth per category in two lines.

Step-by-step thought process: 1. Lead with the data-shape assumption. The frame must be sorted by category then month for `pct_change` within group to mean anything sensible. State this explicitly — the interviewer will check whether you know the sort precondition. 2. Show the pattern.

`df.sort_values(['category', 'month'])`, then `df['mom_growth'] = df.groupby('category')['revenue'].pct_change()`. The groupby ensures the change is computed within category, not across categories. 3. Explain the NaN semantics. The first row in each category has no prior row in that category, so `pct_change` returns NaN there. Downstream code must handle the NaN explicitly — `.fillna(0)` if you want the first month to be zero growth, or `.dropna()` if you want only periods with a baseline. 4. Mention the `periods` parameter. Default is 1 (compare to the immediately prior row). `periods=12` would compute year-on-year growth on a monthly frame, assuming twelve rows per category per year and proper sort. Useful at the senior level.

Edge cases to mention: - If your data has gaps (missing months), `pct_change` does not know that — it just compares row to prior row. For a true month-on-month with gap handling, reindex to a full date range per category first. - Negative or zero baseline values produce infinities or NaN in `pct_change`. Inspect with `np.isinf` if your inputs can be zero. - `pct_change` returns a fraction (0.20 means 20% growth), not a percent. Multiply by 100 if you want the display value.

What NOT to say: - "You need a custom function for this" — pandas has `pct_change` built in and it respects the groupby boundary. - "It only works on time series indexes" — it works on any sequential rows in the group; the meaning is "MoM" only when the rows are in month order.

Common follow-up: "What if the data has a missing month for some category — say June is absent?" — `pct_change` ignores the absence and compares May to July as if they were adjacent. To handle this honestly, build a full month index per category, reindex with NaN for missing months, then call `pct_change`. That way missing months produce NaN baselines, not phantom growth values.


```
df = pd.DataFrame({
    'category': ['Apparel', 'Apparel', 'Apparel', 'Electronics', 'Electronics'],
    'month': ['2026-01', '2026-02', '2026-03', '2026-01', '2026-02'],
    'revenue': [100, 120, 150, 500, 450],
})

df = df.sort_values(['category', 'month'])
df['mom_growth'] = df.groupby('category')['revenue'].pct_change()
print(df)
```

#	category	month	revenue	mom_growth
# 0	Apparel	2026-01	100	NaN
# 1	Apparel	2026-02	120	0.200000
# 2	Apparel	2026-03	150	0.250000
# 3	Electronics	2026-01	500	NaN
# 4	Electronics	2026-02	450	-0.100000

Q19 · Lag and lead within group via `shift`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Goldman · Topic: shift · Lag · Lead

The question (interviewer's verbatim phrasing):

Per customer, I want the previous order's amount and the next order's amount as new columns.
Show me the pandas pattern.

The 30-second answer: Sort by `(customer_id, order_date)`, then `df['prev_amount'] = df.groupby('customer_id')['amount'].shift(1)` for the lag and `df['next_amount'] = df.groupby('customer_id')['amount'].shift(-1)` for the lead. `shift(1)` rolls each customer's amount column down by one row inside the group; `shift(-1)` rolls up. The first row of each customer's slice gets NaN for `prev_amount`, the last row gets NaN for `next_amount` — same as SQL's `LAG` and `LEAD` window functions.

Step-by-step thought process: 1. State the sort precondition first. `shift` is positional — it shifts by row position within each group, not by date. The result is meaningful only when the rows inside each group are in the order you want to lag against. 2. Show both directions. `shift(1)` is lag (look back one row); `shift(-1)` is lead (look forward one row). The sign convention is the same as SQL but the parameter is positional offset rather than a window function. 3. Mention the alignment guarantee. The shifted Series has the same length and same index as the original — pandas aligns everything for you, including the NaN-padding at group boundaries. 4. Bring up the canonical interview problem: time-between-orders. `df['days_since_prev'] = (df['order_date'] - df.groupby('customer_id')['order_date'].shift(1)).dt.days`. One line, vectorised, handles per-customer boundaries.

Edge cases to mention: - `shift` defaults to filling with NaN. Override via `fill_value=0` if you want zero-filled output (useful for the first-order case where you want to treat NaN as "no prior order"). - For a multi-period lag (e.g. compare to two orders back), pass `shift(2)`. For lead two rows, `shift(-2)`. - `shift` works on any column dtype — Datetimes, strings, integers. The NaN at group boundaries becomes NaT for datetime columns.

What NOT to say: - "Use `apply(lambda g: g.shift(1))`" — that works but is slower than the vectorised groupby + shift form. - "Shift only works on integers" — it works on any Series, including datetimes and strings.

Common follow-up: "How would you compute the days between consecutive orders per customer?" — Sort, then `df['days_since'] = df.groupby('customer_id')['order_date'].diff().dt.days`. `.diff()` is conceptually `(this row) - (shift(1))` on the same Series; it gives you the time delta directly.

```
df = pd.DataFrame({
    'customer_id': [1, 1, 1, 2, 2],
    'order_date': pd.to_datetime(['2026-01-01', '2026-01-15', '2026-02-01',
                                   '2026-01-10', '2026-01-20']),
    'amount': [500, 700, 900, 300, 400],
}).sort_values(['customer_id', 'order_date'])

df['prev_amount'] = df.groupby('customer_id')['amount'].shift(1)
df['next_amount'] = df.groupby('customer_id')['amount'].shift(-1)
df['days_since_prev'] = (
    df['order_date'] - df.groupby('customer_id')['order_date'].shift(1)
).dt.days
print(df)
```

Q20 · Cumulative sum within group — running totals per customer

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, PhonePe · Topic: cumsum · Running totals

The question (interviewer's verbatim phrasing):

Per customer, give me a running total of order amount sorted by order date. The first order shows its own amount, the second shows the sum of order one and two, and so on.

The 30-second answer: Sort by `(customer_id, order_date)`, then `df['running_total'] = df.groupby('customer_id')['amount'].cumsum()`. `cumsum` on a grouped Series computes the cumulative sum within each group, resetting at every group boundary. Same pattern as the SQL window `SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)`.

Step-by-step thought process: 1. State the sort dependency. Cumulative sums respect row order within each group; the meaningful order is `order_date` ascending. Without the sort, the cumsum is computed in whatever order the rows happen to be in. 2. Show the one-line pattern and explain why it works. Group-wise cumulative functions in pandas — `cumsum`, `cumprod`, `cummax`, `cummin`, `cumcount` — all respect the group boundary by resetting their accumulator at each new group. 3. Mention the SQL parallel. The window-function equivalent is `SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)`. Knowing the parallel helps in mixed-stack interviews where the interviewer pivots from pandas to SQL mid-question. 4. Note the sibling functions briefly. `cumcount` gives the row index within the group (0, 1, 2, ...), useful for "the Nth order of each customer" patterns. `cummax` gives the running maximum, useful for "highest order so far per customer" reports.

Edge cases to mention: - `cumsum` returns the same length as the input, with the original index — assignable directly to a new column. - NaN values participate: `NaN + 500 = NaN`. Filter or `.fillna(0)` before cumsum if you want NaN treated as zero. - The cumulative functions do not honour `skipna=False` as agg does; they always propagate NaN forward unless you fill first.

What NOT to say: - "You need apply for this" — `cumsum` is a vectorised GroupBy method; no apply needed. - "It sorts the data for you" — it does not; you must sort first or the running total is meaningless.

Common follow-up: "Give me each customer's order number — first order is 1, second is 2, and so on." — `df['order_num'] = df.groupby('customer_id').cumcount() + 1`. `cumcount` starts at 0, so add 1 for the human-readable count.

```
df = pd.DataFrame({
    'customer_id': [1, 1, 1, 2, 2],
    'order_date': pd.to_datetime(['2026-01-01', '2026-01-15', '2026-02-01',
                                   '2026-01-10', '2026-01-20']),
    'amount': [500, 700, 900, 300, 400],
}).sort_values(['customer_id', 'order_date'])

df['running_total'] = df.groupby('customer_id')['amount'].cumsum()
df['order_num'] = df.groupby('customer_id').cumcount() + 1
print(df)
```

Q21 · Rank within group — `rank` and the `method` parameter

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Goldman, Walmart Labs · Topic: rank · Within group

The question (interviewer's verbatim phrasing):

Per region, rank customers by total revenue from highest to lowest. Ranks should be 1, 2, 3, ... and ties should each get the same rank with no gap to the next number.

The 30-second answer: Use `rank` with `method='dense'` and `ascending=False` :
`df['rank_in_region'] = df.groupby('region')['revenue'].rank(method='dense', ascending=False)` . The dense method gives ties the same rank and the next rank is the next integer (no gap). Other methods are `'min'` (ties share the lowest rank, next rank skips), `'max'` (ties share the highest rank), `'first'` (break ties by frame order), and `'average'` (the default — ties get the mean of the would-be ranks).

Step-by-step thought process: 1. State the headline result first. `rank` runs per group when called on a GroupBy Series. `ascending=False` puts the highest revenue at rank 1. 2. Walk through the five method options and what each means for ties. Most interview problems want `'dense'` (1, 1, 2, 3) or `'min'` (1, 1, 3, 4). The default `'average'` produces fractional ranks (1.5, 1.5) which is usually not what the question wants. 3. Tie this to the top-N pattern (Q16, Q17). Once you have ranks, `query('rank_in_region <= 3')` keeps the top three per region — equivalent to the sort + head pattern but with explicit ranks attached. 4. Mention the SQL parallel. `RANK()` in SQL is `method='min'`, `DENSE_RANK()` is `method='dense'`, `ROW_NUMBER()` is `method='first'`. The vocabulary maps cleanly.

Edge cases to mention: - `rank` returns float values by default — even when ranks are integral, the dtype is float64 because the average method can produce fractions. Cast to int after if you need integer ranks. - `na_option` controls how NaN values are ranked: `'keep'` (default — NaN stays NaN), `'top'` (NaN ranks first), `'bottom'` (NaN ranks last). - `pct=True` returns the rank as a percentile (0.0 to 1.0) — useful for "top 10% of customers per region" filters.

What NOT to say: - "rank gives integers" — it returns float64 by default, even for clean rank methods. - "ascending defaults to True" — true, but for ranking problems where 1 should mean "best", you almost always want `ascending=False` .

Common follow-up: "What's the difference between `method='min'` and `method='dense'`?" — On ties at rank 1, both put the tied rows at rank 1. The next rank differs: `'min'` skips the consumed ranks (next is 3 if two tied at 1), `'dense'` does not skip (next is 2). SQL `RANK` is `'min'` ; SQL `DENSE_RANK` is `'dense'` .

```
agg = pd.DataFrame({
    'region': ['North', 'North', 'North', 'South', 'South'],
    'customer_id': [1, 2, 3, 4, 5],
    'revenue': [500, 1200, 1200, 700, 900],
})

agg['rank_dense'] = agg.groupby('region')['revenue'].rank(method='dense', ascending=False)
agg['rank_min'] = agg.groupby('region')['revenue'].rank(method='min', ascending=False)
print(agg)
```

#	region	customer_id	revenue	rank_dense	rank_min
# 0	North	1	500	2.0	3.0
# 1	North	2	1200	1.0	1.0
# 2	North	3	1200	1.0	1.0
# 3	South	4	700	2.0	2.0
# 4	South	5	900	1.0	1.0

Q22 · Percentile / quantile per group

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Cred, JP Morgan · Topic: quantile · Per group

The question (interviewer's verbatim phrasing):

Per region, give me the median order amount and the 95th percentile order amount in one call.

The 30-second answer: Use `quantile` with the named-aggregation form:

`df.groupby('region').agg(median=('amount', lambda s: s.quantile(0.5)), p95=('amount', lambda s: s.quantile(0.95)))` . Or for a single percentile, `df.groupby('region')['amount'].quantile(0.95)` is the shorter path. For multiple quantiles in one shot, pass a list: `df.groupby('region')['amount'].quantile([0.5, 0.95])` — output is a MultiIndex with region and quantile as the levels.

Step-by-step thought process: 1. Show the simple single-quantile path first.

`groupby(...).quantile(0.5)` is the median, returns one row per group. Idiomatic and fast. 2. Show the list-of-quantiles path. `quantile([0.5, 0.95])` returns a MultiIndex: outer level is the region, inner level is the quantile. Useful when you want multiple cutoffs in one call. 3. Show the named-aggregation lambda form. When you want flat output columns with custom names (`median` , `p95`), the named form with lambdas is the cleanest. Lambdas are fine for quantile because the function is short and pandas optimises around them well. 4. Mention the `interpolation` parameter. Default is `'linear'` (mathematically standard); other options are `'nearest'` , `'lower'` , `'higher'` , `'midpoint'` . For interview questions, linear is almost always what is wanted.

Edge cases to mention: - `quantile` requires numeric columns; on object columns it raises. Use `dropna()` first if your column has NaN you do not want to influence the percentile. - The MultiIndex output from list-quantile can be unstacked via `.unstack()` to get the percentiles as columns — useful for downstream merging. - For very large groups, `quantile` builds a sort internally per group; performance scales with group size.

What NOT to say: - "You need a custom percentile function" — pandas has it built in and respects the group boundary. - "Median is different from quantile(0.5)" — the median is exactly quantile 0.5 by definition; the methods are the same call.

Common follow-up: "What does `quantile([0.25, 0.5, 0.75])` give back for a grouped Series?" — A Series with a MultiIndex: the outer level is the group key, the inner level is the quantile (0.25, 0.5, 0.75). Three rows per group. Reshape with `.unstack()` to get quantiles as columns.

```
df = pd.DataFrame({
    'region': ['North'] * 5 + ['South'] * 5,
    'amount': [100, 200, 300, 400, 500, 150, 250, 350, 450, 550],
})

# Single quantile
df.groupby('region')['amount'].quantile(0.95)

# Multiple quantiles, MultiIndex output
df.groupby('region')['amount'].quantile([0.5, 0.95])

# Named aggregation – flat output columns
df.groupby('region').agg(
    median=('amount', lambda s: s.quantile(0.5)),
    p95=('amount', lambda s: s.quantile(0.95)),
)
```

Q23 · First and last per group — with sort precondition

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: first · last · idxmax

The question (interviewer's verbatim phrasing):

Per customer, get the first order date and the last order date — and also the amount on each of those orders.

The 30-second answer: Sort by `(customer_id, order_date)`, then

`df.groupby('customer_id').agg(first_order=('order_date', 'first'), last_order=('order_date', 'last'), first_amount=('amount', 'first'), last_amount=('amount', 'last'))`. The `'first'` and `'last'` aggregations pick the first and last non-null value from each group as they appear in the frame's row order — which is why the sort matters.

Step-by-step thought process: 1. State the sort precondition. `'first'` and `'last'` are positional within each group, not "by date" by default. You sort first or you get whatever order the frame happens to be in. 2. Show the named aggregation form. Each of the four output columns picks a different source column with `'first'` or `'last'`. Clean, flat, named output. 3. Mention NaN handling. `'first'` returns the first non-null value, `'last'` the last non-null. If you want literally the first row regardless of nulls, that is `head(1)` per group instead. 4. Bring up `idxmax` and `idxmin` for the "row where some column is maximum" pattern. `df.loc[df.groupby('customer_id')['amount'].idxmax()]` returns the entire row with the maximum amount per customer — useful when you want all the columns of the max-amount row, not just the amount itself.

Edge cases to mention: - `'first'` and `'last'` are aliases for `.first()` and `.last()` methods on GroupBy. They behave identically. - For multi-key sorts, the order of keys matters for what "first" means. Sort by the date column, not by customer_id alone, when you want chronological first. - `idxmax` requires a unique index for the result to be unambiguous. If the index has duplicates, the returned positions may collide.

What NOT to say: - "first and last give the smallest and largest" — they give the positional first and last, not the min and max. min and max are the aggregations for that. - "It returns NaN if any value is null" — it skips nulls by default.

Common follow-up: "What if I want the whole row where a customer placed their highest-value order?" — Use `idxmax` : `df.loc[df.groupby('customer_id')['amount'].idxmax()]`. That gets the row index of the max-amount row per customer and `loc` materialises the full row.

```
df = pd.DataFrame({
    'customer_id': [1, 1, 1, 2, 2],
    'order_date': pd.to_datetime(['2026-01-01', '2026-01-15', '2026-02-01',
                                   '2026-01-10', '2026-01-20']),
    'amount': [500, 700, 900, 300, 400],
}).sort_values(['customer_id', 'order_date'])

df.groupby('customer_id').agg(
    first_order=('order_date', 'first'),
    last_order=('order_date', 'last'),
    first_amount=('amount', 'first'),
    last_amount=('amount', 'last'),
)

# Whole row where amount is max per customer
df.loc[df.groupby('customer_id')['amount'].idxmax()]
```

Q24 · Filter groups by aggregate condition — HAVING in pandas

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Postman · Topic: filter · groupby

The question (interviewer's verbatim phrasing):

Give me only the regions where total revenue exceeds 10,000. SQL would use HAVING — what is the pandas equivalent?

The 30-second answer: Two clean paths. Option one: aggregate first, then filter the result. `agg = df.groupby('region')['amount'].sum(); agg[agg > 10000]`. Option two: `df.groupby('region').filter(lambda g: g['amount'].sum() > 10000)` — this keeps

every row of qualifying groups, not just the per-group aggregate. Use option one when you want the aggregate output; option two when you want all original rows from groups that pass the condition.

Step-by-step thought process: 1. State the two paths and what each returns. Aggregate-then-filter gives a Series of per-region totals filtered to qualifying regions. `.filter` gives the original rows back, scoped to qualifying groups. 2. Walk through option one's mental model. It is the SQL HAVING pattern split into two pandas steps — GROUP BY + aggregate first, then filter the aggregated frame. 3. Walk through option two. `.filter` runs the predicate function on each group's frame and returns the original rows of groups where the predicate is True. The function must return a boolean scalar (not a per-row Series). 4. Note the performance difference. Aggregate-then-filter is one pass to aggregate, one pass to filter — fast and idiomatic. `.filter` runs the predicate once per group in Python, similar perf to `.apply` — slower at scale.

Edge cases to mention: - `.filter`'s predicate must return a scalar boolean. Returning a Series raises; returning a DataFrame raises. If your condition is per-row, you want `transform` plus boolean indexing instead. - The output index from `.filter` preserves the original row positions, not the group order. - `.transform` + boolean mask is the third path for the per-row variant: `mask = df.groupby('region')['amount'].transform('sum') > 10000; df[mask]`. Vectorised, faster than `.filter` at scale.

What NOT to say: - "There is no HAVING in pandas" — there are at least three good idioms; pick the one that matches your output need. - "filter is the same as agg + boolean" — they return different shapes; the choice depends on whether you want per-group output or per-row output.

Common follow-up: "Which path is faster for a frame with a million groups?" — Aggregate-then-filter, or the `transform + mask` path. `.filter` runs Python per group and dominates the runtime; the vectorised paths are 10x to 50x faster.

```
df = pd.DataFrame({
    'region': ['North'] * 3 + ['South'] * 2 + ['East'] * 4,
    'amount': [4000, 5000, 6000, 1000, 2000, 3000, 3500, 4000, 4500],
})

# Path 1 – aggregate then filter
agg = df.groupby('region')['amount'].sum()
print(agg[agg > 10000])

# Path 2 – original rows from qualifying groups
df.groupby('region').filter(lambda g: g['amount'].sum() > 10000)

# Path 3 – vectorised per-row mask
mask = df.groupby('region')['amount'].transform('sum') > 10000
df[mask]
```


Q25 · Group-wise mean centering — subtract group mean from each value

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Goldman · Topic: transform · Mean centering

The question (interviewer's verbatim phrasing):

Per region, subtract the regional mean revenue from each row's revenue. Add this as a new column called `revenue_vs_region_mean`.

The 30-second answer: `df['revenue_vs_region_mean'] = df['revenue'] - df.groupby('region')['revenue'].transform('mean')`. The transform call broadcasts each region's mean back to every row of that region, so the subtraction aligns one-to-one. One line, vectorised, indexed identically to the source frame.

Step-by-step thought process: 1. Recognise that this is a transform problem, not an agg problem. We want one output value per input row (input shape preserved), not one row per region. 2. Show the pattern. Compute the per-group mean with `.transform('mean')`; that returns a Series the same length as `df` with each row holding its region's mean. Subtract column-wise. 3. State the equivalent slower forms for contrast. `.apply(lambda g: g['revenue'] - g['revenue'].mean())` works but runs per-group Python. `.agg('mean')` followed by `merge` works but is two steps. Transform is the one-liner that does the right thing. 4. Mention the use-case bucket. Group-wise mean centering, group-wise normalisation (subtract mean, divide by std), group-wise log offsets — all use the same transform + broadcast pattern. Once you internalise transform, these collapse to one line each.

Edge cases to mention: - The result Series from `.transform` has the same dtype as the input — be aware of integer overflow on large sums. - If a group has only one row, its mean equals its value, so the centered value is exactly zero. Some downstream models treat zero as "no signal" — confirm the use-case allows it. - For mean centering across multiple grouping levels (e.g. region and category), pass the list: `df.groupby(['region', 'category'])['revenue'].transform('mean')`.

What NOT to say: - "Use apply for this" — transform is the right tool; apply is slower for the same operation. - "You need to merge the means back" — transform does the alignment for you; no merge needed.

Common follow-up: "Now normalise instead of just centering — subtract the group mean and divide by the group standard deviation." — `(df['revenue'] - df.groupby('region')['revenue'].transform('mean')) / df.groupby('region')['revenue'].transform('std')`. Two transforms; pandas re-uses the GroupBy if you cache it in a variable first.

```
df = pd.DataFrame({
    'region': ['North', 'North', 'North', 'South', 'South'],
    'revenue': [100, 200, 300, 400, 600],
})

df['revenue_vs_region_mean'] = (
    df['revenue'] - df.groupby('region')['revenue'].transform('mean')
)
print(df)
```

#	region	revenue	revenue_vs_region_mean
# 0	North	100	-100.0
# 1	North	200	0.0
# 2	North	300	100.0
# 3	South	400	-100.0
# 4	South	600	100.0

Section D — groupby gotchas

The last five questions hit the classes of bug that ship to production from pandas groupby code. NaN keys silently disappearing from the output. MultiIndex columns that downstream code cannot handle. Categorical groupings producing rows for unobserved categories. The apply-versus-agg performance cliff that turns a 200 ms job into a 20-second job overnight. These are the questions the senior interviewer asks at the end of the round to separate the candidates who have written real production pandas from the candidates who have only written tutorial pandas.

Q26 · NaN keys in groupby — silent row loss

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, JP Morgan, Walmart Labs · Topic: groupby gotchas · NaN handling

The question (interviewer's verbatim phrasing):

I run `df.groupby('city')['amount'].sum()` and the total comes out lower than `df['amount'].sum()`. What is happening?

The 30-second answer: `groupby` drops rows where the grouping key is NaN by default. Any row with `city` as NaN is silently excluded from the per-city sums, so the per-city totals add up to less than the overall total. Fix it by passing `dropna=False` to `groupby`: `df.groupby('city', dropna=False)['amount'].sum()`. That keeps NaN-keyed rows together as a NaN-labelled group.

Step-by-step thought process: 1. Name the cause precisely. The default `dropna=True` on `.groupby()` excludes rows whose key is NaN. There is no warning; the rows simply disappear from the aggregation. 2. Demonstrate the symptom. A frame with three NaN-city rows and seven valid-city rows produces a sum over the seven; the candidate sees `groupby_sum < df_sum` and is puzzled. 3. Show the fix. `dropna=False` retains the NaN group with `NaN` as its label. Now `groupby_sum == df_sum` (when accounting for all-NaN observations being zero-summed). 4. State the production implication. In reporting code, this default has caused real misreported totals. Senior reviewers grep for `groupby()` without `dropna` whenever the metric being reported is a sum that must reconcile with a row-level total.

Edge cases to mention: - For multi-key groupbys, `dropna=True` drops rows where ANY of the keys is NaN. A row with valid city but NaN product is dropped from `groupby(['city', 'product'])`. - The `dropna` parameter was added in pandas 1.1; older code may not have it. In ancient pandas you had to `fillna` the key first. - The NaN group label in the output is `NaN` itself, which you can filter or rename downstream.

What NOT to say: - "Groupby includes everything by default" — it explicitly excludes NaN keys. - "This is a bug in pandas" — it is documented behaviour, just an opinionated default that surprises new users.

Common follow-up: "What other defaults silently change the answer compared to a `SELECT * SUM` in SQL?" — `groupby` drops NaN keys; `merge` defaults to inner join; `pivot_table` defaults to mean rather than sum. All three are common sources of "the numbers don't match SQL" bug reports.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', None, 'Mumbai', None],
    'amount': [100, 200, 300, 400, 500],
})

# Default – NaN city dropped, total is 700, not 1500
print(df.groupby('city')['amount'].sum())
# city
# Delhi      200
# Mumbai     500

# With NaN preserved – total reconciles
print(df.groupby('city', dropna=False)['amount'].sum())
# city
# Delhi      200
# Mumbai     500
# NaN        800

print(df['amount'].sum())           # 1500
print(df.groupby('city', dropna=False)['amount'].sum().sum()) # 1500
```

Q27 · MultiIndex result vs flat columns — and how to flatten

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman · Topic: MultiIndex · Flattening

The question (interviewer's verbatim phrasing):

I called `df.groupby('city').agg({'amount': ['sum', 'mean']})` and the result has weird two-level column headers. How do I get flat columns back?

The 30-second answer: The dict + list form of `.agg()` produces a MultiIndex on the columns — outer level is the source column, inner level is the aggregation name. Flatten with `result.columns = ['_'.join(col).strip('_') for col in result.columns]` (join the tuple, strip trailing underscores). Or avoid the MultiIndex entirely by using the named-aggregation form from Q10: `df.groupby('city').agg(amount_sum=('amount', 'sum'), amount_mean=('amount', 'mean'))` gives flat columns directly.

Step-by-step thought process: 1. Diagnose the cause. The MultiIndex columns appear because the dict-with-list-value form `.agg({'amount': ['sum', 'mean']})` returns one column per `(source_column, aggregation)` pair, organised as a two-level header. 2. Show the flattening recipe. Iterate over `result.columns`, join each tuple with an underscore, strip trailing underscores in case some tuples have an empty second element. The output is a list of strings ready to assign back to `result.columns`. 3. Show the preferred avoidance pattern. Named aggregation never produces a MultiIndex when the kwargs have unique names — it is the right tool for new code where you control the output schema. 4. Mention `.reset_index()` does not help here. It flattens the row index (the group keys), not the column index. Column MultiIndex needs the join-tuples recipe.

Edge cases to mention: - The MultiIndex may have empty strings at the inner level for single-aggregation columns. The `.strip('_')` removes the resulting trailing underscore. - `.swapslevel(axis=1)` swaps the outer and inner column levels — useful if you want the aggregation name on the outer level instead of the source column. - Once flattened, the columns are plain strings — downstream `.merge()` and `.assign()` work as expected.

What NOT to say: - "MultiIndex columns are always wrong" — they are a deliberate output form; some pandas users prefer them for the hierarchical labelling. Avoid them by choice, not by reflex. - "You can't flatten them" — the list comprehension recipe is the canonical one-liner.

Common follow-up: "Could you also use `result.columns.map`?" — Yes: `result.columns = result.columns.map('_'.join)`. Shorter when every tuple has the same length. Use the comprehension form when you need the strip step too.

```
df = pd.DataFrame({
    'city': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi'],
    'amount': [499, 1299, 799, 250],
    'quantity': [1, 2, 3, 1],
})

# Produces MultiIndex columns
out = df.groupby('city').agg({'amount': ['sum', 'mean'], 'quantity': 'sum'})
print(out.columns)
# MultiIndex([('amount', 'sum'),
#            ('amount', 'mean'),
#            ('quantity', 'sum')])

# Flatten
out.columns = ['_'.join(col).strip('_') for col in out.columns]
print(out.columns)
# Index(['amount_sum', 'amount_mean', 'quantity_sum'])

# Avoid the MultiIndex in the first place
df.groupby('city').agg(
    amount_sum=('amount', 'sum'),
    amount_mean=('amount', 'mean'),
    quantity_sum=('quantity', 'sum'),
)
```

Q28 · Categorical groupby — `observed=True` vs `observed=False`

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan, Goldman · Topic: Categorical · groupby gotchas

The question (interviewer's verbatim phrasing):

I have a categorical column with five categories defined but only three appear in the data. When I groupby that column I sometimes see five rows in the output and sometimes three — what controls that?

The 30-second answer: The `observed` parameter on `.groupby()` controls it. With `observed=True`, only categories that actually appear in the data show up in the output (three rows). With `observed=False`, every defined category in the Categorical dtype appears, with NaN aggregates for categories that have no rows (five rows). Default behaviour changed in pandas 2.1 — `observed=False` was the historical default; pandas 2.1 deprecated that default and is moving toward `observed=True`. Set the parameter explicitly to lock the behaviour.

Step-by-step thought process: 1. State that the column is a `Categorical` dtype, not a plain object/string. The parameter only affects categorical groupings; for plain object columns, only observed values

are ever returned. 2. Walk through what each value of `observed` produces. `True` → only categories present in the data. `False` → every defined category, even ones with zero rows, returning NaN aggregates for the empty categories. 3. Explain the use-case for each. `observed=False` is useful for fixed-cardinality reports where the schema must show every category every period (e.g. monthly report with all five tiers even when nobody is in Platinum this month). `observed=True` is the usual analytic choice and avoids confusing rows of NaN. 4. Mention the pandas 2.x deprecation. The default switched in 2.1; FutureWarnings nudge users to set it explicitly. New code should always pass `observed=` to lock the behaviour against future default changes.

Edge cases to mention: - `observed=False` on a multi-key groupby with two categorical columns of cardinality 100 produces a 10,000-row output (Cartesian product), even if most combinations have no rows. That can blow up memory unexpectedly. - For pivot tables, the same parameter exists and behaves the same way — `pd.pivot_table(..., observed=True)`. - The Categorical dtype is rarely used by mistake; it usually appears when someone explicitly converted with `pd.Categorical` or read from a Parquet file that preserved the dtype.

What NOT to say: - "It does not matter" — it changes the row count of the output, which downstream code may rely on. - "It is the same as `dropna`" — `dropna` controls NaN keys; `observed` controls unobserved Categorical categories. Different parameters, different bugs.

Common follow-up: "How do you check whether a column is Categorical before grouping?" — `df['col'].dtype.name == 'category'` or `isinstance(df['col'].dtype, pd.CategoricalDtype)`. If it is, decide on `observed=` explicitly before grouping.

```
import pandas as pd

cats = pd.Categorical(
    ['North', 'South', 'North'],
    categories=['North', 'South', 'East', 'West', 'Central'],
)
df = pd.DataFrame({
    'region': cats,
    'amount': [100, 200, 300],
})

# observed=True — only categories present
print(df.groupby('region', observed=True)['amount'].sum())
# region
# North      400
# South      200

# observed=False — every defined category, NaN where empty
print(df.groupby('region', observed=False)['amount'].sum())
# region
# North      400.0
# South      200.0
# East         NaN
# West         NaN
# Central      NaN
```

Q29 · The `.apply` perf cliff — when groupby gets slow at scale

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Walmart Labs, JP Morgan, Target · Topic: Performance · groupby gotchas

The question (interviewer's verbatim phrasing):

A pipeline that took 200 milliseconds on the dev sample now takes 30 seconds on the production frame. The pandas code is `df.groupby('user_id').apply(lambda g: some_function(g))`. What is going wrong?

The 30-second answer: `.apply` runs the Python function once per group, so the cost scales linearly with the number of groups. On a dev sample with 200 user_ids the apply is 200 Python calls — negligible. On a production frame with 200,000 user_ids it is 200,000 Python calls — that is where the 30 seconds comes from. The fix is to rewrite the function as an `.agg()` or `.transform()` if possible, or as a vectorised pre-computation that lifts the per-group work out of the loop.

Step-by-step thought process: 1. Name the cause: per-group Python overhead. Each `apply` invocation pays interpreter dispatch, function-frame setup, attribute lookup. None of those scale with

data; they scale with number of groups. 2. State the rule of thumb. If group count is in the low thousands or fewer, apply is fine. Above ten thousand groups, the cumulative Python overhead dominates and the runtime gets ugly. 3. Walk through the three remediations in order of preference. (a) Replace the lambda with `.agg()` if the function is reducible to one row per group. (b) Replace with `.transform()` if it produces one value per input row. (c) Vectorise a precomputation: pull the column-level math out of the loop, do it once across the whole frame, then aggregate the result. 4. Tie this to the Q14 weighted-average example. Same shape of fix: instead of computing `(price * qty).sum() / qty.sum()` inside an apply, precompute `weighted = price * qty` as a column on the source frame, then `.agg(wsum=('weighted', 'sum'), qsum=('qty', 'sum'))` and divide at the end. The multiplication runs in C, not in Python per group.

Edge cases to mention: - `numba.jit` decorations on aggregation functions can speed up apply by 10–20× for numeric-only paths. `df.groupby('key').agg(numba_fn, engine='numba')` is the syntax. Works only for purely numeric vectorisable functions. - Some apply uses are unavoidable — returning a different-shaped frame per group, for example. In that case, see whether you can reduce the group count first by aggregating to a smaller frame. - Dask and modin parallelise the per-group work across cores or machines. Worth the complexity only when single-machine pandas is provably the bottleneck.

What NOT to say: - "Apply is broken" — it is not; it is doing exactly what it advertises. The wart is that beginners use it for things `.agg` and `.transform` would do faster. - "You need to switch to a different library" — most performance problems can be fixed in pandas itself with the vectorised forms.

Common follow-up: "Walk me through profiling a slow groupby to find which part is the bottleneck." — Use `%timeit` or `cProfile` to time the groupby alone (cheap), the aggregation alone (cheap for vectorised, expensive for Python), and the assembly. Almost always the user-supplied function is the cost; the groupby itself is fast.

```
import time, pandas as pd, numpy as np

n_groups = 200_000
df = pd.DataFrame({
    'user_id': np.repeat(np.arange(n_groups), 10),
    'amount': np.random.rand(n_groups * 10),
})

# Slow – per-group Python
t0 = time.perf_counter()
df.groupby('user_id')['amount'].apply(lambda s: s.sum())
print(f"apply: {time.perf_counter() - t0:.2f}s")

# Fast – vectorised
t0 = time.perf_counter()
df.groupby('user_id')['amount'].sum()
print(f"sum: {time.perf_counter() - t0:.2f}s")
# apply: ~6 to 15 seconds
# sum: ~0.05 to 0.2 seconds
```


Q30 · `groupby` on a column versus on the index — `level=` parameter

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Postman · Topic: groupby gotchas · MultiIndex

The question (interviewer's verbatim phrasing):

The grouping key I want is already in the row index, not a regular column. Do I have to `.reset_index()` first, or is there a shorter path?

The 30-second answer: You do not need `.reset_index()`. Pass `level=` to `.groupby()` instead of a column name: `df.groupby(level='date').agg(...)` if your index has a named level called `date`, or `df.groupby(level=0)` for the outermost level by position. The level form treats the index level exactly as a column for grouping purposes, with no shape change to the source frame.

Step-by-step thought process: 1. State the shortcut first. `groupby(level=...)` is the parallel of `groupby('col')` for index levels. Same lazy GroupBy object out, same aggregation methods on it. 2. Show both addressing styles. `level=0` is by position (outermost). `level='date'` is by name (requires the level to be named). For MultiIndex frames where you want to group by the second level, both `level=1` and `level='product'` work. 3. Show the multi-level form. `level=[0, 1]` or `level=['date', 'product']` groups by multiple index levels at once, the same way passing a list of column names does for columns. 4. Mention that you can combine column and index grouping. `df.groupby([df.index.get_level_values('date'), 'category'])` works but is clunky. The cleaner pandas 2.x form is to keep index-level groupings via `level=` and column groupings via column names in separate calls if mixing is needed.

Edge cases to mention: - If your index does not have a name, `level=0` (positional) is the only addressing form. - `pd.Grouper(level='date', freq='M')` is the right tool when you want to group by a time-frequency derived from a datetime index — monthly, weekly, daily buckets — without first creating a month column. - The `Grouper` form composes with column groupings: `df.groupby([pd.Grouper(level='date', freq='M'), 'category'])` gives monthly per-category aggregates from a DatetimeIndex frame.

What NOT to say: - "You have to reset_index first" — you do not; `level=` is the direct path. - "level= is only for MultiIndex" — it works on single-level indexes too, with `level=0`.

Common follow-up: "How would you group a DatetimeIndex frame by month?" — `df.groupby(pd.Grouper(level='date', freq='M'))` or, if the index is unnamed, `df.groupby(pd.Grouper(freq='M'))`. The `Grouper` resamples by frequency without first materialising a month column. For more on time-frequency aggregation see Chapter 7 in this Data Track on datetime handling.

```
import pandas as pd

idx = pd.MultiIndex.from_tuples(
    [('2026-01', 'A'), ('2026-01', 'B'), ('2026-02', 'A'), ('2026-02', 'B')],
    names=['month', 'category'],
)
df = pd.DataFrame({'revenue': [100, 200, 150, 250]}, index=idx)

# Group by index level
df.groupby(level='month')['revenue'].sum()
# month
# 2026-01    300
# 2026-02    400

# Group by both levels – equivalent to no groupby at all here, but illustrative
df.groupby(level=['month', 'category'])['revenue'].sum()

# Time-frequency grouping with Grouper on a DatetimeIndex
dts = pd.date_range('2026-01-01', periods=90, freq='D')
ts = pd.DataFrame({'revenue': range(90)}, index=dts)
ts.groupby(pd.Grouper(freq='ME'))['revenue'].sum() # month-end buckets
```

End of Chapter 2 — 30 questions on pandas groupby and aggregation patterns.

Next chapter: Chapter 3 — merge / join / concat (30 Qs). The five join types, `on=` versus `left_on/right_on`, `validate=` for catching duplicate-key bugs, suffix handling, and the "this merge produced ten times more rows than the input" scenario that is the most-asked merge gotcha at Razorpay, Swiggy, and Flipkart.

Chapter 3 — Merge, Join, Concat

Data Track Library · 30 Questions · Python Interview Prep 2026 Edition · Learn & Excel

If `groupby` is the most-asked pandas verb at Indian data interviews, `merge` is the second — and it is the one that produces the most interview-round disasters. A typical Swiggy analytics-engineer round will hand you a `users` DataFrame and an `orders` DataFrame and ask you to "join them and tell me the average order value per user." If you reach for `pd.merge(users, orders)` without thinking about join type, key cardinality, NaN behaviour, and validation, you are one stray `m:m` away from a row count that quietly explodes from 50k to 5 million. The interviewer will not stop you. They will ask "are you sure that's right?" and watch you pick the answer out of the wreckage.

This chapter has thirty questions across four sections. Section A is `merge` — the five join types, the keyword arguments that matter (`on` , `left_on` / `right_on` , `suffixes` , `indicator` , `validate`), and the difference between "I want SQL `INNER JOIN` " and "I want every row from the

left." Section B is `join` and the index — when `df.join` is a clean win over `merge`, when it isn't, and how `MultiIndex` joins actually work. Section C is `concat` — vertical vs horizontal stacking, the `keys=` trick for `MultiIndex` tagging, why `concat` -in-a-loop is the canonical pandas performance bug, and what `join='inner'` vs `'outer'` does to alignment. Section D is the gotchas every senior interview ends on — row explosion, float-key merges, silent inner-join drops, NaN treatment in keys, and categorical alignment.

Each question follows the same Step-by-Step Answer format: tags, the question, the 30-second answer, step-by-step thought process, edge cases, what NOT to say, common follow-up, and a runnable code block. All examples are pandas 2.x and assume `import pandas as pd, numpy as np` at the top of the file.

Table of Sections

- Section A — `merge` basics: join types, keys, suffixes, indicator, validate (Q1 to Q10)
- Section B — `join` and the index: shortcut semantics, `MultiIndex`, `set_index` strategy (Q11 to Q15)
- Section C — `concat`: axis, keys, ignore_index, join, alignment, perf (Q16 to Q25)
- Section D — Common gotchas: m:m explosions, float keys, silent drops, NaN, categoricals (Q26 to Q30)

Section A — `merge` basics

`pd.merge` is the workhorse. It is the function you reach for when two DataFrames live side by side and one of them needs columns from the other. The shape of `merge` is `pd.merge(left, right, how=, on=, left_on=, right_on=, suffixes=, indicator=, validate=)` and every keyword in that signature is the answer to at least one interview question. Section A walks all of them.

Q1 · The five join types in `pd.merge`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Swiggy, Razorpay, Walmart Labs · Topic: Merge · Join Types

The question (interviewer's verbatim phrasing):

Walk me through every join type pandas supports in `pd.merge`. For each one, tell me what happens to non-matching rows on the left and on the right.

The 30-second answer: Five join types — `inner`, `left`, `right`, `outer`, `cross`. `inner` keeps only the rows whose key appears in both frames. `left` keeps every row from the left frame and fills NaN where the right has no match. `right` is the mirror — every row from the right, NaN on the left. `outer` is a full union — keeps everything from both sides and fills NaN where one side is missing. `cross` is the Cartesian product with no key at all — every row in left paired with every row in right. The default is `how='inner'`, which silently drops non-matching rows — this is the join-type question every interviewer wants to hear you call out.

Step-by-step thought process: 1. Name all five in one breath — `inner`, `left`, `right`, `outer`, `cross`. Interviewers tick them off as you go and the ones who get to all five before the prompt move into the second question. 2. Explain the row-survival rule for each. `inner` = intersection of keys. `left` / `right` = preserve one side. `outer` = union of keys. `cross` = every-with-every. 3. Call out the default. `how='inner'` is the dangerous default — if your left frame has 1,000 customers and only 600 have orders, `pd.merge(customers, orders)` silently returns 600 rows. Senior interviewers want to hear you say "I always pass `how=` explicitly." 4. Mention `cross` is the newest addition (pandas 1.2+, no key column needed) and is the right choice when you want every combination — typical for generating date × store grids before joining sparse sales data back in.

Edge cases to mention: - `outer` with no key match between any rows returns the union with NaN everywhere — the row count is `len(left) + len(right)`. - `cross` ignores `on=` entirely; passing `on=` with `how='cross'` raises `ValueError`. - For `left` and `right`, duplicate keys on the "other" side multiply rows — a left join of `users` (1,000 rows) to `addresses` (where some users have 3 addresses) returns more than 1,000 rows. - `inner` and `outer` produce the same result only when every key on one side matches every key on the other side.

What NOT to say: - "Pandas merge is the same as SQL JOIN." Mostly true but glosses over `cross` semantics and the fact that pandas does not enforce the key columns being non-null by default — NaN keys do not match. - "The default is left join." It is not — the default is `inner`, and this is the question Indian product-co interviewers use to filter the prepared from the assumed-prepared.

Common follow-up: "In a `left` merge, if a left row has no match on the right, what does pandas put in the right-side columns?" — `NaN` (or `pd.NA` if the right column is a nullable dtype). For object columns it's `NaN`, for `Int64` (capital I) it is `pd.NA`, and for `datetime64` it is `NaT`. Always check `.dtypes` after a left merge — silent dtype promotions are common.

```
import pandas as pd

users = pd.DataFrame({"user_id": [1, 2, 3], "name": ["aarav", "diya", "kabir"]})
orders = pd.DataFrame({"user_id": [1, 1, 2, 4], "amount": [499, 199, 899, 299]})

pd.merge(users, orders, how="inner", on="user_id")
# user 3 dropped (no orders), user 4 dropped (no user record) – 3 rows

pd.merge(users, orders, how="left", on="user_id")
# all 3 users kept, user 3 gets NaN amount – 4 rows (user 1 has 2 orders)

pd.merge(users, orders, how="right", on="user_id")
# all 4 orders kept, user 4 gets NaN name – 4 rows

pd.merge(users, orders, how="outer", on="user_id")
# union of user_ids: 1, 2, 3, 4 – 5 rows

pd.merge(users, orders, how="cross") # no on=
# every user x every order – 12 rows
```

Q2 · `on=` vs `left_on=` and `right_on=` — when each one is the right call

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Flipkart, Zoho · Topic: Merge · Keys

The question (interviewer's verbatim phrasing):

When you call `pd.merge(left, right)`, when do you use `on=` and when do you use `left_on=` / `right_on=`? What happens if you forget both?

The 30-second answer: Use `on=` when the key column has the exact same name in both frames — `pd.merge(orders, customers, on="customer_id")`. Use `left_on=` and `right_on=` when the names differ — `pd.merge(orders, customers, left_on="cust_id", right_on="customer_id")`. If you forget both, pandas falls back to "merge on every column that exists in both frames" — which is almost always wrong and the source of countless data-quality bugs.

Step-by-step thought process: 1. State the rule in one line — `on=` for matching names, `left_on=` / `right_on=` for mismatched names. 2. Explain the default behaviour and why it is dangerous. When you call `pd.merge(left, right)` with neither, pandas finds the intersection of column names and uses all of them as join keys. If both frames have `created_at` and `customer_id`, you accidentally join on both — which can silently lose rows. 3. Mention the column-survival pattern. With `on=`, the join column appears once in the result. With `left_on=` / `right_on=`, both columns survive — you usually want to `.drop` one of them after. 4. Cover the list

form — `on=["customer_id", "store_id"]` for compound keys. This is the form you want for any real-world join because single-column joins on a non-unique id are rarely what business analysts mean.

Edge cases to mention: - `left_on=` and `right_on=` can take lists of equal length for compound keys with different names on each side. - Passing `on=` together with `left_on=` / `right_on=` raises `MergeError`. - If `right_on=` includes the index, you typically want `right_index=True` instead — see Q11. - When both `left_on` and `right_on` produce the same column name, pandas keeps both with the standard suffix rules from Q3.

What NOT to say: - "Pandas figures out the join automatically." It does, but the automatic behaviour is "join on every shared column name," which is rarely what you want. Always pass `on=` explicitly. - "left_on= and right_on= are slower." They are not — same operation, just different names.

Common follow-up: "You have a left frame with `cust_id` and a right frame with `customer_id`. After merging with `left_on='cust_id'`, `right_on='customer_id'`, you see both columns in the output. Why?" — Because the keys had different names, pandas keeps both. Drop one with `.drop(columns="customer_id")` or rename one of them before the merge.

```
orders = pd.DataFrame({"cust_id": [1, 2, 3], "amount": [499, 199, 899]})
customers = pd.DataFrame({"customer_id": [1, 2, 4], "city": ["BLR", "BOM", "DEL"]})

# different names - both columns survive
merged = pd.merge(orders, customers, left_on="cust_id", right_on="customer_id", how="left")
print(merged.columns.tolist())
# ['cust_id', 'amount', 'customer_id', 'city']

merged = merged.drop(columns="customer_id") # clean up

# compound key, same names on both sides
sales = pd.DataFrame({"store_id": [1, 1, 2], "date": ["2026-05-01"]*3, "amt": [100, 200, 300]})
targets = pd.DataFrame({"store_id": [1, 2], "date": ["2026-05-01"]*2, "tgt": [500, 400]})
pd.merge(sales, targets, on=["store_id", "date"])
```

Q3 · **suffixes=** — what happens when both frames have a column with the same name

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Goldman · Topic: Merge · Column Collisions

The question (interviewer's verbatim phrasing):

Both your left and right DataFrames have a column called `amount` that is not the join key. What does pandas do, and how do you control the output column names?

The 30-second answer: Pandas appends `_x` to the left column and `_y` to the right column by default. You override with `suffixes=("_left", "_right")` or any other pair. If you want the left column to keep its original name and the right one to get a suffix, pass `suffixes=("", "_right")` — but an empty suffix on both raises `MergeError` because the columns would collide.

Step-by-step thought process: 1. State the default — `_x` and `_y`. Most interviewers want to hear you call out the default because it is the silent giveaway in production code that someone copy-pasted a merge without thinking. 2. Show the override syntax — `suffixes=("_orders", "_payments")` produces `amount_orders` and `amount_payments`. The pair must be a 2-tuple, never a single string. 3. Explain the empty-suffix trick. `suffixes=("", "_old")` keeps the left column's original name and suffixes the right. Useful when you are enriching the left frame with reference data and want the original schema preserved. 4. Cover the failure mode. If both suffixes are empty and there is a collision, pandas raises `MergeError: Passing 'suffixes' which cause duplicate columns`. Either give one of them a non-empty suffix or drop one of the colliding columns before merging.

Edge cases to mention: - Suffixes only apply to columns that collide. Unique columns on each side are unchanged. - Join keys (the columns in `on=`) do not get suffixes — they appear once. - If you pass `left_on=` / `right_on=` with different names, neither key column gets a suffix; they survive as-is. - Index columns with the same name on both sides do get suffixed when you `reset_index` after a merge.

What NOT to say: - "Pandas just keeps both columns." Not quite — it suffixes them so you can tell them apart, and the default suffix `_x` is so generic it usually needs replacing. - "You can drop one before merging." You can, but if the values differ between frames, you have just thrown away information. Suffix them and inspect.

Common follow-up: "After a merge, both `amount_x` and `amount_y` are present. The product team wants `amount_orders` and `amount_payments`. How do you control it from the merge call?" — Pass `suffixes=("_orders", "_payments")` directly in the merge call. Cleaner than renaming after.

```
orders = pd.DataFrame({"order_id": [1, 2], "amount": [499, 199], "currency": ["INR", "INR"]})
payments = pd.DataFrame({"order_id": [1, 2], "amount": [499, 150], "method": ["upi", "card"]})

# default suffixes
merged = pd.merge(orders, payments, on="order_id")
print(merged.columns.tolist())
# ['order_id', 'amount_x', 'currency', 'amount_y', 'method']

# custom suffixes
merged = pd.merge(orders, payments, on="order_id", suffixes=("_ordered", "_paid"))
print(merged.columns.tolist())
# ['order_id', 'amount_ordered', 'currency', 'amount_paid', 'method']

# keep left as-is, suffix right
merged = pd.merge(orders, payments, on="order_id", suffixes=("", "_paid"))
# 'amount' (from orders) and 'amount_paid' (from payments)
```


Q4 · `indicator=True` — telling which frame a row came from

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, JP Morgan · Topic: Merge · Provenance

The question (interviewer's verbatim phrasing):

You did an outer merge and you want to know, for every output row, whether it came from the left frame, the right frame, or both. What's the cleanest pandas way to get that?

The 30-second answer: Pass `indicator=True` to `pd.merge`. It adds a `_merge` column to the output with three categorical values: `left_only`, `right_only`, and `both`. You then filter — `merged[merged["_merge"] == "left_only"]` gives you the rows that exist in the left frame but had no match on the right. The column is a pandas `Categorical` with the three values fixed, so it sorts predictably and uses minimal memory.

Step-by-step thought process: 1. State the keyword — `indicator=True`. The interviewer expects you to know it exists; most candidates reach for `isin` or anti-join hacks instead. 2. Explain the three values — `left_only`, `right_only`, `both`. These are the three possible origin states for any row in an outer-style merge. 3. Cover the common use case — finding orphans. After an outer merge of `users` and `orders`, `_merge == "left_only"` is "users with no orders" and `_merge == "right_only"` is "orders with no matching user." This is the standard data-quality query at fintech and ecommerce shops. 4. Mention the string-override form. `indicator="origin"` names the column `origin` instead of `_merge` — useful when `_merge` would conflict with an existing column or read awkwardly in a downstream report.

Edge cases to mention: - `indicator=True` works with any `how=`, not just `outer`. With `how='inner'` every row is `both`, which is useless but not an error. - The `_merge` column is `Categorical` — to sort or group, treat it like one. Comparing with `==` to a string works because of category equality semantics. - After `indicator=True`, if you `.drop(columns="_merge")` you lose the information; do your filtering first. - The indicator column has no effect on row count — it only labels the rows that an outer merge would have produced anyway.

What NOT to say: - "I'd compare keys with `isin`." That works, but it's two passes through the data and you have to do it on both sides separately. `indicator=True` is one merge, one column. - "I'd use `df.merge(...).query('amount.notna()')`." Fragile — it conflates "this column was NaN before" with "this row had no match."

Common follow-up: "How would you get all users who never placed an order using `indicator=True`?" — `pd.merge(users, orders, on="user_id", how="left", indicator=True)` then filter `_merge == "left_only"`. Cleaner than `users[~users["user_id"].isin(orders["user_id"])]` because it survives nulls correctly.


```

users = pd.DataFrame({"user_id": [1, 2, 3], "name": ["aarav", "diya", "kabir"]})
orders = pd.DataFrame({"user_id": [1, 1, 4], "amount": [499, 199, 899]})

merged = pd.merge(users, orders, on="user_id", how="outer", indicator=True)
print(merged)
#   user_id  name  amount  _merge
# 0        1  aarav   499.0    both
# 1        1  aarav   199.0    both
# 2        2   diya     NaN  left_only
# 3        3  kabir     NaN  left_only
# 4        4   NaN    899.0  right_only

# users with no orders
no_orders = merged[merged["_merge"] == "left_only"]["user_id"]

# custom column name
merged = pd.merge(users, orders, on="user_id", how="outer", indicator="origin")

```

Q5 · **validate=** — catching m:m row explosions at merge time

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman, Walmart Labs, Goldman ·

Topic: Merge · Validation

The question (interviewer's verbatim phrasing):

You're merging two DataFrames and you expect every customer to appear exactly once on the left and exactly once on the right. How do you make pandas verify that for you instead of silently accepting duplicates?

The 30-second answer: Pass `validate="1:1"`. The four legal values are `1:1`, `1:m`, `m:1`, and `m:m`. Pandas checks the assumption before the merge and raises `MergeError` if it doesn't hold. Available since pandas 0.21 and still under-used — every senior data engineer at a fintech should be passing `validate=` on every merge in production code because an unexpected `m:m` join is the single most common cause of row-count blowups.

Step-by-step thought process: 1. Name the keyword and the four values. `1:1` = unique on both sides; `1:m` = unique on left, possibly duplicates on right; `m:1` = duplicates on left, unique on right; `m:m` = duplicates on both sides (explicit acknowledgement, no check). 2. Explain why you want it. Without `validate=`, an `m:m` join with 100 dups on each side gives you $100 \times 100 = 10,000$ rows where you expected 100. The bug is silent and the row count looks "kind of right" until someone checks. 3. Show the failure mode. If you pass `validate="1:1"` and the left has duplicates, pandas raises `MergeError: Merge keys are not unique in left dataset; not a one-to-one merge`. You then go find the dups. 4. Cover the practical pattern. In a Jupyter notebook, you guess

`1:1`, hit `MergeError`, then either dedupe or relax to `1:m` after understanding why duplicates exist. Treat `validate=` as a contract you write before the merge, not after.

Edge cases to mention: - `validate=` is checked before the merge runs, so it adds a small overhead — typically negligible compared to the merge itself. - `m:m` is the default (no check) and exists mostly for backward compatibility. It does not actually validate anything; it just acknowledges that you know the merge can blow up. - If your "unique" key is unique only after a filter (e.g., active users), apply the filter to the input frame first, then merge with `validate="1:1"`. - `validate=` checks the key columns, not the rest of the frame. If a frame has the same key twice with different other-column values, it still trips the check.

What NOT to say: - "I'd check uniqueness with `nunique` first." You can, but it's two passes — `validate=` does it in one and raises with a clear message. - "validate is just for debugging." It is production-grade. Senior data engineers leave it in shipped code.

Common follow-up: "Your merge of `customers` to `orders` is supposed to be 1:m and pandas raises `MergeError`. What does that tell you about the customers frame?" — Customers has duplicates on the join key. Either run `customers["customer_id"].duplicated()` to find them, or `customers.drop_duplicates(subset="customer_id", keep="first")` before the merge if dedup is intended.

```
customers = pd.DataFrame({"customer_id": [1, 2, 3], "city": ["BLR", "BOM", "DEL"]})
orders = pd.DataFrame({"customer_id": [1, 1, 2, 2, 3], "amount": [499, 199, 899, 299, 150]})

# correct: 1:m – one customer, many orders
pd.merge(customers, orders, on="customer_id", validate="1:m") # passes

# wrong assumption: 1:1 raises
try:
    pd.merge(customers, orders, on="customer_id", validate="1:1")
except pd.errors.MergeError as e:
    print("caught:", e)
# Merge keys are not unique in right dataset; not a one-to-one merge

# accidental m:m – duplicates on left too
customers_bad = pd.DataFrame({"customer_id": [1, 1, 2], "city": ["BLR", "BLR-old", "BOM"]})
# pd.merge(customers_bad, orders, on="customer_id") -> 5 rows silently
# pd.merge(customers_bad, orders, on="customer_id", validate="1:m") -> MergeError
```

Q6 · `how='cross'` — when you want every combination

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Walmart Labs, Target · Topic: Merge · Cartesian Product

The question (interviewer's verbatim phrasing):

You have 30 stores and 31 days in May. You want a frame with one row per (store, day) pair, regardless of whether that store had a sale that day. How do you build it with `pd.merge` ?

The 30-second answer: Use `pd.merge(stores, dates, how="cross")` — the cross-join was added in pandas 1.2 and produces the full Cartesian product, no key column needed. Result: $30 \times 31 = 930$ rows. You then left-merge your sales data onto that grid and fill `NaN` with zero for the no-sale (store, day) combinations. This is the canonical "complete the calendar" pattern at any retail or marketplace company.

Step-by-step thought process: 1. Name the operation — cross join, full Cartesian product. Some interviewers will check that you know the historical workaround (assign a dummy `_key=1` column to each side and merge on it) — pandas 1.2 made that obsolete. 2. Explain the use case. Sparse sales data has only the (store, day) pairs with at least one sale. Reports demand zeros for the no-sale days. You build the full grid with `cross`, left-merge the sales, fill `NaN`. 3. Watch the row count. Cross joins explode by definition — $1,000 \text{ stores} \times 365 \text{ days} = 365,000 \text{ rows}$. Always do the cross between small frames (dimension tables), never between fact tables. 4. Note the keyword restrictions. `how='cross'` is incompatible with `on=`, `left_on=`, `right_on=`, `left_index=`, `right_index=`. The merge has no key to match on; if you pass one, pandas raises `ValueError`.

Edge cases to mention: - Before pandas 1.2 (Dec 2020), you faked a cross join by adding a constant column `df1["_k"] = 1`, doing `pd.merge(df1, df2, on="_k")`, then dropping `_k`. You'll still see this in older Indian codebases — it's the same result, more verbose. - The output index is reset to a new `RangeIndex` regardless of the input indices. - A cross join of an empty frame with any other frame returns an empty frame. - Cross joins respect column ordering — left columns come first, right columns follow.

What NOT to say: - "I'll loop with `itertools.product`." Works but is slow and loses the `DataFrame` type. Stay in pandas. - "Cross joins are dangerous." Only if the sides are large. For a (stores $\times$ dates) dimension grid they are exactly the right tool.

Common follow-up: "After you build the (store, day) grid and left-merge sales, the `amount` column has `NaN` for no-sale days. What is the standard cleanup?" — `df["amount"] = df["amount"].fillna(0)`. For 2.x with nullable `Int64` you'd use `df["amount"] = df["amount"].fillna(0).astype("Int64")` if you want the integer dtype back.

```
stores = pd.DataFrame({"store_id": [101, 102, 103]})
dates = pd.DataFrame({"date": pd.date_range("2026-05-01", "2026-05-03")})

grid = pd.merge(stores, dates, how="cross")
print(len(grid)) # 9 rows: 3 stores x 3 days

sales = pd.DataFrame({
    "store_id": [101, 101, 103],
    "date": pd.to_datetime(["2026-05-01", "2026-05-02", "2026-05-01"]),
    "amount": [499, 199, 299],
})

full = pd.merge(grid, sales, on=["store_id", "date"], how="left")
full["amount"] = full["amount"].fillna(0)
# 9 rows, 6 of them with amount = 0
```

Q7 · `how='left'` is the safest default — when (and why) to prefer it

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Zoho, Postman · Topic: Merge · Defaults

The question (interviewer's verbatim phrasing):

You're enriching a left DataFrame with lookup columns from a right DataFrame. The interviewer asks you to write the merge call. Which `how=` do you reach for and why?

The 30-second answer: `how='left'`. The intent of "enrichment" is "do not lose any rows from the left frame, just attach columns from the right where matches exist." `inner` would silently drop unmatched left rows. `outer` would add right-only rows you don't want. `left` is the only join type that preserves the left frame's row count and order (assuming the right keys are unique). It is the safest default for any enrichment pipeline and the go-to choice when the left frame is your source of truth.

Step-by-step thought process: 1. Restate the intent — enrichment means "keep left, add columns." That single framing tells you `left` is the right join. 2. Compare against the other choices. `inner` loses rows. `outer` adds rows. `cross` ignores keys. None of those match "enrichment." 3. Add the row-count guarantee. `pd.merge(left, right, how="left", validate="m:1")` is the gold-standard enrichment call — same row count as `left`, columns added from `right`. Pair it with `validate=` and you have a self-documenting merge. 4. Mention the order. A left merge preserves the row order of the left frame in pandas 2.x when `sort=False` (the default). Earlier pandas versions sometimes reordered; 2.x is stable.

Edge cases to mention: - If the right frame has duplicate keys, a left merge will multiply left rows. Use `validate="m:1"` to catch this — that's the contract "right is unique." - Left rows with NaN in the join key never match (NaN != NaN), so they survive as left rows with NaN in the right columns. - A left merge

with `right` having extra columns adds those columns; columns present in both get suffixed per Q3. - `pd.merge` with default `sort=False` is faster than `sort=True` on large frames; the default in 2.x is `False`.

What NOT to say: - "Inner is safer because it's strict." Inner is silently destructive — it removes rows without telling you. Strict ≠ safe. - "I always do outer to be safe." Outer is data-additive — it brings in right-only rows you didn't ask for and can pollute downstream reports.

Common follow-up: "You did a left merge expecting same row count and got 1.5x. What happened?" — The right frame had duplicate keys, so each left row matched multiple right rows. Either `right.drop_duplicates(subset="join_key")` first, or pass `validate="m:1"` to fail fast.

```
orders = pd.DataFrame({"order_id": [1, 2, 3], "user_id": [10, 20, 30], "amount": [499, 199, 899]})
users = pd.DataFrame({"user_id": [10, 20, 40], "city": ["BLR", "BOM", "DEL"]})

# enrich orders with user city
enriched = pd.merge(orders, users, on="user_id", how="left", validate="m:1")
print(enriched)
#   order_id  user_id  amount  city
# 0         1       10     499   BLR
# 1         2       20     199   BOM
# 2         3       30     899   NaN    <- user 30 has no row in users
# row count preserved: 3
```

Q8 · Merging on multiple columns (composite keys)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, JP Morgan, Walmart Labs · Topic: Merge · Composite Keys

The question (interviewer's verbatim phrasing):

Your fact table has `(store_id, date)` as the natural key. Your targets table has the same. Write the merge to join targets onto sales for end-of-month reporting.

The 30-second answer: Pass a list to `on=` — `pd.merge(sales, targets, on=["store_id", "date"], how="left", validate="m:1")`. Pandas treats the list as a composite key and matches rows only where both columns agree. Order in the list does not affect the result, but it affects the order of the join columns in the output. The composite-key form is what you reach for on any time-series-by-dimension join; single-column merges on a non-unique id are almost never what business reports actually want.

Step-by-step thought process: 1. State the syntax — `on=["store_id", "date"]`. Composite keys are the rule, not the exception, in real-world data work. 2. Explain the matching rule. A row in

`left` matches a row in `right` only if every column in `on=` agrees. Any disagreement on any column = no match. 3. Cover the dtype trap. If `date` is `datetime64[ns]` on one side and `object` (string) on the other, no rows match and pandas silently returns an empty (or all-NaN) result. Always check `.dtypes` on both join keys before merging. 4. Mention `left_on=` / `right_on=` lists. If column names differ — `left_on=["store_id", "txn_date"]`, `right_on=["store_id", "date"]` — pass two lists of equal length.

Edge cases to mention: - Composite-key merges are slower than single-key because pandas hashes the tuple. On 10M-row frames, sort-merge can be faster — see Q9. - `validate="1:1"` on a composite key checks that the **combination** is unique on each side, not each column independently. - NaN in any one of the composite key columns kills the match for that row (NaN != NaN). - For very wide composite keys (5+ columns), consider building a single hash column first; cleaner and faster.

What NOT to say: - "Just merge on the first column and drop dups." That's a different operation and silently loses data. - "Composite keys are slow so I avoid them." Slower than single-column, but correct beats fast every time — and "fast and wrong" gets you fired.

Common follow-up: "You merge on `["store_id", "date"]` and get fewer rows than expected. What's the most likely cause?" — dtype mismatch on `date` — string on one side, datetime on the other. Convert both with `pd.to_datetime` before the merge.

```
sales = pd.DataFrame({
    "store_id": [101, 101, 102],
    "date": pd.to_datetime(["2026-05-01", "2026-05-02", "2026-05-01"]),
    "amount": [499, 199, 899],
})
targets = pd.DataFrame({
    "store_id": [101, 102],
    "date": pd.to_datetime(["2026-05-01", "2026-05-01"]),
    "target": [1000, 500],
})

merged = pd.merge(sales, targets, on=["store_id", "date"], how="left", validate="m:1")
print(merged)
```

#	store_id	date	amount	target
# 0	101	2026-05-01	499	1000.0
# 1	101	2026-05-02	199	NaN
# 2	102	2026-05-01	899	500.0

Q9 · `merge` vs `merge_ordered` vs `merge_asof` — which one for time series

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Zerodha, PhonePe · Topic: Merge · Time-Series

The question (interviewer's verbatim phrasing):

You have a trades frame with millisecond timestamps and a quotes frame with millisecond timestamps. For each trade you want the most recent quote at or before the trade time. Which pandas function and why?

The 30-second answer: `pd.merge_asof`. It's an "as-of" join — for every row in the left frame, it finds the closest row in the right frame on a sorted key, with controls for direction (`backward` , `forward` , `nearest`) and tolerance. Plain `pd.merge` requires an exact key match; `merge_asof` is the right tool when "exact" is unrealistic, like joining trades to the most recent prior quote. `pd.merge_ordered` is for ordered joins with optional `fill_method` — used less often, mostly for aligning multi-source time series.

Step-by-step thought process: 1. Reject `pd.merge` for this case. Exact-key joins won't match because trade timestamps almost never equal quote timestamps to the millisecond. 2. Introduce `pd.merge_asof`. The classic financial use case — trade-to-quote alignment — is literally why pandas added it. Default direction is `backward`: for each left row, find the right row with the largest key $\leq$ left key. 3. Cover the keyword arguments. `direction='backward'` (default) for "most recent prior," `'forward'` for "next after," `'nearest'` for "closest in either direction." `tolerance=pd.Timedelta("1s")` limits how far back/forward to look. 4. Note the sort requirement. Both frames must be sorted on the merge key. `merge_asof` will not sort them for you; it raises `ValueError` if either side is unsorted. Add `df.sort_values()` before the merge.

Edge cases to mention: - `by=` lets you do an `as-of` join within groups — for each (trade, ticker), find the most recent (quote, ticker). Same idea, partitioned. - `tolerance` defaults to no limit. For high-frequency data, set a sensible upper bound to avoid joining a 9:15 AM trade to an 8:00 AM quote. - `merge_asof` works with numeric keys too — useful for joining sensor readings by sample index. - The right frame's key column does not need to be unique — `merge_asof` picks the closest match per the `direction=` rule.

What NOT to say: - "Just use `pd.merge` and filter." That's an $n \times m$ blowup before filtering — terrible on millions of trades. - "Time-series joins are a SQL window function." They are, but the question is about pandas; reach for the right pandas tool.

Common follow-up: "How do you join trades to the most recent quote within 100 ms, by ticker?" — `pd.merge_asof(trades, quotes, on='time', by='ticker', tolerance=pd.Timedelta('100ms'), direction='backward')`. Sort both frames on `time` first.

```
trades = pd.DataFrame({
    "time": pd.to_datetime(["09:15:00.500", "09:15:01.200", "09:15:01.800"]),
    "ticker": ["RELI", "RELI", "TCS"],
    "qty": [100, 200, 50],
}).sort_values(["ticker", "time"])

quotes = pd.DataFrame({
    "time": pd.to_datetime(["09:15:00.000", "09:15:01.000", "09:15:01.500"]),
    "ticker": ["RELI", "RELI", "TCS"],
    "bid": [2400.0, 2400.5, 3850.0],
}).sort_values(["ticker", "time"])

result = pd.merge_asof(trades, quotes, on="time", by="ticker", direction="backward")
# every trade gets the most recent prior quote for its ticker
```

Q10 · `sort=` in `pd.merge` — what it does and when it matters

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Cred, Goldman · Topic: Merge · Sorting Behavior

The question (interviewer's verbatim phrasing):

What does the `sort=` keyword in `pd.merge` do, and why is the default `False` in modern pandas?

The 30-second answer: `sort=False` (the default) preserves the row order of the left frame in `how='left'`, the right in `how='right'`, and an internal hash order in `how='inner'` and `how='outer'`. `sort=True` sorts the output by the join key. The default flipped to `False` because sorting after a merge is wasted work for the common enrichment pattern where the caller already wants left-frame order, and sorting a large frame is expensive.

Step-by-step thought process: 1. State the default — `sort=False` since pandas 0.23. Older docs and tutorials still show `sort=True`; ignore them. 2. Explain when `sort=True` matters. If you need the result in key order for a downstream merge or for a join-then-window operation, sorting upfront avoids a separate `.sort_values()` call. But sorting is $O(n \log n)$; doing it as part of the merge is usually no cheaper than after. 3. Cover the order-preservation guarantee. `how='left'` with `sort=False` preserves left-frame order in pandas 2.x. This is the contract that makes left joins behave like SQL `LEFT JOIN ... ORDER BY <left order>` without an explicit sort. 4. Mention the small performance impact. On a 10M-row merge, `sort=True` can add 1-3 seconds. Not catastrophic, but unnecessary if you don't need the sort.

Edge cases to mention: - For `how='outer'`, `sort=False` produces an order that depends on the underlying hash — predictable per pandas version but not stable across versions. Sort explicitly if you

ship that data. - `sort=True` sorts by the join key columns in the order they appear in `on=`. - `merge_asof` requires sorted input and produces sorted output; `sort=` is not a parameter there. - If you set the join key as the index and use `df.join`, the result is automatically sorted by index — equivalent to `sort=True` on a merge.

What NOT to say: - "sort=True is faster." It is not — it adds work. - "The default has always been False." It defaulted to `True` in older pandas and flipped in 0.23 (2018). Newer code assumes False; older code may not.

Common follow-up: "You did a left merge and the row order changed. What went wrong?" — Either you passed `sort=True`, or you used `df.join` (which sorts by index), or your left frame's index was not what you thought. With `sort=False` and `how='left'`, left-frame order is preserved.

```
left = pd.DataFrame({"id": [3, 1, 2], "x": ["c", "a", "b"]})
right = pd.DataFrame({"id": [1, 2, 3], "y": [10, 20, 30]})

# default: sort=False – preserves left order
pd.merge(left, right, on="id", how="left")
#    id  x   y
# 0   3  c  30
# 1   1  a  10
# 2   2  b  20

# sort=True: by join key
pd.merge(left, right, on="id", how="left", sort=True)
#    id  x   y
# 0   1  a  10
# 1   2  b  20
# 2   3  c  30
```

Section B — `join` and the index

`df.join` is `pd.merge`'s index-aware shortcut. Whenever the join key is already the index of at least one of the frames, `df.join` saves you the `right_index=True` boilerplate and the awkward column-name juggling that comes with index-as-key merges. This section covers when `join` is the cleaner verb and when you should drop back to `merge`.

Q11 · `df.join` — what it is and how it relates to `merge`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Zoho · Topic: Join · Index Semantics

The question (interviewer's verbatim phrasing):

What does `df.join(other)` do and how is it different from `pd.merge(df, other, ...)` ?

The 30-second answer: `df.join(other)` is a shortcut for joining on the indexes of both frames. By default it does a left join on the left frame's index and the right frame's index. It's equivalent to `pd.merge(df, other, left_index=True, right_index=True, how="left")` — just with cleaner syntax. The `how=` default differs: `merge` defaults to `inner`, `join` defaults to `left`. Use `join` when both frames already have meaningful indexes; use `merge` for any column-based join.

Step-by-step thought process: 1. State the mechanic — `join` is `merge` with `left_index=True, right_index=True` defaults. That one fact answers most follow-ups. 2. Call out the default `how=` difference. `merge` defaults to `inner`; `join` defaults to `left`. The `join` default fits the most common "enrich the left frame with reference data" use case, which is why pandas chose it. 3. Show the `on=` extension. `df.join(other, on="customer_id")` joins the left frame's `customer_id` column to the right frame's index. Useful when the right frame is a lookup table whose index is the natural key. 4. Mention the multi-frame form. `df.join([df2, df3])` joins multiple frames at once on the index — clean for assembling a wide frame from several keyed dimension tables.

Edge cases to mention: - `df.join(other)` raises `ValueError` if column names collide and you don't pass `lsuffix=` or `rsuffix=`. Unlike `merge`'s suffixes, `join` requires you to pick a side. - The multi-frame form requires unique column names across all frames; collisions raise. - `df.join(other, how="inner")` is valid and useful when you want intersection of indexes. - Joining when one side has a `MultiIndex` requires the other side's index to align by name — see Q14.

What NOT to say: - "join and merge are the same thing." They aren't — they have different defaults and different ergonomics for index-keyed joins. - "join is faster than merge." Not on its own. Same operation under the hood; `join` is just sugar.

Common follow-up: "Both frames have a column named `value`. You call `df.join(other)`. What happens?" — `ValueError: columns overlap but no suffix specified: Index(['value'], dtype='object')`. Pass `df.join(other, lsuffix="_left", rsuffix="_right")` or rename one column first.

```
orders = pd.DataFrame({"amount": [499, 199, 899]}, index=[101, 102, 103])
orders.index.name = "order_id"

shipping = pd.DataFrame({"city": ["BLR", "BOM"]}, index=[101, 103])
shipping.index.name = "order_id"

# clean join on shared index name
orders.join(shipping, how="left")
#           amount  city
# order_id
# 101           499   BLR
# 102           199   NaN
# 103           899   BOM

# equivalent merge form
pd.merge(orders, shipping, left_index=True, right_index=True, how="left")
```

Q12 · When to `set_index` before joining

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman · Topic: Join · Index Strategy

The question (interviewer's verbatim phrasing):

When does it make sense to `set_index` a column on both DataFrames before joining instead of just merging on the column directly?

The 30-second answer: Three cases. First, when you'll join the same right frame to multiple left frames repeatedly — `set_index` once on the lookup, then `df.join(lookup)` for each call beats rehashing on every merge. Second, when the join key is the natural identity of the rows and you want index-based slicing (`df.loc[customer_id]`) downstream. Third, when chaining several index-based joins — `set_index` once at the start, then `.join(a).join(b).join(c)` is far cleaner than the equivalent merge chain. Otherwise, plain `merge` is fine.

Step-by-step thought process: 1. List the three triggers — repeated lookups, identity-as-index, and join chains. Most pandas code does not need indexed joins; reach for them only when one of these triggers fires. 2. Explain the perf intuition. `set_index` builds a hash index once. Subsequent `df.join` calls reuse that hash. A `merge` on a column rebuilds the hash every call. On large lookup tables joined many times (e.g., a SKU master joined to ten daily fact tables), the difference is real. 3. Cover the `df.loc[key]` benefit. Once `customer_id` is the index, `df.loc[42]` returns the row for customer 42 in O(1). With a column-keyed frame you'd need `df[df["customer_id"] == 42]`, which is a full-column scan. 4. Mention the trade-off. Setting the index hides the column from `df.columns` (it lives in `df.index` instead). Downstream code that expects to see `customer_id` as a regular column will break — `df.reset_index()` to restore it.

Edge cases to mention: - `set_index` with `drop=False` keeps the column as both index and regular column — useful when you want both index access and a normal column for filters. - Setting a non-unique column as the index is legal but breaks `df.loc[key]` (returns multiple rows) and slows index-based merges. - `inplace=True` is discouraged in pandas 2.x; the idiom is `df = df.set_index("customer_id")`. - A frame with a `MultiIndex` can still be the right side of a `df.join` if the other side's `on=` names match the index levels — see Q14.

What NOT to say: - "Always set the index for joins." Adds complexity for no win on one-off merges. Set when there's a reason. - "set_index is faster than merge." Only amortised across many joins; one-shot `set_index + join` is the same speed as `merge`.

Common follow-up: "You set `customer_id` as the index, then call `df["customer_id"]`. What happens?" — `KeyError`. The column is gone from `.columns` — it's in `.index` now. Either `df.index` for the values or `df.reset_index()["customer_id"]` to restore as a column.

```
# repeated lookup pattern
products = pd.DataFrame({
    "sku": ["A1", "A2", "A3"],
    "category": ["electronics", "apparel", "books"],
    "gst": [18, 12, 5],
}).set_index("sku")

# fact tables for different days
day1 = pd.DataFrame({"sku": ["A1", "A2"], "qty": [10, 5]}).set_index("sku")
day2 = pd.DataFrame({"sku": ["A1", "A3"], "qty": [7, 3]}).set_index("sku")

# clean join chain
day1.join(products).head() # adds category and gst
day2.join(products).head() # same products, no re-hash
```

Q13 · `df.join` with `on=` — joining a column to the right frame's index

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Zoho · Topic: Join · Hybrid

The question (interviewer's verbatim phrasing):

Your left frame has `customer_id` as a regular column. Your right frame has `customer_id` as the index. Show me the cleanest pandas call to join them.

The 30-second answer: `left.join(right, on="customer_id", how="left")`. The `on=` keyword tells pandas to join the left frame's `customer_id` column to the right frame's index. This is the cleanest form when you have a typical "fact + dimension" pair — the fact table has the key as a column, the dimension table has it as an index for fast lookup.

Step-by-step thought process: 1. Identify the asymmetry. Left has the key as a column; right has it as the index. The `on=` argument to `join` exists for exactly this case. 2. Show the call.

`left.join(right, on="customer_id")`. Note that `on=` here means "the left frame's column" — different from `pd.merge`'s `on=` which means "shared column name on both sides." Slight terminology overlap; the docs are explicit. 3. Compare to `pd.merge`. The equivalent is `pd.merge(left, right, left_on="customer_id", right_index=True, how="left")`. Same result, more verbose. `df.join(on=)` is the shorthand for exactly this pattern. 4. Mention the default `how='left'`. If you want `inner`, pass it explicitly. Left is the default because enriching a fact table with dimension columns is the dominant use case.

Edge cases to mention: - If the left frame's `on=` column has NaN, those rows survive but get NaN in the right columns (NaN cannot match any index value). - The right frame's index must be unique for a clean `how='left'`. Duplicates multiply rows just like in a regular merge. - Multiple `on=` columns work for joining to a `MultiIndex` — see Q14. - The result preserves the left frame's index, which is what you usually want.

What NOT to say: - "I'd `reset_index` on the right first and then merge." Works, but loses the perf benefit of having an indexed lookup table. - "on= in join is the same as on= in merge." Subtle difference — `merge`'s `on=` requires the column on both sides; `join`'s `on=` is "left column to right index."

Common follow-up: "You want to enrich `orders` with `users.city` where `users` is indexed by `user_id`. Write it." — `orders.join(users[["city"]], on="user_id", how="left")`. Selecting `users[["city"]]` keeps only the column you want; saves cleanup after.

```
orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4],
    "customer_id": [10, 20, 10, 99],
    "amount": [499, 199, 899, 299],
})

customers = pd.DataFrame(
    {"city": ["BLR", "BOM", "DEL"], "tier": ["gold", "silver", "bronze"]},
    index=[10, 20, 30],
)
customers.index.name = "customer_id"

# left frame column joined to right frame index
enriched = orders.join(customers, on="customer_id", how="left")
#   order_id  customer_id  amount  city  tier
# 0         1           10     499  BLR  gold
# 1         2           20     199  BOM  silver
# 2         3           10     899  BLR  gold
# 3         4           99     299  NaN   NaN
```

Q14 · Multi-level index joins

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Swiggy, Zerodha · Topic: Join · MultiIndex

The question (interviewer's verbatim phrasing):

Both your frames have a `MultiIndex` of `(store_id, date)`. How do you join them, and what happens when one frame has only one of the two levels?

The 30-second answer: When both indexes have matching level names, `left.join(right)` does the right thing — it aligns on shared level names. If only one level matches, pandas joins on that level alone and broadcasts the partial-index frame across the unmatched levels. The explicit form is `pd.merge(left, right, left_index=True, right_index=True)`. For partial joins, pass `level=` or use `join` with one side's `MultiIndex` and the other's regular index — the level-name match drives alignment.

Step-by-step thought process: 1. State the alignment rule — `MultiIndex` joins by level name. Both sides must have at least one level name in common for the join to find a match. 2. Show the full-match case. Two frames, both indexed `(store_id, date)`, same names — `left.join(right)` works. Cleaner than the merge equivalent. 3. Cover the partial match. Frame A is `(store_id, date)`, frame B is `(store_id,)`. `left.join(right)` joins on `store_id` and broadcasts the store-level data across all `(store_id, date)` combinations on the left. The store-level row "follows" every date for that store. 4. Mention the cross-type form. Joining a `MultiIndex` frame to a column-indexed frame via `on=` requires `on=["store_id", "date"]` matching the level names. Pandas will raise if names don't align.

Edge cases to mention: - Unnamed `MultiIndex` levels (the default `level_0`, `level_1`) make joins ambiguous — name your levels explicitly before joining. - Reordering levels with `swaplevel` does not change join behaviour — pandas matches by level name, not position. - A partial-match join can blow up row count if the partial side has duplicates — `validate=` works for `MultiIndex` merges too. - `reset_index().merge(...).set_index(...)` is the safe fallback when the level-name logic is confusing — flatten, merge on columns, restore.

What NOT to say: - "MultiIndex joins are too complicated, just flatten everything." Flattening loses the index ergonomics. Learn the level-name match rule. - "Level order matters." It doesn't — names match by name, not position.

Common follow-up: "You have a `(store_id, date)`-indexed sales frame and a `store_id`-indexed targets frame. Join them so every row has a `target` column." — `sales.join(targets)`. Pandas joins on `store_id` (the shared level name) and broadcasts `target` across all dates for that store.

```

sales = pd.DataFrame(
    {"amount": [100, 200, 150]},
    index=pd.MultiIndex.from_tuples(
        [(101, "2026-05-01"), (101, "2026-05-02"), (102, "2026-05-01")],
        names=["store_id", "date"],
    ),
)

targets = pd.DataFrame({"target": [1000, 500]}, index=pd.Index([101, 102], name="store_id"))

merged = sales.join(targets)
#           amount  target
# store_id date
# 101      2026-05-01    100    1000
# 101      2026-05-02    200    1000
# 102      2026-05-01    150     500

```

Q15 · `df.join` vs `df.merge` vs `pd.concat([df1, df2], axis=1)` for column-stacking

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, TCS, Goldman · Topic: Join · Alignment

The question (interviewer's verbatim phrasing):

You want to stick two DataFrames side by side, aligning on the index. You could use `join`, `merge`, or `pd.concat(axis=1)`. When does each one win?

The 30-second answer: `pd.concat([a, b], axis=1)` is the fastest for aligning on the index when both frames already share an index — pure column stacking, no key resolution. `df.join` is the cleanest when you have one "primary" frame and want to attach columns from one or more others — `df.join([b, c, d])` is concise. `df.merge` is the right call when the join key is a column (not the index) or when you need explicit `how=`, `validate=`, `indicator=`, or `suffixes=` control. The decision tree: shared index = `concat`; primary + lookups = `join`; column-keyed or needs validation = `merge`.

Step-by-step thought process: 1. State the decision tree. The three operations overlap in capability but each one is cleanest in a specific shape. 2. Show `concat` for index-aligned stacking. Both frames indexed by `user_id`, no key-resolution needed — `pd.concat([profile, prefs], axis=1)` aligns and stacks. Faster than `join` for this exact case because no hash lookup. 3. Cover `join` for one-to-many lookups. `orders.join([users, products, regions], on=[...])` is the chain shorthand. Use when one frame is the spine and the others are reference data. 4. Reserve `merge` for column keys and validation. If the join key is a column, or you need `validate="1:1"` to assert uniqueness, or you want an indicator column — reach for `merge`. The keyword surface is richer.

Edge cases to mention: - `pd.concat(axis=1)` with non-matching indexes does an outer alignment by default — fills with NaN on either side. Pass `join='inner'` to keep only shared keys. - `df.join` defaults to `how='left'`; `pd.concat(axis=1)` defaults to `join='outer'`. Different defaults catch people off guard. - For very wide column-stacking (10+ frames), `pd.concat([df1, ..., df10], axis=1)` is the only one of the three with a clean N-ary signature. - Order of columns in the output follows the order in the input list for `concat`; `join` follows the spine frame first, then attached frames in the order given.

What NOT to say: - "They're all the same thing." They have different defaults, different ergonomics, and different perf profiles for different shapes. - "Just use merge for everything." Works but is verbose for index-aligned cases. Pick the cleanest tool for the shape.

Common follow-up: "You have ten DataFrames, all indexed by `user_id`, each with one column. You want a single 10-column frame. Which call?" — `pd.concat(dfs, axis=1)`. One call, no key resolution, fastest.

```
profile = pd.DataFrame({"name": ["aarav", "diya"], "city": ["BLR", "BOM"]}, index=[1, 2])
prefs = pd.DataFrame({"theme": ["dark", "light"], "lang": ["en", "hi"]}, index=[1, 2])

# concat – fastest for index-aligned column stacking
pd.concat([profile, prefs], axis=1)

# join – clean for primary + lookups
profile.join(prefs)

# merge – needed only if more control required
pd.merge(profile, prefs, left_index=True, right_index=True, how="outer", validate="1:1")
```

Section C — `concat`

`pd.concat` is the stacking verb. Vertical stacking (`axis=0`) is the most common use — appending one frame to another, often in a loop over chunks. Horizontal stacking (`axis=1`) is the column-side equivalent. The shape of `pd.concat` is `pd.concat(objs, axis=, join=, keys=, ignore_index=, sort=)` and each keyword answers at least one interview question. This section covers all of them, plus the canonical perf pitfall.

Q16 · `axis=0` vs `axis=1` — vertical vs horizontal

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Zoho, Postman · Topic: Concat · Axis Semantics

The question (interviewer's verbatim phrasing):

What does `pd.concat([df1, df2], axis=0)` do differently from `pd.concat([df1, df2], axis=1)` ? Which is the default?

The 30-second answer: `axis=0` (the default) stacks vertically — rows of `df2` go below rows of `df1`. Columns are aligned by name; missing columns get NaN. `axis=1` stacks horizontally — columns of `df2` go to the right of `df1`. Rows are aligned by index; missing index values get NaN. The mental model is "axis is the axis you stack along" — `axis=0` grows the row axis; `axis=1` grows the column axis.

Step-by-step thought process: 1. State the default — `axis=0`, vertical stacking. The default matches the most common use case: appending new rows to an existing frame. 2. Explain the alignment rule for each direction. `axis=0` aligns by column name; the columns are the "other" axis. `axis=1` aligns by index; the index is the "other" axis. Misaligned entries become NaN. 3. Cover the dtype impact. Concatenating frames with different dtypes for the same column promotes to the common type — `int64 + float64 → float64`; `int64 + Int64 → Int64`; `int64 + object → object`. Always check `.dtypes` after concat. 4. Mention the row-count math. `axis=0` total rows = `sum(len(df) for df in dfs)`. `axis=1` total rows = `len(union of indexes)`.

Edge cases to mention: - `axis=0` with `join='inner'` keeps only columns present in all frames; `join='outer'` (default) keeps all columns and fills NaN. - `axis=1` with non-unique indexes does a Cartesian-style alignment per duplicate key — usually wrong; deduplicate first. - Empty DataFrames in the list are skipped, but their column types contribute to the dtype resolution if `join='outer'`. - `pd.concat` works on `Series` too — `axis=0` stacks vertically into a longer Series; `axis=1` produces a DataFrame with one column per Series.

What NOT to say: - "axis=1 appends rows." It doesn't — it adds columns. - "axis= is the same as numpy's." Conceptually yes, but pandas adds index/column alignment on top, which numpy doesn't do.

Common follow-up: "You concat `axis=0` two frames with the same columns but different column orders. What's the result column order?" — Pandas uses the column order of the first frame, then appends any columns from later frames not in the first. With `sort=True`, it sorts the union alphabetically.

```
df1 = pd.DataFrame({"a": [1, 2], "b": [10, 20]})
df2 = pd.DataFrame({"a": [3, 4], "b": [30, 40]})

# axis=0 - rows stacked
pd.concat([df1, df2], axis=0)
#    a    b
# 0  1   10
# 1  2   20
# 0  3   30    <- duplicate index from df2
# 1  4   40

# axis=1 - columns stacked
pd.concat([df1, df2], axis=1)
#    a    b    a    b    <- duplicate columns
# 0  1   10    3   30
# 1  2   20    4   40
```

Q17 · `ignore_index=True` — when and why

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Swiggy, Walmart Labs · Topic: Concat · Index Reset

The question (interviewer's verbatim phrasing):

After `pd.concat([df1, df2])`, the result index has duplicate values like 0, 1, 0, 1. How do you fix it inside the concat call?

The 30-second answer: Pass `ignore_index=True`. Pandas throws away the existing indexes and assigns a fresh `RangeIndex(0, len(result))` to the output. This is the right choice when the original indexes were not meaningful — like the default `RangeIndex` on fresh DataFrames — and you want a clean 0..N-1 index for downstream code. If the original indexes carry meaning (timestamps, IDs), do not use `ignore_index`; either keep them or use `keys=` per Q19.

Step-by-step thought process: 1. State the keyword and the result. `ignore_index=True` → fresh `RangeIndex`. Same as `.reset_index(drop=True)` on the output, but baked into the concat call. 2. Explain when it is the right call. Stacking chunks read from CSV, appending rows from multiple parallel jobs, or anywhere the original `RangeIndex` was a meaningless 0..N artefact of how the frame was loaded. 3. Cover the case when it's wrong. If your frames are indexed by `customer_id` or `order_date`, `ignore_index=True` throws away the identity of every row. Use `keys=` instead to tag origin while preserving the original index. 4. Mention the perf detail. `ignore_index=True` avoids the cost of computing the (often duplicate) merged index — slightly faster than the default for huge concats.

Edge cases to mention: - `ignore_index=True` works with `axis=1` too — it gives the columns a fresh `RangeIndex` instead of preserving the original column labels. Rarely what you want; the default is to keep column names. - The keyword interacts with `keys=`. If both are passed, `keys=` wins for index labeling and `ignore_index` is silently ignored. - After `ignore_index=True`, downstream code that assumed the original index labels (timestamps, ids) will break — check downstream selects before applying. - `verify_integrity=True` checks for duplicate index values and raises — useful when you want to guarantee a unique index without `ignore_index`.

What NOT to say: - "Always use `ignore_index`." It silently drops information when the index was meaningful. - "`ignore_index` is the same as `reset_index`." Mostly yes, but `reset_index()` by default moves the old index to a column; `ignore_index=True` discards it entirely.

Common follow-up: "You concat with `ignore_index=True` and then realize the original timestamps were meaningful. How do you recover?" — You can't from the result alone. Concat again with `ignore_index=False` (the default) or with `keys=` to preserve origin. Treat `ignore_index` as destructive.

```
df1 = pd.DataFrame({"x": [1, 2]}, index=[10, 20])
df2 = pd.DataFrame({"x": [3, 4]}, index=[30, 40])

# default - preserves original index
pd.concat([df1, df2])
#      x
# 10  1
# 20  2
# 30  3
# 40  4

# duplicate indexes scenario
df1b = pd.DataFrame({"x": [1, 2]}) # index 0, 1
df2b = pd.DataFrame({"x": [3, 4]}) # index 0, 1

pd.concat([df1b, df2b])
#      x
# 0  1
# 1  2
# 0  3  <- duplicate
# 1  4

pd.concat([df1b, df2b], ignore_index=True)
#      x
# 0  1
# 1  2
# 2  3  <- fresh RangeIndex
# 3  4
```

Q18 · `join='inner'` vs `join='outer'` in `concat`**Tags:** Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Goldman · Topic: Concat · Alignment**The question (interviewer's verbatim phrasing):**

You concat two frames vertically. The first has columns `[a, b, c]`, the second has `[b, c, d]`. What does the result look like with `join='outer'` versus `join='inner'`?

The 30-second answer: `join='outer'` (the default) keeps the union of columns — `[a, b, c, d]` — and fills NaN where a frame didn't have that column. `join='inner'` keeps only the intersection — `[b, c]` — and drops `a` from the first frame and `d` from the second. Outer is permissive (no data loss but introduces NaN); inner is strict (loses columns but no NaN). The same logic applies to `axis=1` concat, except the alignment is on the row index instead of column names.

Step-by-step thought process: 1. State the default — `join='outer'`. Union of columns (or indexes, for `axis=1`). 2. Walk the two outcomes. Outer gives you everything-with-NaN. Inner gives you the intersection-no-NaN. 3. Cover when each is right. Outer is the right call when the frames are partial views of the same logical schema (logs from multiple services). Inner is right when you only care about columns common to all sources (a comparison report). 4. Mention the perf implication. Inner is faster on wide concat because the result has fewer columns to allocate. For 50 frames each with 100 columns and only 20 shared, inner is much smaller than outer.

Edge cases to mention: - `join='inner'` on `axis=1` concat keeps only rows whose index is in every frame's index — common when aligning multi-source time series and you only want the dates with full coverage. - Inner join with no shared columns (or indexes) returns an empty frame, not an error. - `sort=False` is the default in pandas 2.x; the column order in `join='outer'` follows the order of first appearance. - Dtype promotion happens for every shared column even in `join='inner'` — same rules as Q16.

What NOT to say: - "join in concat is the same as join in merge." Conceptually similar (set semantics on the "other" axis), but `concat` has only `inner` and `outer`, no `left` or `right`. - "Outer is dangerous because of NaN." Not dangerous, just permissive. NaN is data, not a bug — handle it explicitly.

Common follow-up: "You want to vertically concat two log frames where you only care about the columns common to both. Which keyword and value?" — `pd.concat([log1, log2], axis=0, join='inner')`. The result has only the shared columns.

```
df1 = pd.DataFrame({"a": [1, 2], "b": [10, 20], "c": [100, 200]})
df2 = pd.DataFrame({"b": [30, 40], "c": [300, 400], "d": [3000, 4000]})

# outer - union of columns
pd.concat([df1, df2], axis=0, join="outer")
#      a    b    c      d
# 0  1.0  10  100    NaN
# 1  2.0  20  200    NaN
# 0   NaN  30  300  3000.0
# 1   NaN  40  400  4000.0

# inner - intersection of columns
pd.concat([df1, df2], axis=0, join="inner")
#      b    c
# 0  10  100
# 1  20  200
# 0  30  300
# 1  40  400
```

Q19 · `keys=` for `MultiIndex` tagging

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, JP Morgan, Swiggy, PhonePe · Topic: Concat · MultiIndex

The question (interviewer's verbatim phrasing):

You concat the daily sales frames for May 1, May 2, and May 3 vertically. You want the output index to remember which row came from which day. How?

The 30-second answer: Pass `keys=["2026-05-01", "2026-05-02", "2026-05-03"]` matching the order of the frames in the list. Pandas builds a `MultiIndex` for the result — the outer level is the key you supplied, the inner level is the original index of each frame. After that, `result.loc["2026-05-01"]` slices back to that day's rows. This is the canonical pattern for stacking frames while preserving origin.

Step-by-step thought process: 1. Show the syntax. `keys=` is a list matching the frame list in length and order. Strings, ints, dates — any hashable works. 2. Explain the output index. The new outer level is the key; the original index becomes the inner level. `result.index` is a `MultiIndex` with two (or more) levels. 3. Cover the slicing payoff. `result.loc[key]` returns just that frame's contribution — cleaner than carrying around a `source` column. 4. Mention the `names=` companion. `pd.concat(..., keys=[...], names=["batch_date", "row_idx"])` labels the levels so downstream code can `.groupby(level="batch_date")`.

Edge cases to mention: - `keys=` requires the list to match the frames list in length; mismatched lengths raise `ValueError`. - Combining `keys=` with `ignore_index=True` ignores the keys silently — pick one. - For `axis=1` concat, `keys=` adds a `MultiIndex` on the columns instead — useful for stacking metrics from multiple sources side by side. - The keys themselves are not sorted; they appear in the order you passed them, which matches the frames list.

What NOT to say: - "Just add a `date` column to each frame before concat." Works, but inflates row width and is harder to slice cleanly than a `MultiIndex`. - "keys= is just for cosmetics." It's a functional index — `loc[key]`, `groupby(level=)`, and serialization all work with it.

Common follow-up: "After the keyed concat, how do you get just May 2's rows?" — `result.loc["2026-05-02"]`. The outer level of the MultiIndex is the date you passed in `keys=`.

```
day1 = pd.DataFrame({"amount": [100, 200]}, index=["A1", "B2"])
day2 = pd.DataFrame({"amount": [300, 400]}, index=["A1", "C3"])
day3 = pd.DataFrame({"amount": [500]}, index=["B2"])

stacked = pd.concat(
    [day1, day2, day3],
    keys=["2026-05-01", "2026-05-02", "2026-05-03"],
    names=["date", "sku"],
)
#
#          amount
# date      sku
# 2026-05-01 A1      100
#           B2      200
# 2026-05-02 A1      300
#           C3      400
# 2026-05-03 B2      500

stacked.loc["2026-05-02"]
#          amount
# sku
# A1      300
# C3      400
```

Q20 · The perf hit of `concat` in a loop

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, Goldman, JP Morgan · Topic: Concat · Performance

The question (interviewer's verbatim phrasing):

Your colleague wrote a loop that calls `pd.concat([result, chunk])` on every iteration and the script is taking 40 minutes on a few hundred MB of data. What's wrong and how do you fix it?

The 30-second answer: Each `pd.concat` call allocates a fresh DataFrame the size of the running result plus the chunk and copies everything. Over N iterations, total work is $O(N^2)$ — the classic "Schlemiel the Painter" anti-pattern. Fix: collect chunks in a Python list, then concat once at the end — `pd.concat(chunks, ignore_index=True)`. One allocation, one copy, $O(N)$ total. A 100x-1000x speedup on real datasets is typical.

Step-by-step thought process: 1. State the anti-pattern. `result = pd.concat([result, chunk])` inside a for loop. The result grows; each concat copies it. 2. Walk the complexity. Iteration 1 copies 1 chunk. Iteration 2 copies 2 chunks. Iteration N copies N chunks. Sum is $N(N+1)/2 = O(N^2)$. 3. Show the fix. `chunks = []` outside the loop; `chunks.append(df_for_this_iter)` inside; `result = pd.concat(chunks, ignore_index=True)` after. One concat, linear. 4. Mention the related anti-pattern. `df.append(chunk)` (deprecated in pandas 1.4, removed in 2.0) had the same issue. Modern code uses the list-then-concat idiom only.

Edge cases to mention: - For streaming or memory-constrained pipelines, you can't hold all chunks in memory. Use `pd.concat` in larger batches (every 100 chunks) instead of every iteration — same idea, just batched. - If chunks have different schemas, the concat-at-end will do dtype promotion across all of them in one pass — cheaper than promoting on every iteration. - For very large datasets (10GB+) consider PyArrow or Polars instead of pandas — pandas still copies even in the batched form. - `pd.concat` of a list is the same operation as the batched version of the loop — there's no faster pandas-native pattern.

What NOT to say: - "It's slow because pandas is slow." It's slow because of $O(N^2)$ allocation; the fix is $O(N)$ at the same per-row cost. - "Use `df.append`." It's removed in pandas 2.0. Use the list-then-concat idiom.

Common follow-up: "Show me the canonical chunked-CSV read pattern." — `chunks = [chunk for chunk in pd.read_csv(path, chunksize=100_000)]; df = pd.concat(chunks, ignore_index=True)`. Or stream-process per chunk if memory is the constraint.

```
import pandas as pd
import time

# anti-pattern: O(N^2)
def bad_concat(n_chunks):
    result = pd.DataFrame()
    for i in range(n_chunks):
        chunk = pd.DataFrame({"x": range(1000)})
        result = pd.concat([result, chunk], ignore_index=True) # NO
    return result

# correct: O(N)
def good_concat(n_chunks):
    chunks = []
    for i in range(n_chunks):
        chunks.append(pd.DataFrame({"x": range(1000)}))
    return pd.concat(chunks, ignore_index=True) # YES

# at n_chunks=500, bad is ~50x slower than good
# at n_chunks=5000, bad is ~500x slower
```

Q21 · Concatenating Series — and when the result is a Series vs a DataFrame

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, TCS · Topic: Concat · Series

The question (interviewer's verbatim phrasing):

What does `pd.concat([s1, s2])` return when `s1` and `s2` are Series? What if you concat them with `axis=1` ?

The 30-second answer: `pd.concat([s1, s2])` with the default `axis=0` returns a longer Series — the values of `s2` appended below `s1`, with index concatenated. `pd.concat([s1, s2], axis=1)` returns a 2-column DataFrame — each Series becomes a column, aligned on the index. The column names come from the Series' `.name` attribute (or 0, 1, ... if names are not set). It is the cleanest way to assemble a DataFrame from a list of independently-computed Series.

Step-by-step thought process: 1. Distinguish the two outputs. `axis=0` keeps the Series-ness; `axis=1` promotes to DataFrame. 2. Cover the naming. If both Series have `.name = "amount"`, `axis=1` gives a DataFrame with both columns named `amount` — collision. Rename one or pass `keys=["a", "b"]` to disambiguate. 3. Show the typical use case. Computing several aggregations independently — `s1 = df.groupby("a")["x"].sum()`, `s2 = df.groupby("a")["x"].mean()` — then `pd.concat([s1, s2], axis=1, keys=["sum", "mean"])` for a side-by-side report. 4. Mention the index alignment. With `axis=1`, the indexes of the Series are unioned. If they don't fully overlap, you get NaN. Pass `join='inner'` to keep only the intersection.

Edge cases to mention: - Concatenating a Series and a DataFrame on `axis=1` adds the Series as one column to the DataFrame, aligned on index. - For Series with the same `.name`, pandas does not auto-disambiguate column names — duplicates survive. Use `keys=` to fix. - `axis=0` concat of differently-named Series resets `.name` to None — the union has no single name. - `pd.concat` of an empty list raises `ValueError`. Guard with `if not chunks: return pd.DataFrame()`.

What NOT to say: - "Just convert Series to DataFrames first." Adds noise; pandas handles Series directly. - "axis=1 always returns a DataFrame." True for Series inputs, but the column count depends on the inputs.

Common follow-up: "You computed three groupby aggregations and want them side by side as columns. Show the concat." — `pd.concat([sums, means, counts], axis=1, keys=["sum", "mean", "count"])`. Clean, named, aligned on the group key.

```
import pandas as pd

s1 = pd.Series([1, 2, 3], index=["a", "b", "c"], name="x")
s2 = pd.Series([10, 20, 30], index=["a", "b", "c"], name="y")

# axis=0 - longer Series
pd.concat([s1, s2])
# a      1
# b      2
# c      3
# a     10
# b     20
# c     30

# axis=1 - DataFrame
pd.concat([s1, s2], axis=1)
#   x  y
# a  1 10
# b  2 20
# c  3 30
```

Q22 · `pd.concat` vs `DataFrame.append` (the removed shortcut)

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy · Topic: Concat · API Currency

The question (interviewer's verbatim phrasing):

You see `df.append(other)` in old code. Does it still work in pandas 2.x? What should it be replaced with?

The 30-second answer: `DataFrame.append` was deprecated in pandas 1.4 and removed in pandas 2.0. Calling it now raises `AttributeError`. The replacement is `pd.concat([df, other], ignore_index=True)` — same operation, less convenient API, but the only one that survives the cleanup. The `append` method had the same $O(N^2)$ anti-pattern as concat-in-a-loop, which is part of why pandas removed it — there was no efficient way to use it correctly.

Step-by-step thought process: 1. State the lifecycle. Deprecated 1.4 (Jan 2022), removed 2.0 (April 2023). Any 2.x code calling `.append` on a DataFrame raises. 2. Show the replacement.

`pd.concat([df, other], ignore_index=True)` is the direct equivalent for the single-frame append case. For loop-style accumulation, use the list-then-concat pattern from Q20. 3. Cover the gotcha — Python's list `.append` is unrelated. `chunks.append(df)` on a Python list still works; the removed method was `pd.DataFrame.append`. 4. Mention the migration audit. When you take over a legacy codebase, grep for `\.append(` on DataFrames and replace. The error message is clear but the bug only surfaces at runtime.

Edge cases to mention: - Some libraries (Streamlit, older tutorials, GeeksForGeeks PDFs) still show `df.append`. Treat any tutorial older than 2023 as suspect. - The replacement preserves the order of frames — `pd.concat([df, other])` not `[other, df]`. - The `ignore_index=True` flag is the equivalent of `df.append(other, ignore_index=True)`. Without it, you get the original (possibly duplicate) indexes. - `Series.append` was removed at the same time and the same replacement pattern applies.

What NOT to say: - "Use `df.append`." It will raise in pandas 2.x. Don't. - "concat is slower than append was." Same operation under the hood; concat is no slower.

Common follow-up: "What does the `AttributeError` look like in pandas 2.x when someone calls `df.append`?" — `AttributeError: 'DataFrame' object has no attribute 'append'`. Same message as for any missing attribute — easy to miss in stack traces.

```
import pandas as pd

df = pd.DataFrame({"x": [1, 2]})
other = pd.DataFrame({"x": [3, 4]})

# Pre-2.0 (no longer works):
# df.append(other, ignore_index=True)
# raises: AttributeError: 'DataFrame' object has no attribute 'append'

# Modern replacement:
pd.concat([df, other], ignore_index=True)
#    x
# 0  1
# 1  2
# 2  3
# 3  4
```

Q23 · Concatenating frames with different dtypes for the same column

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan · Topic: Concat · Dtype Promotion

The question (interviewer's verbatim phrasing):

Frame A has column `amount` as `int64`. Frame B has it as `float64`. You concat them. What does the result column dtype look like, and what about `int64` plus `object`?

The 30-second answer: `int64 + float64 → float64`. Pandas finds the common supertype that holds both. `int64 + object → object` — string-ish dtypes win. `int64 + Int64` (the nullable integer) → `Int64`. `datetime64[ns] + object → object`, which is almost always a bug — fix the dtype on one side before concat. Always run `result.dtypes` after a concat to catch silent promotions.

Step-by-step thought process: 1. Walk the promotion rules. Numeric-to-numeric promotion follows numpy: smallest type that holds both sides. Integer + float = float. Integer + integer of different widths = wider integer. 2. Cover the object trap. `object` is pandas' catch-all dtype; once a column has object, it loses all the numeric optimisations. Concat with one object-dtype side promotes the whole column to object. 3. Mention nullable types. The 2.x story: `int64 + Int64` (nullable) gives `Int64`. `bool + boolean` (nullable) gives `boolean`. Pandas 2.x is gradually pushing toward nullable everywhere. 4. Show the diagnostic. After concat, `result.dtypes` is one line, fast, and tells you immediately if any column promoted unexpectedly.

Edge cases to mention: - `int64 + datetime64 → object`, which is broken. Either cast both sides to datetime first or you'll get a column of mixed Python objects. - Categorical dtypes are not preserved across concat by default — pandas converts to object. Use `pd.api.types.union_categoricals` for clean categorical concat. - For nullable Int64 columns, NaN becomes `pd.NA`, not `np.nan` — different identity and different comparison rules. - `pd.concat` of frames with mixed Arrow and numpy backends prefers numpy unless all frames are Arrow — version-specific behaviour, check `pd.options.mode`.

What NOT to say: - "The dtype stays the same." It promotes to the common supertype; check after every concat. - "Just use `astype` after." That repairs the damage; better to fix the dtype on the source frames before concat.

Common follow-up: "After concat, an `amount` column is `object` dtype. Why?" — At least one of the input frames had `amount` as `object` (strings, mixed types). Find the source frame with `df.dtypes`, fix it (`pd.to_numeric` with `errors='coerce'`), then concat again.

```
import pandas as pd

a = pd.DataFrame({"amount": [1, 2, 3]})           # int64
b = pd.DataFrame({"amount": [4.5, 5.5, 6.5]})     # float64
c = pd.DataFrame({"amount": ["7", "8", "9"]})     # object

print(pd.concat([a, b]).dtypes["amount"]) # float64
print(pd.concat([a, c]).dtypes["amount"]) # object – BUG candidate

# fix before concat
c_fixed = c.assign(amount=pd.to_numeric(c["amount"])) # int64
print(pd.concat([a, c_fixed]).dtypes["amount"]) # int64
```

Q24 · `axis=1` concat with non-unique indexes — the Cartesian trap

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Swiggy, JP Morgan · Topic: Concat · Index Hygiene

The question (interviewer's verbatim phrasing):

You concat two DataFrames on `axis=1`. The left frame has duplicate index values (e.g., the index value 42 appears twice). What does pandas produce?

The 30-second answer: In pandas 2.x and later, `pd.concat([left, right], axis=1)` raises `InvalidIndexError: Reindexing only valid with uniquely valued Index objects` whenever one side has duplicate index values that the other side does not match — no need to pass `verify_integrity=True`. The Cartesian-style expansion only survives when *both* sides have matching duplicates on the same key (e.g., left has 42 twice, right has 42 twice → 4 rows for key 42, silently). The old 1.x story of "any duplicate triggers a silent row explosion" no longer holds. Either way, watch out for duplicate indexes before reaching for `axis=1` concat — deduplicate or `reset_index(drop=True)` first.

Step-by-step thought process: 1. State the new rule. In 2.x, `axis=1` concat aligns by index; a duplicate on one side that does not have a matching duplicate count on the other raises `InvalidIndexError` straight from the reindex call. This is the common case the buyer hits in practice. 2. Cover the surviving silent case. When both sides have the same key repeated (left has 42 twice, right has 42 twice), pandas does the $m \times m$ alignment per key — you still get a row explosion, just symmetric. The asymmetric case (left has 42 twice, right has 42 once) used to do this silently in 1.x; in 2.x it errors instead. 3. Cover the diagnostic. `df.index.is_unique` returns `True` or `False`. If `False`, fix before concat. `df.index.value_counts().head()` shows the worst offenders. 4. Mention the fix. Either `df = df.reset_index(drop=True)` if the index was meaningless, or `df = df.groupby(level=0).first()` (or `.last()`, `.mean()`) to pick a deterministic row per key.

Edge cases to mention: - `pd.concat([a, b], axis=1, verify_integrity=True)` raises if the *result* has duplicate index values — still useful for the symmetric-duplicate case where 2.x would otherwise expand silently. - For `MultiIndex`, duplicates across the full tuple are what matter, not duplicates on any single level. - `Series.reindex(unique_index)` is another way to fix the duplicate key problem before concat. - The same alignment story applies in `df.join` and `pd.merge` with `left_index=True, right_index=True` — duplicate-index alignment is consistent across the index-aligned merge family.

What NOT to say: - "Pandas just picks one." It does not — either it errors (mismatched-duplicate case) or expands per duplicate group. - "axis=1 concat is safe because it doesn't grow rows." It can grow rows when both sides carry matching duplicates, and it raises when only one side does.

Common follow-up: "How do you detect the problem before calling concat?" — `df.index.is_unique` returns `False`. Or pass `verify_integrity=True` and let pandas raise on the duplicate result. Or just call the concat — in 2.x the mismatched-duplicate case raises on its own.

```
import pandas as pd

left = pd.DataFrame({"x": [1, 2, 3]}, index=[42, 42, 99])
right = pd.DataFrame({"y": [10, 20]}, index=[42, 99])

# 2.x: raises InvalidIndexError on the mismatched-dup case
pd.concat([left, right], axis=1)
# InvalidIndexError: Reindexing only valid with uniquely valued Index objects

# Symmetric-duplicate case still expands silently:
left2 = pd.DataFrame({"x": [1, 2]}, index=[42, 42])
right2 = pd.DataFrame({"y": [10, 20]}, index=[42, 42])
pd.concat([left2, right2], axis=1)
# 4 rows for key 42 — per-key Cartesian-style alignment

# verify_integrity=True catches duplicate-in-result up front
pd.concat([left2, right2], axis=1, verify_integrity=True)
# ValueError: Indexes have overlapping values: Index([42], ...)
```

Q25 · `keys=` with `axis=1` — stacking metrics from multiple sources

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, JP Morgan, Swiggy · Topic: Concat · Reporting Patterns

The question (interviewer's verbatim phrasing):

You have three DataFrames — `sales` , `returns` , `cancellations` — each with the same index (date) and the same single `count` column. You want a 3-column wide frame where each source is its own column with a clean label. How?

The 30-second answer: `pd.concat([sales, returns, cancellations], axis=1, keys=["sales", "returns", "cancellations"])`. The `keys=` builds a `MultiIndex` on the columns — outer level is the source name, inner level is the original column name. Result columns look like `(sales, count)` , `(returns, count)` , `(cancellations, count)`. Slice with `result["sales"]` to get back the sales-only sub-frame. The pattern is the canonical reporting builder at any KPI dashboard pipeline.

Step-by-step thought process: 1. Show the call. `axis=1` to put each frame as columns; `keys=` to label them at the outer column level. 2. Explain the column index. The result has a 2-level `MultiIndex` on columns — outer is the key you supplied, inner is the original column names of each frame. 3. Cover the slicing payoff. `result["sales"]` returns the sales sub-DataFrame with the original column names. Cleaner than carrying redundant prefixes like `sales_count` , `returns_count`. 4. Mention the alternative. `pd.concat([s.rename("count") for s in (sales_series, returns_series, cancellations_series)], axis=1, keys=["sales", ...])` works for Series inputs too.

Edge cases to mention: - For ergonomic single-column reporting, you can rename after: `result.columns = result.columns.droplevel(1)` flattens the MultiIndex columns back to a flat list of keys. - Mixed-shape frames (different column sets per source) work fine — the MultiIndex columns expose every (source, column) tuple, with NaN where a source didn't have a column. - `keys=` interacts cleanly with `groupby` on the column MultiIndex — `result.groupby(level=0, axis=1).sum()` aggregates per source. - For very wide reports (50+ sources), this beats writing a manual `merge` chain for the same shape.

What NOT to say: - "I'd rename each column to `source_metric` and concat." Works but is brittle if columns differ across sources. - "keys= only works on axis=0." It works on both — outer index level for `axis=0` , outer column level for `axis=1`.

Common follow-up: "After the keyed concat, how do you flatten the columns to `sales_count` , `returns_count` , etc.?" — `result.columns = ["_".join(c) for c in result.columns]` . Or use `result.columns.map("{0[0]}_{0[1]}".format)` for the same effect.

```
import pandas as pd

dates = pd.date_range("2026-05-01", periods=3)
sales = pd.DataFrame({"count": [100, 110, 120]}, index=dates)
returns = pd.DataFrame({"count": [5, 8, 6]}, index=dates)
cancels = pd.DataFrame({"count": [2, 4, 3]}, index=dates)

report = pd.concat(
    [sales, returns, cancels],
    axis=1,
    keys=["sales", "returns", "cancellations"],
)
print(report)
#           sales returns cancellations
#           count      count          count
# 2026-05-01    100         5             2
# 2026-05-02    110         8             4
# 2026-05-03    120         6             3

report["sales"] # sales sub-frame with original columns
```

Section D — Common gotchas

The last five questions are the patterns that catch experienced pandas users off guard. Every one of these has produced a production bug at some Indian fintech, marketplace, or analytics shop — they show up in interview rounds because the interviewer has personally lost a sprint to them.

Q26 · Accidental m:m merge — the row explosion gotcha

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, Postman, Goldman, JP Morgan · Topic: Merge · Gotchas

The question (interviewer's verbatim phrasing):

Your merge produced 10× more rows than you expected. Walk me through what most likely happened and how you'd diagnose and prevent it next time.

The 30-second answer: You did an unintended `m:m` merge — both sides had duplicates on the join key, so every left duplicate matched every right duplicate. If left had 3 dups for key K and right had 4, you got 12 rows for K alone. Diagnose by checking `left["key"].duplicated().sum()` and the same

on right. Prevent by always passing `validate=` — pick the cardinality you expect (`1:1` , `1:m` , or `m:1`) and pandas raises if the data violates it.

Step-by-step thought process: 1. Describe the failure mode. m duplicates on left $\times$ n duplicates on right = $m \times n$ output rows per key. Even small duplicate counts blow up fast. 2. Walk the diagnosis. `df["key"].duplicated().sum()` on both sides shows duplicate counts. `df.groupby("key").size().sort_values(ascending=False).head()` shows the worst offenders. 3. Show the prevention. `validate="1:1"` , `"1:m"` , or `"m:1"` baked into every merge call. The runtime cost is negligible; the bug-prevention value is enormous. 4. Cover the root-cause patterns. Most accidental $m:m$ joins come from one of three sources: forgetting to dedup a slowly-changing dimension table, joining on a non-natural key that has historical duplicates, or joining two fact tables (which is almost always wrong).

Edge cases to mention: - `validate=` is checked before the merge runs — the failure is early and obvious, not a "wait for OOM" failure. - If duplicates exist legitimately (a true $m:m$ relationship), you may want `m:m` — explicitly. Acknowledge it, don't accept it silently. - The fix usually isn't dedup — it's `groupby(key).agg({...})` on one side first, so the merge is `1:m` cleanly. - `indicator=True` plus row-count post-check is a manual variant: `assert len(merged) == len(left)` after a `how='left'` merge.

What NOT to say: - "I'll check row counts after." Reactive; the bug already happened. Use `validate=` . - "Merges should never explode rows." They can, legitimately — but never accidentally.

Common follow-up: "How do you collapse duplicates on the right side before a left merge so the cardinality becomes $1:m$?" — `right_clean = right.groupby("key").agg({"value": "sum"})` . Pick the aggregation that matches your business semantics (sum for amounts, last for most-recent attributes, count for occurrence counting).


```
import pandas as pd

orders = pd.DataFrame({
    "order_id": [1, 1, 2, 2, 3],
    "amount": [499, 499, 199, 199, 899], # duplicates
})

payments = pd.DataFrame({
    "order_id": [1, 1, 2, 3, 3],
    "txn": ["t1", "t2", "t3", "t4", "t5"], # duplicates on right too
})

# silent explosion: 2 left x 2 right for order 1, etc.
bad = pd.merge(orders, payments, on="order_id")
print(len(bad)) # 9 rows for what looks like 3 orders

# guard
try:
    pd.merge(orders, payments, on="order_id", validate="1:m")
except pd.errors.MergeError as e:
    print("caught:", e)
# Merge keys are not unique in left dataset; not a one-to-many merge

# clean fix: aggregate one side first
orders_unique = orders.drop_duplicates(subset="order_id")
clean = pd.merge(orders_unique, payments, on="order_id", validate="1:m")
print(len(clean)) # matches expectation
```

Q27 · Float-key merges — why they silently miss rows

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Zerodha, PhonePe · Topic: Merge · Floating Point

The question (interviewer's verbatim phrasing):

You merge two DataFrames on a `float64` column and the result has fewer rows than expected, even though the values "look the same." What's happening?

The 30-second answer: Floating-point equality is unreliable. `0.1 + 0.2 != 0.3` exactly, and a value that came from a CSV ("0.3") is not bit-identical to a value computed as `0.1 + 0.2` even if they print the same. Pandas merges by hash equality on the key, and two floats that differ in the last bit hash to different buckets — so they don't match. Fix: cast both sides to a stable type before merging. For integer-like floats, cast to `Int64` (capital I, nullable). For genuinely fractional values, cast to a fixed-precision string or to `Decimal`. Never merge on raw `float64`.

Step-by-step thought process: 1. State the underlying bug. IEEE 754 binary floats can't represent all decimal fractions exactly. A value that "is" 0.3 in your head can be `0.29999999999999998` in memory. 2. Walk how it manifests in merge. Hash-based join: the two values hash to different buckets, so they don't match. The merge silently returns fewer rows than expected. 3. Show the fix path. Inspect with `df["key"].iloc[0].hex()` — the hex shows the actual bit pattern. If both sides differ, you have a float-equality bug. 4. Cover the safe alternatives. `Int64` for integer-like floats (Indian payment IDs that came in as `123.0` from JSON). `pd.Categorical` or `str` for categorical-feeling joins. Round both sides to the same precision and cast to `int64` for amount-like values.

Edge cases to mention: - `np.isclose` solves the equality problem but doesn't help with merge — pandas merges by hash, not by approximate equality. - Pandas 2.x with Arrow backend uses Decimal for some float columns by default — partially mitigates the issue but does not eliminate it. - The float-key bug is more common than you'd think. JSON-from-Java APIs often turn integer IDs into floats when null-handling is involved. - For time-series merges by float timestamps (Unix epoch as float), use `pd.to_datetime` to cast to `datetime64[ns]` before merging — far more reliable.

What NOT to say: - "Pandas matches floats by approximate equality." It does not — strict bit-equality via hash. - "Just round both sides." Works but is fragile — you have to pick the precision and apply to both sides every time.

Common follow-up: "You have payment IDs stored as `float64` because of a JSON quirk. What's the cleanest one-line fix before merging?" — `df["id"] = df["id"].astype("Int64")` on both sides. `Int64` (capital I) is pandas' nullable integer; it survives NaN in the source. Then merge normally.

```
import pandas as pd
import numpy as np

# silent bug
left = pd.DataFrame({"key": [0.1 + 0.2, 1.0], "x": [1, 2]})
right = pd.DataFrame({"key": [0.3, 1.0], "y": [10, 20]})

print(left["key"].iloc[0] == right["key"].iloc[0]) # False
# Even though both print as 0.3

merged = pd.merge(left, right, on="key")
print(len(merged)) # 1 – the 0.3 row was lost

# fix: integer ids that came in as floats
ids_left = pd.DataFrame({"id": [1.0, 2.0, 3.0], "x": ["a", "b", "c"]})
ids_right = pd.DataFrame({"id": [1.0, 3.0], "y": [10, 30]})

ids_left["id"] = ids_left["id"].astype("Int64")
ids_right["id"] = ids_right["id"].astype("Int64")
clean = pd.merge(ids_left, ids_right, on="id", how="left")
```

Q28 · Default `how='inner'` drops rows silently

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, TCS, Infosys, Postman · Topic: Merge · Defaults

The question (interviewer's verbatim phrasing):

You merged customers to orders and the row count dropped from 10,000 customers to 6,500. Walk me through how this happened and what the fix should be.

The 30-second answer: You used the default `how='inner'` and customers without any orders were dropped from the result. Inner keeps only customers whose `customer_id` appears in the orders frame; the other 3,500 customers vanish silently because pandas does not warn. The fix is either `how='left'` (if you want every customer, with NaN for those without orders) or `how='right'` / `how='outer'` depending on which side is your source of truth. The deeper fix: always pass `how=` explicitly so the silent default never bites again.

Step-by-step thought process: 1. State the failure. Default `how='inner'` is the silent killer. Pandas does what you asked; you asked for the intersection. 2. Show the diagnostic. `len(left)` before and `len(result)` after — if they differ in a `left` merge case, you used the wrong `how=`. 3. Walk the fix. For "every customer," use `how='left'`. For "every order," use `how='right'`. For "every customer and every order," use `how='outer'`. 4. Cover the prevention. Senior pandas hygiene: pass `how=` to every merge call, even when the default matches your intent. The next maintainer (or you in three months) will not remember the default.

Edge cases to mention: - The silent drop only happens when at least one side has keys absent from the other. `inner` and `outer` give the same row count when keys overlap fully. - `pd.merge` does not emit a warning for inner-joined drops by default — it's the most-requested missing feature on the pandas tracker. - `indicator=True` post-hoc reveals the drops: do `how='outer', indicator=True`, then count `_merge == "left_only"` to see what an inner would have dropped. - A row-count assertion right after a merge — `assert len(result) == len(left)` for a left-style enrichment — is a cheap production guard.

What NOT to say: - "I always use outer to be safe." Outer adds right-only rows you didn't want. Pick the join type that matches the intent. - "Pandas should warn." It should, but it doesn't — own the explicit-`how=` discipline.

Common follow-up: "You did `how='left'` and the row count went up, not down. Why?" — Right frame has duplicates on the join key — each left row matched multiple right rows. `validate='m:1'` would have caught it.

```
import pandas as pd

customers = pd.DataFrame({"customer_id": range(1, 11)}) # 10 customers
orders = pd.DataFrame({"customer_id": [1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7], "amt": [100]*14})

# silent drop with default inner
default_merge = pd.merge(customers, orders, on="customer_id")
print(default_merge["customer_id"].nunique()) # 7 - lost customers 8, 9, 10

# explicit left preserves customer set
left_merge = pd.merge(customers, orders, on="customer_id", how="left")
print(left_merge["customer_id"].nunique()) # 10

# diagnose what got dropped
audit = pd.merge(customers, orders, on="customer_id", how="outer", indicator=True)
print(audit[audit["_merge"] == "left_only"]["customer_id"].tolist()) # [8, 9, 10]
```

Q29 · NaN treatment in merge keys

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Swiggy, JP Morgan, PhonePe · Topic: Merge · NaN

The question (interviewer's verbatim phrasing):

What happens to rows where the join key is NaN on one side or both? Do those rows match each other?

The 30-second answer: NaN never matches anything, including another NaN. Pandas follows IEEE 754 — `NaN == NaN` is `False`. So a left row with NaN in the key column survives in a `how='left'` merge with NaN in the right columns (no match found), and is dropped in `how='inner'`. Two NaN keys, one on each side, do not match each other. If you genuinely want NaN-NaN matching (rare, usually a sign of bad data), fill the keys first with `df["key"] = df["key"].fillna(-1)` and merge on the cleaned column.

Step-by-step thought process: 1. State the rule. NaN does not equal NaN. Pandas merge respects this strictly. 2. Walk the consequences for each `how=`. In `inner`, NaN-keyed rows drop on both sides. In `left`, left NaN rows survive with NaN right columns. In `outer`, NaN rows survive on whichever side had them. 3. Cover the gotcha with `pd.NA`. For nullable Int64 columns, missing values are `pd.NA` not `np.nan`. Same equality rule — `pd.NA == pd.NA` returns `pd.NA` (not True), which still fails the merge match. 4. Show the fix. If NaN keys are bugs (should have been filled upstream), fix upstream. If NaN keys are legitimate "unknown" markers and you want them to group together, `.fillna(sentinel)` with a value that can't collide with real keys (-1 for ids, "UNKNOWN" for strings).

Edge cases to mention: - `IS NULL` style matching does not exist in pandas merge directly. The `fillna`-with-sentinel approach is the standard workaround. - The categorical dtype follows the same rule — `NaN` category does not equal `NaN` category. - `pd.merge` on datetime columns treats `NaT` the same way — does not match itself. - The "NaN match NaN" workaround is `df["key"] = df["key"].fillna("__NAN__")` (object/string columns) or numeric sentinel for numeric columns; remember to undo it after if needed.

What NOT to say: - "NaN matches NaN if both are NaN." It does not — IEEE 754 explicitly says NaN is unequal to everything including itself. - "Just drop NaN keys before merging." That's destructive; understand why they're there first.

Common follow-up: "You have NaN in the join key on both sides and they should match (legacy data where NaN means 'system-generated'). What's the cleanest pre-processing?" — `for df in (left, right): df["key"] = df["key"].fillna("__SYS__")`. Pick a sentinel that cannot collide with real values, merge normally, optionally restore the NaN after if downstream cares.

```
import pandas as pd
import numpy as np

left = pd.DataFrame({"key": [1, 2, np.nan, 4], "x": ["a", "b", "c", "d"]})
right = pd.DataFrame({"key": [1, np.nan, 4], "y": [10, 20, 40]})

# NaN does not match NaN
pd.merge(left, right, on="key", how="left")
#   key  x    y
# 0  1.0  a  10.0
# 1  2.0  b   NaN
# 2  NaN  c   NaN    <- left NaN survives but does not match right NaN
# 3  4.0  d  40.0

# if NaN must match: fillna with a sentinel
left["key_f"] = left["key"].fillna(-1)
right["key_f"] = right["key"].fillna(-1)
pd.merge(left, right, on="key_f", how="left").drop(columns="key_f")
# now the NaN-keyed rows on both sides merge as if they were -1
```

Q30 · Categorical merge alignment — the category mismatch pitfall

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Walmart Labs · Topic: Merge · Categoricals

The question (interviewer's verbatim phrasing):

You merge two DataFrames on a `Categorical` key. After the merge, the result column dtype became `object` and a warning was printed. What happened and how do you avoid it?

The 30-second answer: The two `Categorical` columns had different category sets — for instance, left had `["small", "medium", "large"]` and right had `["medium", "large", "xlarge"]`. Pandas cannot merge them as a single `Categorical` because the merged column would need a unified category set. Default behaviour: promote to `object` and emit `FutureWarning`. Fix: union the categories explicitly before merging — `df["k"] = df["k"].cat.add_categories(...)` on both sides — so they share the same category set, or use `pd.api.types.union_categoricals` to build a unified `Categorical`.

Step-by-step thought process: 1. Diagnose the symptom. `result.dtypes["k"]` is `object` post-merge, when it was `category` on both sides going in. 2. Explain why. `Categorical` equality is checked at the index level — two categoricals with different category lists are considered different dtypes for merge purposes. 3. Show the explicit alignment.

`pd.api.types.union_categoricals([left["k"], right["k"]])` returns a unified `Categorical`. Or call `df["k"] = df["k"].astype(pd.CategoricalDtype(categories=combined))` on both sides. 4. Cover the perf reason this matters. `Categorical` merges are noticeably faster than `object` merges on string-heavy keys — losing the dtype to `object` can multiply merge time on large frames.

Edge cases to mention: - If both sides have the same category list in the same order, pandas keeps the `Categorical` dtype across the merge. - `ordered=True` categoricals add another constraint — both sides must have the same order, or pandas demotes to `object`. - For wide-character or large string keys (long SKU codes, IST timestamps as strings), `Categorical` merges are 5x-10x faster than `object` merges — worth aligning the categories explicitly. - pandas 2.x's Arrow-backed string dtype (`string[pyarrow]`) is another option for fast string-key merges with cleaner alignment than `Categorical`.

What NOT to say: - "Categoricals always merge fine." They don't if the category sets differ. - "Just convert to `object` before merging." Loses the perf benefit; align categories instead.

Common follow-up: "What's the perf difference between merging on `object` strings and on aligned `Categoricals` on a 10-million-row frame?" — Aligned `Categorical`: typically 3-5x faster, especially when the string values are long (long SKU codes, UUIDs). The hashing happens on category codes (small ints), not the string values.

```
import pandas as pd

left = pd.DataFrame({
    "k": pd.Categorical(["small", "medium", "large"], categories=["small", "medium", "large"]),
    "x": [1, 2, 3],
})
right = pd.DataFrame({
    "k": pd.Categorical(["medium", "large", "xlarge"], categories=["medium", "large", "xlarge"]),
    "y": [20, 30, 40],
})

# different category sets - dtype demoted
merged = pd.merge(left, right, on="k", how="inner")
print(merged["k"].dtype) # object - not category any more

# align before merging
all_cats = sorted(set(left["k"].cat.categories) | set(right["k"].cat.categories))
unified = pd.CategoricalDtype(categories=all_cats)
left["k"] = left["k"].astype(unified)
right["k"] = right["k"].astype(unified)

clean = pd.merge(left, right, on="k", how="inner")
print(clean["k"].dtype) # category - preserved
```

Chapter wrap

Thirty questions and you have the merge toolkit Indian product-co interviews actually test. The five join types in `pd.merge`, the keyword arguments that turn merges from "best effort" into validated contracts (`validate=`, `indicator=`, `suffixes=`), `df.join` for the index-aligned cases, `pd.concat` for stacking with the list-then-concat perf discipline, and the five gotchas that catch experienced practitioners — row explosions, float keys, silent inner drops, NaN behaviour, and categorical alignment.

If you remember nothing else from this chapter, remember three rules. One: always pass `how=` explicitly — never let the default `inner` decide. Two: always pass `validate=` — the contract you write before the merge is worth ten audits after. Three: never concat in a loop — collect chunks in a list and concat once. Three lines of discipline that eliminate the majority of pandas merge bugs you'll ever ship.

Next up — Chapter 4: pivot, melt, reshape. The same alignment-and-stacking concepts in a different verb family.

Chapter 4 — Pivot / Melt / Reshape (20 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

This is the reshape chapter. Twenty questions on the operations that take a DataFrame and rearrange its axes — `pivot`, `pivot_table`, `melt`, `stack`, `unstack`, `crosstab`, `explode`, `json_normalize`, and the round-tripping patterns that move data from long form to wide form and back. These are the questions Indian Data-role interviews lean on in round one to check whether you understand that a DataFrame is not just a 2D array but an indexed, axis-labelled structure where reshaping is a different operation from aggregation, even though pandas conflates them in spots. Get these twenty right and you will not be the candidate who reaches for a `for` loop when the question wants a one-line `pivot_table`.

Each question follows the same shape — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, the edge cases to surface, what NOT to say, and the common follow-up. Answers assume pandas 2.2+ on Python 3.12 unless a version is explicitly called out, and code examples use copy-on-write semantics throughout because that is now the default for new projects.

Table of Sections

- Section A — `pivot` vs `pivot_table` (Q1–Q5)
- Section B — `melt` and Long-Form Data (Q6–Q10)
- Section C — `stack` and `unstack` (Q11–Q15)
- Section D — Reshape Patterns: `crosstab`, `explode`, `json_normalize` (Q16–Q20)

Section A — `pivot` vs `pivot_table`

The first five questions are about the two operations that take a long-form DataFrame and turn it into a wide-form one — `pivot` and `pivot_table`. They look interchangeable at a glance, and that is exactly why interviewers ask about them. `pivot` is a pure reshape — no aggregation — and it crashes the moment you have duplicate (index, column) pairs. `pivot_table` aggregates duplicates with a function you supply, defaulting to `mean`, which is the reason a candidate who reaches for `pivot_table` "to be safe" often produces silently wrong numbers. The five questions below cover when each is the right tool, what `margins=True` does, and the failure modes that show up in production.

Q1 · `pivot` vs `pivot_table` — When to Use Which

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, Razorpay, Meesho, Walmart Labs · Topic: pivot fundamentals

The question (interviewer's verbatim phrasing):

What is the difference between `df.pivot` and `df.pivot_table` in pandas? When would you reach for one over the other?

The 30-second answer: `pivot` is a pure reshape — it takes one column as the new index, another as the new column header, and a third as the cell value, with no aggregation. If any (index, column) pair appears more than once in the source frame, `pivot` raises `ValueError`. `pivot_table` is the same reshape with an aggregation function layered on top — `aggfunc="mean"` by default — so duplicates get collapsed by the function instead of crashing. Use `pivot` when the long form has one row per (index, column) pair and you just want it rearranged; use `pivot_table` when there can be multiple rows per pair and you want a summary.

Step-by-step thought process: 1. Open with the structural difference. `pivot` is reshape-only; `pivot_table` is reshape-plus-aggregate. The names suggest interchangeability but the semantics diverge the moment duplicates exist. 2. Walk through the failure case. If you have an `orders` frame with two rows for `(customer_id=42, month='2026-04')` — say, two separate orders that month — `pivot(index="customer_id", columns="month", values="amount")` raises `ValueError: Index contains duplicate entries, cannot reshape`. `pivot_table` would silently take the mean of the two amounts, which is rarely what you want for revenue. 3. State the practitioner default. When you know the long form has one row per cell — for example, a resampled time series or a pre-aggregated summary — `pivot` is correct and `pivot_table` is overkill (and dangerous because the default `mean` can mask a data bug). When the long form has potential duplicates, `pivot_table` with an explicit `aggfunc="sum"` (or `count`, or `first`) is the right tool. 4. Mention the underlying truth. `pivot` is implemented as a thin wrapper around `set_index` plus `unstack`. `pivot_table` is implemented as `groupby` plus `unstack`. The mental model is "pivot is reshape, pivot_table is groupby plus reshape."

Edge cases to mention: - `pivot_table` defaults to `aggfunc="mean"` — silently averaging duplicates is a very common silent bug; always set `aggfunc` explicitly in production code so a reader knows the intent. - `pivot_table` returns float dtypes even for integer columns because the mean of two ints can be a float; explicit `aggfunc="sum"` on int columns returns int. - Both operations preserve the index name and column name on the result; `pivot_table` additionally exposes `margins=True` and `dropna=True/False` knobs.

What NOT to say: - "`pivot_table` is just `pivot` with an aggregation argument" — close but misleading; the failure-on-duplicates vs silently-aggregating-duplicates difference is a real semantic gap,

not a flag. - "Always use `pivot_table` to be safe" — no; defaulting to `mean` is the wrong-answer factory. Use the operation whose contract matches your data.

Common follow-up: "What happens if you call `pivot` on a frame that has duplicate (index, column) pairs?" — pandas raises `ValueError: Index contains duplicate entries, cannot reshape`. That is the signal to either deduplicate the source, pick a tie-breaking column for the index, or switch to `pivot_table` with an explicit aggregation function.

```
import pandas as pd

orders = pd.DataFrame({
    "customer_id": [1, 1, 2, 2, 3],
    "month":       ["2026-04", "2026-05", "2026-04", "2026-04", "2026-05"],
    "amount":      [1200, 800, 500, 700, 1500],
})

# pivot — fine for unique (customer_id, month) pairs, crashes on duplicates
wide_safe = orders.drop_duplicates(["customer_id", "month"]).pivot(
    index="customer_id", columns="month", values="amount"
)

# pivot_table — explicitly sum, never default to mean for money
wide_sum = orders.pivot_table(
    index="customer_id", columns="month", values="amount", aggfunc="sum", fill_value=0,
)
```

Q2 · Why does `pivot` crash on duplicate index/column pairs, and how do you fix it?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Meesho, JP Morgan · Topic: pivot duplicates

The question (interviewer's verbatim phrasing):

Your colleague ran `df.pivot(index='user_id', columns='product', values='qty')` and got `ValueError: Index contains duplicate entries, cannot reshape`. What does that error mean and what are her options?

The 30-second answer: The error means the long form has more than one row for at least one (user_id, product) combination, so `pivot` cannot decide which value to place in the cell. There are three fixes. One — deduplicate the source first if the duplicate is a data quality bug. Two — use `pivot_table` with `aggfunc="sum"` (or `count`, `first`, `last`) to collapse the duplicates

with a deterministic rule. Three — add another column to the index — say `order_id` — so the (index, column) pair becomes unique again. Pick by intent: bug vs aggregation vs more-than-2D shape.

Step-by-step thought process: 1. Name the contract `pivot` is enforcing. A wide-form DataFrame has at most one value per (row label, column label) intersection. If the long form has duplicates on those keys, `pivot` would have to silently pick one, average them, or concatenate — none of which it does. It raises. 2. Walk through each fix and when to use it. Deduplication is right when the source has a genuine duplicate row that should not exist — for example a join that exploded the count. Use `df.drop_duplicates(subset=["user_id", "product"], keep="last")` to keep the most recent row before pivoting. 3. `pivot_table` is right when duplicates are real and you want a summary — multiple orders per (user, product) and you want the sum of quantities. Be explicit about `aggfunc` so the next reader of the code knows it's a sum and not a mean by accident. 4. Adding a column to the index is the right answer when the source has another natural axis you forgot about. If the same user bought the same product across many days, pivoting on `(user_id, date)` as the index and `product` as columns keeps each cell unique.

Edge cases to mention: - Sometimes the duplicate isn't actually a duplicate — two rows differ in a column you weren't selecting. Inspect the duplicates with `df[df.duplicated(subset=["user_id", "product"], keep=False)].sort_values(["user_id", "product"])` before deciding which fix applies. - `pivot_table` will silently drop entirely-`NaN` groups unless `dropna=False` is passed; that can surprise you if a sparse product is dropped from the columns. - If duplicates exist only because of a leading or trailing whitespace bug in the column key (`'apple'` vs `'apple '`), the duplicate-detection step uncovers the bug — fix the strings instead of working around the symptom.

What NOT to say: - "Just use `pivot_table`, it always works" — it works, but it silently averages by default, which often masks the very data issue you should be investigating. - "Drop the duplicates before pivoting" — only if duplicates are genuinely bad rows. If they are real measurements that need aggregation, `drop_duplicates` discards information.

Common follow-up: "How would you check which (user_id, product) pairs are the duplicates?" — `df.groupby(["user_id", "product"]).size().pipe(lambda s: s[s > 1])` returns the offending keys with their duplicate counts. That output is usually enough to decide whether it's a data bug or a real aggregation case.

```
import pandas as pd

bad = pd.DataFrame({
    "user_id": [1, 1, 2, 2, 1],
    "product": ["A", "A", "B", "B", "A"],
    "qty":      [3, 5, 2, 1, 1],
})

# inspection step before deciding the fix
dups = bad.groupby(["user_id", "product"]).size().loc[lamba s: s > 1]
print(dups)
# user_id  product
# 1        A        3
# 2        B        2

# fix path 1 – aggregation (real duplicates, sum is the intent)
wide = bad.pivot_table(index="user_id", columns="product", values="qty", aggfunc="sum", fill_value=0)
```

Q3 · What does `margins=True` do in `pivot_table`, and what is `margins_name`?

Tags: Difficulty: Easy · Asked at: Flipkart, Cred, Swiggy, Walmart Labs, Target · Topic: `pivot_table` margins

The question (interviewer's verbatim phrasing):

I want my pivot table to include row totals and column totals. How do I do that in one call?

The 30-second answer: Pass `margins=True` to `pivot_table`. It adds an extra row and column to the result, each containing the aggregation across the other axis — by default labelled `"All"`. The label is configurable via `margins_name="Total"` (or whatever you want). Margins use the same `aggfunc` as the body of the table, which is the gotcha — `aggfunc="mean"` produces a margin row that is the mean of the underlying cells, not the mean of the means; pandas computes it from the raw rows, so the math is right but reviewers sometimes assume it is wrong.

Step-by-step thought process: 1. State the syntax directly. `df.pivot_table(index=..., columns=..., values=..., aggfunc=..., margins=True)`. The argument is binary; pass `True` to get the totals row and totals column. 2. Mention the rename. `margins_name="All"` is the default; many Indian reports prefer `"Total"`, `"Grand Total"`, or `"India"` for a country-rollup pivot. Pass the string you want; pandas does not localize. 3. Walk through how the totals are computed. They are NOT computed from the body cells — they are computed from the raw long-form rows. So `aggfunc="mean"` gives you the true mean of all underlying rows in the margin, not the mean of the cell means. For `sum` and `count`, the two would coincide anyway; for `mean`, `median`, and `std`

they differ and the from-row behavior is what you want. 4. Note the layout. The totals row appears at the bottom; the totals column appears at the right. The intersection — the grand total — is included.

Edge cases to mention: - With `aggfunc=["sum","count"]` (a list of functions), `margins=True` produces a margin per function — each gets its own totals row and column. - If your data has NaN values, the margin computation respects pandas's `skipna=True` default; this is consistent with `sum` and `mean` but can produce a margin that differs from a naive `df.sum().sum()` check. - Categorical columns with unused categories will get an `All` row that includes only observed values; the unobserved categories stay as empty rows if `observed=False`.

What NOT to say: - "`margins=True` is the same as `df.append(df.sum())`" — no; the margins are computed from the source rows, not from the body of the pivot. - "Margins only work with `aggfunc='sum'`" — wrong; they work with any `aggfunc`, including a list of functions.

Common follow-up: "If `aggfunc='mean'`, why does the bottom-right `All` cell sometimes look 'off' to the reviewer?" — Because it is the mean of all underlying rows, not the mean of the cell means. The two coincide only when each cell has the same row count. Showing that distinction is what a Senior Data-role candidate is expected to do.

```
import pandas as pd

sales = pd.DataFrame({
    "region": ["North","North","South","South","West","West","North"],
    "channel": ["online","store","online","store","online","store","online"],
    "revenue": [1200, 800, 1500, 600, 900, 1100, 400],
})

table = sales.pivot_table(
    index="region", columns="channel", values="revenue",
    aggfunc="sum", margins=True, margins_name="Total", fill_value=0,
)
print(table)
# channel  online  store  Total
# region
# North      1600    800   2400
# South      1500    600   2100
# West        900   1100   2000
# Total      4000   2500   6500
```

Q4 · How do you pivot on multiple value columns or multiple aggregation functions?

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Microsoft India, Goldman · Topic: `pivot_table` multi-value

The question (interviewer's verbatim phrasing):

I want a pivot table where each cell shows both the sum and the count for the revenue column, broken down by region and month. How do I do that?

The 30-second answer: Pass a list to `aggfunc` . `df.pivot_table(index="region", columns="month", values="revenue", aggfunc=["sum","count"])` returns a DataFrame with a two-level column MultiIndex — the outer level is the function name (`sum` , `count`) and the inner level is the month. Similarly, passing a list to `values` — `values=["revenue","units"]` — gives you both metrics side by side, again with a two-level column index. Mixing both (`values=["revenue","units"]` and `aggfunc=["sum","mean"]`) produces a three-level column index, which is fine but starts to feel like a report nobody will read.

Step-by-step thought process: 1. State the rule clearly. Both `values` and `aggfunc` accept either a scalar or a list. A scalar gives a single-level column index; a list gives a two-level column index whose outer level reflects the list. 2. Walk through the two-aggfunc case. `aggfunc=["sum","count"]` on `values="revenue"` puts `sum` and `count` at the top level and `month` underneath. Access individual blocks with `result["sum"]` or `result[("sum","2026-04")]` . 3. Walk through the multi-values case. `values=["revenue","units"]` on `aggfunc="sum"` puts `revenue` and `units` at the top level. Useful when you want two related metrics in one report. 4. Mention `aggfunc` as a dict for per-column functions. `aggfunc={"revenue":"sum","units":"mean"}` lets each value column have its own aggregation — useful when revenue should be summed but quantity should be averaged.

Edge cases to mention: - Dict-form `aggfunc` requires that every key in the dict appear in `values` ; passing a column that is not in `values` is silently ignored in older versions and raises in newer ones. - Multi-level column results are wide; pandas does not always render them well in Jupyter. `result.swaplevel(axis=1).sort_index(axis=1)` reorders the levels for a cleaner look. - `margins=True` combined with multi-level columns produces a margin per top-level group, which is usually what you want but the result can be confusing when reviewed.

What NOT to say: - "You have to do two separate `pivot_table` calls and merge them" — wrong; one call with a list `aggfunc` is the right tool. - "Multi-level column results are not real DataFrames" — they are; they just have a `MultiIndex` on the columns, which all pandas operations support.

Common follow-up: "How do you flatten a two-level column MultiIndex into a single-level for export to CSV or Excel?" — `result.columns = ["_".join(map(str, col)).strip("_") for col in result.columns.values]` joins the levels with an underscore. Or use `result.columns = result.columns.map("_".join)` if both levels are strings. Either way, you usually want to flatten before `.to_excel` .

```
import pandas as pd

sales = pd.DataFrame({
    "region": ["N","N","S","S","W","W","N","S"],
    "month": ["Apr","May","Apr","May","Apr","May","Apr","Apr"],
    "revenue": [1200, 800, 1500, 600, 900, 1100, 400, 700],
    "units": [3, 2, 5, 1, 4, 2, 1, 2],
})

table = sales.pivot_table(
    index="region", columns="month",
    values=["revenue","units"],
    aggfunc={"revenue":"sum","units":"mean"},
    fill_value=0,
)
# columns are a 2-level MultiIndex: (revenue/units) × (Apr/May)
```

Q5 · The "Data Track · the pivot they ask" question — pivot a transaction log by month

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Swiggy, Meesho, Walmart Labs · Topic: pivot scenario

The question (interviewer's verbatim phrasing):

Here is a transactions DataFrame with `customer_id`, `txn_date`, and `amount`. Give me a pivot table where rows are customers, columns are calendar months, cells are total spend, and missing months show as zero. Walk me through it.

The 30-second answer: Extract the month from `txn_date` into a new column (`Period[M]` is the cleanest dtype because it sorts and prints nicely), call `pivot_table` with `index="customer_id"`, `columns="month"`, `values="amount"`, `aggfunc="sum"`, and `fill_value=0`. That's six lines including the dtype conversion. Then sort the columns to make sure months print in chronological order, and reset the index if you need a flat frame for export.

Step-by-step thought process: 1. Talk through the dtype step first. `txn_date` is almost always parsed as a `datetime64[ns]` or a string. You want the month part. `.dt.to_period("M")` gives you a `Period[M]` series whose values sort and render as `"2026-04"`; alternatively `.dt.strftime("%Y-%m")` gives strings — fine for output, slightly slower to sort. 2. Call `pivot_table` with `aggfunc="sum"`. Be explicit — mean of money is the default-aggfunc trap that catches careless candidates. `fill_value=0` is the right zero because for revenue, missing means no spend, not unknown spend. 3. After the pivot, sort the columns. `pivot_table` orders columns in the order they appear in the source; if the data has April rows before March rows, the result reads April-then-

March. `.sort_index(axis=1)` fixes this. 4. Optional last step: `reset_index()` if downstream code wants `customer_id` as a regular column rather than the index. Most reporting code is happier with a flat frame.

Edge cases to mention: - Customers who never transacted will not appear at all in the pivot result. If the requirement is "all customers, even silent ones," `merge` the result with the full customer list with `how="right"` and `fillna(0)`. - If `txn_date` is in IST but stored as UTC, the month boundary at 12:30 AM IST can shift a transaction into the previous month silently. Convert to the business timezone before extracting the month: `df["txn_date"].dt.tz_convert("Asia/Kolkata").dt.to_period("M")`. - For a multi-year dataset, months from different years collide if you use `.dt.month` (which returns 1–12) instead of `.dt.to_period("M")` (which returns "2026-04"). Use the period dtype to avoid the collision.

What NOT to say: - "I'll use a `for` loop to bucket transactions by month" — that is the wrong answer; pivot exists for exactly this shape and the loop will be 50–500× slower on a real-size dataset. - "I'll do a `groupby` and then transpose with `.T`" — works for a single-column result, but loses you `fill_value` and produces a less idiomatic call. `pivot_table` is the one-line tool.

Common follow-up: "What if the question asks for cumulative spend per customer up to and including that month?" — Run the same pivot, then apply `.cumsum(axis=1)` on the result. The pivot gives you per-month spend; the cumsum across columns turns it into running total. Two lines, both vectorized.

```
import pandas as pd

txn = pd.DataFrame({
    "customer_id": [101, 101, 102, 102, 103, 101],
    "txn_date":    pd.to_datetime(["2026-04-03", "2026-05-12", "2026-04-08", "2026-04-20", "2026-05-01", "2026-04-15"]),
    "amount":     [1200, 800, 500, 700, 1500, 300],
})

txn["month"] = txn["txn_date"].dt.to_period("M")

monthly = txn.pivot_table(
    index="customer_id", columns="month", values="amount",
    aggfunc="sum", fill_value=0,
).sort_index(axis=1)

cumulative = monthly.cumsum(axis=1)
```

Section B — `melt` and Long-Form Data

The next five questions go the other direction — from wide to long. `melt` is the inverse operation of `pivot` for the simple cases. It takes a frame with many value columns and stacks them into two columns: one holding what used to be the column header, one holding what used to be the cell value.

Long form is the natural input for plotting libraries (seaborn, plotly), for SQL-like analysis (`groupby` on the variable name), and for relational storage where wide tables fight against schema evolution. The five questions below cover the `id_vars` vs `value_vars` split, the `var_name` and `value_name` renames, and the round-trip with `pivot` that takes you back.

Q6 · What does `pd.melt` do, and what are `id_vars` and `value_vars` ?

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Postman, Walmart Labs, Target · Topic: melt fundamentals

The question (interviewer's verbatim phrasing):

Explain what `pd.melt` does. What are `id_vars` and `value_vars` ?

The 30-second answer: `melt` reshapes a wide DataFrame into long form. `id_vars` is the list of columns that should stay as-is — they become the identifier columns on the long-form result. `value_vars` is the list of columns whose names should become row values in a new `variable` column, with their cell contents going into a new `value` column. The result has one row per (original row × value_var), so a 1000-row × 5-value-col frame becomes a 5000-row × (n_id_vars + 2) frame.

Step-by-step thought process: 1. Lead with the mental model. `melt` says "unstack these columns into a tall frame." For each row in the source and each column in `value_vars`, emit one row in the result that carries the `id_vars` values, the source column name, and the cell value. 2. Walk through the two parameters. `id_vars` is what you keep — typically the keys: `customer_id`, `region`, `date`. `value_vars` is what you unpivot — typically the metric columns: `q1_revenue`, `q2_revenue`, `q3_revenue`, `q4_revenue`. If you omit `value_vars`, every column not in `id_vars` is melted; if you omit `id_vars`, every column is a value_var. 3. State the canonical use case. Plotting libraries like seaborn want long form — one row per observation, one column per metric. Reports and SQL exports usually arrive in wide form. `melt` is the bridge. 4. Mention the result shape. If the source is `n` rows by `(k id_vars + m value_vars)` columns, the result is `n × m` rows by `(k + 2)` columns — the two extra columns being `variable` and `value`. This expansion can balloon memory if `m` is large; watch the row count.

Edge cases to mention: - If `value_vars` columns have different dtypes — say one is int and another is string — the resulting `value` column gets the `object` dtype, which is slower and uses more memory than a homogeneous typed column. Melt within compatible dtype groups when memory matters. - `melt` does not enforce that `value_vars` columns are the same dtype; mixing is permitted but you pay for it. - The default column names — `variable` and `value` — are usually not what you want in a finished report; rename them with `var_name=` and `value_name=` (next question).

What NOT to say: - "`melt` is the same as `stack`" — they are related but not identical. `melt` is column-based and produces a flat DataFrame; `stack` is index-based and produces a Series with a MultiIndex. Different tools, different shapes. - "`melt` aggregates values" — it does not; `melt` is pure reshape. Aggregation comes from `groupby` after the melt if you need it.

Common follow-up: "What happens if you call `melt` with no arguments?" — Every column is treated as a `value_var`; the result has only the `variable` and `value` columns. Almost never what you want, but pandas does not error — the implicit defaults can produce a surprising result, so always pass at least `id_vars` explicitly.

```
import pandas as pd

wide = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "q1_revenue": [1200, 800, 1500],
    "q2_revenue": [1100, 950, 1400],
    "q3_revenue": [1000, 1100, 1300],
})

long = pd.melt(wide, id_vars=["customer_id"], value_vars=["q1_revenue", "q2_revenue", "q3_revenue"])
# customer_id  variable  value
# 1           q1_revenue  1200
# 1           q2_revenue  1100
# 1           q3_revenue  1000
# 2           q1_revenue   800
# ...
```

Q7 · How do you rename the `variable` and `value` columns produced by `melt` ?

Tags: Difficulty: Easy · Asked at: Flipkart, Cred, Razorpay, Postman, Target · Topic: melt var_name value_name

The question (interviewer's verbatim phrasing):

When I call `pd.melt`, I get columns named `variable` and `value`. How do I rename them inline?

The 30-second answer: Pass `var_name="quarter"` and `value_name="revenue"` to the `melt` call. These two parameters set the column labels for the two new columns the operation creates. The defaults — "`variable`" and "`value`" — are rarely what a downstream consumer expects, so naming them inline is good practice. The alternative is to

`.rename(columns={"variable": "quarter", "value": "revenue"})` after the call, which works but adds a step and reads less clearly.

Step-by-step thought process: 1. Name the parameters. `var_name` (singular) sets the column that holds the old column names; `value_name` (singular) sets the column that holds the cell values. Both default to `"variable"` and `"value"` respectively. 2. Explain why this matters in the report. A column literally named `variable` reads like a placeholder. Renaming to `quarter`, `metric`, `period`, or whatever the domain dictates makes downstream `groupby` and join code self-documenting. 3. Show the typical Indian-report shape. Frame has `customer_id`, `q1_revenue`, `q2_revenue`, `q3_revenue`, `q4_revenue`; the melt should produce `customer_id`, `quarter`, `revenue`. Setting `var_name="quarter"` and `value_name="revenue"` is the difference between a clean output and one that needs a rename pass. 4. Mention the gotcha when `var_name` collides with an existing `id_vars` column. pandas raises in pandas 2.x if the chosen `var_name` overlaps with an `id_vars` entry. Pick a name that does not already exist on the frame.

Edge cases to mention: - `var_name` accepts only a string in current pandas; older docs mention list support for multi-level `value_vars` but that path is now via `pd.wide_to_long` (next question) instead. - The renamed `variable` column is still an `object` dtype carrying the original column names. If those names follow a pattern — `"q1_revenue"`, `"q2_revenue"` — you usually want to extract the quarter and the metric into two columns with `.str.extract` after the melt. - Setting `value_name` to a name that already exists in `id_vars` raises a `ValueError`.

What NOT to say: - "Just rename after with `.rename`" — works but it's a two-step path when one step would do. Senior candidates use `var_name` / `value_name` inline. - "The defaults `variable` and `value` are fine" — fine for a notebook scratch cell, not for a frame that will be joined, exported, or read by another human.

Common follow-up: "What if the wide columns follow a pattern like `q1_revenue`, `q1_units`, `q2_revenue`, `q2_units` — how do you split that into two long-form columns?" — Use `pd.wide_to_long` with `stubnames=["revenue", "units"]`, or melt then `.str.extract(r"q(\d+)_(\w+)")` to split the variable column into `quarter` and `metric`. `wide_to_long` is more declarative; the extract path is more flexible. Both are covered in the next two questions.

```
import pandas as pd

wide = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "q1_revenue": [1200, 800, 1500],
    "q2_revenue": [1100, 950, 1400],
})

long = pd.melt(
    wide,
    id_vars=["customer_id"],
    value_vars=["q1_revenue", "q2_revenue"],
    var_name="quarter",
    value_name="revenue",
)
```

Q8 · `pd.melt` vs `pd.wide_to_long` — when do you reach for which?

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, Walmart Labs, Postman, Razorpay ·

Topic: `wide_to_long`

The question (interviewer's verbatim phrasing):

Suppose I have a frame with columns `revenue_q1`, `revenue_q2`, `units_q1`, `units_q2`, and I want long form with `quarter`, `revenue`, and `units` as separate columns. Why is `wide_to_long` better than `melt` here?

The 30-second answer: `wide_to_long` understands stub-name patterns. If your columns are `revenue_q1`, `revenue_q2`, `units_q1`, `units_q2`, you can pass `stubnames=["revenue","units"]`, `i="customer_id"`, `j="quarter"`, and `suffix="q\\d+"`, and pandas extracts the suffix as the new `quarter` column and keeps `revenue` and `units` as separate value columns — exactly the shape you want. `melt` would put both metrics into one `value` column with `variable` carrying `revenue_q1`, `revenue_q2`, etc., and you'd then have to split and pivot back to recover the two-metric shape. `wide_to_long` is the one-call answer when columns follow a `stub_suffix` pattern.

Step-by-step thought process: 1. Anchor on the problem `wide_to_long` solves. `melt` reduces everything to one variable column and one value column. When you have two related metrics — revenue and units — that share a suffix axis — q1, q2 — you don't want them collapsed into a single value column. 2. Walk through the four key arguments. `stubnames` is a list of the prefixes that become value columns. `i` is the identifier column (same role as `id_vars` in `melt`). `j` is the name of the new column that captures the suffix. `suffix` is a regex that matches the part of the column name to extract. 3. Show the contract. A column `revenue_q1` is matched against `stubname="revenue"` +

`sep="_"` (default `_`) + `suffix="q\\d+"`. Pandas extracts the suffix, sets `j="quarter"` to `"q1"`, and assigns the cell value to the `revenue` column in the result. The same row gets the `units_q1` cell in the `units` column. 4. Mention the `sep` argument. Default is `_` (underscore between stub and suffix). If the columns use a different separator or none at all, override `sep`.

Edge cases to mention: - `wide_to_long` is strict about the `i` column. After the reshape, the result has a MultiIndex of `(i, j)`. Call `.reset_index()` if you want a flat frame. - The default `suffix=r"\d+"` matches only integer suffixes (`revenue_1`, `revenue_2`); for `q1`, `q2`, override to `suffix=r"q\d+"` as in the example. - If a stub appears with some suffix values but not others (e.g., `revenue_q1`, `revenue_q2`, `units_q1` but not `units_q2`), the missing combination shows up as NaN in the result. That's usually correct, but it's a thing to scan for.

What NOT to say: - "`wide_to_long` is the same as `melt` with extra arguments" — they are different operations with different result shapes; `wide_to_long` produces a multi-metric long form, `melt` does not. - "I'll do a `melt` then `pivot` to get the multi-metric shape" — works but it's a three-step path when `wide_to_long` does it in one declarative call.

Common follow-up: "*What if I have three metrics — revenue, units, returns — across four quarters? Same pattern?*" — Yes, just pass `stubnames=["revenue","units","returns"]`. The mechanism scales to any number of stubs as long as they share the same suffix axis.

```
import pandas as pd

wide = pd.DataFrame({
    "customer_id": [1, 2],
    "revenue_q1": [1200, 800],
    "revenue_q2": [1100, 950],
    "units_q1": [3, 2],
    "units_q2": [2, 3],
})

long = pd.wide_to_long(
    wide, stubnames=["revenue","units"], i="customer_id", j="quarter", sep="_", suffix=r"q\d+",
).reset_index()

# customer_id  quarter  revenue  units
# 1           q1       1200      3
# 1           q2       1100      2
# 2           q1        800      2
# 2           q2        950      3
```

Q9 · Round-trip: pivot a melted frame back to wide form

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Cred, Postman, JP Morgan · Topic: pivot melt round-trip

The question (interviewer's verbatim phrasing):

I have a long-form DataFrame with `customer_id`, `quarter`, and `revenue`. How do I get back to the wide form with one column per quarter?

The 30-second answer: Call `df.pivot(index="customer_id", columns="quarter", values="revenue")`. That's the inverse of the `melt` you did to produce the long form. If the long form has only one row per (customer, quarter) pair, `pivot` works directly; if there are duplicates, use `pivot_table` with `aggfunc="sum"` (or whatever the right aggregation is). The round-trip is intentional and well-supported — `melt` and `pivot` are designed to be inverses for the clean case.

Step-by-step thought process: 1. Name the inverse relationship. `melt` takes wide to long; `pivot` takes long back to wide. As long as each (id_var, variable) pair occurs once in the long form, the round-trip is exact and reversible. 2. Walk through the call. `df.pivot(index="customer_id", columns="quarter", values="revenue")` — `index` is what was your `id_vars`, `columns` is your `variable` column, `values` is your `value` column. 3. Cover the duplicate-collision case. If you melted a frame that had two rows for the same customer-quarter — perhaps the wide form had two columns mapped to the same quarter accidentally — `pivot` will crash. `pivot_table` with `aggfunc="sum"` is the recovery path, with the same warning as Q2 about the default `mean`. 4. Note the post-pivot cleanup. The result of `pivot` has `customer_id` as the index and `quarter` as the columns name. Call `.reset_index()` to flatten, and `.rename_axis(columns=None)` if you want to drop the `quarter` label from the column axis name for export.

Edge cases to mention: - The column order in the result is determined by the order in which the values appear in the source, not by alphabetic sort. `.sort_index(axis=1)` after pivot if you need chronological columns. - If the long form is the result of a `groupby` + aggregation, the round-trip is not exact in dtype — counts might come back as int, sums as float, etc. Re-cast if downstream code is strict about dtypes. - For very wide round-trips — say 5000 distinct quarters — pivot allocates a 5000-column frame; that's usually fine, but on memory-constrained workers it can balloon. Stay in long form when the cardinality of the column axis is large.

What NOT to say: - "The round-trip isn't reliable" — it is, for the clean case. Be clear about when it's reliable (no duplicates) vs when you need `pivot_table`. - "Just iterate and reconstruct manually" — no; pivot is the right tool and is vectorized.

Common follow-up: "How do you write the wide form to a CSV with `quarter` as a header row, not as the column axis name?" —

`df.pivot(...).reset_index().rename_axis(columns=None).to_csv(...)`. The `rename_axis(columns=None)` removes the `quarter` label from the columns axis so the CSV header row is clean.

```
import pandas as pd

long = pd.DataFrame({
    "customer_id": [1, 1, 2, 2],
    "quarter":     ["q1", "q2", "q1", "q2"],
    "revenue":     [1200, 1100, 800, 950],
})

wide = long.pivot(index="customer_id", columns="quarter", values="revenue").reset_index().rename_axis(None, axis=1)

# customer_id  q1    q2
# 1            1200  1100
# 2             800   950
```

Q10 · Why does long form work better as input for plotting and SQL-like analysis?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Microsoft India, Walmart Labs, Postman · Topic: long form rationale

The question (interviewer's verbatim phrasing):

Why would you melt a perfectly readable wide-form report into long form before plotting it in seaborn or grouping it for analysis?

The 30-second answer: Long form has one row per observation. That maps directly to the relational data model: each row carries **who**, **when**, **what metric**, **what value**. seaborn, plotly, and SQL all expect this shape because every aesthetic mapping (x, y, color, facet) becomes a column reference, and every aggregation (groupby, window) becomes a column key. Wide form forces the consumer to iterate over column names — clumsy in pandas, impossible in SQL. Long form is also more schema-stable: adding a new quarter is a row append, not a column add, which keeps downstream consumers from breaking.

Step-by-step thought process: 1. Frame the rationale as "one observation per row." A row in long form carries everything needed to identify that observation — the id, the variable name, the value. Wide form scatters the same observation across columns; the variable identity is embedded in the column header, which is awkward to extract programmatically. 2. Connect to plotting. `sns.lineplot(data=df, x="quarter", y="revenue", hue="customer_id")` — every argument is a column name. Wide form would force `sns.lineplot(data=df[["q1_revenue", "q2_revenue", "q3_revenue"]])` which loses the customer-id and quarter axes as first-class data. 3. Connect to SQL. SQL works in long form natively. `SELECT customer_id, quarter, SUM(revenue) FROM ... GROUP BY 1, 2` is a sensible query; the equivalent on a wide table requires `UNION ALL` of one query per column, which is brittle and explodes the query length. 4. Connect to schema stability. A wide frame with one column per quarter changes shape every quarter — your ETL has to add a column, your views have to be edited,

your dashboards need re-binding. A long frame with a `quarter` column just gets new rows; the schema is invariant. That is why warehouses store data long and ship it wide only at the BI layer.

Edge cases to mention: - Long form is larger on disk and in memory — $n_rows \times n_metrics$ rows vs $n_rows \times 1$ row — but the overhead is usually small once you account for compression and that wide form often has many NaN cells. - Some traditional reporting tools (Excel pivots, old BI dashboards) want wide form. The pattern is to store long, pivot to wide only at the render layer. - For very large multi-metric frames (50+ metrics), long form can be slower for column-wise operations because the metric axis becomes a row-wise filter. The trade is real; long form is the default but not the universal answer.

What NOT to say: - "Long form is always better" — it's the right default, but not absolute. Wide form is right for human-readable reports and small dashboards. - "Wide form is wrong" — it's correct for the right consumer; the issue is matching the form to the consumer.

Common follow-up: "How does long form interact with `groupby` ? Why is it the natural input?" — Because `groupby("variable")` (or `groupby(["customer_id", "quarter"])`) gives you exactly the grouping the analysis needs. The metric axis is just another column you can group by, filter on, or join with. Wide form makes that grouping impossible without melt-first.

```
import pandas as pd
import seaborn as sns

wide = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "q1_revenue": [1200, 800, 1500],
    "q2_revenue": [1100, 950, 1400],
    "q3_revenue": [1000, 1100, 1300],
})
long = wide.melt(id_vars="customer_id", var_name="quarter", value_name="revenue")

# seaborn now reads as if it were SQL
# sns.lineplot(data=long, x="quarter", y="revenue", hue="customer_id")
# the same call on `wide` would not work — hue cannot reference customer_id across columns
```

Section C — `stack` and `unstack`

The next five questions are about `stack` and `unstack` — the MultiIndex-aware sisters of `melt` and `pivot`. They operate on the column index and the row index respectively, moving levels between the two axes. They produce Series or DataFrames with MultiIndex labels rather than the flat shape `melt` and `pivot` give. They are the right tool when the source already has a MultiIndex (typical after a `groupby` with multiple keys) and you want to flip a level from rows to columns or back. The five questions cover the level parameter, the `dropna` interaction, the relationship with `pivot`, and the patterns that recur in Indian Data-role interviews.

Q11 · What does `.stack()` do to a DataFrame, and what does `.unstack()` do?

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Walmart Labs, Target · Topic: stack unstack basics

The question (interviewer's verbatim phrasing):

Explain `.stack()` and `.unstack()` in pandas. What does each do to the index?

The 30-second answer: `stack` takes a column level and pushes it into the row index, producing a longer, narrower frame (often a Series if you started with a single column level). `unstack` is the inverse — it takes a row index level and pulls it up into the column index, producing a wider frame. They operate on MultiIndex levels rather than on data columns, so they are the natural reshape for results that already have a hierarchical index — typically what `groupby(['a','b']).sum()` returns.

Step-by-step thought process: 1. State the structural difference from melt/pivot. `melt` and `pivot` work with data columns by name. `stack` and `unstack` work with index levels by position or name. They move information between the row index and the column index without naming individual columns. 2. Walk through a concrete example. Start with `groupby(["region","month"])` `["revenue"].sum()` — that gives a Series with a 2-level MultiIndex on the rows. `.unstack("month")` pulls the `month` level up into the columns, producing a DataFrame with regions on rows and months on columns. That is the same shape `pivot_table` would have produced from the original frame, just arrived at via a different path. 3. Mention the inverse. `.unstack().stack()` should return to the original Series for the clean case. If there are missing combinations, the round-trip drops NaN cells by default; pass `dropna=False` to preserve them. 4. Note the return type rules. `stack` on a single-level-column DataFrame returns a Series. `unstack` on a single-level-index Series returns a DataFrame. `stack` and `unstack` on MultiIndex inputs return MultiIndex outputs — the level count changes by one in each direction.

Edge cases to mention: - `.stack()` without an argument stacks the innermost (last) column level. To stack a specific level, pass `level=` by name or position. - `unstack` drops the value for any (row index combination, column value) that doesn't exist in the source; in older pandas this fell back to NaN with `dropna=True` interpretation, which dropped all-NaN rows or columns. - In pandas 2.1+, `stack` was given a `future_stack=True` flag that changes some NaN behavior; in 2.2 that became the default. Use `future_stack=True` for forward-compatible code.

What NOT to say: - "`stack` and `melt` are the same" — they overlap but are not the same. `melt` produces a flat DataFrame; `stack` produces a Series or a MultiIndex DataFrame. - "`stack` flattens the column index" — no; it moves one level of the column index into the row index. The column index loses a level but the row index gains one.

Common follow-up: "If you have a DataFrame whose columns are a 2-level MultiIndex, what does `df.stack()` produce by default?" — A DataFrame with the inner column level moved into the row

index. The result is one level taller on the rows and one level shorter on the columns. The values are unchanged; only the indexing shape changed.

```
import pandas as pd

sales = pd.DataFrame({
    "region":  ["N","N","S","S"],
    "month":   ["Apr","May","Apr","May"],
    "revenue": [1200, 1100, 800, 950],
})

# groupby produces a Series with a 2-level MultiIndex
grouped = sales.groupby(["region","month"])["revenue"].sum()

wide = grouped.unstack("month")      # month moves to columns
back = wide.stack(future_stack=True) # month moves back to rows; original Series shape
```

Q12 · What does the `level=` parameter on `stack` and `unstack` control?

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Postman, Razorpay, Microsoft India · Topic: stack level parameter

The question (interviewer's verbatim phrasing):

A DataFrame has columns at three levels — `metric`, `quarter`, `currency`. How do I unstack the `quarter` level specifically rather than just the innermost?

The 30-second answer: Pass `level="quarter"` (or the integer position) to `.unstack()`. The `level` parameter picks which level of the column index to move to the rows (for `stack`) or row index to move to the columns (for `unstack`). It accepts a string (the level name), an integer (the position from the outside, with 0 being the outermost), or a list of either to move several levels at once. The default is the innermost level — index `-1` — which is the most common case but rarely the only useful one in real Indian-report DataFrames.

Step-by-step thought process: 1. State the default. Without `level=`, `stack` stacks the last (innermost) column level; `unstack` unstacks the last (innermost) row level. That is the convenient default for the typical post-`groupby` shape. 2. Walk through the override. `unstack(level="quarter")` moves the quarter level to the columns regardless of where it sits in the MultiIndex hierarchy. `unstack(level=0)` moves the outermost level. `unstack(level=[0,1])` moves both top levels at once, producing two new column levels. 3. Anchor on the use case. After a `groupby(["region","month","channel"]).sum()`, the result has all three keys on the row index. If the question wants regions on rows but month and channel as a 2-level column header, `unstack(level=["month","channel"])` is the one-call answer. 4. Cover the

names-vs-positions trade. Strings are safer because they survive level reordering; positions are shorter but break if a level is added or moved upstream. For interview code, prefer names.

Edge cases to mention: - Passing a name that does not exist on the MultiIndex raises `KeyError`.

The error message includes the available level names — useful when you're debugging. -

`stack(level=[0, 2])` is legal and stacks two levels at once into the row index, producing a result with two extra row index levels. - After unstacking, the new column MultiIndex may have NaN-filled gaps for combinations that did not exist in the source; use `fill_value=` to override.

What NOT to say: - "`level=` is just for picking the order of unstacking" — it picks which level, not just the order. - "You always pass an integer" — strings are safer; recommend them in production code.

Common follow-up: "What does `unstack(level=[0,1])` do compared to `unstack(level=0).unstack(level=0)`?" — They produce the same result in most cases but the single-call form is faster and clearer. The two-call form is occasionally needed when you want to perform an operation in between; otherwise prefer the single call with a list.

```
import pandas as pd

idx = pd.MultiIndex.from_product(
    [{"N","S"}, ["Apr","May"], ["online","store"]],
    names=["region","month","channel"],
)
s = pd.Series([1200, 800, 1100, 700, 1500, 600, 950, 1100], index=idx, name="revenue")

by_region_month = s.unstack(level=["month","channel"])
by_region_month_outer = s.unstack(level=0) # region as columns
```

Q13 · How do NaN values behave when you unstack a sparse MultiIndex?

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs, Postman, Microsoft India ·

Topic: stack unstack NaN

The question (interviewer's verbatim phrasing):

I have a Series indexed by (customer_id, month) where not every customer has every month. When I unstack, where do the gaps go and how do I control them?

The 30-second answer: The gaps become NaN cells in the resulting wide DataFrame. Every (customer_id, month) combination that does not exist in the source becomes a NaN in the unstacked result. You control them in two ways: `fill_value=0` replaces NaN with a constant at unstack time; `dropna=False` (which is the default in newer pandas) preserves all-NaN columns or rows. For

currency or count data, `fill_value=0` is usually the right answer; for measurements that genuinely could be missing — sensor data, survey responses — leaving NaN is correct.

Step-by-step thought process: 1. State the cause. `unstack` produces the full Cartesian product of the row-index level being moved with the existing column levels. Cells that don't appear in the source can't have a value, so pandas fills them with NaN. 2. Walk through `fill_value`.

`unstack(fill_value=0)` replaces NaN with 0 in the result. This is the right answer for revenue, units, counts — any quantity where "missing" semantically means "zero". Note that this changes the dtype: an integer series unstacked with `fill_value=0` stays int, but with default NaN fill, it gets promoted to float (because NaN is float). 3. Walk through `dropna`. The `dropna` flag controls whether all-NaN columns or rows in the result get dropped. Default behavior changed across pandas versions; in 2.2 with `future_stack=True` semantics, the behavior is to keep all-NaN entries, which is more predictable. 4. Note the dtype consequence. NaN forces float dtype on integer columns. If you want to preserve int dtype, use `fill_value=0` (or another int) at unstack time, or use a nullable integer dtype (`Int64`) which supports NaN natively.

Edge cases to mention: - Categorical columns with unused categories will appear as NaN columns in the unstacked result. Use `.cat.remove_unused_categories()` first if you don't want them. - `fill_value` is applied only to cells created by the unstack operation, not to existing NaN values in the source. Use `.fillna(0)` on the source first if you want to fill pre-existing NaN as well. - For datetime columns, `fill_value=pd.NaT` is the equivalent of `fill_value=0` for numeric columns.

What NOT to say: - "Just use `.fillna(0)` after unstack" — works but is less efficient; `fill_value=0` at unstack time avoids the dtype promotion to float in the first place. - "NaN cells mean the source has bad data" — no; they mean the Cartesian combination did not exist. That's expected for sparse data and is information you might want to preserve.

Common follow-up: "What's the dtype of an unstacked int Series with default NaN fill vs `fill_value=0`?" — Default: float64 (because NaN forces float). With `fill_value=0`: int64 (no NaN introduced). For Indian revenue or count data, this matters because downstream code may assume int.

```
import pandas as pd

s = pd.Series(
    [1200, 1100, 800, 950, 1500],
    index=pd.MultiIndex.from_tuples(
        [("Anil","Apr"), ("Anil","May"), ("Beena","Apr"), ("Beena","May"), ("Chetan","Apr")],
        names=["customer","month"],
    ),
    name="revenue",
)

with_nan = s.unstack("month")          # Chetan/May is NaN; dtype is float64
with_zero = s.unstack("month", fill_value=0)  # Chetan/May is 0; dtype is int64
```

Q14 · `stack` and `unstack` vs `pivot` and `melt` — when do you pick which pair?

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Microsoft India, Postman, Goldman · Topic: stack vs pivot

The question (interviewer's verbatim phrasing):

Both `pivot` and `unstack` can produce a wide-form DataFrame from a long-form one. Both `melt` and `stack` can go the other way. Which pair do you reach for?

The 30-second answer: Pick by the shape you start with. If you start with a flat DataFrame and want a wide-form report, use `pivot` or `pivot_table` — both take data columns as arguments. If you start with a MultiIndex Series or DataFrame (typical after `groupby` with multiple keys), use `unstack` — it takes index levels. Going the other way, `melt` starts from a flat DataFrame; `stack` starts from a MultiIndex DataFrame. The operations overlap in capability but not in argument shape; matching the operation to the input shape produces cleaner, faster code.

Step-by-step thought process: 1. Anchor on input shape. `pivot` and `melt` operate on column names. `unstack` and `stack` operate on index levels. The input shape — flat vs MultiIndex — is the deciding factor. 2. Walk through the post-groupby case. `df.groupby(["region", "month"])["revenue"].sum()` returns a Series with a 2-level MultiIndex. The natural reshape is `.unstack("month")` because `month` is an index level, not a column. Reaching for `pivot` here forces a `.reset_index()` first. 3. Walk through the flat-frame case. A long-form DataFrame with `region`, `month`, `revenue` columns is naturally pivoted with `df.pivot(index="region", columns="month", values="revenue")`. Using `unstack` here forces a `.set_index(["region", "month"])` first. 4. Note the equivalence. The two paths produce the same result on the same data, just with different setup. Match the operation to the input; don't fight the shape.

Edge cases to mention: - `pivot_table` is the only one of the four that aggregates. `stack`, `unstack`, `pivot`, and `melt` are all pure reshapes; they do not collapse duplicates. - For multi-level reshapes — moving two levels at once — `unstack(level=[0,1])` is the cleaner call; `pivot` does not support multi-level pivots without chained calls. - After `unstack`, the result often has a column index axis name (e.g., `month`) that downstream code (CSV export, plotting) doesn't expect. `.rename_axis(columns=None)` clears it.

What NOT to say: - "They're all the same, just use whichever" — they overlap in result but not in argument shape; using the wrong one adds boilerplate. - "`pivot` is always faster" — performance is comparable; the right pick is by readability and fit to input shape, not by speed.

Common follow-up: "What does the result of `groupby([...]).sum().reset_index().pivot(...)` produce vs `groupby([...]).sum().unstack(...)` — are they the same?" — Same numerical result; different

intermediate steps. The `unstack` path avoids the `.reset_index()` and is one fewer line. Both are correct.

```
import pandas as pd

sales = pd.DataFrame({
    "region":  ["N","N","S","S","W","W"],
    "month":   ["Apr","May","Apr","May","Apr","May"],
    "revenue": [1200, 1100, 800, 950, 900, 1100],
})

# Path A – flat frame, pivot
wide_pivot = sales.pivot(index="region", columns="month", values="revenue")

# Path B – MultiIndex via groupby, unstack
wide_unstack = sales.groupby(["region","month"])["revenue"].sum().unstack("month")

# both produce the same DataFrame
```

Q15 · How do you flatten a MultiIndex columns result after `unstack` for CSV export?

Tags: Difficulty: Easy · Asked at: Cred, Flipkart, Swiggy, Walmart Labs, Postman · Topic: MultiIndex flatten

The question (interviewer's verbatim phrasing):

After `pivot_table` or `unstack`, my DataFrame has a 2-level column MultiIndex and I need to write it to CSV. How do I flatten the columns?

The 30-second answer: Assign a new `columns` attribute. The idiom is `df.columns = ["_".join(map(str, col)).strip("_") for col in df.columns.values]`, which joins each tuple of column levels with an underscore and strips leading or trailing underscores left by empty levels. For two-string-level cases, the shorter form is `df.columns = df.columns.map("_".join)`. After flattening, the column index is single-level and `.to_csv` produces a clean header row.

Step-by-step thought process: 1. Acknowledge the problem.

`pivot_table(values=["revenue","units"], ...)` produces columns like `("revenue","Apr"), ("revenue","May"), ("units","Apr"), ("units","May")`. Writing that to CSV produces two header rows, which most consumers do not expect. 2. Walk through the comprehension version. `["_".join(map(str, col)).strip("_") for col in df.columns.values]` handles the mixed-type case where one level is a string (`"revenue"`) and

another is a non-string (a date, a number). `map(str, col)` coerces both; `.strip("_")` cleans up if one level is the empty string (which happens for total rows from `margins=True`). 3. Walk through the cleaner version. If both levels are strings, `df.columns = df.columns.map("_".join)` is shorter and reads better. Use this when you control the dtypes upstream. 4. Note the alternative: `df.columns = df.columns.get_level_values(0)`. This drops one level entirely instead of joining. Useful when one level is just a redundant header (e.g., everything is `revenue`).

Edge cases to mention: - For a 3-level column MultiIndex, the same comprehension works because `col` is a tuple of length 3. The join handles any arity. - `margins=True` results in margin column tuples like `("revenue", "All")` or sometimes `("All", "")`. The `.strip("_")` handles the empty-string case cleanly. - If you'd rather flatten by keeping one level only, `df.columns = df.columns.droplevel(0)` removes the outer level and leaves the inner.

What NOT to say: - "Just call `df.to_csv` and pandas will figure it out" — pandas will write the multi-level header, which is technically valid CSV but most consumers (Excel, downstream parsers, Indian-MNC reporting tools) treat as two rows of headers and misalign. - "Use `pd.DataFrame(df.values, columns=...)`" — re-creates the frame from values, losing the index. A simple `df.columns = ...` assignment is in-place and cheaper.

Common follow-up: "What if my second level is a Period or Timestamp, not a string?" — `map(str, col)` handles it because `str(pd.Period("2026-04", "M"))` returns `"2026-04"` and `str(pd.Timestamp("2026-04-01"))` returns `"2026-04-01 00:00:00"`. For periods you usually want the period string; for timestamps you might want `col[1].strftime("%Y-%m-%d")` to drop the time.

```
import pandas as pd

sales = pd.DataFrame({
    "region": ["N", "N", "S", "S", "W", "W"],
    "month":  ["Apr", "May", "Apr", "May", "Apr", "May"],
    "revenue": [1200, 1100, 800, 950, 900, 1100],
    "units":   [3, 2, 5, 1, 4, 2],
})

wide = sales.pivot_table(
    index="region", columns="month", values=["revenue", "units"], aggfunc="sum", fill_value=0,
)

wide.columns = ["_".join(map(str, c)).strip("_") for c in wide.columns.values]
# columns are now: revenue_Apr, revenue_May, units_Apr, units_May
wide.reset_index().to_csv("/tmp/sales_wide.csv", index=False)
```


Section D — Reshape Patterns: crosstab, explode, json_normalize

The last five questions cover the reshape operations that sit outside the pivot/melt/stack triangle but show up constantly in Indian Data-role interviews. `crosstab` is the frequency-table specialist — a pivot that counts (or aggregates) and supports a normalize knob for proportions. `explode` turns a list-column into multiple rows, the canonical fix for "we stored a CSV inside a cell". `json_normalize` and the `.to_list()` plus `DataFrame()` pattern handle the "column of dicts" problem that API responses force on you. Five questions, five recurring patterns.

Q16 · What is `pd.crosstab` and how does it differ from `pivot_table` ?

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Walmart Labs, Postman, JP Morgan · Topic: crosstab basics

The question (interviewer's verbatim phrasing):

Explain `pd.crosstab` . How is it different from `pivot_table` and when do you use it?

The 30-second answer: `crosstab` is a specialized `pivot_table` for frequency analysis. It takes Series (or columns) as `index=` and `columns=` arguments and counts the co-occurrences by default — exactly what you want for a frequency table. It also supports `values=` plus `aggfunc=` for an aggregation other than count, but the calling convention treats inputs as columnar Series rather than as references inside a DataFrame. Use `crosstab` when you want a frequency table; use `pivot_table` when you want a value-aggregation table from an already-flat DataFrame.

Step-by-step thought process: 1. State the default behavior. `pd.crosstab(df["region"], df["channel"])` returns a frequency table — how many rows had each (region, channel) combination. That is functionally `df.groupby(["region", "channel"]).size().unstack()` in one call. 2. Walk through the `values + aggfunc` extension. `pd.crosstab(df["region"], df["channel"], values=df["revenue"], aggfunc="sum")` aggregates revenue by (region, channel) — same as `pivot_table` but with the Series-style call. 3. State the comparison plainly. `crosstab` and `pivot_table` overlap; the practical difference is the convention. Reach for `crosstab` when the question is "how often did X co-occur with Y", and for `pivot_table` when the question is "what's the sum/mean/etc of Z by X and Y". 4. Mention the `normalize` parameter. This is `crosstab` 's killer feature for frequency analysis — `normalize="index"` returns row proportions, `normalize="columns"` returns column proportions, `normalize="all"` returns proportions over the grand total. `pivot_table` has no equivalent.

Edge cases to mention: - `crosstab` accepts lists for `index` and `columns` to produce MultiIndex rows or columns — `pd.crosstab([df["region"], df["state"]],`

`df["channel"])` gives a 2-level row index. - The default `aggfunc` for `crosstab` is implicitly `count` when no `values` is given. When `values` is given, you must also pass `aggfunc`; otherwise pandas raises. - `margins=True` works the same way as on `pivot_table` — adds a totals row and column. `margins_name="Total"` renames the default `"All"`.

What NOT to say: - "`crosstab` is just a shortcut for `pivot_table`" — close but misleading; the default count behavior and the `normalize` parameter make it a distinct tool worth knowing by name. - "`crosstab` only counts" — no; it aggregates when you pass `values` and `aggfunc`.

Common follow-up: "How is `pd.crosstab(a, b)` different from `df.groupby([a, b]).size().unstack()`?" — Functionally equivalent for the count case. The `crosstab` call is shorter and supports `normalize`, `margins`, and `margins_name` directly. The groupby path is more flexible if you need intermediate steps. For a one-shot frequency table, `crosstab` is the right tool.

```
import pandas as pd

txn = pd.DataFrame({
    "region": ["N", "N", "S", "S", "W", "W", "N", "S"],
    "channel": ["online", "store", "online", "store", "online", "store", "online", "online"],
    "revenue": [1200, 800, 1500, 600, 900, 1100, 400, 700],
})

# frequency table
freq = pd.crosstab(txn["region"], txn["channel"])
# channel  online  store
# region
# N          2      1
# S          2      1
# W          1      1

# aggregated table (matches pivot_table)
rev = pd.crosstab(txn["region"], txn["channel"], values=txn["revenue"], aggfunc="sum")
```

Q17 · How does the `normalize=` parameter on `crosstab` work?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Postman, Microsoft India · Topic: `crosstab` `normalize`

The question (interviewer's verbatim phrasing):

Walk me through `normalize="index"` vs `normalize="columns"` vs `normalize="all"` in `pd.crosstab`. What do the resulting cells represent?

The 30-second answer: `normalize="all"` divides every cell by the grand total — each cell is the share of the whole. `normalize="index"` divides each cell by its row sum — each cell is its share of the row, so rows sum to 1.0. `normalize="columns"` divides each cell by its column sum — each cell is its share of the column, so columns sum to 1.0. Pick the one whose denominator answers the question you're actually trying to answer.

Step-by-step thought process: 1. Name each option's denominator. `"all"` → grand total. `"index"` → row sum. `"columns"` → column sum. `True` is an alias for `"all"`. 2. Walk through a concrete read. For a (region, channel) crosstab of order counts, `normalize="index"` reads as "within each region, what fraction of orders came from each channel". `normalize="columns"` reads as "within each channel, what fraction of orders came from each region". They answer different business questions; picking the wrong one gives a misleading dashboard. 3. Show the math. If the table has row sums `[100, 200, 50]` and you pick `normalize="index"`, every cell in row 1 is divided by 100, every cell in row 2 by 200, and so on. The result is rows that each sum to 1.0 — proportions within row. 4. Note that `normalize` plays with `margins=True` — the margins are also normalized, which can be confusing. The bottom-right cell is always 1.0 when `normalize="all"`; the right margin is row sums of the normalized table, which is 1.0 for every row under `normalize="index"`.

Edge cases to mention: - `normalize=True` is the same as `normalize="all"` — kept for historical reasons. - The result is always float; integer counts are promoted because proportions are real numbers. No way to keep ints because rows or columns might not sum to integer multiples. - For a sparse crosstab (many zero cells), `normalize="index"` on a row that sums to zero produces NaN (division by zero). Handle with `.fillna(0)` if appropriate.

What NOT to say: - "`normalize` just makes the numbers look nicer" — it changes the meaning of every cell; this is a semantic operation, not cosmetic. - "All three normalize options give the same answer, just on different axes" — they give different answers; picking the wrong one is a real reporting error.

Common follow-up: "If I want each cell to show as a percentage rather than a proportion, what do I do?" — Multiply by 100 — `crosstab(..., normalize="index") * 100` — and either format with `.style.format("{:.1f}%")` for display or apply `.round(1)` for a numeric column.

```
import pandas as pd

txn = pd.DataFrame({
    "region": ["N","N","S","S","W","W","N","S"],
    "channel": ["online","store","online","store","online","store","online","online"],
})

share_by_region = pd.crosstab(txn["region"], txn["channel"], normalize="index")
# rows sum to 1.0 – within-region share
# channel  online  store
# region
# N         0.6667  0.3333
# S         0.6667  0.3333
# W         0.5000  0.5000

share_by_channel = pd.crosstab(txn["region"], txn["channel"], normalize="columns")
# columns sum to 1.0 – within-channel share
```

Q18 · How do you turn a list-column into multiple rows? Walk me through `.explode()`.

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, Microsoft India, Cred · Topic: explode

The question (interviewer's verbatim phrasing):

I have a DataFrame where one column holds Python lists — for example, `tags` per article. How do I turn each list element into its own row?

The 30-second answer: Call `df.explode("tags")`. For each row in the source, pandas emits one row per element of the list in the named column, repeating the other columns. If the list is empty, the row is emitted once with NaN in the exploded column unless `ignore_index=True` and an empty list is treated as a single NaN row by default. If multiple columns hold parallel lists of the same length, pass a list of column names — `df.explode(["tags", "scores"])` — which preserves the positional pairing.

Step-by-step thought process: 1. Name the operation. `explode` is the canonical fix for "we stuffed a comma-separated string into one cell" or "we stored a list per row". It converts vertical-of-lists into vertical-of-rows. 2. Walk through the row-expansion. A row with `tags=["python","pandas","interview"]` becomes three rows after explode, each carrying the original `article_id`, `title`, etc., and one tag in the `tags` column. The original index is preserved by default — you get duplicate index labels. Pass `ignore_index=True` for a fresh sequential index. 3. Cover the empty-list case. By default, a row with an empty list `[]` becomes a single row with NaN in the exploded column — the row is preserved. If you want empty-list rows

dropped entirely, filter with `df[df["tags"].str.len() > 0]` before exploding. 4. Cover the multi-column case. `df.explode(["tags", "scores"])` works only if both columns hold lists of the same length per row; otherwise pandas raises. Useful when the two columns are positional pairs.

Edge cases to mention: - If the column dtype is `object` and some cells are lists, some are scalars, some are NaN — `explode` handles the mixed case but the result types may not be what you expect. For a clean explode, pre-normalize cells to lists (with `.apply(lambda x: x if isinstance(x, list) else [x])` if needed). - Exploding does not deduplicate. A row with `tags=["python", "python"]` produces two rows with `tags="python"`. - Strings are NOT exploded by default — `"python,pandas"` stays as one cell. Split first with `.str.split(",")` to convert to a list, then explode.

What NOT to say: - "I'll iterate over rows and append to a new list of records" — works but is $O(\text{rows} \times \text{avg_list_length})$ of Python overhead; explode is vectorized and several orders of magnitude faster on real data. - "Use `.str.split` and then `.melt`" — `melt` doesn't unstack lists; the right tool is `explode`.

Common follow-up: "How do you explode a column where the cells are comma-separated strings rather than lists?" — Two steps: `df["tags"] = df["tags"].str.split(",")` to convert the string to a list, then `df.explode("tags")`. Often combined with `.str.strip()` between the two to clean whitespace around tags.

```
import pandas as pd

articles = pd.DataFrame({
    "article_id": [1, 2, 3],
    "title":      ["Decorators 101", "Pandas patterns", "GIL explained"],
    "tags":       [["python", "decorators"], ["python", "pandas", "data"], ["python", "gil"]],
})

exploded = articles.explode("tags", ignore_index=True)
# 7 rows total: 2 + 3 + 2

# from comma-string source
csv_like = pd.DataFrame({"id": [1, 2], "tags": ["python,pandas", "django,fastapi,api"]})
csv_like["tags"] = csv_like["tags"].str.split(",").map(lambda lst: [t.strip() for t in lst])
exploded_csv = csv_like.explode("tags", ignore_index=True)
```

Q19 · How do you turn a column of dicts into multiple columns? Walk me through `pd.json_normalize`.

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, PhonePe, Microsoft India, Goldman · Topic: `json_normalize`

The question (interviewer's verbatim phrasing):

An API gave me back a list of dicts like `[{"name":"Anil","city":"Bangalore","plan":{"tier":"gold","mrr":499}}, ...]`. How do I get a flat DataFrame with `name`, `city`, `plan.tier`, `plan.mrr` as columns?

The 30-second answer: Call `pd.json_normalize(records)`. It flattens nested dicts into dot-separated column names by default — your example produces columns `name`, `city`, `plan.tier`, `plan.mrr` in one call. Control the separator with `sep="_"` if you'd rather have `plan_tier` and `plan_mrr` (which is friendlier for SQL exports). For deeply nested structures, `max_level=` caps the recursion. For a `record_path` that is itself a list inside each top-level record (line items inside an order), pass `record_path="line_items"` and `meta=["order_id","customer_name"]` to explode the lines while keeping the parent fields.

Step-by-step thought process: 1. Name the operation. `pd.json_normalize` exists because the natural shape of API responses — nested JSON — does not map cleanly to a flat DataFrame. The function does the flattening for you. 2. Walk through the flat case. Pass a list of dicts. Each dict becomes a row. Nested dicts are flattened into dot-separated column names. `plan.tier` is the default; `sep="_"` rewrites to `plan_tier`. 3. Cover the `record_path` case. When each record has a list of sub-records (`order` with `line_items=[...]`), you can pass `record_path="line_items"` to expand the lines into rows, and `meta=["order_id","customer_id"]` to copy the parent fields onto every child row. That's the "explode the line items but keep the order metadata" pattern. 4. Mention the limits. Deep nesting beyond a few levels gets unwieldy; cap with `max_level=`. Lists inside dicts that aren't at the `record_path` are left as-is (a column whose cells contain lists), often followed by `.explode` for the final flatten.

Edge cases to mention: - `json_normalize` accepts a single dict, a list of dicts, or a pandas Series of dicts. For a Series — say a column of API response payloads — pass `pd.json_normalize(df["payload"])` and then concat with the original frame's other columns. - Keys missing from some records become NaN columns in the result, which is what you usually want for API data with optional fields. - For very large arrays of records, `json_normalize` can be memory-heavy; chunking with `pd.read_json(..., lines=True, chunksize=10_000)` is the pattern for JSONL inputs.

What NOT to say: - "I'll write a list comprehension that flattens each dict" — works for small data, slow and brittle at scale; `json_normalize` is vectorized and handles edge cases (missing keys, nested lists) correctly. - "Just use `pd.DataFrame(records)` and pandas will flatten" — it won't; `pd.DataFrame(records)` keeps the nested dicts as object-typed cells.

Common follow-up: "What's the difference between `pd.json_normalize(df['col'])` and `df['col'].apply(pd.Series)`?" — `apply(pd.Series)` works for a single level of nesting and is slower because it dispatches per row. `json_normalize` handles arbitrary nesting depth, supports `sep`, `max_level`, `record_path`, and `meta`, and is much faster on large series. For nested JSON, always `json_normalize`.

```
import pandas as pd

records = [
    {"name": "Anil", "city": "Bangalore", "plan": {"tier": "gold", "mrr": 499}},
    {"name": "Beena", "city": "Mumbai", "plan": {"tier": "silver", "mrr": 199}},
    {"name": "Chetan", "city": "Delhi", "plan": {"tier": "gold", "mrr": 999}},
]

df = pd.json_normalize(records, sep="_")
# columns: name, city, plan_tier, plan_mrr

# record_path pattern – orders with line items
orders = [
    {"order_id": 1, "customer": "Anil", "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 1}]},
    {"order_id": 2, "customer": "Beena", "items": [{"sku": "A", "qty": 1}]},
]
lines = pd.json_normalize(orders, record_path="items", meta=["order_id", "customer"])
# columns: sku, qty, order_id, customer – one row per line item
```

Q20 · The "Data Track · the reshape they ask" question — flatten an API response with mixed nesting

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, PhonePe, Microsoft India, Walmart Labs · Topic: reshape scenario

The question (interviewer's verbatim phrasing):

Here is a list of API responses where each record has a customer, a list of orders, and each order has a list of items. Give me a flat DataFrame with one row per item, carrying the customer name, the order id, the item sku, and the item quantity. Walk me through it.

The 30-second answer: This is a two-level explode plus a metadata copy. Call `pd.json_normalize(records, record_path=["orders", "items"], meta=["customer", "orders", "order_id"])`. The `record_path` traverses two list levels — first into `orders`, then into `items` per order. The `meta` list copies the outer `customer` field onto every row and, for the nested `order_id`, uses the bracketed list-path form to pull it from inside each order. The result is one row per item with all the parent identifiers attached.

Step-by-step thought process: 1. Diagnose the shape. Each top-level dict is a customer. Each customer has a list of orders. Each order has a list of items. We want one row per item, so the leaf level is `items` and we need everything above it (customer name, order id) carried down. 2. Set up `record_path`. The list form `["orders", "items"]` tells `json_normalize` to traverse from each record into `orders`, then from each order into `items`. The cells of `items` become the rows

of the result. 3. Set up `meta`. Strings in `meta` refer to top-level keys — `"customer"` copies the customer name onto every row. List paths inside `meta` — `["orders","order_id"]` — pull a value from a deeper level, so each item row gets its parent order's id. The exception is that `meta` paths must each terminate at a non-list value (or a value `json_normalize` can stringify). 4. Verify the result shape. If the input has 2 customers, with 3 + 2 orders, and the orders have 2 + 1 + 3 + 1 + 2 items respectively, the output has 9 rows. Sanity-check with a small example before scaling up.

Edge cases to mention: - If a customer has zero orders, that customer is dropped from the result — there's no item row to attach the customer name to. Track these separately if you need them. - If an order has no items, same problem. Pre-process to inject a sentinel item if missing rows would cause downstream bugs. - For deeply nested structures where `record_path` would need to traverse 4+ levels, `json_normalize` still works but the call gets unwieldy. At that point, two `json_normalize` calls — one per level — joined with `pd.merge` is often more readable.

What NOT to say: - "I'll write three nested `for` loops to flatten this" — works but is the wrong tool. `json_normalize` is vectorized and handles edge cases for you. - "Just `pd.DataFrame(records)`" — keeps nested data in cells; the question explicitly asks for flat rows.

Common follow-up: "What if I also want a column showing the total amount for the parent order, copied onto each item row?" — Add `["orders","amount"]` to the `meta` list. Or, after the flatten, `groupby("order_id")["amount"].transform("sum")` to compute and broadcast the per-order total back to every item row.

```
import pandas as pd

api_records = [
    {
        "customer": "Anil",
        "orders": [
            {"order_id": 101, "items": [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 1}]},
            {"order_id": 102, "items": [{"sku": "C", "qty": 4}]},
        ],
    },
    {
        "customer": "Beena",
        "orders": [
            {"order_id": 201, "items": [{"sku": "A", "qty": 1}, {"sku": "D", "qty": 3}]},
        ],
    },
]

items = pd.json_normalize(
    api_records,
    record_path=["orders","items"],
    meta=["customer", ["orders","order_id"]],
)
items = items.rename(columns={"orders.order_id": "order_id"})
# columns: sku, qty, customer, order_id — one row per item
# 4 rows total: (Anil×3) + (Beena×2) = adjusts to 5 rows
```

Chapter Wrap-Up

Twenty questions across pivot, melt, stack, unstack, crosstab, explode, and json_normalize. The thread that runs through all of them is that reshape operations are about moving information between rows, columns, index levels, and embedded structures — and that pandas gives you several tools whose argument shape, not whose capability, decides which is the right pick.

A summary of the picking rules, restated:

- `pivot` when you have a flat long frame with one row per (index, column) pair and no aggregation needed.
- `pivot_table` when duplicates exist and you want to aggregate them; always set `aggfunc` explicitly.
- `melt` when you have a flat wide frame and want long form for plotting, SQL-like analysis, or schema stability.
- `wide_to_long` when the wide columns follow a stub-suffix pattern with multiple metrics.
- `stack` and `unstack` when you already have a MultiIndex (typically after `groupby` with multiple keys) and want to move levels between the row index and column index.
- `crosstab` when the question is a frequency table, especially with `normalize` for within-row or within-column proportions.
- `explode` for list-columns; combined with `.str.split` for comma-separated strings.
- `json_normalize` for nested JSON; use `record_path` and `meta` for two-level exploding with parent-field carry-down.

These reshape operations come up in round one of every Indian Data-role interview — Flipkart, Razorpay, Swiggy, PhonePe, the GCC quants at Goldman and JP Morgan, the data platform teams at Walmart Labs and Target. A candidate who reaches for the right tool in one line — and who can explain why `pivot_table` defaulted to mean and silently corrupted last quarter's report — is the candidate who clears round one and gets to the harder rounds where the questions are about scaling, lineage, and design.

Next chapter: Apply, map, vectorize, and the performance cliff that comes with `.apply` on big frames.

Chapter 5 — apply / map / vectorize + Performance (20 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

This is the chapter that separates the candidate who has "used pandas" from the candidate who has shipped pandas to production. Every Data-role interviewer in India has a stash of `apply` questions ready, and they ask them for the same reason — a single misplaced `df.apply(fn, axis=1)` on a ten-million-row frame turns a five-second query into a forty-minute job, and the candidate who reaches for `apply` as the first hammer is signalling they have never run their own code at real scale. The 2026 version of this conversation is sharper than the 2022 version because pandas 2.x ships an Arrow backend whose string and datetime methods are 3 to 10 times faster than the NumPy-backed equivalents, because `df.eval` and `df.query` have stabilised, and because polars and Modin have raised the floor of what counts as "acceptable performance" on a Razorpay or Swiggy bench test. The questions below are the ones that get asked at Flipkart's Data Engineer screen, Cred's risk-analytics round, Postman's growth-analytics panel, Walmart Labs's data-platform interview, JP Morgan and Goldman's quant-data desks, and the Service MNC ML-engineer pools at TCS, Infosys, and Wipro.

Twenty questions, four sections. Section A is `apply` itself — the basics, the perf cliff, the axis confusion that costs candidates a round, the groupby-apply quirk where the group itself is a DataFrame, the `result_type` knob, and the early-return pattern. Section B is the vectorisation toolkit — NumPy ufuncs, `np.where`, `pd.cut` / `pd.qcut`, `.str` and `.dt` accessors. Section C is the performance toolkit — `%timeit`, copy-vs-view discipline, the Arrow backend, `eval` / `query` for complex expressions, and the escape hatches (`numba`, polars, swifter) for when pandas genuinely cannot get there. Each Q follows the Step-by-Step Answer shape — interviewer's verbatim phrasing, 30-second answer, the out-loud thought process, edge cases, what NOT to say, the likely follow-up. Code blocks run on pandas 2.2 / NumPy 1.26 / Python 3.12 and the output comments are what the interpreter actually prints.

Sections in this chapter

- Section A — `apply` basics and the perf cliff (Q1–Q10)
- Section B — Vectorisation patterns (Q11–Q15)
- Section C — Performance optimisation and escape hatches (Q16–Q20)

Section A — `apply` Basics and the Performance Cliff

Q1 · `apply` on a Series vs `apply` on a DataFrame

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Walmart Labs · Topic: pandas apply

The question (interviewer's verbatim phrasing):

What does `apply` do when you call it on a Series versus a DataFrame? Are they the same method?

The 30-second answer: They share a name but they behave differently. `Series.apply(fn)` calls `fn` once per element — the input to `fn` is a scalar value, the output builds a new Series of the same length. `DataFrame.apply(fn)` calls `fn` once per column (the default) or once per row (`axis=1`) — the input to `fn` is an entire Series, the output is a Series (if `fn` returns a scalar) or a DataFrame (if `fn` returns a Series). The Series form is element-wise; the DataFrame form is column-wise or row-wise.

Step-by-step thought process: 1. State the axis defaults loud and clear. `Series.apply` has no axis — it iterates elements. `DataFrame.apply` defaults to `axis=0`, which iterates columns. Many candidates assume `axis=0` means rows because "rows are the zeroth dimension" — that intuition is wrong. 2. Walk through the input shape. On a Series, `fn` receives a scalar each call. On a DataFrame with `axis=0`, `fn` receives a Series — the entire column. With `axis=1`, `fn` receives a Series — the entire row, indexed by column name. 3. Mention the obvious one — for pure element-wise work on a Series, `apply` and `map` produce the same result; `map` is the older name and accepts a dict or Series for substitution as well as a callable. For DataFrame-wide element-wise work pre-2.1 you reached for `applymap`; in 2.1+ the same operation is `DataFrame.map` (and `applymap` is deprecated). 4. Land on the practical implication. If `fn` does anything non-trivial — even a single string operation — the column-wise form on a DataFrame is the right default, because it amortises the per-call overhead across an entire column instead of paying it per element.

Edge cases to mention: - `Series.apply` with a function that returns a list builds a Series of lists; it does not auto-explode. You need `.explode()` after. - `DataFrame.apply(fn, axis=1)` on a frame with mixed dtypes silently upcasts the per-row Series to `object`, which makes the function slower than a pure-numeric column-wise apply. - `Series.agg` and `Series.apply` look almost identical but `agg` also accepts a list of functions and a dict mapping column to function; `apply` does not.

What NOT to say: - "They're the same method on different types" — they're not; `DataFrame.apply` has an `axis` argument and `Series.apply` does not. - "`apply` is always element-wise" — only on a Series; on a DataFrame it's column-wise or row-wise.

Common follow-up: "In pandas 2.1, what replaced `applymap`?" — `DataFrame.map`. The behaviour is identical (element-wise across every cell), the name is now aligned with `Series.map`, and `applymap` raises a `FutureWarning`.

```
import pandas as pd

s = pd.Series([1, 2, 3])
print(s.apply(lambda x: x * 2).tolist())    # [2, 4, 6] — element-wise

df = pd.DataFrame({"a": [1, 2, 3], "b": [10, 20, 30]})

# axis=0 (default): fn gets each column as a Series
print(df.apply(sum).to_dict())             # {'a': 6, 'b': 60}

# axis=1: fn gets each row as a Series
print(df.apply(sum, axis=1).tolist())      # [11, 22, 33]

# Element-wise across every cell (pandas 2.1+)
print(df.map(lambda x: x + 1).to_dict())   # {'a': [2,3,4], 'b': [11,21,31]}
```

Q2 · `axis=0` vs `axis=1` — Resolving the Confusion Once

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cred, Swiggy, JP Morgan · Topic: pandas apply

The question (interviewer's verbatim phrasing):

When I write `df.apply(fn, axis=0)`, what does `fn` actually receive? And what's the difference with `axis=1`?

The 30-second answer: `axis=0` means "operate along the row index" — the function receives each column as a Series, one at a time. `axis=1` means "operate along the column index" — the function receives each row as a Series, indexed by column name. The mnemonic that sticks is "axis is the one that disappears": `df.sum(axis=0)` collapses rows and leaves you with one value per column; `df.sum(axis=1)` collapses columns and leaves you with one value per row.

Step-by-step thought process: 1. State the mnemonic first because it survives interview pressure. Axis is the dimension that vanishes. `axis=0` makes rows go away; `axis=1` makes columns go away. 2. Apply it to `apply`. `axis=0` means the rows-going-away direction — each column-Series gets fed to `fn` one at a time, and the output has one value per column. `axis=1` means the columns-going-away direction — each row-Series gets fed to `fn`, and the output has one value per row. 3. The same logic explains `df.mean`, `df.sum`, `df.max`. `axis=0` is the default for aggregations, which is why `df.mean()` returns per-column means. 4. The `axis=1` form is where most of the row-wise `apply` perf trouble comes from, because the per-row Series has to be constructed for every row. For a million-row frame, that is a million Series allocations on top of a million Python-level calls.

Edge cases to mention: - `axis="rows"` is a synonym for `axis=0`; `axis="columns"` is a synonym for `axis=1`. Some style guides prefer the strings for readability. - `pd.concat([df1,`

`df2]`, `axis=0`) and `axis=1` follow the same mnemonic — `axis=0` stacks vertically (rows grow), `axis=1` stitches horizontally (columns grow). - `dropna(axis=1)` drops columns that contain NaN; `dropna(axis=0)` drops rows. Same direction logic.

What NOT to say: - "`axis=0` means rows because rows are first" — that intuition fails the mnemonic and gets you the wrong direction on every method that takes `axis`. - "I always pass `axis=1` for safety" — this is exactly the path to the row-wise `apply` perf cliff.

Common follow-up: "Which axis would you use to compute the per-customer total across columns `q1_revenue`, `q2_revenue`, `q3_revenue`, `q4_revenue`?" — `axis=1`, because you want one value per customer (one per row), and that means the column dimension has to disappear.

```
import pandas as pd

df = pd.DataFrame({"q1": [100, 200], "q2": [150, 250], "q3": [120, 220]})
print(df.sum(axis=0).to_dict())          # {'q1': 300, 'q2': 400, 'q3': 340} — collapse rows
print(df.sum(axis=1).tolist())           # [370, 670] — collapse columns

# apply mirrors the same direction
print(df.apply(lambda col: col.max(), axis=0).to_dict()) # per-column max
print(df.apply(lambda row: row.max(), axis=1).tolist())  # per-row max
```

Q3 · When `apply` Is Fine vs When `apply` Is the Bug

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, PhonePe, Walmart Labs, Microsoft India
· Topic: pandas perf

The question (interviewer's verbatim phrasing):

Your colleague wrote `df["margin"] = df.apply(lambda row: row["revenue"] - row["cost"], axis=1)` on a ten-million-row DataFrame and it's running for forty minutes. Walk me through what's wrong and what you'd replace it with.

The 30-second answer: `apply` with `axis=1` calls the lambda once per row, which on ten million rows is ten million Python-level function calls plus ten million per-row Series constructions. The vectorised replacement is one line: `df["margin"] = df["revenue"] - df["cost"]`. That pushes the subtraction into the C layer of NumPy or Arrow and finishes in under a second on the same hardware. The rule of thumb is that `apply` on rows is two to three orders of magnitude slower than the vectorised equivalent whenever a vectorised equivalent exists, which for pure arithmetic, comparisons, masking, and standard string and datetime methods is always.

Step-by-step thought process: 1. Open with the mechanics. `apply(axis=1)` is a Python for-loop dressed in pandas syntax. It pays the per-row overhead of constructing a Series, dispatching a Python

function call, and unpacking the return value, for every row. On a million-row frame the per-row overhead dominates. 2. Vectorisation pushes the work into a tight C loop. `df["a"] - df["b"]` is a single NumPy ufunc call that subtracts two contiguous memory blocks in one pass. The Python interpreter is involved exactly once. 3. The empirical gap is large — 100 to 1000x on a million-row frame for arithmetic, comparisons, and standard string methods. Quote a number the candidate has actually measured if asked. 4. The actionable rule: any time you write `apply`, stop and ask "can I express this with operators or accessor methods on the columns directly?" If yes, do that. `apply` belongs in the fallback bucket, not the default toolbox.

Edge cases to mention: - `apply` is fine — sometimes the only option — when the per-row logic genuinely cannot be vectorised: calling a non-vectorised external library, doing per-row stateful transformation that depends on multiple columns in a non-arithmetic way, or invoking a Python function with no NumPy or Arrow equivalent. - For per-column work on a wide DataFrame, `apply(axis=0)` is usually fine — the loop runs over columns (typically dozens), not rows (typically millions). The per-call overhead is amortised. - "Vectorised" does not mean "fast no matter what" — `df.str.contains(pattern, regex=True)` is vectorised but still slow on a large frame with a complex regex; the Arrow backend's contiguous string layout helps here.

What NOT to say: - "`apply` is slow because pandas is slow" — pandas is fast; `apply(axis=1)` is slow because it's a row-wise Python loop. Vectorised pandas is C-speed. - "I'd use `swifter` to parallelise the apply" — that's a constant-factor speedup (typically 2 to 8x). Vectorisation is a different complexity class — orders of magnitude faster.

Common follow-up: "What if the per-row logic is a multi-branch if/else that can't be a single arithmetic expression?" — Reach for `np.where`, `np.select`, or `pd.cut` first. They handle multi-branch logic vectorised. `apply` is the fallback after those don't fit, not the default.

```
import pandas as pd
import numpy as np

n = 10_000_000
df = pd.DataFrame({"revenue": np.random.rand(n), "cost": np.random.rand(n)})

# Slow — Python-level call per row (~40 seconds on this size)
# df["margin"] = df.apply(lambda r: r["revenue"] - r["cost"], axis=1)

# Fast — single C-level ufunc (~50 milliseconds)
df["margin"] = df["revenue"] - df["cost"]

# Multi-branch logic stays vectorised with np.where
df["bucket"] = np.where(df["margin"] > 0.5, "high",
                        np.where(df["margin"] > 0.1, "mid", "low"))
```

Q4 · `apply` That Returns a Series — and `result_type='expand'`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs, Goldman Sachs · Topic: pandas apply

The question (interviewer's verbatim phrasing):

If my `apply` function returns a Series, what does the output of `df.apply(fn, axis=1)` look like? And what does `result_type='expand'` change?

The 30-second answer: If `fn` returns a Series, `df.apply(fn, axis=1)` builds a DataFrame whose columns come from the returned Series's index — so a function that returns a two-element Series with index `["a", "b"]` produces a DataFrame with columns `a` and `b`.

`result_type='expand'` is the explicit form: if `fn` returns a list or tuple, pandas expands it into multiple columns named `0`, `1`, `2`. Without `expand`, returning a list packs the whole list into a single object-dtype column.

Step-by-step thought process: 1. Start with the default. When `fn` returns a Series with a named index, pandas treats that index as column names in the output DataFrame. No knob needed. 2. When `fn` returns a tuple or list, the default is to pack the return value into a single column of object dtype — each cell holds a tuple. That is usually not what the user wants. 3. `result_type='expand'` says "treat the return value as multiple columns, expand it across the column axis." For a tuple of length `k`, the output gets `k` columns named by integer position. 4. The three knobs to know:

`result_type='expand'` (multi-column), `result_type='reduce'` (force a Series result even when each return is array-like), and `result_type='broadcast'` (replicate scalar results back to the original shape). Default is `None`, which picks the best fit but can surprise you on edge cases.

Edge cases to mention: - If `fn` returns Series with different indexes across rows, pandas aligns them — you end up with NaN in the cells where a particular column was missing for that row. -

`result_type='expand'` only matters when the return value is a tuple, list, or ndarray. For a Series return, the column names already come from the Series index. - Returning a dict and using `result_type='expand'` does not give you dict-keyed columns — it gives you integer-named columns. Use `pd.Series(d)` inside `fn` for dict-keyed output.

What NOT to say: - "`result_type` is for type conversion" — no, it controls the shape of the assembled output, not the dtype. - "Returning a list from `apply` gives you a DataFrame" — only with `result_type='expand'`; the default packs it into one object-dtype cell.

Common follow-up: "What's the perf difference between returning a Series and using `result_type='expand'` with a tuple?" — Tuples are slightly faster because there is no per-row Series construction in the user code, but the gap is small. Both pay the row-wise `apply` tax; the choice is about clarity of column naming.

```
import pandas as pd

df = pd.DataFrame({"x": [10, 20, 30], "y": [1, 2, 3]})

# Function returns a Series – columns named by the Series index
def stats(row):
    return pd.Series({"total": row["x"] + row["y"], "ratio": row["x"] / row["y"]})

print(df.apply(stats, axis=1))
#    total  ratio
# 0     11   10.0
# 1     22   10.0
# 2     33   10.0

# Function returns a tuple – without expand, packed into one column
print(df.apply(lambda r: (r["x"] + r["y"], r["x"] / r["y"]), axis=1).iloc[0])
# (11, 10.0)          – single object-typed cell per row

# With result_type='expand' – split across columns named 0, 1
print(df.apply(lambda r: (r["x"] + r["y"], r["x"] / r["y"]),
                axis=1, result_type='expand'))
#    0     1
# 0  11  10.0
# 1  22  10.0
# 2  33  10.0
```

Q5 · The GroupBy `apply` Quirk — The Group Is a DataFrame

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, JP Morgan, Walmart Labs ·
Topic: pandas groupby apply

The question (interviewer's verbatim phrasing):

What does the function passed to `df.groupby("user_id").apply(fn)` actually receive?
People sometimes get this wrong.

The 30-second answer: It receives a full DataFrame — the subset of rows that share the group key — not a row, not a scalar, not a Series. So `fn(group)` is being called with `group` as a DataFrame of however many rows that user had. Inside `fn`, you write whatever DataFrame logic you want — return a scalar to build a Series-of-scalars output, return a Series to build a DataFrame output, return a DataFrame to stitch back into a larger DataFrame. The mental model is "for each group, here's the slice — do what you want."

Step-by-step thought process: 1. Anchor on the input shape. `groupby().apply(fn)` calls `fn` once per group, and the argument to `fn` is the full subframe, indexed exactly as in the original

DataFrame. Many candidates think the argument is a row or a scalar because they confuse it with `apply` on a plain DataFrame. 2. Walk through the output shape — pandas tries to stitch the per-group returns back together. Scalars become a Series indexed by group key; Series become a DataFrame; DataFrames become a larger DataFrame with a hierarchical index. 3. Mention `transform` and `agg` as the two cousins. `transform` is faster when you want one value per row aligned back to the original index; `agg` is faster when you want one scalar per group. `apply` is the catch-all when neither shape fits. 4. Land on the 2.2 / 2.3 perf note. `groupby().apply()` in current pandas applies `fn` once per group plus once on the first group again (to figure out the output shape). Set `include_groups=False` (the 2.2+ argument that becomes the default in a future release) to skip the duplicate first-group call.

Edge cases to mention: - The group keys are by default included as columns in the subframe passed to `fn`. Set `include_groups=False` to exclude them — pandas 2.2 deprecated the old auto-include behaviour. - Returning a scalar inside `apply` gives you `Series` output; returning the input frame unchanged gives you the same frame back (no-op); returning an empty DataFrame drops the group from the output. - The order of rows inside each group matches the order in the original frame unless you sort inside `fn`. Don't assume any per-group sort happens automatically.

What NOT to say: - "The function receives a row" — no, it receives the entire subframe for that group. - "`apply` is the fastest way to do per-group work" — `transform` and `agg` are faster for their respective shapes; `apply` is the slowest and most general.

Common follow-up: "You want per-group z-scores — the value minus the group mean over the group std. Which of `apply`, `transform`, `agg` would you reach for?" — `transform`. It returns one value per input row aligned back to the original index, and the per-group math is vectorised inside the transform call. `apply` would also work but is slower because of the per-group function-call overhead.


```
import pandas as pd

df = pd.DataFrame({
    "user_id": [1, 1, 1, 2, 2, 3],
    "amount": [10, 20, 30, 15, 25, 50]
})

# The function receives a DataFrame — the slice for this user
def top_two(group):
    return group.nlargest(2, "amount")

result = df.groupby("user_id", group_keys=False).apply(top_two,
                                                         include_groups=False)

print(result)
#    amount
# 1      30
# 2      20
# 3      25
# 4      15
# 5      50

# transform is faster for "one value per row" shape
df["zscore"] = df.groupby("user_id")["amount"].transform(
    lambda x: (x - x.mean()) / x.std(ddof=0)
)
```

Q6 · The `result_type` Knob in Detail — Expand vs Reduce vs Broadcast

Tags: Difficulty: Medium · Asked at: Postman, Cred, Walmart Labs, Microsoft India, Goldman Sachs ·
Topic: pandas apply

The question (interviewer's verbatim phrasing):

Walk me through every value of `result_type` on `df.apply` — what does each one do and when would I pick each?

The 30-second answer: Four values. `None` (default) lets pandas infer the output shape from the first return value, which is convenient but can surprise you on edge cases. `'expand'` says "the return is array-like; split it across multiple columns." `'reduce'` says "force a single Series result even when the return is array-like — pack the array into one cell." `'broadcast'` says "the return is a scalar; broadcast it back across the same shape as the input row or column." In practice you reach for `'expand'` 90% of the time and the default the rest of the time.

Step-by-step thought process: 1. Cover the default first. With `result_type=None`, pandas tries to be smart — a scalar return collapses to a Series; a Series return becomes a DataFrame; a list return

packs into an object-dtype Series. The smartness sometimes guesses wrong. 2. `'expand'` is the explicit "split into columns" form. Use it when `fn` returns a tuple, list, or ndarray and you want each element in its own column. 3. `'reduce'` is the opposite — when `fn` returns an array-like and you want it packed into one cell. Useful when you want object-dtype output deliberately, or when the array structure matters per row. 4. `'broadcast'` is the niche one. When `fn` returns a scalar and you want that scalar to fill out the same shape as the input row or column. Almost never used in modern pandas — the same effect is achieved more clearly with `.assign` or a vectorised expression.

Edge cases to mention: - The default behaviour changed subtly between pandas 1.x and 2.x for ambiguous return shapes. If you care about deterministic shape, pass `result_type` explicitly. - `'expand'` ignores the `name` attribute on a returned Series — the columns come from the Series index, not the name. To use the name, return a one-element Series. - `'broadcast'` is only meaningful with `axis=1` on a frame with mixed dtypes — for pure numeric work, just write the assignment directly.

What NOT to say: - "`result_type='reduce'` makes it faster" — it doesn't; it controls shape, not speed. - "Always pass `result_type='expand'`" — wrong default for Series-returning functions, where the index naming already does the right thing.

Common follow-up: "Show me the difference between returning `pd.Series({'a': 1, 'b': 2})` and returning `(1, 2)` with `result_type='expand'`." — Both produce a two-column DataFrame. The Series form gives columns named `a` and `b`; the tuple-with-expand form gives columns named `0` and `1`. The Series form is preferred when column names matter, which is almost always.

```
import pandas as pd

df = pd.DataFrame({"x": [1, 2, 3], "y": [10, 20, 30]})

# 'expand' — tuple return becomes multiple columns
r1 = df.apply(lambda r: (r["x"] + r["y"], r["x"] * r["y"]),
              axis=1, result_type='expand')
print(r1.columns.tolist())          # [0, 1]

# 'reduce' — keep the tuple packed in one cell
r2 = df.apply(lambda r: (r["x"], r["y"]), axis=1, result_type='reduce')
print(type(r2), r2.iloc[0])         # <class 'pd.Series'> (1, 10)

# Series return with column names — usually cleanest
r3 = df.apply(lambda r: pd.Series({"sum": r["x"] + r["y"],
                                  "prod": r["x"] * r["y"]}), axis=1)
print(r3.columns.tolist())          # ['sum', 'prod']
```

Q7 · Early Return From an `apply` Lambda — and Why It Won't Help You

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, Walmart Labs · Topic: pandas perf

The question (interviewer's verbatim phrasing):

Someone told me that putting an early `return None` inside the `apply` lambda would speed it up by skipping work for irrelevant rows. Does that work?

The 30-second answer: No — and the reasoning matters. `apply` calls the function for every row regardless; an early return inside the lambda just exits the function body slightly sooner. The expensive part — the Python-level dispatch, the per-row Series construction, the return-value boxing — is paid before the function body starts and after it ends. Skipping work inside the body saves a tiny amount of per-row CPU but does not change the order-of-magnitude perf profile. The right move is to filter the rows out before the `apply`, not return early inside it.

Step-by-step thought process: 1. State the cost structure. The per-row tax in `apply(axis=1)` has three parts — function dispatch, Series construction, return-value boxing — and they are paid every call. An early return saves CPU only on the function body itself. 2. Walk through the right pattern. Filter first, `apply` to the smaller subframe, then merge or reindex the result back.

`df[df["needs_work"]].apply(fn, axis=1)` is dramatically faster than `df.apply(lambda r: fn(r) if r["needs_work"] else None, axis=1)`. 3. If the early-return is for a vectorisable case ("skip if amount is zero"), the right move is to express the whole thing as a vectorised conditional with `np.where` or boolean masking. You stop using `apply` entirely. 4. The candidate signal interviewers look for here is whether you understand that `apply` overhead is constant per row, not scaling with the body. People who think early return helps have not profiled `apply` at scale.

Edge cases to mention: - Filtering before applying can change the result if the `apply` has side-effects on rows you would have skipped — but `apply` with side-effects is a code-smell already. -

`apply(axis=0)` has a different cost shape — the per-call cost is per-column, not per-row, so it's amortised. Early return inside a column-wise `apply` does help slightly when the column is large. - If the filter itself is expensive, do `np.where` or boolean masking on the result of a vectorised computation; do not filter then `apply` then re-merge.

What NOT to say: - "Early return inside the lambda is a free perf win" — no; the per-row tax is paid regardless of the body. - "I'll use `try/except` to skip bad rows" — exceptions inside `apply` are expensive and obscure the bug; filter first.

Common follow-up: "Show me the right way to apply a function only to rows where status is 'paid'." — Boolean-mask the DataFrame first: `mask = df["status"] == "paid"; df.loc[mask, "out"] = df.loc[mask].apply(fn, axis=1)`. The `apply` runs over the smaller frame; the unaffected rows keep their existing value.

```
import pandas as pd

df = pd.DataFrame({
    "amount": [100, 0, 250, 0, 500],
    "status": ["paid", "failed", "paid", "failed", "paid"],
})

# Wrong – apply runs for every row, early return saves nothing meaningful
df["fee_wrong"] = df.apply(
    lambda r: r["amount"] * 0.02 if r["status"] == "paid" else None,
    axis=1
)

# Right – filter first, apply to the smaller subframe
mask = df["status"] == "paid"
df.loc[mask, "fee_right"] = df.loc[mask].apply(
    lambda r: r["amount"] * 0.02, axis=1
)

# Better still – vectorise the whole thing
df["fee_best"] = df["amount"].where(df["status"] == "paid") * 0.02
```

Q8 · `Series.map` vs `Series.apply` vs `Series.replace`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Walmart Labs · Topic: pandas apply

The question (interviewer's verbatim phrasing):

I want to replace every value in a Series using a lookup dictionary. What's the difference between `map`, `apply`, and `replace`?

The 30-second answer: `map` is the cleanest for dict-based substitution and is the fastest on a Series — it does a hash lookup per element, no Python-level function call. `apply` requires a callable, so for a dict you have to write `apply(lambda x: d.get(x, x))` which is slower because of the per-element call. `replace` works on Series and DataFrame, accepts dicts and regex, and is more flexible but also slower than `map` for plain dict substitution. Rule of thumb: dict on a Series → `map`; callable on a Series → `apply`; regex substitution or partial matching → `replace`.

Step-by-step thought process: 1. Start with `map`. It is element-wise on a Series, accepts a dict, a Series, or a callable. With a dict, it does a vectorised hash lookup — the fastest path. Unmapped values become NaN by default. 2. `apply` is the more general version on a Series — it only accepts a callable. To use a dict, you wrap with `lambda x: d.get(x, x)`, which costs you a Python-level call per element. For element-wise work, `map` is usually faster. 3. `replace` is the most general — works on Series and DataFrame, accepts dict-of-dict for per-column rules, accepts regex via `regex=True`, and

preserves unmapped values by default (so it does not silently introduce NaN). It is slower than `map` for plain dict substitution because of the regex compilation and per-element fallback path. 4. The candidate-signal layer: knowing that `map` returns NaN for unmapped values while `replace` keeps the original is the kind of detail that catches you in a Guess-the-Output round.

Edge cases to mention: - `Series.map` with a callable is identical to `Series.apply` for that case. The dict form is where `map` pulls ahead. - `replace` with `regex=True` compiles the regex once per pattern and runs it via `re.sub` — vectorised, but the regex engine is the bottleneck for complex patterns. - For DataFrame-wide element-wise substitution in 2.1+, `DataFrame.map` replaces `applymap`. `replace` still works on the whole frame and accepts the per-column dict form.

What NOT to say: - "`map` and `apply` are the same on a Series" — they accept the same callable input but `map` also accepts dicts and Series, which `apply` does not. - "`replace` is for missing-data filling" — `replace` is general substitution; for NaN-filling specifically, `fillna` is the right tool.

Common follow-up: "What happens if a value isn't in the lookup dict — for each of `map`, `apply` -with- `.get`, and `replace`?" — `map` returns NaN, `apply(lambda x: d.get(x, x))` returns the original value, `replace` returns the original value. The default behaviours differ — be deliberate.

```
import pandas as pd

s = pd.Series(["mumbai", "delhi", "blr", "hyd"])
zone = {"mumbai": "west", "delhi": "north", "blr": "south"}

# map — dict lookup, fastest, unmapped becomes NaN
print(s.map(zone).tolist())           # ['west', 'north', 'south', nan]

# apply — needs a callable, lambda fills in default
print(s.apply(lambda x: zone.get(x, "unknown")).tolist())
# ['west', 'north', 'south', 'unknown']

# replace — preserves unmapped, supports regex
print(s.replace(zone).tolist())       # ['west', 'north', 'south', 'hyd']
```

Q9 · `apply` With a Function That Mutates External State — Why It's a Smell

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, JP Morgan · Topic: pandas apply

The question (interviewer's verbatim phrasing):

Your apply function appends to a list defined outside its scope. Is that a bug?

The 30-second answer: It's a smell, sometimes a bug. `apply` is meant to be a pure transform — input row, output value — and pandas does not guarantee call order, may call the function on the first group twice (for shape inference), and gives no parallel-safety guarantees if the user opts into something like `swifter`. Mutating external state from inside `apply` works in basic single-threaded pandas but breaks the moment shape-inference runs the function twice, the moment you parallelise, or the moment a developer reads the code six months later and assumes purity. Write the for-loop instead.

Step-by-step thought process: 1. Open with the purity expectation. The pandas API is built around pure transforms — `apply`, `map`, `transform`, `agg` all assume the function has no side-effects. Code that breaks this assumption is fragile. 2. Walk through the specific failure modes. Shape inference may call the function twice on the first group of a `groupby().apply`. Parallel wrappers like `swifter`, `pandarallel`, and `modin` rely on side-effect freedom and silently double-count under accumulating state. The new free-threaded Python build (3.13t) makes this worse. 3. State the right alternative. If the operation is fundamentally accumulating ("running total", "row id stamping"), use a vectorised cumulative method — `cumsum`, `cummax`, `expanding`, `rolling`. If the operation is bookkeeping ("count how many rows hit this branch"), do the count via a boolean mask after the fact, not via mutation. 4. Land on the readability angle. A function with side-effects buried in an `apply` is invisible to the reader. A for-loop wears its intent on its sleeve.

Edge cases to mention: - Logging inside `apply` is technically a side-effect but the cost is usually fine in dev; in prod, it slows the `apply` by another order of magnitude. Log on the aggregate, not per row. - Counting hits via `nonlocal counter += 1` inside `apply` works on single-threaded plain pandas; it silently undercounts under any parallel backend. - Modifying the row passed in (`row["x"] = ...`) does not modify the original DataFrame — `apply` gives you a copy of the row — so this is silent no-op, which is its own kind of bug.

What NOT to say: - "Side-effects inside `apply` are fine because pandas is single-threaded" — not under any parallel wrapper, not under free-threaded Python, not under the chained-assignment patterns pandas 2.x is moving toward. - "I'll use `global`" — strictly worse; the side-effect is now even harder to reason about.

Common follow-up: "What's the vectorised equivalent of accumulating a running total of a column?" — `df["running"] = df["amount"].cumsum()`. Or `groupby("user_id")["amount"].cumsum()` for per-user running totals. Both are C-speed.

```
import pandas as pd

df = pd.DataFrame({"amount": [10, 20, 30, 40]})

# Smell – external state mutated inside apply
log = []
def bad(row):
    log.append(row["amount"])          # side-effect – may run more than once
    return row["amount"] * 2

df["doubled"] = df.apply(bad, axis=1)
# log might have 4 entries – or 5, if pandas re-runs the first row

# Clean – pure transform, no external state
df["doubled_ok"] = df["amount"] * 2

# Cumulative – use a vectorised method, not external accumulation
df["running"] = df["amount"].cumsum()
print(df["running"].tolist())          # [10, 30, 60, 100]
```

Q10 · `apply` on a GroupBy Returning a Scalar vs Series vs DataFrame

Tags: Difficulty: Hard · Asked at: Flipkart, Swiggy, Razorpay, JP Morgan, Walmart Labs, Goldman Sachs · Topic: pandas groupby apply

The question (interviewer's verbatim phrasing):

Walk me through what `df.groupby(key).apply(fn)` returns when `fn` returns a scalar, a Series, and a DataFrame. The shape changes — why?

The 30-second answer: Pandas stitches the per-group outputs together based on their shape. Scalar returns become a Series indexed by group key — one value per group. Series returns become a DataFrame — the group key becomes the row index, the Series index becomes the column axis. DataFrame returns become a larger DataFrame with a MultiIndex — the outer level is the group key, the inner level is whatever index the per-group DataFrame had. The choice of shape inside `fn` controls the shape of the assembled result.

Step-by-step thought process: 1. Start with the scalar case. Cheap, common, and unambiguous — one value per group, indexed by the group key. 2. Move to the Series case. Pandas takes each per-group Series and assembles a DataFrame by stacking them — the Series index becomes the column index of the result. If the Series indexes differ across groups, pandas aligns them and fills the missing cells with NaN. 3. The DataFrame case is the most general. Each per-group DataFrame is stacked, and the result has a MultiIndex — outer level is the group key, inner level is the per-group DataFrame's row index. To flatten, call `.reset_index()` or pass `group_keys=False` to the original groupby. 4. Mention the

2.2 perf detail. Pandas calls `fn` on the first group twice (once for shape inference, once for real). To avoid this on expensive `fn`, pass `include_groups=False` and structure `fn` to be shape-stable.

Edge cases to mention: - `group_keys=False` on the groupby suppresses the outer MultiIndex level — useful when you want the result to share the original DataFrame's index, common when assigning back to the original frame. - Returning a single-row DataFrame versus a single-element Series from `fn` gives slightly different output shapes — both work, the difference is whether you get a column-axis from the Series index or from the DataFrame columns. - `agg` and `transform` are the lower-overhead alternatives for the scalar and aligned-Series cases respectively. `apply` is the fallback when neither fits.

What NOT to say: - "`groupby.apply` is always slower than `groupby.agg`" — `apply` is slower when `agg` fits the shape, but `apply` is the only option for arbitrary per-group transformations. - "The output shape doesn't matter, pandas figures it out" — pandas does figure it out, but you should know which shape you're producing so the assembled result matches what your downstream code expects.

Common follow-up: "If `fn` is expensive and you only need a per-group scalar, which method should you pick?" — `agg`. It runs `fn` once per group with no shape-inference re-run and lets you pass a dict of `{column: function}` for multi-column aggregates in one call.


```
import pandas as pd

df = pd.DataFrame({
    "user_id": [1, 1, 1, 2, 2, 3],
    "amount": [10, 20, 30, 15, 25, 50],
    "city": ["mum", "mum", "del", "blr", "blr", "hyd"],
})

# Scalar return – Series indexed by group key
print(df.groupby("user_id")["amount"].apply(lambda s: s.max()).to_dict())
# {1: 30, 2: 25, 3: 50}

# Series return – DataFrame, columns from Series index
def stats(g):
    return pd.Series({"total": g["amount"].sum(), "n": len(g)})

print(df.groupby("user_id").apply(stats, include_groups=False))
#           total  n
# user_id
# 1           60  3
# 2           40  2
# 3           50  1

# DataFrame return – MultiIndex result
def top1(g):
    return g.nlargest(1, "amount")

print(df.groupby("user_id", group_keys=False)
      .apply(top1, include_groups=False))
#   amount city
# 2     30  del
# 4     25  blr
# 5     50  hyd
```

Section B — Vectorisation Patterns

Q11 · NumPy Ufuncs for Arithmetic — Why They Are 100x Faster Than `apply`

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Postman, Walmart Labs · Topic: numpy vectorisation

The question (interviewer's verbatim phrasing):

Explain why `df["a"] + df["b"]` is much faster than `df.apply(lambda r: r["a"] + r["b"], axis=1)`.

The 30-second answer: The first form is a NumPy ufunc — a single function call that operates on two contiguous memory blocks in a tight C loop, no Python interpreter involvement per element. The second form is a Python-level call per row, with all the dispatch, Series construction, and return boxing overhead the interpreter adds. On a million-row frame the first form runs in milliseconds; the second runs in tens of seconds. The gap is roughly 100 to 1000x for pure arithmetic and grows with the dtype regularity — pure float64 columns vectorise faster than mixed-dtype columns.

Step-by-step thought process: 1. Name the mechanism. A ufunc — universal function — is a NumPy C function that operates element-wise on arrays. `np.add`, `np.subtract`, `np.multiply`, `np.maximum`, `np.where` — all ufuncs. They take typed memory blocks, operate in one tight loop, and return a typed memory block. 2. Contrast with the Python loop cost. Each iteration of a Python for-loop pays: instruction dispatch, attribute lookup, boxing of the operand into a Python object, the operation itself, and unboxing of the result. The actual arithmetic is a tiny fraction of the total time. 3. Highlight the dtype angle. Ufuncs are fastest when the input arrays are contiguous and of a single dtype — float64, int64, bool. Mixed-dtype object columns force per-element dispatch, which closes the gap with `apply`. The Arrow backend in pandas 2.x extends ufunc-speed semantics to strings and timestamps. 4. Land on the actionable rule. For any operation that can be written as `df[col] op df[col]` or a NumPy function applied to columns, that form will beat any `apply`-based version by orders of magnitude. Reach for `apply` only when no ufunc or accessor method fits.

Edge cases to mention: - Ufuncs broadcast — `df["a"] * 2` works because NumPy broadcasts the scalar across the array. Same with `df["a"] + np.array(...)` when shapes align. - Some pandas ops look ufunc-shaped but are not — `df.apply(np.sum, axis=1)` still pays the per-row Python loop tax. Use `df.sum(axis=1)` instead. - Ufuncs respect NaN propagation — `np.nan + 1 == nan`. Use the `nanX` family (`np.nansum`, `np.nanmax`) when you want NaN-skipping arithmetic, or the equivalent pandas method which handles it by default.

What NOT to say: - "Vectorised pandas just hides a for-loop" — no, vectorised pandas dispatches to a C function that does not enter the Python interpreter per element. - "Ufuncs are NumPy-only — pandas can't use them" — pandas operations on numeric columns dispatch to NumPy ufuncs internally; that is exactly why they are fast.

Common follow-up: "Show me a one-liner that computes the row-wise standard deviation across columns `q1`, `q2`, `q3`, `q4`." — `df[["q1","q2","q3","q4"]].std(axis=1)`. Vectorised, single C call. Beats any `apply`-based version.

```
import pandas as pd
import numpy as np

n = 1_000_000
df = pd.DataFrame({"a": np.random.rand(n), "b": np.random.rand(n)})

# Vectorised – single ufunc call, milliseconds
df["sum_vec"] = df["a"] + df["b"]

# Equivalent via explicit ufunc
df["sum_ufunc"] = np.add(df["a"].values, df["b"].values)

# Slow – Python-level loop, seconds (don't run on a million rows)
# df["sum_slow"] = df.apply(lambda r: r["a"] + r["b"], axis=1)

# Other ufunc patterns
df["max_pair"] = np.maximum(df["a"], df["b"])
df["clipped"] = np.clip(df["a"], 0.1, 0.9)
```

Q12 · `np.where` and `np.select` – Vectorised Conditional Logic

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Cred, Postman, Walmart Labs, JP Morgan ·

Topic: numpy vectorisation

The question (interviewer's verbatim phrasing):

Your apply has a three-branch if/else inside it. Show me how to vectorise that without losing the branch logic.

The 30-second answer: For two branches, `np.where(cond, value_if_true, value_if_false)` — a single ufunc call that runs at C speed. For three or more branches, `np.select([cond1, cond2, cond3], [val1, val2, val3], default=val_else)` — same speed, cleaner than nested `np.where`. Both replace the row-wise apply with a single C-level vectorised pass. The result is 100 to 1000x faster on million-row frames and reads cleaner than the nested ternary form.

Step-by-step thought process: 1. Start with `np.where`. Signature is `np.where(cond, a, b)` — for each position, picks from `a` where `cond` is True, from `b` where False. The condition and both arms are arrays of the same shape; the operation is a single C ufunc. 2. For multi-branch, nested `np.where` becomes unreadable. `np.select(condlist, choicelist, default=...)` takes a list of conditions evaluated in order, with the first matching condition's choice taken. Same C speed, much clearer code. 3. `pd.Series.where` is the sibling — it keeps values where the condition is True and replaces others with NaN by default (or with a supplied value). Useful for "mask in place" patterns. 4.

Land on the perf number. Replacing an apply-with-branching with `np.select` on a million-row frame typically goes from 30 seconds to 50 milliseconds. That gap shows up on every Data interview perf question because it is the cleanest demo of the cliff.

Edge cases to mention: - `np.where` evaluates both arms even if the condition would skip one — so `np.where(x != 0, 1/x, 0)` still computes `1/x` for the zero positions and emits a warning. Mask the input or use `np.divide` with `where=` for division-by-zero safety. - `np.select` is greedy in condition order — the first matching condition wins. Put the most specific condition first; the default applies to whatever falls through. - For string output, use object-dtype values in the choice list. For Arrow-backed strings, `np.select` outputs object dtype, which you may want to cast back via `.astype("string[pyarrow]")`.

What NOT to say: - "I'd use a nested ternary inside `apply`" — that gets you back on the cliff. - "`np.where` is short-circuit" — it is not; both arms are evaluated for every position.

Common follow-up: "Bucket revenue into 'high' (>1L), 'mid' (10K-1L), 'low' (<10K). Write it three ways: nested `np.where`, `np.select`, and `pd.cut`. Which would you ship?" — `pd.cut` for binning, because it expresses intent ("these are bins") and produces a Categorical that downstream code can group by efficiently. `np.select` for irregular branches that are not bins. `np.where` only for true two-way conditions.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"revenue": [5_000, 50_000, 250_000, 8_000]})

# Two-branch — np.where
df["is_premium"] = np.where(df["revenue"] > 100_000, "yes", "no")

# Three-branch — np.select (cleaner than nested np.where)
conds = [
    df["revenue"] > 100_000,
    df["revenue"] > 10_000,
]
choices = ["high", "mid"]
df["tier"] = np.select(conds, choices, default="low")

print(df.to_dict(orient="records"))
# [{'revenue': 5000, 'is_premium': 'no', 'tier': 'low'},
#  {'revenue': 50000, 'is_premium': 'no', 'tier': 'mid'},
#  {'revenue': 250000, 'is_premium': 'yes', 'tier': 'high'},
#  {'revenue': 8000, 'is_premium': 'no', 'tier': 'low'}]
```

Q13 · `pd.cut` and `pd.qcut` — Binning Without `apply`

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, PhonePe, Cred, JP Morgan · Topic: pandas binning

The question (interviewer's verbatim phrasing):

You want to bucket users into deciles of spending. Which pandas function would you reach for, and what's the difference between `pd.cut` and `pd.qcut` ?

The 30-second answer: `pd.qcut` for deciles or any quantile-based binning — it places bin edges so each bin holds roughly the same number of observations. `pd.cut` for fixed-value bin edges — you supply `[0, 100, 1000, 10000]` and rows land in whichever range they fall into. `qcut` is the right tool for "split into N equal-sized buckets" (deciles, quartiles, percentiles); `cut` is the right tool for "split at business-meaningful thresholds" (low/mid/high tiers, age brackets, fixed price ranges). Both are vectorised and return a Categorical column that groups efficiently downstream.

Step-by-step thought process: 1. State the distinction clearly. `cut` takes value-based bin edges; `qcut` takes quantile-based bin counts (or quantile edges). The output of both is a Categorical Series — labelled bins that play nicely with `groupby` and `value_counts`. 2. Mention the `labels=` argument — you can pass strings to get readable bin names instead of the default `(0, 100]` interval form. Useful for analytics output and dashboards. 3. The duplicate-edge gotcha: `qcut` on a column with many ties at one value (heavily right-skewed revenue, for example) will raise about duplicate bin edges. Pass `duplicates="drop"` to consolidate. 4. Land on the perf angle. `pd.cut` and `pd.qcut` are both vectorised — a million-row binning runs in well under a second. The `apply`-based version (`df.apply(lambda x: 'high' if x > 1L else ...)`) takes 20 to 30 seconds and reads worse.

Edge cases to mention: - `pd.cut(x, bins=3)` with an integer `bins` argument splits the value range into 3 equal-width intervals — not equal-size; for equal-size, use `qcut` with `q=3`. - The first interval in `cut` is left-open by default — set `include_lowest=True` if your minimum value should land in bin 1, not be silently flagged NaN. - `qcut` returns NaN for values that fall outside any bin (rare unless `duplicates="drop"` consolidates everything).

What NOT to say: - "`cut` is for numeric and `qcut` is for categorical" — both take numeric input; the difference is bin-edge style. - "I'd just write `apply` with a custom function for clarity" — that is the perf cliff again, and `cut` / `qcut` already express intent more clearly.

Common follow-up: "How would you do per-group deciles — deciles of revenue within each city?" — `df["decile"] = df.groupby("city")["revenue"].transform(lambda s: pd.qcut(s, 10, labels=False, duplicates="drop"))`. The transform aligns each group's decile result back to the original index.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"revenue": np.random.exponential(10_000, 1000)})

# Fixed-edge binning – business thresholds
df["tier"] = pd.cut(
    df["revenue"],
    bins=[0, 10_000, 100_000, np.inf],
    labels=["low", "mid", "high"],
    include_lowest=True,
)

# Quantile-based – equal-sized buckets
df["decile"] = pd.qcut(df["revenue"], q=10, labels=False, duplicates="drop")

print(df["tier"].value_counts())
print(df.groupby("decile")["revenue"].agg(["min", "max"]).head(3))
```

Q14 · The `.str` Accessor — Vectorised String Operations

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Swiggy, Walmart Labs, Microsoft India ·

Topic: pandas strings

The question (interviewer's verbatim phrasing):

Convert this column of email addresses to lowercase and split each on `@` to extract the domain. Walk me through the pandas-idiomatic way.

The 30-second answer: Use the `.str` accessor — `df["email"].str.lower()` to lowercase, `df["email"].str.split("@").str[1]` to grab the domain part. Both are vectorised — the operations run in a single C-level pass over the column, no per-row Python call. On pandas 2.x with the Arrow backend, the `.str` methods are 3 to 10 times faster than the NumPy-backed equivalents on large columns because Arrow stores strings contiguously and the C loop does not pay per-string Python-object overhead.

Step-by-step thought process: 1. Lead with the accessor pattern. Every standard Python string method has a `.str` equivalent that operates element-wise on a Series — `lower`, `upper`, `strip`, `replace`, `contains`, `startswith`, `endswith`, `split`, `slice`, `cat`. They are vectorised; calling them is the pandas-idiomatic way to do string work on a column. 2. Walk through `.str.split` carefully. It returns a Series of lists by default; chaining `.str[1]` or `.str.get(1)` selects the element at index 1 from each list. To get separate columns instead of a list-Series, pass `expand=True`. 3. Highlight `.str.contains` and `.str.match` for filtering — they accept regex

by default and return a boolean Series ready for boolean indexing. Far faster than apply with a regex inside. 4. Land on the Arrow angle. Switching the column to `"string[pyarrow]"` dtype makes the same `.str` calls 3 to 10x faster on large frames and uses 30 to 50% less memory because Arrow dictionary-encodes repeated values.

Edge cases to mention: - `.str` operations return NaN for rows where the input is NaN — they do not raise. To get the original string back where missing, chain with `.fillna(...)`. - Chained `.str` calls are common — `df["x"].str.strip().str.lower().str.replace("@@", "@")`. Each step is vectorised; the chain is still C-speed. - For very complex regex on huge frames, the `pyarrow` engine in pandas 2.x is materially faster than the default Python `re` engine.

What NOT to say: - "I'd use `apply(lambda x: x.lower())`" — that's the per-row Python call again; `.str.lower()` is the vectorised idiom. - "`.str` methods are slow because they go through Python strings" — they don't; with NumPy backend they unbox once per row, with Arrow backend they do not unbox at all.

Common follow-up: "How would you split the email column into two new columns `local` and `domain` in one line?" — `df[["local", "domain"]] = df["email"].str.split("@", n=1, expand=True)`. The `expand=True` returns a DataFrame; assigning to a two-column slice lands each part in its own column.

```
import pandas as pd

df = pd.DataFrame({
    "email": ["Anita@Razorpay.com", "ravi@swiggy.IN", "priya@cred.club"]
})

# Vectorised string cleanup
df["clean"] = df["email"].str.strip().str.lower()
df["domain"] = df["clean"].str.split("@", n=1, expand=True)[1]

# Arrow-backed for 3–10x speedup on large columns
df = df.astype({"email": "string[pyarrow]"})

# Boolean filter via .str.contains
mask = df["clean"].str.contains(r"^[a-z]+@[a-z.]+$", regex=True)
print(df[mask][["clean", "domain"]].to_dict(orient="records"))
# [{'clean': 'anita@razorpay.com', 'domain': 'razorpay.com'},
#  {'clean': 'ravi@swiggy.in', 'domain': 'swiggy.in'},
#  {'clean': 'priya@cred.club', 'domain': 'cred.club'}]
```

Q15 · The `.dt` Accessor — Vectorised Datetime Operations

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Flipkart, Swiggy, PhonePe, JP Morgan · Topic: pandas datetime

The question (interviewer's verbatim phrasing):

Extract the year, month, day-of-week, and hour from a timestamp column. What's the pandas-idiomatic way?

The 30-second answer: Use the `.dt` accessor on a datetime Series —

`df["ts"].dt.year`, `.dt.month`, `.dt.day_name()`, `.dt.hour`. All vectorised, all C-speed. The same pattern handles timezones (`.dt.tz_convert("Asia/Kolkata")`), date arithmetic (`.dt.to_period("M")` for monthly buckets), and formatting (`.dt.strftime("%Y-%m-%d")`). Reaching for `apply(lambda ts: ts.year)` is the same per-row tax as on any other accessor — orders of magnitude slower than `.dt.year`.

Step-by-step thought process: 1. Confirm the dtype first. `.dt` only works on a datetime-typed Series (`datetime64[ns]` or `datetime64[ns, tz]`). If you read CSV without parsing dates, the column is object — call `pd.to_datetime()` first. 2. Walk through the common attributes. `year`, `month`, `day`, `hour`, `minute`, `second`, `weekday` (Monday=0), `dayofweek` (alias), `day_name()`, `month_name()`, `quarter`, `is_month_start`, `is_month_end`. All vectorised. 3. Mention timezone handling. `.dt.tz_localize("Asia/Kolkata")` attaches a timezone to a naive series; `.dt.tz_convert("UTC")` changes the timezone of an already-aware series. `pd.NaT` is the datetime NaN equivalent. 4. The 2.x Arrow detail — datetime columns under `"timestamp[ns, tz=Asia/Kolkata][pyarrow]"` are even faster for the same operations, particularly `.dt.floor`, `.dt.ceil`, `.dt.round` for time-bucketing analytics.

Edge cases to mention: - `dt.day_name()` returns localised names if `locale` is set; for portable output use `dt.weekday` (integer 0-6) or `dt.day_name(locale="en_US")` explicitly. - Naive vs aware mixing raises — you cannot subtract a naive datetime from an aware one. Use `dt.tz_localize` to align. - `pd.NaT` in a datetime Series behaves like NaN — comparisons return False, arithmetic propagates. Filter with `.isna()`.

What NOT to say: - "I'd parse the string with `apply` and a `strptime`" — `pd.to_datetime` is vectorised and supports `format=`, `errors="coerce"`, and a fast Arrow-backed path. - "Timezones are an application-layer concern" — for analytics across multiple regions, the database stores UTC, pandas converts on read; getting this wrong silently double-counts day boundaries.

Common follow-up: "How would you bucket timestamps into hourly windows for a rolling-volume calculation?" — `df["hour"] = df["ts"].dt.floor("h")` to get the hourly bucket, then `df.groupby("hour")["amount"].sum()`. For non-aligned windows use `df.resample("h", on="ts").sum()`.


```
import pandas as pd

df = pd.DataFrame({
    "ts": pd.to_datetime([
        "2026-05-30 09:15:00+05:30",
        "2026-05-30 14:30:00+05:30",
        "2026-05-31 23:45:00+05:30",
    ])
})

df["year"]    = df["ts"].dt.year
df["month"]   = df["ts"].dt.month
df["dow"]     = df["ts"].dt.day_name()
df["hour"]    = df["ts"].dt.hour
df["bucket"]  = df["ts"].dt.floor("h")
df["in_utc"]  = df["ts"].dt.tz_convert("UTC")

print(df[["year", "dow", "hour", "in_utc"]].to_dict(orient="records"))
# [{'year': 2026, 'dow': 'Saturday', 'hour': 9, 'in_utc': '...03:45:00+00:00'},
#  {'year': 2026, 'dow': 'Saturday', 'hour': 14, 'in_utc': '...09:00:00+00:00'},
#  {'year': 2026, 'dow': 'Sunday', 'hour': 23, 'in_utc': '...18:15:00+00:00'}]
```

Section C — Performance Optimisation and Escape Hatches

Q16 · Profiling pandas Code — `%timeit`, `%prun`, and `line_profiler`

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs, Microsoft India, Goldman Sachs · Topic: pandas perf

The question (interviewer's verbatim phrasing):

Your colleague says their pandas pipeline is slow. How do you figure out which step is the bottleneck before you start rewriting?

The 30-second answer: Three tools, three altitudes. `%timeit` in a notebook for a single expression — it runs the expression many times, reports mean and standard deviation, and is the right tool for "is operation A faster than operation B." `%prun` (cProfile) for a whole function — shows time per called function, useful for spotting which pandas method is the actual bottleneck. `line_profiler` (`%lprun`) when you need per-line timing inside a long function — most expensive line jumps out. Profile first, rewrite second; intuition about pandas perf is famously wrong, even for senior engineers.

Step-by-step thought process: 1. State the "profile first" rule. Pandas perf intuition is unreliable because the cost depends on dtype, frame size, copy-vs-view semantics, and the specific operation.

Measure before you rewrite. 2. Walk through `%timeit`. Best for comparing two expressions.

`%timeit df["a"] + df["b"]` versus `%timeit df.apply(lambda r: r["a"] + r["b"], axis=1)`. The output is a per-call mean, which is what you want for relative comparisons. 3. `%prun`

for whole-function profiling. The output is a per-function call count and cumulative time. Useful for identifying which pandas method is being called the most or for the longest cumulative time. 4.

`line_profiler` for line-level. `%lprun -f my_func my_func(args)` shows time per line inside `my_func`. The most expensive line is usually the one to rewrite first. For 80% of cases `%timeit` + `%prun` is enough.

Edge cases to mention: - `%timeit` warmup matters — pandas does a lot of one-time setup on the first call (Arrow dtype caching, index hashing). Warm-up runs first and use `-n` to set the number of executions for tight expressions. - `%prun` output is dominated by pandas internals — filter with `-l` to limit output to the top N functions or grep for your own function names. - Memory profiling is a separate tool — `memory_profiler` for line-level memory, `df.memory_usage(deep=True)` for per-column memory of a DataFrame.

What NOT to say: - "I'd just rewrite anything with `apply`" — `apply` is sometimes the right call (per-group apply on a small number of groups), and the bottleneck may be elsewhere (a `merge` on an unsorted key, a `concat` that copies everything). - "I'll add prints and measure timestamps myself" — works but noisy; `%timeit` and `%prun` are the standard tooling and give comparable numbers across machines.

Common follow-up: "What's the perf model for `df.merge` on a million-row left frame and a hundred-thousand-row right frame?" — Sort-merge if the keys are unsorted (the sort dominates); hash-join if pandas can build a hash on the smaller side. Setting the merge key as the index ahead of time often gives a 2-5x speedup because pandas can skip the hash build.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"a": np.random.rand(100_000), "b": np.random.rand(100_000)})

# In Jupyter
# %timeit df["a"] + df["b"]
# 200 microseconds per call

# %timeit df.apply(lambda r: r["a"] + r["b"], axis=1)
# 1.5 seconds per call (about 7500x slower)

# %prun -l 10 do_pipeline(df)
# top 10 functions by cumulative time

# %lprun -f do_pipeline do_pipeline(df)
# per-line timing inside do_pipeline
```

Q17 · Avoiding Unnecessary Copies — Copy-on-Write and View Discipline

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Swiggy, Walmart Labs, Microsoft India, JP Morgan · Topic: pandas perf

The question (interviewer's verbatim phrasing):

Your pandas pipeline allocates 3 GB of memory and your laptop has 8 GB free. Walk me through where the allocations are coming from and how to reduce them.

The 30-second answer: The four big sources of unnecessary copies are `df[df["x"] > 0]` (boolean filtering creates a new frame), chained assignment that triggers `SettingWithCopyWarning` (sometimes makes a hidden copy you did not ask for), `pd.concat` of many small frames (each concat allocates the full output), and `df.merge` with a non-indexed key (sort plus copy). Reduce them with copy-on-write semantics (`pd.options.mode.copy_on_write = True` in pandas 2.2; default in 3.0), in-place operations where the semantics are clear, pre-sizing via `pd.concat` once at the end instead of in a loop, and `.merge` on indexed keys.

Step-by-step thought process: 1. Anchor on the dominant pattern. Pandas operations default to returning a new DataFrame; chained operations stack copies. A pipeline of ten `.filter().assign().merge().drop()` calls can multiply memory by 10x at peak. 2. Walk through copy-on-write. Enabled via `pd.options.mode.copy_on_write = True` in 2.2, default in 3.0. Pandas internally tracks which frames share memory and only copies when a write would actually alter data the user did not own. The user-visible effect: every derived frame appears independent; chained-assignment warnings disappear; memory drops materially in practice. 3. Mention the in-place option but with caveats. `df.drop(...)` returns a new frame by default; `df.drop(..., inplace=True)` modifies in place. The pandas team has been steering users away from `inplace=True` because the perf benefit is small and the semantics-clarity cost is large. Under CoW, `inplace=True` does not save memory — pandas already avoids the unneeded copy. 4. The concat-in-a-loop antipattern. Each `pd.concat([acc, new_df])` inside a loop allocates the full size of `acc + new_df`. The right pattern is to collect frames in a list and `pd.concat(list)` once at the end — one allocation, not N.

Edge cases to mention: - `df.loc[mask, "col"] = value` is the right way to set in place under CoW. `df[mask]["col"] = value` is the broken chained pattern that has either silently copied or raised in pre-CoW pandas. - Reading data with `dtype_backend="pyarrow"` typically uses 30 to 50% less memory for string-heavy frames and is the single biggest memory win on real-world DataFrames. - `df.memory_usage(deep=True)` is your tool for seeing per-column memory; the `deep=True` argument is required to account for object-dtype data, which lives off-row.

What NOT to say: - "Just use `inplace=True` everywhere" — the pandas team is deprecating that pattern; CoW is the modern answer. - "Memory doesn't matter on cloud machines" — it does for bench tests, for laptops, for kernel-killed notebooks, and for the cost of holding a million rows in pandas vs a hundred million in polars.

Common follow-up: "What's the difference between pandas CoW and explicit `.copy()`?" — CoW makes the copy lazily and only when a write demands it; explicit `.copy()` allocates immediately and unconditionally. Under CoW, you almost never need explicit `.copy()` anymore.

```
import pandas as pd

pd.options.mode.copy_on_write = True      # opt in (default in 3.0)

df = pd.DataFrame({"a": range(100_000), "b": range(100_000)})

# Right way to write under CoW
df.loc[df["a"] > 50_000, "b"] = -1

# Wrong (chained – broken in CoW; raised pre-CoW with SettingWithCopyWarning)
# df[df["a"] > 50_000]["b"] = -1

# Concat in a loop – antipattern, O(n²) memory and time
# acc = pd.DataFrame()
# for f in frames:
#     acc = pd.concat([acc, f])  # allocates full size each iter

# Concat once at the end – O(n) memory and time
acc = pd.concat(frames, ignore_index=True)

# See per-column memory
print(df.memory_usage(deep=True))
```

Q18 · The Arrow Backend — Why pandas 2.x Is Faster Than 1.x

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Razorpay, Postman, Cred, Microsoft India · Topic: pandas Arrow

The question (interviewer's verbatim phrasing):

Tell me about the Arrow backend in pandas 2.x. What is it, what gets faster, and how do you opt in?

The 30-second answer: pandas 2.0 introduced Arrow — the Apache Arrow columnar memory format — as an alternative storage backend to NumPy. The API shape did not change; the memory layout did. Strings become contiguous instead of one Python object per row, missing values get a single universal marker (`pd.NA`) instead of three different sentinels, and operations that previously paid per-row Python overhead now run in tight C loops. Opt in with `pd.read_csv("...", dtype_backend="pyarrow")` or `df.convert_dtypes(dtype_backend="pyarrow")` on an existing frame. The win is typically 3 to 10x on `.str` methods, 30 to 50% less memory on string-heavy

frames, and 2 to 4x faster Parquet reads. Mid-2026 it is interview-table-stakes at Indian product cos and GCCs.

Step-by-step thought process: 1. Name the change. NumPy backend stored DataFrames as a dict of NumPy arrays; strings were stored as object dtype, one Python str per row. Arrow backend stores strings contiguously in a single buffer plus an offsets array — same C-level layout as Parquet on disk. 2. Walk through the four wins the candidate should be able to articulate. (a) String operations are 3 to 10x faster on large columns because the work runs as a single C-level scan, not a Python-object-per-element loop. (b) Memory drops 30 to 50% on string-heavy frames because Arrow dictionary-encodes repeats. (c) Parquet reads are 2 to 4x faster because the in-memory layout matches the on-disk layout — no per-row conversion. (d) `pd.NA` is the universal missing-value marker, which removes the `np.nan` vs `None` vs `NaT` confusion. 3. Mention the opt-in mechanics. On read: `pd.read_csv(path, dtype_backend="pyarrow")`. On an existing frame: `df.convert_dtypes(dtype_backend="pyarrow")`. Column-specific: `df.astype({"city": "string[pyarrow]", "ts": "timestamp[ns][pyarrow]"})`. 4. Land on the gotchas. Some pandas methods still fall back to NumPy internally; for those the Arrow layout is a temporary perf win that pays a conversion cost. The free-threaded 3.13 build interacts with Arrow differently than NumPy — Arrow buffers are not GIL-protected the same way. For greenfield work, the Arrow backend is the default in 2026.

Edge cases to mention: - The `engine="pyarrow"` argument on `read_csv` (separate from `dtype_backend`) uses Arrow's C++ CSV parser — typically 5 to 10x faster than pandas's native parser on 1 GB+ files. - Arrow strings are immutable; in-place mutation of a string buffer triggers a full copy. For workloads heavy on `.str.replace` chains, the perf gap with NumPy narrows. - Some seaborn / matplotlib plot calls do not yet accept Arrow-backed Series cleanly. Convert via `.astype(object)` for display.

What NOT to say: - "Arrow is just for big data" — Arrow is for any frame where strings, missing values, or Parquet I/O matter, which is most Data work. - "Switching to Arrow is risky" — for new code it is the right default; for existing code the migration is mostly transparent and the perf wins are large.

Common follow-up: "What's the perf difference between `engine='pyarrow'` on `read_csv` and `dtype_backend='pyarrow'`?" — `engine` controls the parser (Arrow C++ vs pandas's Python); `dtype_backend` controls the in-memory storage after parsing. They are independent and you usually want both for the maximum win.

```
import pandas as pd

# Read with Arrow parser AND Arrow-backed dtypes
df = pd.read_csv("orders.csv",
                 engine="pyarrow",
                 dtype_backend="pyarrow")

# Convert existing NumPy-backed frame
df_arrow = df.convert_dtypes(dtype_backend="pyarrow")

# Per-column opt-in
df = df.astype({
    "city": "string[pyarrow]",
    "created_at": "timestamp[ns][pyarrow]",
    "amount": "double[pyarrow]",
})

# pd.NA is the universal missing marker under Arrow
print(df["city"].isna().sum())
```

Q19 · `df.eval` and `df.query` — When Complex Expressions Run Faster as Strings

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs, JP Morgan, Goldman Sachs, Microsoft India · Topic: pandas perf

The question (interviewer's verbatim phrasing):

You have a complex boolean filter with five conditions across three columns on a ten-million-row frame. Would `df.query("...")` be faster or slower than the boolean-mask version?

The 30-second answer: On a large frame with a multi-term expression, `df.eval` and `df.query` can be 2 to 3x faster than the equivalent boolean-mask chain because they fuse the operations through `numexpr` — a library that combines multiple ufunc passes into a single C-level pass and avoids materialising intermediate boolean arrays. The break-even is roughly 100k rows and 3+ terms in the expression. Below that, the parsing overhead of `eval` and `query` makes them slower than the direct boolean form. For one-off ad-hoc queries, the boolean-mask form reads cleaner; for hot-path code, `eval` and `query` deliver real wins.

Step-by-step thought process: 1. Explain the mechanism. `df.query(expr)` and `df.eval(expr)` parse the expression string, hand it to `numexpr`, which compiles the operation chain into a single C-level loop. The optimisation comes from fusing — `(a + b) > c & (a - b) < d` becomes one pass instead of four intermediate arrays. 2. State the break-even. Numexpr's fusion

only pays off when (a) the frame is large enough that the per-element work dominates the parsing overhead, typically 100k rows or more, and (b) the expression has multiple operations to fuse — a single comparison gets no win. 3. Mention the readability angle.

`df.query("amount > 1000 and city == 'mumbai' and not refunded")` reads like a sentence; the boolean form gets verbose for 5+ conditions. For interactive analysis the readability is a real win even when the perf is a wash. 4. Land on the gotchas. `query` uses Python-like syntax but is not Python — `and` / `or` / `not` are spelled out, `in` works for membership, you reference Python variables with `@var`, and you cannot call arbitrary functions. For complex logic that does not fit `numexpr`'s surface, the boolean mask is the right tool.

Edge cases to mention: - `df.query` does not work on Arrow-backed columns in all pandas 2.x versions — check the release notes before relying on it for Arrow data. - `df.eval` returns a Series or DataFrame depending on the expression; for assignment use `df.eval("c = a + b", inplace=True)` to add a column. - Backtick column names with `df.query("`column with space`")` for non-identifier column names.

What NOT to say: - "`df.query` is always faster" — for small frames or simple expressions, the parsing overhead dominates and the boolean form wins. - "`df.query` runs arbitrary Python" — it does not; the expression is constrained to a `numexpr`-compatible subset.

Common follow-up: "*Why does pandas's docs recommend `df.eval` for chained arithmetic over the direct form?*" — Memory. The fused single-pass form does not materialise intermediate arrays; for a 10-million-row frame with three arithmetic terms, the eval form can save 80 MB of intermediate allocation per call.

```
import pandas as pd
import numpy as np

n = 10_000_000
df = pd.DataFrame({
    "amount": np.random.rand(n) * 10_000,
    "city": np.random.choice(["mumbai", "delhi", "blr"], n),
    "refund": np.random.rand(n) < 0.05,
})

# Boolean mask – readable for 2–3 terms, slower for many
mask = (df["amount"] > 1000) & (df["city"] == "mumbai") & (~df["refund"])
print(df[mask].shape)

# query – sentence-like, faster on large frames with multi-term filters
print(df.query("amount > 1000 and city == 'mumbai' and not refund").shape)

# eval for fused arithmetic – saves intermediate arrays
df.eval("score = amount * 0.02 + (amount > 5000) * 100", inplace=True)
```


Q20 · When pandas Isn't Fast Enough — `numba`, `polars`, and the Honest Tradeoff

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, Walmart Labs, Microsoft India, Goldman Sachs · Topic: pandas perf

The question (interviewer's verbatim phrasing):

You've vectorised everything you can, you're on the Arrow backend, and your pipeline is still too slow. What are your escape hatches?

The 30-second answer: Three escape hatches, in order of effort. `numba`'s `@jit` for a single hot per-row function — JIT-compiles your Python to native code, typically 10 to 100x faster, useful when the work genuinely cannot vectorise. `polars` for whole-pipeline rewrites — written in Rust, lazy evaluation, multi-threaded by default, 5 to 30x faster than pandas on most analytic workloads, lighter memory. Daft or PySpark for distributed work when the data does not fit on one machine. The honest tradeoff: `numba` adds a JIT-warmup cost and works best on numeric loops; `polars` has a different API surface and your team has to learn it; PySpark is justified only when single-node solutions exhaust memory.

Step-by-step thought process: 1. Start with `numba`. Decorate a function with `@jit`, pass NumPy arrays in. The first call compiles (warmup cost, often a few seconds); subsequent calls run at compiled-C speed. Useful for genuinely non-vectorisable per-row logic that you cannot express as ufuncs. `numba.vectorize` gives you a custom ufunc with broadcasting. 2. `Polars` is the modern alternative. Written in Rust, columnar, multi-threaded by default, lazy evaluation lets the optimiser fuse operations the way SQL planners do. Same shape of API as pandas (`DataFrame`, `GroupBy`, expressions) but the methods are different. Migration is a real cost but the perf wins on analytic workloads are large — typically 5 to 30x faster, 30 to 50% lower memory. 3. For data that does not fit on a single machine, PySpark or Daft (newer, lighter than Spark). The boundary is around 10 to 50 GB of data depending on the laptop; below that, `polars` on one machine beats Spark on a cluster on most workloads because there is no shuffle cost. 4. Land on the practitioner judgement. Most pandas perf problems are not "pandas is slow" — they are "I wrote `apply` where I should have vectorised" or "I am on NumPy backend where Arrow would help." Exhaust those before reaching for `numba` or `polars`. The reach-for-`polars` threshold is "my vectorised pandas pipeline takes 30+ seconds on hardware I cannot upgrade, and I cannot find more vectorisation."

Edge cases to mention: - `numba`'s JIT warmup is real — for short-running scripts the warmup can outweigh the runtime win. Profile the warmup cost into your decision. - `Polars` has lazy and eager APIs. The lazy API (`pl.scan_csv`, `.collect()`) lets the optimiser fuse the whole pipeline; the eager API is closer to pandas but loses the optimisation. Most `polars`-using code at Razorpay and Postman uses the lazy form for the analytic pipelines. - `swifter`, `pandarallel`, and `modin` exist as drop-in `apply` accelerators. They give 2 to 8x speedups by parallelising the per-row loop. They do not change the order-of-magnitude analysis — vectorisation is still better when it fits.

What NOT to say: - "Polars is always faster, just use it" — for code that has to integrate with existing pandas-using teams and notebooks, the rewrite cost is real. The decision is a team judgement, not a

tech one. - "I'd reach for Spark for any 1 GB job" — single-node polars beats single-cluster Spark up to about 50 GB on typical analytic workloads.

Common follow-up: "Show me a `@njit` function that computes a custom per-row metric on a million-row frame." — The function takes NumPy arrays as input, not a DataFrame, and writes its output to a pre-allocated array. The DataFrame boundary is for I/O; the numba boundary is for hot loops. Use `.to_numpy()` to convert at the boundary.

```
import pandas as pd
import numpy as np
from numba import njit

df = pd.DataFrame({
    "a": np.random.rand(1_000_000),
    "b": np.random.rand(1_000_000),
})

# numba - JIT-compiled per-row loop, no Python overhead
@njit
def custom_metric(a, b):
    out = np.empty_like(a)
    for i in range(a.shape[0]):
        if a[i] > b[i]:
            out[i] = a[i] * 0.95 + b[i] * 0.05
        else:
            out[i] = a[i] * 0.05 + b[i] * 0.95
    return out

df["metric"] = custom_metric(df["a"].to_numpy(), df["b"].to_numpy())

# polars - Rust-backed, lazy, multi-threaded
# import polars as pl
# pdf = pl.from_pandas(df)
# result = (pdf.lazy()
#           .filter(pl.col("a") > 0.5)
#           .group_by("city")
#           .agg(pl.col("amount").sum())
#           .collect())
```

Chapter wrap

The arc of this chapter is one rule sharpened twenty different ways — `apply` is the fallback, vectorisation is the default, profiling is the discipline. The candidate who internalises that arc clears the perf segment of any Data interview because the rule scales. A two-line subtraction beats a two-page `apply`; `np.select` beats nested `if/else`; the `.str` and `.dt` accessors beat element-wise string and datetime work; `pd.cut` and `pd.qcut` beat hand-rolled bucketing; the Arrow backend gives 3 to 10x on string operations for the cost of one keyword argument; `eval` and `query` deliver a

fused single-pass win on multi-term expressions; copy-on-write removes a whole class of memory-bloat bugs; and when pandas truly is not fast enough, `numba` for hot loops and polars for whole pipelines are the escape hatches. The Indian Data-interview gradient — TCS / Infosys at the reading-comprehension end, Razorpay / Flipkart / Swiggy / Postman / Cred at the perf-reasoning end, JP Morgan / Goldman / Walmart Labs / Microsoft India at the deeper-systems end — is covered by mastering these twenty Qs in order. Read them once for the pattern; come back to them the week before the interview for the muscle memory.

The next chapter (Chapter 6 — NumPy core) goes one level down — ndarray internals, broadcasting rules, axis semantics, fancy indexing, and the pitfalls that catch even experienced pandas users when they drop into raw NumPy.

Chapter 6 — numpy core (30 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

This is the numpy chapter of the Data Track Library. Thirty questions covering the four surfaces every Indian data interviewer probes: the `ndarray` object itself (dtype, shape, strides, views vs copies), broadcasting (the rules, the legal shapes, the error message you keep seeing), axis semantics (the difference between `axis=0` and `axis=1`, `keepdims`, when `axis=None` is meant), indexing (basic vs fancy, view vs copy, boolean masks, the `np.ix_` pattern), and the small numerics block (`np.dot`, `@`, `np.linalg.solve` vs `inv`, the modern `default_rng` API, `astype`). Each question follows the format you've seen across the pack — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases to surface, what NOT to say, the common follow-up, and runnable code. Answers assume numpy 1.26 / 2.0 on Python 3.12 unless a version is called out.

Table of Sections

- Section A — `ndarray` Basics: dtype, shape, strides, order, views (Q1–Q5)
 - Section B — Broadcasting: rules, legal shapes, `np.newaxis` (Q6–Q15)
 - Section C — `axis=` Semantics: reductions, `keepdims`, `axis=None` (Q16–Q20)
 - Section D — Indexing: basic, fancy, boolean masks, `np.ix_` (Q21–Q25)
 - Section E — Numerics + Linear Algebra: ufuncs, `@`, `linalg.solve`, `default_rng`, `astype` (Q26–Q30)
-

Section A — ndarray Basics

The first five questions are the ones every data-role interviewer opens with at companies like Swiggy, Razorpay, PhonePe, Flipkart, and Walmart Labs. They look basic, but candidates who slip on `dtype` choice, who confuse `shape` with `size`, or who don't know whether a slice gave them a view or a copy will spend the rest of the interview defending themselves. The `ndarray` object is a small C struct with a few critical attributes — `dtype`, `shape`, `strides`, `flags`, `data` — and almost every numpy bug you ship to production comes from misreading one of those five.

Q1 · What attributes does an `ndarray` actually carry, and what's the difference between `shape`, `size`, and `ndim`?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Walmart Labs · Topic: ndarray basics

The question (interviewer's verbatim phrasing):

When you create an array with `np.zeros((3, 4))`, what attributes does the resulting object expose? Walk me through `shape` versus `size` versus `ndim` versus `dtype`.

The 30-second answer: An `ndarray` is a thin Python wrapper over a contiguous C buffer plus a few metadata fields. The four you must name are `shape` — the tuple of dimensions, like `(3, 4)`; `size` — the total element count, the product of the shape, here `12`; `ndim` — the number of dimensions, the length of `shape`, here `2`; and `dtype` — the element type, here `float64`. There are three more worth knowing: `strides` (bytes to skip per axis), `itemsize` (bytes per element), and `nbytes` (`size * itemsize`).

Step-by-step thought process: 1. Lead with the headline picture: `ndarray` is a struct over a flat C buffer. Almost every attribute is about how that flat buffer gets reshaped into the n-dimensional view you see in Python. 2. Walk the interviewer through the four-of-seven that get asked about. `shape` is the tuple. `size` is the count of cells. `ndim` is `len(shape)`. `dtype` is the element type. Get the relationship out loud — `size == prod(shape) == prod of the dimensions`. 3. Mention `strides`, `itemsize`, `nbytes` even if not asked — these signal you've read the docs. `strides` is how numpy implements views without copying, and it's the foundation of broadcasting, transposes, and slicing all returning views. 4. Anchor it in practice: `arr.shape[0]` is row count, `arr.shape[1]` is column count for 2-D. `arr.size` is what you'd use when computing memory or sanity-checking element counts. `arr.ndim` is what you'd branch on if your function accepts 1-D and 2-D inputs.

Edge cases to mention: - A 0-dimensional array has `shape=()`, `ndim=0`, `size=1` — yes, size is 1 not 0, because there's still one scalar inside. `np.array(5)` is the canonical example. - For a 1-D array of 100 elements, `shape` is `(100,)` with the trailing comma — that's a tuple, not `(100)`. Beginners write `(100)` and get back an int from `np.zeros((100))` failing — call this out. - `size`

is a Python `int`, not a numpy scalar, and it can be very large — for a `(1_000_000, 1_000)` array, `size` is `10**9`. Use it before allocating to sanity-check you're not about to OOM.

What NOT to say: - "`shape` and `size` are the same thing" — they aren't; `shape` is a tuple of dimensions, `size` is a single integer count. - "`ndim` is the row count" — `ndim` is the count of axes, not rows. For a `(3, 4)` array `ndim` is `2`, not `3`.

Common follow-up: "How would you check if an array is 2-D before applying a 2-D-only operation?" — `if arr.ndim == 2:` is the idiomatic check. `assert arr.ndim == 2, f"expected 2-D, got {arr.ndim}-D"` is what you'd write at the top of a function that's strict about shape.

```
import numpy as np

a = np.zeros((3, 4))
print(a.shape)      # (3, 4)
print(a.size)       # 12
print(a.ndim)       # 2
print(a.dtype)      # float64
print(a.itemsize)   # 8
print(a.nbytes)     # 96 (12 * 8)
print(a.strides)    # (32, 8) - 32 bytes to next row, 8 bytes to next column
```

Q2 · How do you pick the right `dtype`, and why does the default `float64` sometimes hurt you?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Goldman, JP Morgan · Topic: dtype choices

The question (interviewer's verbatim phrasing):

When you write `np.zeros(1_000_000)`, numpy gives you a `float64` array by default. When would you override that, and what dtype would you pick instead?

The 30-second answer: `float64` is the safe default — 8 bytes per element, full IEEE 754 double precision, the same precision as a Python `float`. You override it when memory matters, when the data is structurally not a float, or when downstream tooling expects a narrower type. Common overrides: `int32` or `int64` for counters and IDs, `float32` for ML feature matrices to halve memory, `bool` for masks, `uint8` for image pixels, `datetime64[ns]` for timestamps. The rule of thumb is: pick the narrowest type that holds your range and your precision, because numpy's vector speed scales inversely with bytes per element.

Step-by-step thought process: 1. Open with the why-default-is-float64: numpy's design choice is to never silently lose precision, and `float64` matches Python's `float`, so `np.array([1, 2,`

3.14]) upcasts to `float64` without surprises. 2. Walk through the four override drivers. Memory — a 10-million-row `float32` array is 40 MB versus 80 MB; matters when you have 30 of them. Semantics — pixel values are `uint8`, IDs are `int64`, flags are `bool`. Downstream tool — PyTorch defaults to `float32`, scikit-learn doesn't care, but a Parquet writer cares a lot. Range — `int8` overflows silently at 128, so don't use it for anything you can't bound. 3. Mention overflow without a warning. `np.array([200], dtype=np.int8) + np.array([100], dtype=np.int8)` returns `[44]` because of wraparound. numpy doesn't raise — it's C-style two's complement. This bites people coming from Python ints. 4. Default to `int64` and `float64` unless you have a specific reason. The memory savings of `int32` versus `int64` is usually not worth the overflow risk on a counter or ID column.

Edge cases to mention: - `np.zeros(5)` is `float64`; `np.zeros(5, dtype=int)` is platform-dependent — on 64-bit Linux it's `int64`, on 64-bit Windows it's `int32`. Always pass the explicit `np.int64` if portability matters. - `bool` dtype takes one byte per element, not one bit — numpy doesn't pack bits. For a million-element mask, that's 1 MB, not 125 KB. - `dtype=object` is the escape hatch — stores Python object pointers per slot, kills all vectorized speed. Tempting for mixed-type arrays; almost always the wrong answer. Use pandas if you have mixed types.

What NOT to say: - "I always use `float32` to save memory" — half the precision often shows up as silent training drift in ML, and downstream code may expect `float64`. Pick deliberately, don't default. - "I'll use `dtype=object` for flexibility" — that signals you don't understand the point of numpy.

Common follow-up: "You have a 100-million-row array of integers that fit in `int16`. Worth using `int16`?" — Yes if memory is constrained — that's 200 MB versus 800 MB for `int64`. Watch for overflow on any arithmetic that could exceed $\pm 32,767$, and cast up explicitly before reductions if you're summing into a number larger than `int16` max.

```
import numpy as np

# default is float64
print(np.zeros(3).dtype)           # float64

# explicit narrower float
arr = np.zeros(1_000_000, dtype=np.float32)
print(arr.nbytes)                  # 4_000_000 — half of float64

# overflow is silent — this is the trap
small = np.array([200], dtype=np.int8)
print(small + small)               # [-112], not [400]
```

Q3 · What are `strides`, and why does the same buffer give you different views?

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Microsoft India, Walmart Labs, Postman · Topic: strides, views

The question (interviewer's verbatim phrasing):

An array has a `strides` attribute that's a tuple of bytes. What does it mean, and how does numpy use it to make `arr.T` instant?

The 30-second answer: `strides` is the tuple of bytes you skip in the underlying buffer to move one step along each axis. For a C-contiguous `(3, 4) float64` array, strides are `(32, 8)` — 32 bytes (4 elements × 8 bytes) to move to the next row, 8 bytes to move to the next column. Transposing the array doesn't copy any data — numpy just swaps the strides to `(8, 32)`, so now "next row" walks the old columns. That's why `arr.T` is O(1) memory-wise and runs instantly regardless of array size.

Step-by-step thought process: 1. Start with the picture: the buffer is a flat 1-D byte sequence; `shape` plus `strides` is the lens that makes it look n-dimensional. 2. Compute the strides for a small example out loud. A `(3, 4) float64` C-contiguous array has 12 elements in row-major order — row 0, then row 1, then row 2. To go from `arr[0, 0]` to `arr[1, 0]` you skip 4 elements = 32 bytes. To go from `arr[0, 0]` to `arr[0, 1]` you skip 1 element = 8 bytes. So `strides=(32, 8)`. 3. Now show the transpose trick. `arr.T` is the same buffer with `strides=(8, 32)` and `shape=(4, 3)`. No copy. Walking down a column of the original is the same byte pattern as walking across a row of the transpose. 4. Mention what breaks the trick: any operation that requires contiguous memory (some C extensions, some numpy functions like older `np.ravel(order='K')` semantics) will quietly copy when handed a non-contiguous view. Use `arr.flags.c_contiguous` and `arr.flags.f_contiguous` to check.

Edge cases to mention: - A slice like `arr[:, :2]` returns a view with the stride doubled — `(64, 8)` instead of `(32, 8)`. That's why slicing is O(1) memory; it's the same trick as transpose. - A reshape that fits the existing strides is a view; a reshape that doesn't is a copy. `arr.reshape(-1)` on a C-contiguous array is a view; on a transposed array, it has to copy. Numpy will copy without warning. - Negative strides happen when you reverse: `arr[::-1]` has `strides=(-8,)` and `data` pointing to the last element. Some C libraries don't accept negative strides; cast with `np.ascontiguousarray` first if exporting.

What NOT to say: - "Strides are an internal implementation detail" — they're surfaced specifically because they explain the view-vs-copy behavior, and interviewers want to hear you know it. - "Transposing copies the data" — it doesn't, and that's the whole point of strides.

Common follow-up: "What does `np.ascontiguousarray(arr)` do, and when would you call it?" — It forces a C-contiguous copy if the input isn't already contiguous. Call it before handing the array to a C extension (e.g., a hand-rolled Cython kernel) that assumes row-major contiguous memory.

```
import numpy as np

a = np.arange(12, dtype=np.float64).reshape(3, 4)
print(a.strides)          # (32, 8)
print(a.flags.c_contiguous) # True

b = a.T
print(b.shape)             # (4, 3)
print(b.strides)           # (8, 32) — swapped, no copy
print(b.flags.c_contiguous) # False
print(b.flags.f_contiguous) # True — Fortran-contiguous after transpose
```

Q4 · C order versus F order — what's the difference, and when does it matter?

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Microsoft India, Swiggy · Topic: memory layout

The question (interviewer's verbatim phrasing):

What's the difference between `np.zeros((3, 4), order='C')` and `np.zeros((3, 4), order='F')` ? When would you actually pick `F` order?

The 30-second answer: C order is row-major — adjacent elements along the last axis sit next to each other in memory. F order is column-major — adjacent elements along the first axis sit next to each other. For a `(3, 4)` array in C order, the buffer is `[r0c0, r0c1, r0c2, r0c3, r1c0, r1c1, ...]`. In F order, the same `(3, 4)` shape stores `[r0c0, r1c0, r2c0, r0c1, r1c1, ...]`. You pick F order when you're interoperating with Fortran-based libraries — LAPACK, classic BLAS interfaces, R, MATLAB — or when you'll be iterating heavily down columns and want cache locality there. For everything else, stick with the default C order.

Step-by-step thought process: 1. Lead with row-major versus column-major. Then immediately ground it in the byte layout — for a `(3, 4) float64` array, C order gives `strides=(32, 8)` and F order gives `strides=(8, 24)`. Same data, different walking order. 2. Explain why it matters: cache locality. When you iterate down a column of a C-order array, you jump 32 bytes per step — that's likely a cache miss per element. Iterate down a column of an F-order array, you walk 8 bytes per step — one cache line holds many. Reverse for row iteration. 3. Call out the interop angle. SciPy's `linalg` wraps LAPACK, which expects F order. NumPy will silently transpose-and-copy when you pass C-order arrays into LAPACK-backed functions, doubling memory briefly. F-order arrays skip that copy. 4. State the default and the practical rule. C order is the default because Python's nested-list literal `[[1,2,3], [4,5,6]]` is row-major. Don't override unless you have a measured reason — a transposed matrix is F-contiguous for free, so just transpose if you need columns fast.

Edge cases to mention: - `arr.T` gives you F-contiguous view of a C-contiguous array without copying. So instead of `np.zeros((m, n), order='F')`, you can write `np.zeros((n, m)).T` if the shape semantics allow. - `arr.reshape(-1, order='C')` flattens in C order regardless of the source layout; `order='K'` keeps the underlying memory order. The default is `'C'`, which surprises people who started with an F-order array. - An array that is neither C-contiguous nor F-contiguous shows up after operations like `arr[:, ::2]` — a strided view that's neither. Check `arr.flags` to confirm.

What NOT to say: - "C and F order are the same, numpy abstracts it away" — they're not, and the abstraction leaks through performance. - "Always use F order, it's faster for linear algebra" — only for column-heavy work and LAPACK-backed calls. For typical pandas-style row processing, C is faster.

Common follow-up: "You're computing column sums on a `(1_000_000, 10)` matrix. Which order is faster?" — F order (or equivalently, transposing a `(10, 1_000_000)` C-order array). Column sums walk down columns; F order makes those walks sequential in memory, so the CPU prefetcher actually helps.

```
import numpy as np

c = np.zeros((3, 4), order='C')
f = np.zeros((3, 4), order='F')
print(c.strides)  # (32, 8) — row-major
print(f.strides)  # (8, 24) — column-major

# the "free" F-order trick
a = np.zeros((4, 3))  # C-contiguous
print(a.T.flags.f_contiguous)  # True — no copy
```

Q5 · `ndarray.copy()` versus a slice — when do you get a view and when do you get a copy?

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, PhonePe, JP Morgan · Topic: views vs copies

The question (interviewer's verbatim phrasing):

If I write `b = a[2:5]` and then `b[0] = 99`, what happens to `a`? When would you have to write `b = a[2:5].copy()` to avoid that?

The 30-second answer: Basic slicing returns a view, so `b = a[2:5]` shares memory with `a` — mutating `b[0]` also mutates `a[2]`. To get an independent copy, use `b = a[2:5].copy()` or `b = np.array(a[2:5])`. The rule is: basic slicing (integer ranges, `:`, `::2`) gives views; fancy

indexing (integer arrays, boolean masks) always gives copies. When in doubt, check `b.base is a` — if it's truthy, `b` is a view of `a`.

Step-by-step thought process: 1. State the rule first, because it's the question. Basic slicing = view. Fancy / boolean indexing = copy. Anything that can be expressed as a stride-and-offset stays a view; anything that requires arbitrary element selection has to copy. 2. Demonstrate the mutation propagation. `a = np.arange(10); b = a[2:5]; b[0] = 99; print(a)` shows `[0,1,99,3,4,5,6,7,8,9]`. That's not a bug; it's the whole point of views. 3. Walk through when you'd want a view (default, fast, no copy) versus a copy (you need to mutate independently, you need the slice to outlive the parent, you're handing the data to code you don't trust to leave it alone). 4. Mention the `.base` attribute. A view's `.base` is the array it views; a copy's `.base` is None. `b.base is a` is the cleanest check.

Edge cases to mention: - Fancy indexing always copies — `a[[1, 3, 5]]` is a new array, mutating it does not touch `a`. - Boolean mask indexing always copies — `a[a > 5]` is a new array. - `arr.reshape(...)` returns a view *if possible*, otherwise a copy. There's no syntactic signal which you got — check `.base` or `np.shares_memory(new, old)`. - Assigning back through fancy indexing DOES write to the original: `a[[1, 3, 5]] = 0` modifies `a`. The right-hand-side selection is a copy, but the left-hand assignment writes through. This trips people up.

What NOT to say: - "Slicing always copies, like Python lists" — no, numpy's design is opposite to Python list slicing precisely so you don't accidentally copy gigabytes. - "I'll just `.copy()` everywhere to be safe" — that defeats the point of numpy's view semantics. Copy when you need to; default to views.

Common follow-up: *"You sliced a 100-MB array, returned the slice from a function, and the original is still in memory. Why?"* — Because the slice is a view holding a reference to the original via `.base`. The original can't be garbage-collected as long as the view exists. Use `.copy()` if you want to release the original.

```
import numpy as np

a = np.arange(10)

# view — shares memory
b = a[2:5]
print(b.base is a)      # True
b[0] = 99
print(a)                 # [ 0  1 99  3  4  5  6  7  8  9]

# copy — independent
c = a[2:5].copy()
print(c.base is None)   # True
c[0] = -1
print(a[2])             # 99, unchanged

# fancy indexing always copies
d = a[[2, 3, 4]]
print(d.base is None)   # True
```

Section B — Broadcasting

The ten questions in this section are the highest-value block in the chapter. Broadcasting is the single feature that distinguishes numpy thinking from list-of-lists thinking, and it's the single feature that produces the most confusing error messages in production. Every product-co interview at Razorpay, Flipkart, Swiggy, Postman, Walmart Labs, and the GCC pool tests broadcasting — and most candidates can recite the rule but freeze on which concrete shapes are legal versus illegal. Master these ten and broadcasting will stop being mysterious; it'll just be a pattern you see.

Q6 · State the broadcasting rules from first principles

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, Microsoft India · Topic: broadcasting rules

The question (interviewer's verbatim phrasing):

Explain numpy broadcasting in your own words. What are the rules, and how do two shapes get aligned?

The 30-second answer: Broadcasting is numpy's rule for combining arrays of different shapes without copying. The rules are three: first, align shapes from the right — pad the shorter one with 1s on the left

until both have the same number of axes. Second, two axes are compatible if they're equal or one of them is 1. Third, when one axis is 1 and the other is N, the size-1 axis is conceptually stretched to N, but no actual data is copied — numpy just reuses the same element via a stride-0 view. If any axis pair fails the compatibility test, you get the "shapes not broadcastable" error.

Step-by-step thought process: 1. Open with what broadcasting *is for*: it lets you write `arr + 5`, or add a row vector to a 2-D matrix, without writing an explicit loop or making a giant copy. That's the whole point. 2. State the three rules cleanly. Right-align. Pad with 1s. Compatible if equal or one is 1. Repeat the third part for emphasis: a size-1 axis stretches to match the other side at zero memory cost. 3. Walk through a concrete shape pair. `(3, 4)` and `(4,)` — right-align gives `(3, 4)` and `(_, 4)`, pad to `(3, 4)` and `(1, 4)`, both axis pairs compatible (3 vs 1 and 4 vs 4), result shape is `(3, 4)`. The `(4,)` row gets reused across all 3 rows. 4. Compare to the failure case. `(3,)` and `(4,)` — right-align gives `(3,)` and `(4,)`, both 1-D, axis pair is 3 vs 4, neither is 1, broadcasting fails. That's the canonical "shapes (3,) (4,) not broadcastable" error.

Edge cases to mention: - Scalars broadcast against any shape — `arr + 5` works for any `arr` because a scalar is treated as shape `()` and rule 1 pads it to as many axes as needed. - A `(1, N)` array and an `(M, 1)` array broadcast to `(M, N)` — this is the outer-product pattern, the basis of pairwise distances and gram matrices. - Three or more arrays in one expression are broadcast pairwise from the right — numpy doesn't do anything magical for more-than-two operands.

What NOT to say: - "Broadcasting copies the smaller array to match the bigger one" — it doesn't; the size-1 axis is a stride-0 view, no copy. - "Broadcasting just works on any two shapes" — it doesn't; mismatched non-1 axes fail.

Common follow-up: "What's `np.broadcast_shapes` for?" — Given a list of shapes, it returns the shape they would broadcast to, or raises if they wouldn't. Use it in library code to validate inputs without actually constructing the broadcasted arrays: `np.broadcast_shapes((3, 4), (4,))` returns `(3, 4)`.

```
import numpy as np

a = np.arange(12).reshape(3, 4) # shape (3, 4)
b = np.array([10, 20, 30, 40]) # shape (4,)
print((a + b).shape)           # (3, 4) — b broadcast across rows

# the failure case
try:
    np.array([1, 2, 3]) + np.array([10, 20, 30, 40])
except ValueError as e:
    print(e)
# operands could not be broadcast together with shapes (3,) (4,)
```

```
# the (M, 1) + (1, N) outer-product pattern
m = np.array([1, 2, 3]).reshape(3, 1)
n = np.array([10, 20, 30, 40]).reshape(1, 4)
print((m + n).shape)           # (3, 4)
```

Q7 · Which of these pairs broadcast: `(3, 4)` and `(4,)` ? `(3, 4)` and `(3,)` ? `(3, 4)` and `(1, 4)` ?

Tags: Difficulty: Easy · Asked at: Swiggy, Razorpay, Flipkart, Walmart Labs, Goldman · Topic: broadcasting shapes

The question (interviewer's verbatim phrasing):

Walk me through whether each of these pairs of shapes can be broadcast together: `(3, 4)` and `(4,)`, `(3, 4)` and `(3,)`, `(3, 4)` and `(1, 4)`, `(3, 4)` and `(3, 1)`.

The 30-second answer: `(3, 4)` and `(4,)` — yes, the `(4,)` gets padded to `(1, 4)` and broadcasts across the 3 rows. `(3, 4)` and `(3,)` — no, the `(3,)` gets padded to `(1, 3)` and the trailing axis is 3 vs 4, which fails; you'd need to reshape to `(3, 1)` first. `(3, 4)` and `(1, 4)` — yes, the `(1, 4)` row gets stretched across 3 rows. `(3, 4)` and `(3, 1)` — yes, the `(3, 1)` column gets stretched across 4 columns. The right-align step is the source of the asymmetry — adding a row-vector "just works" but adding a column-vector requires explicit reshape.

Step-by-step thought process: 1. Walk through each pair using the algorithm, out loud, in interview tempo. 2. Pair 1: `(3, 4)` and `(4,)`. Right-align: `(3, 4)` and `(_, 4)`. Pad: `(3, 4)` and `(1, 4)`. Axis 0: 3 vs 1, compatible. Axis 1: 4 vs 4, compatible. Result: `(3, 4)`. 3. Pair 2: `(3, 4)` and `(3,)`. Right-align: `(3, 4)` and `(_, 3)`. Pad: `(3, 4)` and `(1, 3)`. Axis 1: 4 vs 3, neither is 1, fails. The fix is `(3, 1)` — `arr[:, np.newaxis]` or `arr.reshape(-1, 1)`. 4. Pair 3 and 4: both work. `(1, 4)` and `(3, 4)` are the row-vector + matrix pattern. `(3, 1)` and `(3, 4)` are the column-vector + matrix pattern. The size-1 axis is what enables stretching.

Edge cases to mention: - A 1-D array of length N is treated as `(N,)` not `(1, N)` — that's why the rightmost axis is what counts in the alignment. `(4,)` aligns to `(_, 4)`, not to `(4, _)`. - The `(3,) + (3, 4)` failure is the most common production bug — engineer wants to add a per-row bias term and forgets to reshape. Always check `bias.shape` before adding to a matrix. - `(N, 1) + (M,)` and `(N, 1) + (1, M)` both broadcast to `(N, M)` — they're equivalent because the trailing `(M,)` pads to `(1, M)`.

What NOT to say: - "`(3, 4)` plus `(3,)` works because 3 matches" — it doesn't; the 3 has to match on the trailing axis, not the leading axis. - "Broadcasting is left-aligned" — no, it's right-aligned. This is the one rule most candidates get backwards.

Common follow-up: "You want to subtract a per-column mean from a `(N, M)` matrix. What shape should the mean array be?" — The column mean is `(M,)` from `arr.mean(axis=0)`. `(M,)` right-aligns to `(_, M)` and broadcasts across rows, so `arr - arr.mean(axis=0)` works without reshape.

```
import numpy as np

a = np.zeros((3, 4))

# yes
print((a + np.zeros(4)).shape)      # (3, 4)
print((a + np.zeros((1, 4))).shape) # (3, 4)
print((a + np.zeros((3, 1))).shape) # (3, 4)

# no – fails
try:
    a + np.zeros(3)
except ValueError as e:
    print(e)
# operands could not be broadcast together with shapes (3,4) (3,)

# the fix
print((a + np.zeros(3).reshape(-1, 1)).shape) # (3, 4)
```

Q8 · What is `np.newaxis`, and what shape does it produce?

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Postman, Walmart Labs · Topic: newaxis

The question (interviewer's verbatim phrasing):

What does `arr[:, np.newaxis]` do to a 1-D array of length 5? And what's the difference between that and `arr[np.newaxis, :]`?

The 30-second answer: `np.newaxis` is an alias for `None` that inserts a size-1 axis at the indicated position. For a 1-D array of length 5, `arr[:, np.newaxis]` inserts the new axis after the existing axis, giving shape `(5, 1)` — a column vector. `arr[np.newaxis, :]` inserts the new axis before, giving `(1, 5)` — a row vector. The data is unchanged; only the shape and strides change. It's the canonical way to set up broadcasting without `reshape`.

Step-by-step thought process: 1. State what `np.newaxis` is: it's literally `None`. `arr[:, None]` and `arr[:, np.newaxis]` are identical. People prefer `np.newaxis` for readability. 2. Walk through the position. `np.newaxis` slots into the indexing tuple at the position you write it. So `(:, np.newaxis)` means "take all of axis 0, then insert a new size-1 axis after that" — result shape `(5, 1)`. 3. Show the pair: `(N, 1)` and `(1, M)` is the outer-product setup. `a[:, None] + b[None, :]` is how you build a `(N, M)` pairwise sum without an explicit loop. This is the most common use. 4. Compare to alternatives: `arr.reshape(-1, 1)` does the same thing, but `np.newaxis` is composable inside indexing expressions, which `reshape` isn't.

Edge cases to mention: - You can insert multiple new axes in one expression:

`arr[None, :, None]` on a 1-D array gives shape `(1, 5, 1)`. Useful for setting up 3-D broadcasting. - `np.expand_dims(arr, axis=N)` is the function-form equivalent and is sometimes more readable in long pipelines. `arr[None, :]` and `np.expand_dims(arr, 0)` produce the same result. - The view returned by `np.newaxis` indexing is a view, not a copy — `arr[:, None].base is arr` is True.

What NOT to say: - "`np.newaxis` is a special index value" — it's literally `None`, no magic; that's worth knowing. - "You can use `reshape` instead, it's the same" — they're equivalent for this use case, but `np.newaxis` is what idiomatic numpy code uses inside indexing expressions.

Common follow-up: "How do you compute the pairwise L2 distance between two sets of points using broadcasting?" — Given `A` of shape `(N, D)` and `B` of shape `(M, D)`, write `np.sqrt(((A[:, None, :] - B[None, :, :])**2).sum(axis=-1))`. The `A[:, None, :]` is `(N, 1, D)`, `B[None, :, :]` is `(1, M, D)`, the difference broadcasts to `(N, M, D)`, sum-of-squares over `axis=-1` gives `(N, M)`.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a.shape)                # (5,)
print(a[:, np.newaxis].shape) # (5, 1)
print(a[np.newaxis, :].shape) # (1, 5)
print(a[:, None].shape)       # (5, 1) — None and np.newaxis are identical

# the outer-product pattern
b = np.array([10, 20, 30])
result = a[:, None] + b[None, :]
print(result.shape)           # (5, 3)
print(result)
# [[11 21 31]
#  [12 22 32]
#  [13 23 33]
#  [14 24 34]
#  [15 25 35]]
```

Q9 · The `(N, 1) + (M,)` pattern — what's it for and what shape do you get?

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Microsoft India, Goldman, Postman · Topic: broadcasting patterns

The question (interviewer's verbatim phrasing):

Given two 1-D arrays `a` of length `N` and `b` of length `M`, how would you compute the matrix of pairwise sums `a[i] + b[j]` ?

The 30-second answer: Reshape `a` to `(N, 1)` and leave `b` as `(M,)` — when added, they broadcast to `(N, M)`, where the `[i, j]` cell is `a[i] + b[j]`. Concretely: `a[:, None] + b`. The `(N, 1)` column broadcasts across the `(M,)` row, producing the full `N×M` outer-sum matrix without any loop. This is the canonical "broadcasting builds an outer-product-style matrix" pattern, used for pairwise distances, gram matrices, similarity scores.

Step-by-step thought process: 1. State the trick: turn one of the 1-D arrays into a column, leave the other as a row, then add. The result is the full pairwise grid. 2. Walk through the alignment.

`a[:, None]` has shape `(N, 1)`. `b` has shape `(M,)`, which right-aligns to `(_, M)` and pads to `(1, M)`. Axis 0: `N` vs `1`, broadcasts to `N`. Axis 1: `1` vs `M`, broadcasts to `M`. Result `(N, M)`. 3. Generalize to other operators. `a[:, None] * b` is the outer product (the same as `np.outer(a, b)`). `a[:, None] - b` is pairwise differences. `np.abs(a[:, None] - b)` is pairwise L1 distances. 4. Mention memory. The result is genuinely `(N, M)` — that *is* allocated. If `N` and `M` are each a million, that's a trillion cells; you'll OOM. Broadcasting saves the inputs, not the output.

Edge cases to mention: - The same result with both inputs explicitly reshaped: `a.reshape(N, 1) + b.reshape(1, M)`. Equivalent, slightly more verbose, sometimes clearer. - `np.add.outer(a, b)` does the same thing for `+`, and `np.multiply.outer(a, b)` for `*`. Some teams prefer those for clarity. - The 3-D variant for pairwise distances on `D`-dimensional points: `A[:, None, :] - B[None, :, :]` gives shape `(N, M, D)`. Watch the memory — for `N = M = 10_000` and `D = 100`, that's 8 GB at `float64`.

What NOT to say: - "Use a nested for loop" — that's what broadcasting exists to avoid. In an interview, a nested loop on numpy arrays is a downgrade signal. - "`a + b` should give the pairwise sum" — no, `a + b` requires same shape (or broadcast-compatible), and for `(N,)` + `(M,)` with `N ≠ M`, it errors.

Common follow-up: "Given two arrays of timestamps, how would you find the time delta between every pair?" — Same pattern: `t1[:, None] - t2[None, :]` gives an `(N, M)` matrix of `timedelta64` values, one for each pair.

```
import numpy as np

a = np.array([1, 2, 3])          # shape (3,)
b = np.array([10, 20, 30, 40])  # shape (4,)

pairwise_sum = a[:, None] + b
print(pairwise_sum.shape)        # (3, 4)
print(pairwise_sum)
# [[11 21 31 41]
#  [12 22 32 42]
#  [13 23 33 43]]

# pairwise absolute differences
pairwise_diff = np.abs(a[:, None] - b)
print(pairwise_diff)
# [[ 9 19 29 39]
#  [ 8 18 28 38]
#  [ 7 17 27 37]]

# np.add.outer is the explicit-named version
print(np.array_equal(np.add.outer(a, b), pairwise_sum)) # True
```

Q10 · You see `ValueError: operands could not be broadcast together with shapes (3,) (4,)`. What's wrong and how do you fix it?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Flipkart, Cognizant · Topic: broadcasting errors

The question (interviewer's verbatim phrasing):

You hit this error in production: "operands could not be broadcast together with shapes (3,) (4,)". What does it mean, and how do you diagnose it?

The 30-second answer: The two 1-D arrays have different lengths and neither has a size-1 axis, so they fail the third broadcasting rule. The fix depends on intent: if you meant element-wise on equal-length arrays, you have a bug upstream that produced the wrong length. If you meant the pairwise grid, reshape one to `(3, 1)` so it becomes a column. The diagnosis is always the same — print `a.shape` and `b.shape` before the operation and check if they line up under the right-align rule.

Step-by-step thought process: 1. Parse the error message. The two shapes printed are the actual shapes at the point of failure, not the declared shapes. Trust them. 2. Apply the rule. Right-align: `(3,)` and `(4,)` are both 1-D. Axis 0: 3 vs 4, neither is 1, fails. That's literally what the error says. 3. Decide intent. Did the engineer want `a[i] + b[i]` for all *i* (element-wise)? Then they have a length-mismatch bug — somewhere upstream, one array is shorter than expected. Did they want `a[i] + b[j]` for all (*i*, *j*) (pairwise)? Then they need to reshape — `a[:, None] + b` gives shape `(3, 4)`. 4. Add a

defensive check. If your function does element-wise math, write `assert a.shape == b.shape, f"shape mismatch: {a.shape} vs {b.shape}"` at the top. Catch it early, not as a confusing `ValueError` six function calls deep.

Edge cases to mention: - The error message format includes the shapes — always read them. Beginners panic at "could not be broadcast" without reading the actual numbers. - For higher-dimensional cases the error includes more shape — `(3, 4)` and `(5, 6)` will print both. The right-align rule still applies; check each axis pair. - A `(1,)` and `(N,)` pair always broadcasts — the size-1 axis stretches. So `np.array([5]) + arr` is `arr + 5` essentially. Sometimes useful, sometimes a sign the engineer meant to scalar-ify.

What NOT to say: - "I'll just `reshape` it to make the error go away" — that masks the underlying shape mismatch and creates a worse bug. Understand intent first. - "The numpy version is buggy" — broadcasting hasn't changed semantically in 15 years. It's almost always the caller's shape that's wrong.

Common follow-up: "Same error, but the shapes printed are `(100, 64)` and `(100,)`. What's the fix?" — The `(100,)` right-aligns to `(_, 100)` and the trailing axis is 64 vs 100, fails. The engineer probably wanted to add a per-row scalar to each row of the matrix. Reshape `(100,)` to `(100, 1)` and broadcasting works: `mat + vec[:, None]`.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30, 40])

try:
    a + b
except ValueError as e:
    print("Error:", e)

# fix 1: element-wise — make them the same length (and fix the upstream bug)
b_trimmed = b[:3]
print((a + b_trimmed).shape)      # (3,)

# fix 2: pairwise — reshape a to (3, 1)
print((a[:, None] + b).shape)     # (3, 4)
```

Q11 · Broadcasting a row vector across a 2-D matrix — show the per-column scaling pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Walmart Labs, Microsoft India · Topic: broadcasting patterns

The question (interviewer's verbatim phrasing):

I have a `(1000, 5)` feature matrix and a length-5 array of column scales. How do I multiply each column by its scale without a loop?

The 30-second answer: Just write `mat * scales`. The matrix is `(1000, 5)`, scales is `(5,)`, which right-aligns to `(_, 5)` and pads to `(1, 5)`. Broadcasting stretches the row of scales across all 1000 rows, multiplying each column by its corresponding scale. No reshape needed because the scales' length matches the trailing axis of the matrix. This is the same shape pattern as adding a per-feature bias, normalizing by per-feature stddev, or any column-wise transform.

Step-by-step thought process: 1. State the answer first: no loop needed, `mat * scales` is the whole solution. Then explain why. 2. Walk the broadcasting alignment. `(1000, 5)` and `(5,)`. Right-align: `(1000, 5)` and `(_, 5)`. Pad: `(1000, 5)` and `(1, 5)`. Axis 0: 1000 vs 1, fine. Axis 1: 5 vs 5, fine. Result: `(1000, 5)`. 3. Generalize to the family of column-wise ops. Subtracting per-column means: `mat - mat.mean(axis=0)`. Dividing by per-column stddev: `(mat - mat.mean(axis=0)) / mat.std(axis=0)` — that's a z-score normalization in one line. 4. Contrast with the row-wise variant. Row-wise scaling requires the scales as `(1000, 1)` so they broadcast across the 5 columns: `mat * row_scales[:, None]`. The transposition of the broadcasting shape is the only difference.

Edge cases to mention: - If `scales` is `(1, 5)` instead of `(5,)`, it still works — the explicit-row form. Some teams prefer this for clarity since the row-ness is visible in the shape. - If `scales` is `(5, 1)` by mistake, you get shape error: `(1000, 5)` and `(5, 1)` — axis 0 is 1000 vs 5, axis 1 is 5 vs 1; axis 0 fails. This is the bug where someone reshapes the wrong way. - For sparse matrices (`scipy.sparse`), broadcasting doesn't work the same way — you need element-wise multiplication via the `.multiply` method.

What NOT to say: - "Use a for-loop over the 5 columns" — that's 5x slower minimum and signals you don't understand vectorization. - "Reshape `scales` to `(1, 5)` first" — works, but unnecessary; the implicit padding does it for you.

Common follow-up: "How would you do row-wise normalization — divide each row by its own L2 norm?" — Compute `norms = np.linalg.norm(mat, axis=1)` which is shape `(1000,)`. Reshape to `(1000, 1)` for broadcasting across columns: `mat / norms[:, None]`. Don't forget the `[:, None]` — without it, you get a shape error because `(1000,)` aligns wrong.

```
import numpy as np

rng = np.random.default_rng(42)
mat = rng.standard_normal((1000, 5))
scales = np.array([1, 10, 100, 1000, 10000], dtype=float)

scaled = mat * scales
print(scaled.shape)           # (1000, 5)
print(scaled[0])              # row 0, each column multiplied by its scale
print(scaled[:, 2].std())     # roughly 100 — column 2 was scaled by 100

# row-wise normalization
norms = np.linalg.norm(mat, axis=1)      # shape (1000,)
unit = mat / norms[:, None]              # each row has L2 norm ~1
print(np.linalg.norm(unit, axis=1)[:5])  # approx [1, 1, 1, 1, 1]
```

Q12 · Why doesn't broadcasting copy the data, and what does "stride-0" mean?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Postman, Razorpay · Topic: broadcasting internals

The question (interviewer's verbatim phrasing):

When numpy broadcasts a length-4 vector against a `(1000, 4)` matrix, does it actually allocate a `(1000, 4)` copy of the vector?

The 30-second answer: No, that's the whole point. Broadcasting is implemented as a stride-0 view — the size-1 axis (or the implicitly-padded axis) gets its stride set to 0, so as numpy walks along that axis it just re-reads the same memory location. The conceptual `(1000, 4)` view of the length-4 vector occupies the same 32 bytes as the original; only the *output* of the operation is a real allocation. This is what makes broadcasting both memory-efficient and fast — no per-row copying, no per-row dispatch.

Step-by-step thought process: 1. State the no-copy fact, then immediately explain the mechanism. A stride is "bytes to skip per step along this axis". Stride 0 means "don't skip — re-read the same byte". Numpy uses this to fake-replicate a small array across a big axis. 2. Show it concretely. `np.broadcast_to(arr, shape)` is the explicit function. `np.broadcast_to(np.array([1, 2, 3, 4]), (1000, 4))` returns a `(1000, 4)`-shaped view of the original 32 bytes, with `strides=(0, 8)` — 0 bytes to move to the next row (you re-read), 8 bytes to move to the next column. 3. Connect it to performance. Because the input isn't actually replicated, broadcasting is cache-friendly — the small array fits in L1 cache and stays there while the big array streams through. Loops over the size-1 axis are essentially free. 4. Note the asymmetry: the inputs are stride-0 views, but the output of `a + b` is a freshly allocated array of the broadcast shape. You save memory on the operands, not the result.

Edge cases to mention: - A broadcast view is read-only. `np.broadcast_to(arr, (1000, 4))[0, 0] = 99` raises because writing through a stride-0 view would invisibly write to all 1000 rows at once. - `np.broadcast_arrays(a, b)` returns the explicit broadcast views of both inputs — useful for understanding what shapes the operation will actually combine. - The stride-0 trick works for any axis, not just leading or trailing. `arr[:, None, :]` with shape `(N, 1, D)` broadcasts to `(N, M, D)` with `strides=(8D, 0, 8)` — middle axis stride is 0.

What NOT to say: - "Broadcasting replicates the smaller array in memory" — it doesn't, and saying so contradicts the memory savings everyone expects from broadcasting. - "Broadcasting is slow because of all the copying" — broadcasting is *fast* precisely because there's no copying.

Common follow-up: *"If broadcasting doesn't copy, why does my memory usage spike when I write `mat - mat.mean(axis=0)` ?"* — The spike isn't the broadcasted mean — that's a stride-0 view of a tiny `(D,)` array. The spike is the *result* of the subtraction, which is a fresh `(N, D)` array the same size as `mat`. Use in-place ops (`mat -= mat.mean(axis=0)`) if you can spare the original.

```
import numpy as np

vec = np.array([1, 2, 3, 4])
broad = np.broadcast_to(vec, (1000, 4))
print(broad.shape)           # (1000, 4)
print(broad.strides)         # (0, 8) — stride 0 means re-read
print(broad.nbytes)          # 32_000 (logical), but underlying is still 32 bytes
print(broad.base.nbytes)     # 32 — the real backing buffer

# read-only
try:
    broad[0, 0] = 99
except ValueError as e:
    print("Error:", e)
# assignment destination is read-only
```

Q13 · Broadcasting a column vector — when is `arr[:, None]` necessary vs not?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Flipkart, Walmart Labs · Topic: broadcasting patterns

The question (interviewer's verbatim phrasing):

When you want to scale each row of a `(N, D)` matrix by its own factor, why must you write `mat * factors[:, None]` and not just `mat * factors` ?

The 30-second answer: Because numpy aligns shapes from the right. A `(N,)` array aligns as a row, not a column — it pads to `(1, N)`, not `(N, 1)`. For row-wise scaling, you need the factors to broadcast across D columns, which means the factors must be shape `(N, 1)`.

`factors[:, None]` is how you turn `(N,)` into `(N, 1)`. Without it, `mat * factors` either errors (if $N \neq D$) or silently scales the wrong axis (if $N == D$). The `[:, None]` is the explicit "this is a column" signal.

Step-by-step thought process: 1. State the rule one more time: right-align. `(N,)` becomes `(_, N)` then `(1, N)`. That's row-shape. 2. Walk through the failure. Matrix `(1000, 5)`, factors `(1000,)`. Right-align: `(1000, 5)` and `(_, 1000)`. Pad: `(1000, 5)` and `(1, 1000)`. Axis 1: 5 vs 1000, fails. Error message: shapes `(1000, 5)` `(1000,)` not broadcastable. 3. Apply the fix. `factors[:, None]` is `(1000, 1)`. Now: `(1000, 5)` and `(1000, 1)`. Axis 0: 1000 vs 1000, fine. Axis 1: 5 vs 1, broadcast. Result: `(1000, 5)`, each row scaled by its factor. 4. State the symmetric case for the other interpretation. If you wanted to scale each *column*, the factors are length D and `mat * factors` works as-is, no reshape — `(N, D)` and `(D,)` aligns naturally.

Edge cases to mention: - The silent-bug case: if $N == D$ (square matrix), `mat * factors` doesn't error — it just scales columns instead of rows, silently producing wrong output. Always write `factors[:, None]` for row-wise, even on square matrices, to make intent explicit. - `factors.reshape(-1, 1)` is equivalent to `factors[:, None]`. Some teams prefer one for readability. - `np.diag(factors) @ mat` is the linear-algebra equivalent — multiply by a diagonal matrix on the left. Same result, much more memory and compute. Don't use it; broadcasting is the right tool.

What NOT to say: - "Broadcasting will figure out my intent" — no, broadcasting is mechanical; right-align rules apply regardless of what you meant. - "I'll transpose, multiply, transpose back" — works, two extra transposes, more code, no win.

Common follow-up: "What about scaling a 3-D tensor along its middle axis?" — Same logic. For shape `(B, N, D)` and per-N factors of length N, write `tensor * factors[None, :, None]`. The `(1, N, 1)` broadcasts across B in axis 0 and D in axis 2, applying `factors[k]` to the entire `tensor[:, k, :]` slice.

```
import numpy as np

mat = np.ones((1000, 5))
factors = np.arange(1, 1001, dtype=float) # shape (1000,)

# this fails
try:
    mat * factors
except ValueError as e:
    print(e)
# operands could not be broadcast together with shapes (1000,5) (1000,)

# this works
scaled = mat * factors[:, None]
print(scaled.shape) # (1000, 5)
print(scaled[0]) # [1, 1, 1, 1, 1] – first factor is 1
print(scaled[999]) # [1000, 1000, 1000, 1000, 1000]
```

Q14 · Demonstrate computing pairwise distances between two sets of points using broadcasting

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Postman, Razorpay · Topic: broadcasting application

The question (interviewer's verbatim phrasing):

Given **A** of shape **(N, D)** and **B** of shape **(M, D)** representing two sets of D-dimensional points, compute the **(N, M)** matrix of Euclidean distances using only broadcasting, no loops.

The 30-second answer: Insert middle axes to make **A** shape **(N, 1, D)** and **B** shape **(1, M, D)**. Subtract to get a **(N, M, D)** tensor of element-wise differences via broadcasting. Square, sum over the last axis, sqrt. The one-liner is `np.sqrt((A[:, None, :] - B[None, :, :]) ** 2).sum(axis=-1))`. There's a more memory-efficient version using the algebraic identity $||a - b||^2 = ||a||^2 + ||b||^2 - 2 a \cdot b$, which avoids the **(N, M, D)** intermediate, but the broadcasting form is what interviewers want to see.

Step-by-step thought process: 1. Set up the shapes. **A[:, None, :]** is **(N, 1, D)**. **B[None, :, :]** is **(1, M, D)**. The subtraction broadcasts axis-0 (N vs 1) to N, axis-1 (1 vs M) to M, axis-2 (D vs D) stays D. Result: **(N, M, D)**. 2. Each cell **(i, j, k)** is **A[i, k] - B[j, k]** — the k-th coordinate difference between point i in A and point j in B. Squaring is element-wise, no shape change. 3. Sum over the last axis: `.sum(axis=-1)` reduces the D dimension, giving **(N, M)** — the squared Euclidean distance from each point in A to each point in B. Sqrt to get the actual

distance. 4. Mention the memory caveat. The `(N, M, D)` intermediate is NMD8 bytes. For $N = M = 10000$ and $D = 100$, that's 80 GB — won't fit. The algebraic trick `(A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T` produces the squared distances in $O(NM + ND + MD)$ memory, only ever materializing `(N, M)`.

Edge cases to mention: - For L1 distance (Manhattan), replace square + sum + sqrt with abs + sum: `np.abs(A[:, None, :] - B[None, :, :]).sum(axis=-1)`. - `scipy.spatial.distance.cdist(A, B)` is the production-grade equivalent — written in C, handles the memory blowup, accepts a `metric=` argument. - Numerical precision: the algebraic form can produce small negative numbers from floating-point cancellation when A and B contain identical rows. Clip to `np.maximum(..., 0)` before the sqrt to avoid `nan`.

What NOT to say: - "Use a nested for-loop, it's clearer" — that's three orders of magnitude slower and signals you don't actually know broadcasting. - "`scipy.spatial.distance.cdist` is the right answer" — true in production, but the interviewer wants to see the broadcasting derivation.

Common follow-up: "How would you find, for each row in A, the index of the closest row in B?" — Compute the distance matrix as above, then `D.argmax(axis=1)` gives a length-N array of indices into B. For just-the-distance (no index), `D.min(axis=1)` is the length-N array of nearest-neighbor distances.

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.standard_normal((5, 3))    # 5 points in 3-D
B = rng.standard_normal((4, 3))    # 4 points in 3-D

# broadcasting form
D_squared = ((A[:, None, :] - B[None, :, :]) ** 2).sum(axis=-1)
D = np.sqrt(D_squared)
print(D.shape)                      # (5, 4)

# algebraic form — same result, less memory
D2_alt = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T
D_alt = np.sqrt(np.maximum(D2_alt, 0))
print(np.allclose(D, D_alt))        # True
```

Q15 · `np.broadcast_to` vs `np.tile` — what's the difference?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India, Goldman, Walmart Labs · Topic: broadcasting tools

The question (interviewer's verbatim phrasing):

You want a `(1000, 4)` array where every row is `[1, 2, 3, 4]`. You could use `np.broadcast_to` or `np.tile`. What's the difference, and which should you reach for?

The 30-second answer: `np.broadcast_to(arr, (1000, 4))` returns a read-only stride-0 view — no data copied, just 32 bytes for the original plus a metadata wrapper. `np.tile(arr, (1000, 1))` actually allocates and fills a 32_000-byte array. They behave the same for read-only consumption, but `broadcast_to` is free in memory while `tile` is `(1000, 4) * 8` bytes. Reach for `broadcast_to` whenever you just need the broadcasted shape for a subsequent operation. Reach for `tile` only when you need a real, writable, independent array.

Step-by-step thought process: 1. State the memory difference upfront: `broadcast_to` is free, `tile` is full-cost. 2. Explain the underlying mechanism. `broadcast_to` uses stride-0 — it tells numpy "as you walk this axis, don't move". `tile` does an honest memcopy, replicating the source `(1000, 4) / (1, 4) = 1000` times in memory. 3. Both produce arrays of the right shape, and arithmetic on them yields the same result. The difference shows up in (a) memory profile of the program, and (b) writeability — `broadcast_to` views are read-only because writing would corrupt the stride-0 trick. 4. Almost always you want `broadcast_to`. The one case for `tile` is when downstream code requires a contiguous writable array (e.g., a C extension that doesn't accept stride-0 views).

Edge cases to mention: - `np.repeat` is a third sibling — it repeats *each element* in place (`np.repeat([1,2,3], 2) == [1,1,2,2,3,3]`) versus `np.tile` which repeats *the whole sequence* (`np.tile([1,2,3], 2) == [1,2,3,1,2,3]`). Both are full copies. - `broadcast_to` raises if the requested shape isn't compatible. `np.broadcast_to(np.zeros(4), (1000, 5))` fails because `4 ≠ 5`. - For most arithmetic, you don't need either — implicit broadcasting in the operator (`+`, `*`, `np.add`) does the broadcast automatically as a stride-0 view internally.

What NOT to say: - "They're the same, just pick one" — they're not; the memory and writeability differ substantially. - "`tile` is faster because it doesn't have to handle stride-0" — no, `broadcast_to` is faster precisely because it doesn't allocate.

Common follow-up: "You called `tile` to produce a `(10_000_000, 100)` array and OOM'd. What's the fix?" — Replace `tile` with `broadcast_to` if the downstream code is read-only. If downstream needs a writable array, the operation itself was probably wrong — most operations that need a tiled array can be reformulated as broadcasting the small array directly into the next computation, skipping the full materialization.


```
import numpy as np

base = np.array([1, 2, 3, 4])

# broadcast_to — view, free
bv = np.broadcast_to(base, (1000, 4))
print(bv.shape)           # (1000, 4)
print(bv.base is base)    # True — same buffer
print(bv.nbytes)          # 32_000 (logical)
print(bv.flags.writeable) # False

# tile — copy, allocates
tv = np.tile(base, (1000, 1))
print(tv.shape)           # (1000, 4)
print(tv.base is None)    # True — independent buffer
print(tv.nbytes)          # 32_000 (real)
print(tv.flags.writeable) # True
```

Section C — `axis=` Semantics

The five questions below cover the single most-confused topic in numpy: what does `axis=0` actually mean. Indian product-co interviewers ask it in every data-role panel because the wrong intuition produces silently-wrong analytics — the candidate who computes "row mean" when asked for "column mean" gets every downstream number wrong. The rule that saves you is one sentence: `axis=k` is the axis that *disappears* from the output shape under a reduction. Everything else falls out of that.

Q16 · `axis=0` vs `axis=1` — what's the difference for a 2-D array?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Walmart Labs · Topic: axis semantics

The question (interviewer's verbatim phrasing):

For a 2-D array of shape `(rows, cols)`, what does `arr.sum(axis=0)` give you versus `arr.sum(axis=1)` ?

The 30-second answer: `axis=0` collapses the rows — you walk down each column and sum, producing a length-`cols` 1-D array of column sums. `axis=1` collapses the columns — you walk across each row and sum, producing a length-`rows` 1-D array of row sums. The rule of thumb: `axis=k` is the axis that disappears from the output shape. Input shape `(rows, cols)`, `axis=0` removes axis 0, output is `(cols,)`. `axis=1` removes axis 1, output is `(rows,)`.

Step-by-step thought process: 1. Say the rule once: `axis=k` is the axis that disappears. Memorize this one sentence and every axis question becomes mechanical. 2. Walk through `axis=0`. For shape `(3, 4)`, `axis=0` means "remove axis 0", output shape `(4,)`. The operation is performed *along* axis 0, meaning we sum across all the row-indices for each column-index. Result element `k` is `sum(arr[i, k] for i in range(3))` — a column sum. 3. Walk through `axis=1`. For shape `(3, 4)`, `axis=1` means "remove axis 1", output shape `(3,)`. We sum across all column-indices for each row-index. Result element `k` is `sum(arr[k, j] for j in range(4))` — a row sum. 4. Generalize. For higher-dimensional arrays, the same rule holds. `(B, N, D)` with `axis=1` gives `(B, D)`. `axis=-1` is the last axis, `axis=-2` is the second-to-last. Negative indexing works the same as in lists.

Edge cases to mention: - `arr.sum()` with no axis is equivalent to `axis=None`, which reduces everything to a scalar. - `arr.sum(axis=(0, 1))` reduces over multiple axes in one call. For a 3-D array `(B, N, D)`, `axis=(0, 1)` gives `(D,)` — sum over batches and rows, leaving feature dimension. - Some functions use `axis=-1` as their default (`np.argmax`, `np.fft.fft`), some use `axis=None` (`np.sum`, `np.mean`). Read the docs; don't assume.

What NOT to say: - "`axis=0` is rows, `axis=1` is columns" — ambiguous; this phrasing is what causes most confusion. Use "axis=k disappears" instead. - "Always pass `axis=` to be safe" — usually right, but call out when no-axis (full reduction) is what you want.

Common follow-up: "For a `(100, 50, 10)` 3-D array of "batch × time × features", how do you average over time to get a `(100, 10)` "batch × features" array?" — Average along the time axis, which is axis 1. `arr.mean(axis=1)` gives shape `(100, 10)`. The axis-1 dimension disappears.

```
import numpy as np

arr = np.arange(12).reshape(3, 4)
print(arr)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]

print(arr.sum(axis=0))      # [12 15 18 21] — column sums, shape (4,)
print(arr.sum(axis=1))      # [ 6 22 38] — row sums, shape (3,)
print(arr.sum())            # 66 — full reduction, scalar
print(arr.sum(axis=(0, 1))) # 66 — same as full reduction for 2-D
```

Q17 · `keepdims=True` — what's it for and when do you need it?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Microsoft India, Goldman · Topic: `keepdims`

The question (interviewer's verbatim phrasing):

What does `arr.sum(axis=0, keepdims=True)` do differently from `arr.sum(axis=0)` ?
When would you want `keepdims=True` ?

The 30-second answer: `keepdims=True` preserves the reduced axis as a size-1 axis instead of dropping it. For a `(3, 4)` input, `arr.sum(axis=0)` returns shape `(4,)`, but `arr.sum(axis=0, keepdims=True)` returns shape `(1, 4)`. You want it whenever the reduction result will be broadcast back against the original array — the size-1 axis aligns naturally with the unreduced shape, so no explicit reshape is needed. The canonical use is row-wise normalization: `arr / arr.sum(axis=1, keepdims=True)` divides each row by its sum.

Step-by-step thought process: 1. State the shape difference: `keepdims=True` keeps the axis as size 1 rather than removing it. Same data, different shape. 2. Walk through why it matters for downstream broadcasting. `arr` is `(3, 4)`. `arr.sum(axis=1)` is `(3,)` — that won't broadcast against `arr` as a column scaler because `(3,)` aligns to `(_, 3)` not `(3, _)`. `arr.sum(axis=1, keepdims=True)` is `(3, 1)` — that broadcasts perfectly. 3. Show the row-normalization example: `arr / arr.sum(axis=1, keepdims=True)` is the one-line softmax denominator, the one-line row-fraction. Without `keepdims`, you'd need `arr / arr.sum(axis=1)[: , None]` — same result, an extra mental step. 4. Generalize: anytime you reduce and immediately broadcast back against the original, `keepdims=True` saves you the reshape. It's purely a convenience feature for shape arithmetic.

Edge cases to mention: - `keepdims=True` with `axis=None` (full reduction) gives a shape of all ones — for a `(3, 4)` input, `arr.sum(axis=None, keepdims=True)` is shape `(1, 1)`. Useful when you want the grand total in a shape that broadcasts. - Works on the operator method too: `np.sum(arr, axis=0, keepdims=True)`. Same behavior. - Some older code uses `axis=0` followed by `[None, :]` to manually re-add the axis. Equivalent, but verbose; prefer `keepdims=True`.

What NOT to say: - "`keepdims` is for display purposes" — no, it's for broadcasting alignment, not visualization. - "I'll always use `keepdims=True`" — only when you're about to broadcast. If you're just storing the result or printing it, the squeezed shape is what you want.

Common follow-up: "Write the row-wise softmax with `keepdims=True` ." —
`exp = np.exp(arr - arr.max(axis=1, keepdims=True))` (subtract max for numerical stability), then `softmax = exp / exp.sum(axis=1, keepdims=True)`. Both reductions use `keepdims=True` so they broadcast back against `arr`'s `(N, K)` shape without explicit reshape.

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
               [5, 6, 7, 8],
               [9, 10, 11, 12]], dtype=float)

print(arr.sum(axis=1))          # [10 26 42], shape (3,)
print(arr.sum(axis=1, keepdims=True)) # [[10] [26] [42]], shape (3, 1)

# row-wise normalize – keepdims makes this one line
row_norm = arr / arr.sum(axis=1, keepdims=True)
print(row_norm.sum(axis=1))      # [1. 1. 1.] – each row sums to 1

# without keepdims, need[:, None]
row_norm2 = arr / arr.sum(axis=1)[:, None]
print(np.allclose(row_norm, row_norm2)) # True – same result
```

Q18 · When does `axis=None` make sense, and what's the default for `np.sum` ?

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Razorpay, Swiggy, Walmart Labs · Topic: `axis=None`

The question (interviewer's verbatim phrasing):

What's the default `axis` for `np.sum` ? When would you explicitly pass `axis=None` ?

The 30-second answer: The default for `np.sum` (and `mean`, `max`, `min`, `prod`, etc.) is `axis=None`, which means reduce over every axis, returning a scalar. You pass `axis=None` explicitly when you want to be clear that "I really do want the grand total, not a per-column or per-row total." For most other functions like `np.argmax` and `np.argmin`, the default is also `axis=None`, and the returned index is into the flattened array, which surprises people.

Step-by-step thought process: 1. State the default. `axis=None` for the reduction functions. Most people don't know this because they always pass an explicit axis. 2. Walk through what `axis=None` does. It treats the array as 1-D (ravels it conceptually) and reduces over the whole thing. Result is a 0-D scalar. 3. Call out the `argmax` gotcha. `arr.argmax()` returns a single integer — the position in the flattened array. For a `(3, 4)` matrix, that integer is between 0 and 11. To get back `(row, col)`, use `np.unravel_index(arr.argmax(), arr.shape)`. 4. Suggest the explicit-axis discipline. Even when `axis=None` is what you want, write it out — `arr.sum(axis=None)` reads more clearly than `arr.sum()` because the next person reading the code knows you didn't forget the axis.

Edge cases to mention: - `arr.sum(axis=None, keepdims=True)` gives a shape of all-ones — `(1, 1)` for a 2-D input. Sometimes useful for broadcasting back against the original. - Some functions, like `np.linalg.norm`, have a different default (`axis=None` but with Frobenius norm for

matrices). Check the docs per function. - For floats, `axis=None` and the result being a numpy scalar (not a Python float) sometimes matters for downstream code — use `.item()` to convert if you need a real Python scalar.

What NOT to say: - "There's no default; you always need to pass `axis`" — there is a default; it's `None`. - "`axis=None` is for 1-D arrays only" — it works on any shape; the meaning is "flatten then reduce".

Common follow-up: "You wrote `idx = arr.argmax()` on a 2-D array and got back a single integer. How do you get `(row, col)`?" — `np.unravel_index(idx, arr.shape)` converts a flat index to a tuple of per-axis indices: `np.unravel_index(7, (3, 4))` returns `(1, 3)`, meaning row 1, column 3.

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

print(arr.sum())           # 78 — implicit axis=None
print(arr.sum(axis=None))  # 78 — explicit, same thing
print(arr.argmax())        # 11 — flat index of 12

# get back (row, col)
flat_idx = arr.argmax()
print(np.unravel_index(flat_idx, arr.shape)) # (2, 3)
print(arr[np.unravel_index(flat_idx, arr.shape)]) # 12
```

Q19 · For a 3-D array `(B, N, D)`, how do you reduce over the middle axis?

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, Razorpay, Swiggy, Postman · Topic: 3-D axis semantics

The question (interviewer's verbatim phrasing):

I have a `(batch, timestep, feature)` shaped tensor. I want the per-batch, per-feature mean across timesteps. What's the call?

The 30-second answer: `arr.mean(axis=1)`. The middle axis is axis 1 — that's the one that gets removed, going from `(B, N, D)` to `(B, D)`. If you want the result back in a `(B, 1, D)` shape so it broadcasts against the original tensor (for example, to subtract the mean from each timestep), pass `keepdims=True`: `arr.mean(axis=1, keepdims=True)` gives shape `(B, 1, D)`.

Step-by-step thought process: 1. Apply the "axis disappears" rule. Input `(B, N, D)`, removing the middle axis (N), output `(B, D)`. Call: `axis=1`. Same logic for `axis=-2`, which is equivalent for a 3-D array. 2. Walk through the semantics. Per batch element `b` and feature index `d`, the result is `mean(arr[b, n, d] for n in range(N))` — the time-average of feature `d` in batch `b`. 3. Show the `keepdims=True` variant if you'll broadcast back. `arr - arr.mean(axis=1, keepdims=True)` centers each batch element's time series around zero, per feature, in one line. 4. Generalize: any axis works the same way. `axis=0` would average across batches (giving `(N, D)`); `axis=2` would average across features (giving `(B, N)`); `axis=(1, 2)` would average over both time and features (giving `(B,)` — one mean per batch).

Edge cases to mention: - Negative axis indices: `axis=-1` is the last axis. For 3-D arrays, `axis=-1` and `axis=2` are equivalent. Negative indices are easier to write when your code might handle inputs of varying dimensionality. - `np.mean` on integer arrays returns a float — `np.array([1, 2, 3]).mean()` is `2.0`, not `2`. This trips people writing typed pipelines. - Reducing over a tuple of axes: `axis=(1, 2)` is the only way to express "reduce over time AND features" in one call. The equivalent step-by-step is `arr.mean(axis=2).mean(axis=1)` — same result, two passes.

What NOT to say: - "The middle axis is axis 2 because Python is 0-indexed" — no, axis 0 is first, axis 1 is middle for 3-D, axis 2 is last. - "Use a for-loop over batches" — never; numpy reductions are designed for exactly this.

Common follow-up: "How do you compute the per-batch, per-feature standard deviation across time?" — Same axis, different function: `arr.std(axis=1)`. Note that numpy's `std` defaults to the population std (`ddof=0`); pass `ddof=1` for the sample std, the one statistics textbooks teach.

```
import numpy as np

rng = np.random.default_rng(0)
arr = rng.standard_normal((4, 100, 8))  # 4 batches, 100 timesteps, 8 features

print(arr.mean(axis=1).shape)           # (4, 8)
print(arr.mean(axis=1, keepdims=True).shape)  # (4, 1, 8)

# center each batch's time series around its own mean
centered = arr - arr.mean(axis=1, keepdims=True)
print(centered.mean(axis=1).round(10))  # all zeros — centered correctly
```

Q20 · `np.sum(arr, axis=0)` versus `arr.sum(axis=0)` — any difference?

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Razorpay, Swiggy, Walmart Labs · Topic: API equivalence

The question (interviewer's verbatim phrasing):

Is there a difference between calling `np.sum(arr, axis=0)` versus the method form `arr.sum(axis=0)` ?

The 30-second answer: For standard numpy arrays, they're equivalent — both dispatch to the same C implementation, both accept the same `axis` , `keepdims` , `dtype` arguments. The functional form is one character shorter and works on lists too (`np.sum([1, 2, 3])`), the method form is one fewer dependency-on-the-numpy-namespace and reads as object-oriented. The one place they diverge is subclasses — `np.sum` always returns a base `ndarray` , the method form can return a subclass (like a `np.ma.MaskedArray` or a `pandas.Series` when you call the latter's `.sum`). Pick one style and stay consistent in a code base.

Step-by-step thought process: 1. State the equivalence. Both forms call into the same underlying reduce-and-sum primitive. There's no performance difference. 2. Walk through the minor differences. Functional form accepts non-array inputs: `np.sum([1, 2, 3])` works; `[1, 2, 3].sum(axis=0)` doesn't (lists don't have `.sum`). Functional form is slightly more discoverable via tab-completion on `np` . 3. The subclass note. For `MaskedArray` subclasses (`np.ma`), `arr.sum()` may dispatch to `MaskedArray.sum` and return a `MaskedArray` ; `np.sum(arr)` also dispatches via `__array_function__` to the masked-array version in modern numpy. So in practice they match. Older numpy versions could differ — worth knowing if you're stuck on 1.16. 4. Style guidance: pick one. pandas-heavy code bases often prefer method-form because pandas uses `.sum()` heavily. ML codebases often use functional form because of consistency with `np.exp` , `np.log` and friends.

Edge cases to mention: - For 0-D arrays (scalars in an array wrapper), both forms work and both return a 0-D result of the same shape. - The functional form accepts a `where=` argument for conditional reduction (since numpy 1.17). The method form also does, but the API is identical. - For libraries that mimic numpy (like `cupy` , `jax.numpy` , `dask.array`), the functional form usually has wider coverage than the method form because of the implementation pattern.

What NOT to say: - "Method form is faster" — no, they're the same. - "Always use one, never the other" — there's no functional reason to prefer one across the board; consistency within a project matters more.

Common follow-up: "Do other reduction functions follow the same pattern?" — Yes — `mean` , `max` , `min` , `prod` , `std` , `var` , `argmax` , `argmin` , `cumsum` , `cumprod` all have both functional (`np.X(arr, ...)`) and method (`arr.X(...)`) forms with the same semantics.

```
import numpy as np

arr = np.arange(12).reshape(3, 4)

# both equivalent
a = np.sum(arr, axis=0)
b = arr.sum(axis=0)
print(np.array_equal(a, b))    # True

# functional accepts lists
print(np.sum([1, 2, 3]))      # 6

# method form doesn't work on lists
try:
    [1, 2, 3].sum(axis=0)
except AttributeError as e:
    print(e)
# 'list' object has no attribute 'sum'
```

Section D — Indexing

The five questions in this section cover the most consequential difference between "basic indexing" and "fancy indexing" — one returns a view, the other returns a copy, and the bug surface that misunderstands this is enormous. Boolean masks are technically fancy indexing; `np.ix_` is the rare cross-product index. Indian product-co interviews lean heavily on indexing because it's where vectorized thinking pays off most clearly — the candidate who reaches for a mask instead of a loop signals real numpy fluency.

Q21 · Basic indexing vs fancy indexing — what's the difference, and which one returns a view?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Microsoft India, Walmart Labs · Topic: indexing

The question (interviewer's verbatim phrasing):

What's the difference between `arr[2:5]` and `arr[[2, 3, 4]]` ? They both seem to give you the same elements.

The 30-second answer: `arr[2:5]` is basic indexing — uses a slice object. It returns a view of the original array, sharing memory. `arr[[2, 3, 4]]` is fancy (advanced) indexing — uses an integer

array as the index. It always returns a copy, never a view. The two produce the same *values* in this example but behave fundamentally differently: mutating the basic slice mutates the original, mutating the fancy result does not. The rule is mechanical: anything expressible via a slice/integer scalar is basic and views; anything using an integer array or boolean array is fancy and copies.

Step-by-step thought process: 1. State the view/copy rule directly. Basic = view, fancy = copy. That's the load-bearing fact. 2. Demonstrate the mutation difference. `a = np.arange(10); b = a[2:5]; b[0] = 99; a` shows `a[2]` is now 99 — view. `a = np.arange(10); c = a[[2, 3, 4]]; c[0] = 99; a` shows `a[2]` is still 2 — copy. 3. Explain why. A basic slice can be expressed as offset + stride into the original buffer, so numpy returns a view at zero cost. A fancy index picks arbitrary elements, which can't be expressed as offset + stride — numpy has to allocate a new buffer and gather elements into it. 4. Mention the assignment-target asymmetry. `a[[2, 3, 4]] = [99, 88, 77]` does modify `a` — the right-hand fancy selection is a copy, but the left-hand fancy assignment writes through. This is intentional but confusing; the assignment is the only way to fancy-index *into* an array.

Edge cases to mention: - Mixing slices and integer arrays — `arr[1:4, [0, 2]]` — is still fancy (because the integer array is fancy). The result is a copy. - Mixing slices and a scalar integer — `arr[1:4, 0]` — is still basic. Scalar integers are basic; arrays of integers are fancy. - Boolean masks — `arr[arr > 5]` — are fancy. Always a copy. - `np.shares_memory(result, arr)` is the runtime check; `result.base is arr` works for direct views but not for indirect ones.

What NOT to say: - "`arr[[2, 3, 4]]` is just slow indexing" — no, it's *fancy*, which copies; it's not slow, it's semantically different. - "Both return views" — wrong on the fancy case; this is the bug-hatchery answer.

Common follow-up: "You did `b = a[[1, 3, 5]]` and modified `b`. You're surprised `a` is unchanged. What gives?" — Fancy indexing returns a copy; mutating the copy doesn't affect the source. If you wanted to mutate the source, use the assignment form: `a[[1, 3, 5]] = b`. If you want to inspect-then-mutate via the same names, use basic indexing if your selection can be expressed as a slice.

```

import numpy as np

a = np.arange(10)

# basic slicing – view
b = a[2:5]
b[0] = 99
print(a)           # [ 0  1 99  3  4  5  6  7  8  9] – a is changed
print(b.base is a) # True – view

# reset
a = np.arange(10)

# fancy indexing – copy
c = a[[2, 3, 4]]
c[0] = 99
print(a)           # [ 0  1  2  3  4  5  6  7  8  9] – a unchanged
print(c.base is None) # True – copy

# but fancy ASSIGNMENT writes through
a[[2, 3, 4]] = [99, 88, 77]
print(a)           # [ 0  1 99 88 77  5  6  7  8  9] – a changed

```

Q22 · Boolean masks — what shape do they need, and what do they return?

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan, Goldman · Topic: boolean indexing

The question (interviewer's verbatim phrasing):

Given an array `prices = np.array([100, 200, 150, 300, 250])`, write code to get just the prices above 200 using a boolean mask. What shape does the result have?

The 30-second answer: `prices[prices > 200]` returns a 1-D array `[300, 250]`. The mask `prices > 200` is a boolean array of the same shape as `prices`. Indexing with that mask selects every position where the mask is True, in flattened order, returning a 1-D copy. The result is always 1-D regardless of the input dimensionality — boolean indexing flattens. To preserve shape, use `np.where(prices > 200, prices, 0)` instead, which keeps shape and replaces False positions with 0.

Step-by-step thought process: 1. State the syntax. The mask is the result of a comparison — `prices > 200` evaluates element-wise to a boolean array of the same shape. 2. Explain the selection. `arr[mask]` returns a 1-D array of every element where `mask` is True. The order is the C-order traversal of the original — row-major for 2-D, etc. 3. Note the always-1-D result. Even if `arr` is 2-D,

`arr[arr > 5]` is 1-D. If you want a 2-D result with masked positions replaced by a sentinel value, use `np.where(mask, arr, sentinel)` — that preserves shape. 4. Mention the in-place assignment form. `arr[arr > 200] = 200` clips every value above 200 to exactly 200. This is the canonical "clip outliers" pattern. The right-hand-side mask is a copy, but the left-hand mask assignment writes through.

Edge cases to mention: - The mask must be the same shape as the array (or broadcast-compatible to it). A length-mismatched mask raises `IndexError`. - Combining masks: use `&`, `|`, `~` for element-wise AND, OR, NOT — NOT `and`, `or`, `not`, which are Python-scalar and raise on arrays.

`(prices > 200) & (prices < 300)` is correct; `prices > 200 and prices < 300` raises. - Use parentheses around comparisons before `&` / `|`: `(a > 0) & (b > 0)`, not `a > 0 & b > 0` — the latter has bitwise-precedence bugs.

What NOT to say: - "Masks return the same shape" — they don't; mask indexing flattens. - "Use `and` / `or` between masks" — that raises `ValueError: The truth value of an array is ambiguous`. Use `&` / `|`.

Common follow-up: "How do you count how many prices are above 200?" — `(prices > 200).sum()` — booleans count as 1 and 0 in a sum. `np.count_nonzero(prices > 200)` is the explicit-name variant; both produce the same result and the same speed.

```
import numpy as np

prices = np.array([100, 200, 150, 300, 250])

# mask is a boolean array
mask = prices > 200
print(mask)           # [False False False  True  True]

# selection — 1-D result
print(prices[mask])   # [300 250]

# combining
print(prices[(prices > 100) & (prices < 300)]) # [200 150 250]

# count
print((prices > 200).sum()) # 2
print(np.count_nonzero(prices > 200)) # 2

# clip via in-place mask assignment
prices[prices > 250] = 250
print(prices)         # [100 200 150 250 250]
```

Q23 · `np.ix_` — what is it for, and why can't you just write `arr[rows, cols]` ?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Postman, Swiggy · Topic: ix_ pattern

The question (interviewer's verbatim phrasing):

I have a matrix and I want to select the sub-matrix at rows `[0, 2, 4]` and columns `[1, 3]`. Why does `arr[[0, 2, 4], [1, 3]]` not give me that, and how does `np.ix_` fix it?

The 30-second answer: `arr[[0, 2, 4], [1, 3]]` is fancy indexing in two axes simultaneously — it zips the two index arrays element-by-element and picks `arr[0, 1]`, `arr[2, 3]`, then would fail because the index arrays have different lengths. Even if lengths matched, it's not what you want — you want the *cross product* of rows and cols, the 3×2 sub-matrix. `np.ix_` is the helper that turns two 1-D index arrays into a broadcasting-compatible pair so the result is the cross product: `arr[np.ix_([0, 2, 4], [1, 3])]` returns shape `(3, 2)`.

Step-by-step thought process: 1. State what numpy actually does with two parallel index arrays. `arr[[i0, i1, ...], [j0, j1, ...]]` returns `[arr[i0, j0], arr[i1, j1], ...]` — *zipped*, 1-D output, one cell per pair. 2. Show why that's not the sub-matrix. The user wants the 6-cell rectangle defined by rows × cols. Zipped indexing picks 3 cells, not 6, and only if the index arrays are the same length. 3. Introduce `np.ix_`. It reshapes the first argument to a column vector and the second to a row vector, so when numpy broadcasts them in the index, it sweeps every (row, col) pair:

`np.ix_([0, 2, 4], [1, 3])` returns a tuple of `(array([0, 2, 4]), array([1, 3]))`. Used as an index, that yields the full cross product. 4. Mention the alternative: `arr[[0, 2, 4][:, [1, 3]]` — index rows first, then columns. Same result, two passes, makes an intermediate copy of the rows. `np.ix_` does it in one pass.

Edge cases to mention: - `np.ix_` works for any number of axes, not just 2. `arr[np.ix_([0, 2], [1, 3], [4, 5])]` on a 3-D array gives a 2×2×2 sub-cube. - The returned arrays from `np.ix_` aren't independent — they're meant to be used together as a single index expression. - For just two axes, `arr[rows][:, cols]` is sometimes more readable. For three or more axes, `np.ix_` is the only clean option.

What NOT to say: - "`arr[[0, 2, 4], [1, 3]]` should error, not pick mismatched pairs" — it errors only when lengths differ; same-length arrays silently pick wrong-shaped output. Know the distinction. - "Just use a nested for-loop" — works, but defeats the point of numpy.

Common follow-up: "How would you select the diagonal of a square matrix using fancy indexing?" — `arr[np.arange(n), np.arange(n)]` — both index arrays are the same and same-length, so this is the zip pattern, picking `arr[0,0]`, `arr[1,1]`, etc. The result is the 1-D diagonal. Equivalent to `np.diag(arr)`.

```
import numpy as np

arr = np.arange(20).reshape(4, 5)
print(arr)
# [[ 0  1  2  3  4]
#   [ 5  6  7  8  9]
#   [10 11 12 13 14]
#   [15 16 17 18 19]]

# zipped indexing – picks pairs, not a sub-matrix
print(arr[[0, 2], [1, 3]])      # [1 13] – picks (0,1) and (2,3)

# cross-product via np.ix_
print(arr[np.ix_([0, 2], [1, 3])])
# [[ 1  3]
#   [11 13]] – shape (2, 2), the full sub-matrix

# alternative – two-step
print(arr[[0, 2][:, [1, 3]])    # same result
```

Q24 · In-place modification via indexing — when does it work and when does it silently fail?

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, Razorpay, Swiggy, Postman · Topic: in-place indexing

The question (interviewer's verbatim phrasing):

I wrote `arr[arr > 0] += 1`. Does that mutate `arr` in place, or does it create a new array? What about `arr[arr > 0] = arr[arr > 0] + 1`?

The 30-second answer: `arr[arr > 0] += 1` mutates `arr` in place. The augmented assignment is special-cased to dispatch through `__setitem__` on the left side, treating the mask as the assignment target — `arr` is written to directly. `arr[arr > 0] = arr[arr > 0] + 1` also mutates `arr`, but it does so by (1) reading the masked elements into a temporary 1-D array, (2) adding 1 to that copy, (3) writing the result back via the masked-assignment form. Both forms end with the same `arr`, but the second creates a transient allocation and is slightly slower. For complex updates, the second form is sometimes the only one that reads clearly.

Step-by-step thought process: 1. Lead with the bottom line: both forms produce the same final `arr`. The difference is whether there's an intermediate copy. 2. Walk through `arr[mask] += 1`. Python's `+=` on a subscripted left-hand side desugars roughly to `arr.__setitem__(mask, arr.__getitem__(mask) + 1)`. Numpy's `__setitem__` with a boolean mask writes the right-

hand values into the masked positions. Net effect: `arr` is updated. 3. Walk through `arr[mask] = arr[mask] + 1`. Step 1: `arr[mask]` is fancy-index-read, allocates a 1-D copy of the masked values. Step 2: `+ 1` produces another 1-D temporary. Step 3: assignment writes back. Two allocations versus zero for the in-place form. For small arrays this is invisible; for a 100-million-row array, it matters. 4. Mention the duplicate-indices gotcha. `arr[[0, 0, 0]] += 1` does NOT add 3 to `arr[0]` — it reads-modifies-writes once, with duplicates collapsed. To add with duplicates, use `np.add.at(arr, [0, 0, 0], 1)`, the explicit "unbuffered" form.

Edge cases to mention: - `arr[mask] = scalar` works for any scalar and is the cleanest clip/replace pattern: `arr[arr < 0] = 0`. - `arr[mask] = other_arr` requires `other_arr` to be broadcast-compatible with the shape of the masked region (which is 1-D, of length `mask.sum()`). - `np.add.at`, `np.multiply.at`, etc. exist for the duplicate-index unbuffered case. They're slower than `+=` but correct for histogram-style accumulation.

What NOT to say: - "`+=` is just sugar for `= ... +`" — for scalars yes; for subscripted left-hand sides on numpy arrays the dispatch is meaningfully different (in-place buffer ops vs copy-and-write). - "`np.add.at` does the same thing as `+=`" — only for unique indices; for duplicates, the two differ and `+=` is wrong if you wanted accumulation.

Common follow-up: "You want to count occurrences of each integer in a length-N array of small integers. How?" — `counts = np.zeros(max_val + 1, dtype=int); np.add.at(counts, arr, 1)` — uses `np.add.at` to correctly accumulate duplicates. Or `np.bincount(arr)`, which is the purpose-built function and faster.

```
import numpy as np

arr = np.array([-2, -1, 0, 1, 2, 3])

# in-place via +=
copy1 = arr.copy()
copy1[copy1 > 0] += 1
print(copy1)           # [-2 -1  0  2  3  4]

# read-modify-write via explicit assignment
copy2 = arr.copy()
copy2[copy2 > 0] = copy2[copy2 > 0] + 1
print(copy2)           # [-2 -1  0  2  3  4] — same result

# the duplicate-index gotcha
buf = np.zeros(3, dtype=int)
buf[[0, 0, 0]] += 1
print(buf)             # [1 0 0] — only added once, not three times

# correct accumulation
buf = np.zeros(3, dtype=int)
np.add.at(buf, [0, 0, 0], 1)
print(buf)             # [3 0 0] — added three times
```

Q25 · Multi-axis indexing — the difference between `arr[1, 2]`, `arr[1][2]`, and `arr[1:2, 2:3]`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan, Walmart Labs · Topic: multi-axis indexing

The question (interviewer's verbatim phrasing):

What's the difference between `arr[1, 2]`, `arr[1][2]`, and `arr[1:2, 2:3]` on a 2-D array? They look similar.

The 30-second answer: `arr[1, 2]` and `arr[1][2]` both return the scalar at row 1, column 2. They produce the same value, but `arr[1, 2]` is one numpy operation while `arr[1][2]` is two — the first creates a 1-D view of row 1, then indexes into that. `arr[1:2, 2:3]` is different: it uses slices, which keep the axes, returning a 2-D array of shape `(1, 1)` containing the same value. For reading, `arr[1, 2]` is the fastest and cleanest. For preserving shape (e.g., when downstream code expects a 2-D), use slices.

Step-by-step thought process: 1. Walk through each. `arr[1, 2]` is a single-step tuple index — numpy directly computes the offset into the buffer. Output: 0-D scalar. 2. `arr[1][2]` is two steps — first `arr[1]` produces a 1-D view of row 1, then `[2]` indexes that view at position 2. Output: same scalar. Slightly slower due to the intermediate view object. Identical for reading. 3. For writing, the two-step form has a subtle gotcha. `arr[1][2] = 99` does write to `arr` because `arr[1]` is a view — writing into the view writes into the buffer. But `arr[fancy_index][2] = 99` does NOT write because `arr[fancy_index]` is a copy. The single-step form `arr[1, 2] = 99` is always correct. 4. `arr[1:2, 2:3]` returns a 2-D `(1, 1)` array. The slice in each axis keeps that axis but reduces it to one element. Useful when you need a 2-D result for downstream broadcasting or reshaping.

Edge cases to mention: - `arr[1, :]` and `arr[1]` both return the same 1-D view of row 1. The trailing `:` is implicit when omitted. - For fancy indexing, `arr[[1, 2]][2]` and `arr[[1, 2], 2]` differ. The first picks rows then indexes; the second is parallel fancy indexing that returns one element from each picked row. Different results. - `np.atleast_2d(arr[1, 2])` is the explicit "give me a 2-D result" wrapper — useful when you don't know the input dimensionality in advance.

What NOT to say: - "`arr[1, 2]` and `arr[1][2]` are different operations" — they produce the same value; the difference is just one numpy step vs two. - "Slicing with `1:2` is the same as `1`" — no, slicing keeps the axis (size 1), scalar indexing drops it.

Common follow-up: "You wrote `arr[fancy_mask][0] = 99` and `arr` didn't change. Why?" — Because `arr[fancy_mask]` is a copy (fancy indexing copies), and `[0] = 99` writes into the copy, not into `arr`. The fix: `arr[fancy_mask][0]` can't be turned into an in-place write through chaining; use `arr[indices_to_modify] = 99` with a directly-constructed index.

```
import numpy as np

arr = np.arange(20).reshape(4, 5)

print(arr[1, 2])          # 7 – scalar
print(arr[1][2])          # 7 – same scalar
print(arr[1:2, 2:3])      # [[7]] – 2-D (1, 1) array
print(arr[1:2, 2:3].shape) # (1, 1)

# in-place write works through view chaining
arr[1][2] = 99
print(arr[1, 2])          # 99

# in-place via tuple form (cleaner)
arr[1, 2] = 88
print(arr[1, 2])          # 88
```

Section E — Numerics + Linear Algebra

The final five questions cover the small but high-value block of pure numerical operations — what makes numpy fast (ufuncs and vectorization), the matrix multiplication operator, the `solve` -versus- `inv` discipline that signals numerical literacy, the modern random API that replaced `np.random.rand`, and `astype` for the dtype conversion that comes up in every ML pipeline. These are the questions that distinguish a candidate who "uses numpy" from one who "thinks in numpy".

Q26 · What's a ufunc, and why is `np.exp(arr)` faster than a Python loop?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India, JP Morgan, Goldman · Topic: ufuncs, vectorization

The question (interviewer's verbatim phrasing):

Explain why `np.exp(arr)` is so much faster than `[math.exp(x) for x in arr]`.
What's a "ufunc" and what does vectorization mean here?

The 30-second answer: A ufunc is a universal function — a numpy primitive that applies an element-wise operation across an entire array in compiled C, with one type-check up front instead of one per element. `np.exp` is a ufunc. The Python list comprehension calls `math.exp` ~N times, each call going through the interpreter, doing a method lookup, unboxing the int, converting to float, computing exp, boxing the result back into a Python float — per element. The ufunc loop in C does all that once at

the top, then runs a tight scalar loop over the raw buffer. The speedup is typically 50–200x for simple ops, and it's why "use numpy operations, not Python loops" is the first rule of numerical Python.

Step-by-step thought process: 1. Name the term. Ufunc = universal function. `np.exp`, `np.sin`, `np.add`, `np.maximum`, `np.where` — all ufuncs. The ufunc machinery is in `numpy.core.umath`. 2. Walk through the per-element cost. Each Python iteration is: bytecode dispatch, scope lookup of `math.exp`, type check of the argument, C-level call, type check of the return, box into Python float, append to result list. That's many tens of nanoseconds even before the actual `exp` call. 3. Walk through the C loop. Numpy resolves the dtype once at the top, picks the right C function pointer, then runs `for (i=0; i < N; i++) out[i] = exp(in[i])`. Each element is sub-nanosecond. The interpreter is never invoked inside the loop. 4. Generalize the rule: any time you find yourself writing a Python for-loop over numpy array elements, look for the ufunc or function that does it in C. If there's no direct match, look for the combination of broadcasting + ufunc + reduction that gets there.

Edge cases to mention: - Custom Python functions can be wrapped with `np.vectorize` for a syntactic shortcut, but that's not a real ufunc — it's still a Python loop under the hood, just dressed up. For real speed, write the operation using existing ufuncs or use `numba` / `cython`. - Ufuncs support `out=` to write into a pre-allocated buffer. `np.exp(arr, out=arr)` does in-place exp, saving the output allocation. Useful in tight memory loops. - Ufuncs support broadcasting natively — `np.add(arr2d, arr1d)` broadcasts the 1-D against the 2-D following the standard rules. So vectorization and broadcasting compose.

What NOT to say: - "`np.vectorize` makes Python functions fast" — it doesn't; it's a wrapper, not a compiler. Real speed comes from real ufuncs or compilation. - "Loops in numpy are fine" — explicit Python for-loops over a million-element array are the most common performance bug in numerical Python.

Common follow-up: "You profiled a function and `np.exp(arr)` is dominating the time. The array has 100M elements. How do you speed it up?" — Options: (a) use `float32` instead of `float64` if precision allows — halves memory and is sometimes ~2x faster on modern CPUs; (b) compute in chunks if you're memory-bound; (c) reach for `numexpr` or `numba` for fused-loop ops; (d) check if a downstream step actually needs the full exp or can use a log-space alternative.

```
import numpy as np
import math
import time

arr = np.linspace(-1, 1, 1_000_000)

# Python loop
t = time.perf_counter()
out_py = [math.exp(x) for x in arr]
print("Python loop:", round(time.perf_counter() - t, 3), "seconds")

# ufunc
t = time.perf_counter()
out_np = np.exp(arr)
print("np.exp:      ", round(time.perf_counter() - t, 3), "seconds")

# typical speedup: 50-100x
```

Q27 · `np.dot` , `@` , and `np.matmul` — which one for what?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India, JP Morgan, Goldman · Topic: matrix multiplication

The question (interviewer's verbatim phrasing):

What's the difference between `np.dot(A, B)` , `A @ B` , and `np.matmul(A, B)` ? Which one should I use for matrix multiplication?

The 30-second answer: For 2-D arrays, all three are equivalent — they all compute the matrix product. The `@` operator (Python 3.5+, PEP 465) is the modern, recommended form for readability.

`np.matmul` is the function form of `@` and supports batched matmul for 3-D+ arrays. `np.dot` is the legacy function — for 1-D arrays it's the inner product, for 2-D it's matrix multiplication, but for 3-D+ it does something different (a sum-product over the last axis of A and second-to-last of B), which is not what most people want. Default to `@` for clarity, use `np.matmul` when you need the function form, avoid `np.dot` for anything 3-D.

Step-by-step thought process: 1. State the rule: `@` is the modern operator, use it for matrix multiplication. 2. Walk through the equivalences for 2-D. `A @ B` , `np.matmul(A, B)` , `np.dot(A, B)` all produce the standard $(m, k) @ (k, n) \rightarrow (m, n)$ matrix product. Same result, same speed. 3. Diverge at 3-D. `A @ B` and `np.matmul(A, B)` treat 3-D arrays as a stack of 2-D matrices, doing batched matmul. For `A: (B, m, k)` and `B: (B, k, n)` , they return `(B, m, n)` . `np.dot` does something else — a contraction over the last axis of A and second-to-last of B, giving `(B, m, B, n)` , which is almost never what you want. 4. State the inner-product

case. For 1-D arrays, `A @ B` and `np.dot(A, B)` and `np.matmul(A, B)` all give the same scalar inner product. The convenience of `@` is that you don't have to remember which function form gives you what.

Edge cases to mention: - `np.inner(A, B)` is the inner product over the last axes — different from `np.dot` for higher dimensions. Useful for specific tensor contraction patterns. - `np.einsum` is the universal hammer for any tensor contraction — `np.einsum('ik,kj->ij', A, B)` is matrix multiplication explicitly. Slower to write but unambiguous about which axes contract. - For very small matrices, `@` has Python-call overhead that dominates. For batched small matmuls, building one big block-diagonal or using `np.einsum` once can be faster than `N @` calls.

What NOT to say: - "`np.dot` is the standard, use it" — it's legacy at this point; `@` is the recommended modern form. - "`@` is just syntactic sugar" — it dispatches to `__matmul__`, which on `ndarray` calls a C implementation; it's a real operator with its own semantics, distinct from `*` (which is element-wise).

Common follow-up: "What does `A * B` do versus `A @ B`?" — `A * B` is element-wise multiplication (Hadamard product) — both operands must broadcast to the same shape. `A @ B` is matrix multiplication — the inner dimensions must match. They're fundamentally different operations; mixing them up is a classic bug.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# all equivalent for 2-D
print(A @ B)
print(np.matmul(A, B))
print(np.dot(A, B))
# [[19 22]
#  [43 50]]

# element-wise – different operation
print(A * B)
# [[ 5 12]
#  [21 32]]

# batched matmul – only @ / matmul, not dot
batch_A = np.random.default_rng(0).standard_normal((10, 3, 4))
batch_B = np.random.default_rng(1).standard_normal((10, 4, 5))
print((batch_A @ batch_B).shape)      # (10, 3, 5) – batched
print(np.matmul(batch_A, batch_B).shape) # (10, 3, 5) – same
# np.dot would give (10, 3, 10, 5) – usually not what you want
```

Q28 · `np.linalg.solve` versus `np.linalg.inv` — why don't you invert?

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Microsoft India, Razorpay, Swiggy · Topic: linear algebra discipline

The question (interviewer's verbatim phrasing):

To solve $Ax = b$, why do experienced numerical engineers always say "use `np.linalg.solve`, not `np.linalg.inv(A) @ b`"?

The 30-second answer: Three reasons, in order of importance. First, accuracy — `solve` uses LU decomposition with partial pivoting on `A` and `b` together, which is numerically more stable than computing the explicit inverse and then multiplying. The accumulated rounding error in `inv(A) @ b` is larger, sometimes catastrophically so for ill-conditioned matrices. Second, speed — `solve` does roughly half the floating-point work because it skips computing `inv(A)` and just does the forward + back substitution. Third, memory — `inv(A)` is a full $N \times N$ allocation; `solve` works without ever materializing it. The rule "don't invert if you can solve" is universal in numerical computing.

Step-by-step thought process: 1. State the rule: solve, never invert. Memorize the catchphrase. 2. Walk through the accuracy argument. LU with pivoting is the gold standard for general linear systems. `inv(A) @ b` does LU twice (effectively — once for the inverse, then a matmul that's also subject to roundoff). The error bound is worse, especially when `A` is close to singular. 3. The speed argument. Computing `inv(A)` is $O(N^3)$ and produces an $N \times N$ result. `solve(A, b)` is $O(N^3)$ for the LU but skips the back-substitution-to-form-inverse step; for a single right-hand side it's measurably faster. 4. The memory argument. For $N = 10000$, `inv(A)` is 800 MB at `float64`. `solve` works in-place on the factored form, never materializing the inverse. This matters for production code at scale.

Edge cases to mention: - If you really need the inverse — say, for a Kalman filter step that uses it explicitly multiple times — `inv(A)` is reasonable. Even then, prefer `scipy.linalg.lu_factor` once, then `scipy.linalg.lu_solve` repeatedly with different RHSes. (Note: LU factor/solve live in `scipy.linalg`, not `numpy.linalg` — numpy exposes `np.linalg.solve` for the one-shot case and `np.linalg.cholesky` for SPD, but not the standalone LU pair.) - For symmetric positive-definite matrices, use `np.linalg.cholesky` or `scipy.linalg.cho_solve` — faster and more stable than LU. - For least-squares ($\min ||Ax - b||^2$ when `A` is rectangular), use `np.linalg.lstsq`, not the normal equations `inv(A.T @ A) @ A.T @ b` — same anti-invert principle.

What NOT to say: - "`inv(A) @ b` and `solve(A, b)` give the same answer" — they don't, in finite precision. The difference shows up in the last few digits for well-conditioned `A` and in the leading digits for ill-conditioned `A`. - "Inverting is fine for small matrices" — habit forms regardless of size; train yourself to reach for `solve`.

Common follow-up: "You need to solve $Ax = b$ for 100 different `b`s with the same `A`. What's the right approach?" — Factor once, solve many times.

`lu, piv = scipy.linalg.lu_factor(A); xs = [scipy.linalg.lu_solve((lu, piv), b) for b in bs]` . This avoids re-factoring `A` 100 times, which is the dominant cost.

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.standard_normal((1000, 1000))
b = rng.standard_normal(1000)

# the right way
x_solve = np.linalg.solve(A, b)

# the wrong way – works for well-conditioned A, but slower and less stable
x_inv = np.linalg.inv(A) @ b

print(np.allclose(x_solve, x_inv))      # True for well-conditioned A

# verify
print(np.linalg.norm(A @ x_solve - b)) # tiny
```

Q29 · `np.random.default_rng` — what's it, and why does it replace `np.random.rand` ?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Microsoft India, Goldman, JP Morgan · Topic: modern random API

The question (interviewer's verbatim phrasing):

What's the difference between `np.random.rand(1000)` and `np.random.default_rng().random(1000)` ? Why does the numpy 1.17+ documentation say to prefer the second?

The 30-second answer: `np.random.rand` and friends use a single global random state — the legacy module-level API. `np.random.default_rng()` returns a `Generator` object with its own independent state. The newer API solves three problems with the global state: reproducibility in parallel code (each worker gets its own generator, no contention), explicit seeding semantics (`default_rng(42)` is unambiguous about which generator gets seeded), and a better default algorithm (PCG64 instead of Mersenne Twister, faster and statistically stronger). For any new code, instantiate a generator with `rng = np.random.default_rng(seed)` and call its methods like `rng.random(1000)` , `rng.integers(0, 100, 1000)` , `rng.standard_normal(1000)` .

Step-by-step thought process: 1. State the API shift. Old API: `np.random.rand(...)` , `np.random.randn(...)` , `np.random.seed(42)` . New API (1.17+): `rng =`

`np.random.default_rng(42); rng.random(...), rng.standard_normal(...)`. 2. Why the change. The global state is a hidden dependency — any library that calls `np.random.seed` affects every other consumer of `np.random`. The new generator object makes the state explicit and per-call-site. 3. Seeding semantics. `np.random.default_rng(42)` creates a fresh generator seeded with 42. `np.random.default_rng()` uses a fresh seed from the OS each time. For reproducible experiments, always pass an explicit seed. 4. The methods on the generator. `random()` for uniform $[0, 1)$. `integers(low, high)` for ints. `standard_normal()` for $N(0, 1)$. `normal(mu, sigma)` for $N(\mu, \sigma)$. `choice(arr, size)` for sampling. `permutation(arr)` for shuffling. The shape goes as a `size=` kwarg.

Edge cases to mention: - `np.random.default_rng` uses PCG64 by default. For a different algorithm, pass `BitGenerator`: `np.random.default_rng(np.random.MT19937(42))` uses the legacy Mersenne Twister. - The two APIs do not share state. `np.random.seed(42)` does NOT affect a generator created with `np.random.default_rng()`. - For parallel workers, use `np.random.default_rng().spawn(N)` to create N independent child generators — guaranteed non-correlated streams.

What NOT to say: - "The old API is fine, no need to change" — legacy code can stay, but new code should use the new API; that's the official recommendation. - "PCG64 isn't a real algorithm" — it is; PCG (Permuted Congruential Generator) was published in 2014 and is now the numpy default.

Common follow-up: "How do you make a numerical experiment fully reproducible?" — Use a single explicit seed: `rng = np.random.default_rng(42)`. Pass `rng` (not a fresh `default_rng()` per call) into every function that needs randomness. For multi-process work, use `rng.spawn(n_workers)` to get independent child generators per worker, then pass each child to its worker.

```
import numpy as np

# old API — works, but global state
np.random.seed(42)
print(np.random.rand(3))           # [0.37454012 0.95071431 0.73199394]

# new API — explicit generator
rng = np.random.default_rng(42)
print(rng.random(3))               # different values; different algorithm

# reproducibility
rng1 = np.random.default_rng(42)
rng2 = np.random.default_rng(42)
print(np.array_equal(rng1.standard_normal(100), rng2.standard_normal(100))) # True

# independent streams for parallel workers
parent = np.random.default_rng(42)
children = parent.spawn(4)
for c in children:
    print(c.random(3))              # 4 independent streams
```

Q30 · `astype` for dtype conversion — what does it do, when does it copy, and what's the safe way to downcast?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Microsoft India, Walmart Labs · Topic: dtype conversion

The question (interviewer's verbatim phrasing):

You have a `float64` array and need to convert it to `float32` to save memory. How do you do it, what happens with precision, and how do you avoid silent overflow when going from `float64` to `int32` ?

The 30-second answer: `arr.astype(np.float32)` returns a new `float32` array, with each value cast — `float64` values that fit are kept as the nearest `float32`, those that don't (because they exceed the smaller exponent range) become `inf`. `astype` always copies by default; pass `copy=False` to get a view back *if* the dtype matches, otherwise it still copies (so it's just a perf hint). For `float → int`, `astype` truncates toward zero — `np.array([1.7]).astype(int)` is `[1]`. For overflow safety on `int` casts, check the value range against the target dtype's `info` first, or clip explicitly: `arr.clip(-2**31, 2**31 - 1).astype(np.int32)`.

Step-by-step thought process: 1. Start with the basic call. `new = arr.astype(np.float32)`. By default, this allocates a new array and casts every element. 2. Walk through what casting does. For float → smaller float: precision is reduced, exponent range is reduced; values outside the range become `inf`. For float → int: truncation (toward zero), not rounding — `(-1.7).astype(int)` is `-1`, not `-2`. For int → smaller int: overflow wraps silently (two's complement) — `np.array([200], np.int8)` from `[200]` actually stores `-56`. 3. The `copy=False` flag. If the target dtype matches the source, `copy=False` returns a view; if it doesn't, it copies anyway. So it's a hint, not a guarantee. Useful in library code where you want to avoid double-copying. 4. The safe-downcast pattern. Before `astype(np.int32)` on a float array, clip to the int32 range. Before `astype(np.float32)`, check `np.abs(arr).max()` against `np.finfo(np.float32).max` if you can't tolerate `inf`s. For ML feature matrices, `float32` is the production standard; the precision loss is negligible vs the 2x memory win.

Edge cases to mention: - `casting=` argument controls strictness. `casting='safe'` raises on lossy conversions; `casting='unsafe'` (default) allows anything. Use `'safe'` in library code that should fail loudly. - For string-to-numeric conversion, `astype` works for clean strings: `np.array(['1', '2']).astype(int)`. For dirty data (NaN strings, commas), use pandas; numpy doesn't handle text parsing well. - `arr.view(np.float32)` is NOT the same as `astype` — `view` reinterprets the same bytes under a different dtype, no casting. `np.array([1.5], np.float64).view(np.float32)` gives garbage. Use `view` only for bitwise reinterpretation, never for type conversion.

What NOT to say: - " `astype` is always safe" — it isn't; it silently truncates ints and produces `inf` on float underflow/overflow. - "Use `arr.view(other_dtype)` instead, it's faster" — that's a different operation (bit reinterpretation), not a cast.

Common follow-up: "You converted a `float64` array with values around $1e-40$ to `float32`. The result is all zeros. Why?" — `float32`'s smallest positive normal value is $\sim 1.18e-38$; anything smaller underflows to 0. The precision range of `float32` is narrower than `float64`; values that fit `float64` but not `float32` either become `inf` (too big) or 0 (too small). Check with `np.finfo(np.float32)` before the cast.

```
import numpy as np

arr = np.array([1.0, 2.5, 3.7, -1.7], dtype=np.float64)

# float64 -> float32
small = arr.astype(np.float32)
print(small.dtype)      # float32
print(small.nbytes)     # 16 — half of float64's 32

# float -> int truncates toward zero
print(arr.astype(int))  # [ 1  2  3 -1]

# silent int overflow
big = np.array([200], dtype=np.int16)
overflow = big.astype(np.int8)
print(overflow)         # [-56] — overflow wraps

# safe downcast via clip
big_f = np.array([1e10, -1e10, 5], dtype=np.float64)
safe = big_f.clip(-2**31, 2**31 - 1).astype(np.int32)
print(safe)             # [ 2147483647 -2147483648  5] — clipped before cast

# casting='safe' raises on lossy
try:
    arr.astype(np.float16, casting='safe')
except TypeError as e:
    print(e)
# Cannot cast array data from dtype('float64') to dtype('float16') according to the rule 'safe'
```

End of Chapter 6.

Thirty questions covering numpy's four critical surfaces: the `ndarray` object itself, broadcasting, axis semantics, and indexing — plus the small but high-value numerics block. If you've worked through all thirty in step-by-step format, you have the depth Indian data-role interviewers expect on numpy in a 2026 panel. Next chapter: wrangling — missing data, dtypes, and datetime handling that builds on the dtype and broadcasting foundations laid here.

Chapter 7 — Wrangling: Missing Data, dtypes & Datetime (20 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

This is the chapter that decides whether you actually own the messy half of data work. Twenty questions across the four surfaces every Data interviewer in India probes once the candidate sails through `groupby` and `merge` : how missing data behaves in pandas 2.x (three different sentinels and a three-valued logic that catches even experienced devs), how to pick the right dtype (the nullable integer / Arrow-backed / category triad that the lander promises), how to parse and operate on datetimes without losing your timezone, and the real-world wrangling reflexes — messy CSVs, mixed-dtype warnings, duplicate detection, string-to-numeric coercion, split-by-delimiter — that separate the candidate who has only practiced toy datasets from the one who has actually shipped a notebook on a deadline. Each question follows the same shape: tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases, what NOT to say, and the common follow-up. Answers assume pandas 2.2 on Python 3.12 unless a version is called out explicitly.

Table of Sections

- Section A — Missing Data (Q1–Q5)
- Section B — dtypes & the pandas 2.x Backend (Q6–Q10)
- Section C — Datetime, the `.dt` accessor & Timezones (Q11–Q15)
- Section D — Real-World Wrangling Reflexes (Q16–Q20)

Section A — Missing Data

The five questions in this section cover the most-asked area of the chapter, by a wide margin. Every Data-role round one at Razorpay, Swiggy, Flipkart, PhonePe, and the analytics teams at JP Morgan India will ask at least one of these, and the fresher panels at TCS, Infosys, and Wipro lean on `isna` / `fillna` / `dropna` as their default "do you actually know pandas" probe. The trap is that pandas now has three missing-value sentinels — `NaN` , `None` , and `pd.NA` — and treats them differently depending on the column dtype. If you only ever say "use `fillna` " you sound like you have done the Kaggle tutorial; if you can explain when `pd.NA` propagates through a comparison and when it doesn't, you sound like you have shipped pandas to production.

Q1 · The three missing-value sentinels in pandas — `NaN` , `None` , and `pd.NA`

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, JP Morgan · Topic: missing data

The question (interviewer's verbatim phrasing):

pandas seems to have three different ways of representing missing values — `np.nan` , `None` , and `pd.NA` . What's the difference, and when does each one show up?

The 30-second answer: `np.nan` is a floating-point `NaN` from IEEE-754 — it lives in `float64` and `Float64` columns. `None` is the Python null and shows up in `object` -dtype columns (typical for strings before you cast to `string` dtype). `pd.NA` is the modern pandas-native missing-value singleton introduced for the nullable extension dtypes — `Int64` , `Float64` , `boolean` , `string` , and the Arrow-backed types. The three are not interchangeable: `np.nan == np.nan` is `False` , `None == None` is `True` , and `pd.NA == anything` returns `pd.NA` itself because pandas uses Kleene three-valued logic for the new sentinel.

Step-by-step thought process: 1. Open with the dtype map. `np.nan` is a `float` value, so any column that holds `np.nan` is forced up to `float64` . That is why an integer column with one missing value silently becomes `float64` — the classic "why are my user ids now 1.0, 2.0, 3.0" surprise. 2. Move to `None` . In an `object` -dtype Series (untyped strings, mixed types), `None` is what you actually see when you read a CSV with empty cells under the old behavior. `pd.isna(None)` returns `True` , but `None == None` returns plain Python `True` — different from how `NaN` compares. 3. Introduce `pd.NA` . It is the missing-value sentinel for the nullable dtypes — `Int64` , `Float64` , `boolean` , `string` , `category` , and the Arrow-backed types. It uses Kleene three-valued logic, so `pd.NA & False` is `False` , `pd.NA | True` is `True` , but `pd.NA == 5` is `pd.NA` and so are `pd.NA + 1` and `pd.NA > 0` . 4. Land the practical rule: `pd.isna()` and `Series.isna()` correctly recognise all three. Equality comparisons do not. If you write `s == np.nan` you always get `False` ; if you write `s == pd.NA` you get a Series of `pd.NA` . Always use `.isna()` .

Edge cases to mention: - A `datetime64[ns]` column uses `pd.NaT` ("Not a Time") as its missing sentinel — that is a fourth one, technically, but it behaves like `NaN` for purposes of `isna()` and is dtype-specific. - The old `np.nan` lives forever in any non-nullable float column you read from a CSV unless you explicitly opt into nullable dtypes via `convert_dtypes()` or the `dtype_backend=` keyword. - `pd.NA in [pd.NA]` is `True` because list `__contains__` uses identity-then-equality, but `pd.NA == pd.NA` is `pd.NA` (not `True`). The list lookup short-circuits on identity first.

What NOT to say: - "They are all the same, pandas handles it" — wrong; the equality semantics differ, and the dtype implications differ. - "Always use `np.nan` " — outdated advice; for any new pandas 2.x code you want the nullable dtypes and `pd.NA` .

Common follow-up: "If I read a CSV with one empty cell in a column that is otherwise integers, what dtype do I get and how do I keep it as an integer?" — With the legacy backend you get `float64`

(because `np.nan` forces float). With

`pd.read_csv(..., dtype_backend='numpy_nullable')` or `dtype_backend='pyarrow'`, you get `Int64` with `pd.NA` in the missing slot — the integers stay integers.

```
import pandas as pd, numpy as np

s_float = pd.Series([1, np.nan, 3])          # float64; np.nan
s_obj    = pd.Series(["a", None, "c"])       # object; None
s_nul    = pd.Series([1, pd.NA, 3], dtype="Int64") # Int64; pd.NA

s_float.isna()          # [False, True, False]
s_obj.isna()            # [False, True, False]
s_nul.isna()            # [False, True, False] – all three caught
np.nan == np.nan        # False
pd.NA == pd.NA          # <NA> (Kleene)
```

Q2 · `isna` vs `isnull` vs `notna` vs `notnull` — are they aliases?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: missing data

The question (interviewer's verbatim phrasing):

What is the difference between `isna()` and `isnull()` in pandas? Are they the same thing or do they behave differently?

The 30-second answer: They are exact aliases — `Series.isnull` is literally `Series.isna` under the hood, same for `notna` / `notnull`. The duplication exists because R uses `is.null` and `is.na` for different things, and early pandas wanted users from both R and SQL backgrounds to land on a method that "feels familiar." Internally, only `isna()` is the canonical name in pandas 2.x; the team prefers it because `pd.NA` is genuinely "NA," not "null." Use `isna()` in new code.

Step-by-step thought process: 1. State the alias fact directly. Open `pandas/core/generic.py` and you can see `isnull = isna` and `notnull = notna` — same implementation, no behavior difference. 2. Explain the historical reason. R distinguishes `NA` (missing) from `NULL` (no value at all). SQL has `NULL`. pandas chose to ship both names so neither audience felt like an outsider. 3. Mention what `isna()` actually checks. It returns `True` for `np.nan`, `None`, `pd.NA`, and `pd.NaT`. Anything else — including the empty string `""`, the integer `0`, and the boolean `False` — is *not* missing. 4. Close with the canonical recommendation. New pandas 2.x docs use `isna` everywhere; the team has hinted (no timeline) at eventually deprecating the `null` spelling, so write `isna()` for forward compatibility.

Edge cases to mention: - Empty strings, whitespace-only strings, and the strings `"NA"` / `"N/A"` / `"null"` are NOT treated as missing by `isna()`. If your CSV uses those as missing markers, pass

`na_values=["NA", "N/A", "null", ""]` to `read_csv`, or `.replace("", pd.NA)` after the fact. - `pd.isna(pd.NaT)` is `True`, but `pd.NaT == pd.NaT` is `False` — same quirk as `NaN`. - On an `object` Series of nested lists, `isna()` checks the top-level value only; a row holding `[np.nan]` is not flagged as missing because the cell itself is a non-null list.

What NOT to say: - "`isnull` is for SQL-style nulls and `isna` is for numpy NaN" — wrong; they are identical. - "I always check `== None`" — broken; `np.nan == None` is `False`, `pd.NA == None` is `pd.NA`. `isna()` is the only correct primitive.

Common follow-up: "How do I count missing values per column?" — `df.isna().sum()` — returns a Series with column names as index and missing-count as values. For percentages:

`df.isna().mean() * 100`.

```
import pandas as pd, numpy as np
df = pd.DataFrame({"a": [1, np.nan, 3], "b": ["x", None, "z"]})

df.isna().equals(df.isnull()) # True — exact aliases
df.isna().sum()               # a: 1, b: 1
df["a"].isna() | df["b"].isna() # row-level OR
```

Q3 · `fillna` — the four patterns interviewers actually probe

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, PhonePe, Walmart Labs · Topic: missing data

The question (interviewer's verbatim phrasing):

Walk me through the different ways you'd call `fillna`. Give me at least four — a scalar, a per-column dict, a forward-fill, and one where the value comes from another column.

The 30-second answer: Scalar — `df.fillna(0)` fills every NA with the same value. Per-column dict — `df.fillna({"price": 0, "city": "Unknown"})` lets you fill each column with a context-appropriate default. Forward-fill — `df.ffill()` (the 2.x name; `df.fillna(method="ffill")` is deprecated) carries the last observed value forward, ideal for time-series gaps. From another column — `df["price"].fillna(df["fallback_price"])` aligns on index and substitutes wherever `price` is missing.

Step-by-step thought process: 1. Start with the scalar case because that is the one every junior knows. Note that `df.fillna(0)` fills *every* NA across every column, which is almost never what you want on a real-world frame — text columns get a `0`, dates get a `0`, the schema goes feral. 2. Move to the per-column dict pattern. `df.fillna({"col_a": value_a, "col_b": value_b})` is the production-safe form — each column gets a context-appropriate default. Columns not in the dict are left alone. 3.

Walk through the directional fills. `df.ffill()` (forward-fill) and `df.bfill()` (backward-fill) are the modern method names; the old `method="ffill"` / `method="bfill"` keyword inside `fillna()` was deprecated in pandas 2.1 (still works in 2.2 with a FutureWarning, scheduled for removal in 3.0). Add `limit=` to cap how many consecutive NAs you'll fill — typical guard against propagating a stale value across an entire missing day. 4. Land the "value from another column" pattern. `df["A"].fillna(df["B"])` aligns on the index and substitutes only where `A` is missing. This is the pandas-idiomatic version of SQL's `COALESCE(A, B)`.

Edge cases to mention: - `fillna(method="ffill")` was deprecated in pandas 2.1 (FutureWarning) and is scheduled for removal in 3.0 — as of 2.2 it still works but warns. Use `df.ffill()` directly to be forward-safe; legacy code with the keyword runs today, errors after the 3.0 upgrade. - `inplace=True` on `fillna` is still supported in 2.x but the pandas team's roadmap is to deprecate `inplace` across the board because of copy-on-write — write `df = df.fillna(...)` instead. - For nullable dtypes (`Int64`, `string`, etc.), you can fill with `pd.NA` to explicitly keep something missing — no-op but useful in conditional pipelines.

What NOT to say: - "`df.fillna(0)` is the safe default" — it is not; it changes the meaning of every text and date column. - "ffill is just for time series" — also useful for any ordered data: ranked categories, sorted-by-id, grouped-then-filled.

Common follow-up: "What if I want to forward-fill but only within each group, like per-user transaction gaps?" — `df.groupby("user_id").ffill()`. The group keys form the boundary; ffill does not leak across users.

```
import pandas as pd, numpy as np
df = pd.DataFrame({
    "price": [100, np.nan, np.nan, 120, np.nan],
    "fallback": [99, 99, 105, 110, 115],
    "city": ["Mumbai", None, "Delhi", None, "Pune"],
})

df.fillna(0)                # scalar – blunt
df.fillna({"price": 0, "city": "Unknown"}) # per-column dict
df.ffill(limit=1)           # carry-forward, max one row
df["price"].fillna(df["fallback"]) # cross-column COALESCE
df.groupby("city").ffill()  # per-group ffill
```

Q4 · `dropna` — `how`, `thresh`, and `subset` decoded

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Meesho, Cred, JP Morgan · Topic: missing data

The question (interviewer's verbatim phrasing):

I have a DataFrame where some columns are critical and some are nice-to-have. How would you drop rows where the critical columns are missing but keep rows that only have some optional columns missing?

The 30-second answer: Use `df.dropna(subset=["critical_a", "critical_b"])`. The `subset=` keyword restricts the missing-check to the columns you list — rows are dropped only when one of those is `NA`. If you want to also tolerate up to N missing optional columns, pass `thresh=` (the minimum non-NA count required to keep a row). `how="any"` drops if *any* of the considered columns is NA (the default); `how="all"` drops only if *every* considered column is NA.

Step-by-step thought process: 1. Establish the default behavior so the interviewer knows you know it. `df.dropna()` drops every row that has *any* NA in *any* column — almost always too aggressive on real data, where you'd lose 80% of rows. 2. Walk through `subset=`. Pass a list of columns and `dropna` only checks those for missingness. This is the production-safe answer: "I'm dropping rows where `user_id` is missing, but I don't care about `referral_source`." 3. Introduce `how`. `"any"` (default) — drop if any of the considered columns is NA. `"all"` — drop only if every considered column is NA. Useful when you have parallel sensors and want to drop rows where all sensors failed but keep rows where at least one fired. 4. Land `thresh=`. It is the underrated keyword — "keep rows that have at least N non-NA values." `df.dropna(thresh=5)` keeps any row with five or more populated cells, regardless of which ones. Pairs beautifully with `subset=` to restrict which columns count toward the threshold.

Edge cases to mention: - `axis=1` flips the whole thing to "drop columns instead of rows" — `df.dropna(axis=1, thresh=len(df) * 0.9)` drops any column where more than 10% of values are missing. Standard cleaning step on wide ad-hoc datasets. - `subset=` on `axis=1` takes a list of *row labels* (the index), not column names — confusing but documented. Most people use `axis=0` and `subset=` for columns. - `how` and `thresh` are mutually exclusive in their effect: if you pass both, `thresh` wins. Don't mix them in the same call.

What NOT to say: - "`dropna()` is safe to call without arguments" — it is the most aggressive form possible. On a real DataFrame with any sparsity, you keep almost nothing. - "`how='all'` means drop everything that's all NA" — close, but specifically "drop the *row* where every considered column is NA," not "drop columns."

Common follow-up: "How do you drop columns that are more than 50% empty?" — `df.dropna(axis=1, thresh=int(len(df) * 0.5))`. That keeps columns with at least 50% non-NA values.

```
import pandas as pd, numpy as np
df = pd.DataFrame({
    "user_id": [1, 2, np.nan, 4],
    "email": ["a@x.in", None, "c@x.in", "d@x.in"],
    "referral": [None, "google", None, None],
    "city": ["Mumbai", "Delhi", None, "Pune"],
})

df.dropna() # drops 3 rows – too much
df.dropna(subset=["user_id", "email"]) # critical-cols only
df.dropna(thresh=3) # keep if 3+ non-NA
df.dropna(axis=1, thresh=int(len(df) * 0.75)) # drop sparse columns
df.dropna(subset=["referral"], how="all") # drop if referral is NA
```

Q5 · Kleene three-valued logic with `pd.NA` — the boolean trap

Tags: Difficulty: Hard · Asked at: Cred, PhonePe, Postman, Razorpay, Goldman · Topic: missing data semantics

The question (interviewer's verbatim phrasing):

If I have a nullable boolean Series with some `pd.NA` values and I write `if s.any():`, what happens? Walk me through Kleene three-valued logic and what it changes in pandas.

The 30-second answer: `if s.any()` raises `TypeError: boolean value of NA is ambiguous` because `pd.NA` cannot collapse to a plain Python boolean — that is Kleene logic refusing to guess. The fix is `s.any(skipna=True)` (the default for the method, but the bare `if` bypasses it) or being explicit with `s.fillna(False).any()`. Kleene three-valued logic says `True | NA == True`, `False & NA == False`, `True & NA == NA`, `NA | NA == NA` — operations short-circuit on the value that determines the outcome, otherwise propagate `NA`.

Step-by-step thought process: 1. Open with what Kleene logic is. It is three-valued: `True`, `False`, `NA`. Operations are defined to short-circuit when the result is determined regardless of the missing value, and to propagate `NA` otherwise. 2. Walk through the truth tables verbally. `True | anything = True`, including `True | NA`. `False & anything = False`, including `False & NA`. But `True & NA = NA` because the result genuinely depends on what NA turns out to be. 3. Explain the Python-boolean ambiguity. Plain Python `if` needs to call `bool(value)` and get `True` or `False`. `bool(pd.NA)` raises `TypeError` because returning either answer would be a guess. Same for `not pd.NA`, `pd.NA and x`, `pd.NA or x` when used as Python control flow. 4. Land the survival rules: use `.any(skipna=True)` / `.all(skipna=True)` (defaults are `skipna=True` so the method call works, only direct `bool()` fails), use `s.fillna(False)` before any

plain-Python conditional, and use `&` / `|` / `~` (bitwise) on the Series itself, which apply Kleene element-wise without collapsing to a single bool.

Edge cases to mention: - `pd.NA == pd.NA` is `pd.NA`, not `True` — same trap as NumPy's `nan != nan` but with the Kleene flavour. Use `.equals()` for whole-Series equality including NA-position equality. - `(s == 5)` on a Series with `pd.NA` gives back a boolean Series with `pd.NA` in the missing slots — not `False`. Filtering with `df[s == 5]` will raise if it hits NA slots; wrap in `(s == 5).fillna(False)` or use `s.eq(5, fill_value=False)`. - The old `np.nan`-based boolean Series (object dtype with NaN) does *not* use Kleene; it uses the legacy "NaN is truthy as a non-empty float" behavior, which is its own footgun. Migrating to `boolean` dtype gives you Kleene and a clean exception instead of a silent wrong answer.

What NOT to say: - "`pd.NA` is just like `None` in boolean contexts" — wrong; `None` evaluates to `False` in `bool()`, `pd.NA` raises. - "Kleene logic is for SQL only" — pandas explicitly adopted Kleene for the nullable dtypes in 1.0; it is now the official semantics.

Common follow-up: "*How do you filter rows where a nullable-boolean column is True, ignoring missing values?*" — `df[df["flag"].fillna(False)]` or `df[df["flag"].eq(True)]` — the comparison itself returns a boolean Series with NA-as-False after the fill.

```
import pandas as pd
s = pd.Series([True, False, pd.NA], dtype="boolean")

s | True          # [True, True, True]    - short-circuit
s & False         # [False, False, False] - short-circuit
s & True          # [True, False, <NA>]    - NA propagates
s.any()           # True - method skips NA by default
# bool(pd.NA)     # TypeError: boolean value of NA is ambiguous
s.fillna(False).all() # False - safe coercion
```

Section B — dtypes & the pandas 2.x Backend

This section is where pandas 2.x diverges hardest from the pandas-1.x tutorials most candidates trained on. The five questions here cover the modern dtype landscape — the nullable numeric and string types, the `category` dtype that turns a million-row column of "Mumbai" / "Delhi" / "Bengaluru" into a fixed dictionary lookup, the Arrow backend that pandas 2.x added as a parallel universe of dtypes, the `convert_dtypes()` inference helper that turns legacy frames into nullable ones, and the `dtype_backend=` keyword on `read_csv` / `read_parquet` that lets you pick your storage at the door. If you can speak fluently about why you would pick `Int64` over `int64`, when `category` actually saves memory, and what `pd.ArrowDtype('string[pyarrow]')` gives you that `string` does not, you sound like someone who has read the 2.0 release notes and the candidate next door does not.

Q6 · `int64` vs `Int64` — nullable integers and why pandas added them

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Swiggy, JP Morgan, Microsoft India · Topic: dtypes

The question (interviewer's verbatim phrasing):

What's the difference between `int64` (lowercase) and `Int64` (capitalised) in pandas? Why would I use the capitalised one?

The 30-second answer: Lowercase `int64` is the NumPy integer dtype — fixed-width, no missing-value support. A single `NaN` forces the whole column up to `float64`. Uppercase `Int64` is the pandas-native nullable integer dtype — it stores integers plus a bitmask of which slots are missing, represented in pandas as `pd.NA`. You use `Int64` when the column is conceptually an integer (user_id, age, count) but might have missing values that you don't want silently coerced to floats.

Step-by-step thought process: 1. Name the two clearly and link each to its origin. `int64` is `numpy.int64` — pandas adopted it directly from NumPy in the 0.x days. `Int64` is `pandas.Int64Dtype` — pandas-native, introduced in 0.24 and stabilised in 1.0. 2. Show the missing-value problem. With `int64`, there is no in-band way to mark a slot missing — IEEE-754 has `NaN` for floats but integers don't. So pandas historically forced an integer column with any NA up to `float64` and put `np.nan` in the gap. Result: user ids become `1.0`, `2.0`, `3.0`, you lose exactness once you cross 2^{53} . 3. Walk through how `Int64` solves it. The dtype is backed by an integer array plus a parallel boolean mask. Missing slots are stored as `pd.NA` in the API but the underlying integer payload stays an integer. Your user ids remain exactly representable. 4. Land the trade-off honestly. `Int64` is slower than `int64` on the hot path — masked arithmetic costs extra. For a column you know is dense (no NAs), stick with `int64`. For any column where missingness is possible, `Int64` keeps the type honest.

Edge cases to mention: - The capitalisation matters and is case-sensitive: `int64` (NumPy), `Int64` (pandas masked), `int64[pyarrow]` (Arrow-backed). Three different things, three different perf profiles. - Operations between `int64` and `Int64` columns return `Float64` if there's any NA risk — pandas widens to the safer nullable type. Mixing them is fine but explicit casting is clearer. - `Series.dtype` printed for nullable types renders as `Int64`, `Float64`, `boolean`, `string` — distinct from the lowercase numpy/object spellings. Easy way to verify what you've got.

What NOT to say: - "They are the same, the capitalisation is just style" — wrong; semantically different. - "Always use `Int64`" — costs perf on dense numeric work; pick based on whether missingness is possible.

Common follow-up: "How do you convert an entire DataFrame's numeric columns to nullable dtypes?" — `df.convert_dtypes()` infers the best nullable dtype per column. Or read with `pd.read_csv(..., dtype_backend='numpy_nullable')` so the frame is born nullable.

```
import pandas as pd, numpy as np

s_legacy = pd.Series([1, 2, np.nan, 4]) # dtype: float64
s_nullable = pd.Series([1, 2, pd.NA, 4], dtype="Int64") # dtype: Int64

s_legacy.dtype # float64 — ids silently floated
s_nullable.dtype # Int64 — still integers
s_nullable + 1 # [2, 3, <NA>, 5] — NA propagates correctly
s_nullable.sum() # 7 — skipna=True default
```

Q7 · The `category` dtype — when it saves memory and when it doesn't

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Meesho, Walmart Labs, Target · Topic: dtypes

The question (interviewer's verbatim phrasing):

I have a column with city names — "Mumbai", "Delhi", "Bengaluru" — and the DataFrame has 50 million rows. Would you cast it to `category` ? Why?

The 30-second answer: Yes, casting to `category` will collapse 50 million repeated strings down to a tiny dictionary plus a 50-million-entry integer code array. Memory drops from ~3 GB to under 200 MB. The trick is low cardinality — `category` wins when the number of unique values is far less than the number of rows. If unique-count is close to row-count (timestamps, user_ids, hashes) `category` is actively *worse* because you pay the dictionary overhead without compression benefit.

Step-by-step thought process: 1. Explain what `category` actually stores. It is a `CategoricalDtype` backed by two things: a `categories` index (the unique values, stored once) and a `codes` array of integer indices into the categories. Each row holds a small int (often `int8` for under 128 categories) instead of the full string object. 2. Do the memory math out loud. A 50M-row `object`-dtype string column with average length 8 bytes is roughly $50M \times (8 \text{ bytes payload} + 64 \text{ bytes Python-object overhead})$ — easily 3+ GB. The same as `category` with say 30 unique cities is $50M \times 1 \text{ byte (codes)} + 30 \times \sim 80 \text{ bytes (categories index)} \approx 50 \text{ MB}$. Order-of-magnitude memory win. 3. Acknowledge the perf wins beyond memory. `groupby` on a categorical column is faster because the join keys are integers. Sorting can use the categorical order. Pickle/parquet round-trip preserves the category set. 4. Spell out the anti-pattern. If the column is high-cardinality — timestamps, UUIDs, free-text comments — every row is a distinct category, so the codes array is the same width as the original *plus* a categories index the size of the original. Net loss. The rule of thumb: cast when unique-count is under ~10% of row-count.

Edge cases to mention: - Categorical columns can be `ordered=True` for ordinal types (Sat < Sun < Mon won't work, but T-shirt sizes XS < S < M < L will) — gives you comparison operators that respect the order. - Adding a string that is not in the categories raises by default; use

`s.cat.add_categories(["new"])` first or convert back to `object`. - Some operations silently widen `category` back to `object` — string methods like `s.str.upper()` return an `object` Series. Re-cast afterwards if you want the dtype back.

What NOT to say: - "Always use `category` for string columns" — wrong; high-cardinality strings are worse as `category`. - "`category` is just like SQL ENUM" — close, but pandas `category` has richer operations (codes accessor, ordered comparisons) and the categories set can grow.

Common follow-up: "How do you check the memory saving you actually got?" —

`df.memory_usage(deep=True)` — the `deep=True` is critical for object/string columns, otherwise you only see the pointer-array size, not the underlying strings.

```
import pandas as pd, numpy as np

cities = np.random.choice(["Mumbai", "Delhi", "Bengaluru", "Pune", "Chennai"], size=5_000_000)
s_obj = pd.Series(cities)                # object dtype
s_cat = s_obj.astype("category")         # category dtype

s_obj.memory_usage(deep=True)           # ~300 MB
s_cat.memory_usage(deep=True)           # ~5 MB — codes are int8

s_cat.cat.categories                    # the 5 unique strings, stored once
s_cat.cat.codes.head()                 # integer codes into categories
```

Q8 · `convert_dtypes()` — what it infers and when to use it

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India, Walmart Labs · Topic: dtypes

The question (interviewer's verbatim phrasing):

I have a legacy DataFrame with `int64`, `float64`, and `object` columns. I want to upgrade everything to the nullable dtypes in one shot. What's the call?

The 30-second answer: `df.convert_dtypes()` — pandas inspects each column and infers the best nullable dtype: integers become `Int64`, floats with no fractional part become `Int64`, true floats become `Float64`, object columns that look like strings become `string`, booleans-disguised-as-objects become `boolean`. The downstream effect is that any missing values are represented as `pd.NA` instead of `np.nan`, and your integer columns stop silently floating when an NA appears.

Step-by-step thought process: 1. State the call and what it returns — a new DataFrame with inferred nullable dtypes. The original frame is untouched (pandas idiomatic, copy-on-write friendly). 2. Walk through the inference rules. Integer-like → `Int64`. Float-like that fits in an integer → `Int64` if the

keyword `convert_integer=True` (the default). Float-like with fractional values → `Float64`. Object → `string` if all non-null entries are strings, `boolean` if all non-null are bools, else stays `object`. 3. Mention the knobs. `convert_string`, `convert_integer`, `convert_boolean`, `convert_floating` — each is a boolean default `True` that you can flip off if you want to preserve a specific column's original dtype. `dtype_backend="pyarrow"` (added in 2.0) routes inference to Arrow-backed dtypes instead of pandas-native masked dtypes. 4. Land the practical use. You inherit a legacy frame, you run `df = df.convert_dtypes()` once at the top of your notebook, every downstream operation sees `pd.NA` instead of `np.nan`, integer columns stay integer, and your `isna()` checks work uniformly. One call, immediate hygiene win.

Edge cases to mention: - `convert_dtypes` does not change `datetime64[ns]` columns — those are already non-nullable safe with `pd.NaT`. If you want `timestamp[ns][pyarrow]`, pass `dtype_backend="pyarrow"`. - An object column with mixed types (some ints, some strings) stays `object` — `convert_dtypes` only promotes when the inference is unambiguous. - The output is a new DataFrame; original is unchanged. If you have a huge frame and memory is tight, this is a noticeable doubling during the call.

What NOT to say: - "`convert_dtypes` casts everything to `object`" — wrong; it does the opposite, lifting object up to specific nullable types. - "It changes the data" — wrong; values are preserved, only dtypes change (and the missing-value sentinel under the hood).

Common follow-up: "What if I want Arrow-backed types specifically — the `string[pyarrow]` and `int64[pyarrow]` ones?" — `df.convert_dtypes(dtype_backend="pyarrow")`. Or read in with Arrow from the start: `pd.read_csv(..., dtype_backend="pyarrow")`.

```
import pandas as pd, numpy as np

df = pd.DataFrame({
    "id": [1, 2, 3, 4],
    "score": [9.5, 8.0, np.nan, 7.5],
    "city": ["Mumbai", "Delhi", None, "Pune"],
    "active": [True, False, None, True],
})

df.dtypes
# id      int64
# score   float64
# city    object
# active  object

df2 = df.convert_dtypes()
df2.dtypes
# id      Int64
# score   Float64
# city    string
# active  boolean
```

Q9 · The `dtype_backend=` keyword on `read_csv` — what changed in pandas 2.x

Tags: Difficulty: Hard · Asked at: PhonePe, Razorpay, Postman, Goldman, Cred · Topic: dtypes, pandas 2.x

The question (interviewer's verbatim phrasing):

pandas 2.0 added a `dtype_backend` parameter to the read functions. What are the options and what does each one do?

The 30-second answer: Two accepted values: `dtype_backend="numpy_nullable"` and `dtype_backend="pyarrow"`. The legacy default is the absence of the keyword — omit it and you get the pre-2.0 behavior (NumPy arrays — `int64`, `float64`, `object` — with `np.nan` for missing). Passing the keyword explicitly: `"numpy_nullable"` reads into pandas-native masked dtypes — `Int64`, `Float64`, `string`, `boolean` — with `pd.NA` for missing. `"pyarrow"` reads directly into Arrow-backed `pd.ArrowDtype` columns — `int64[pyarrow]`, `string[pyarrow]`, `timestamp[ns][pyarrow]` — with Arrow's null support. (Trying to pass `"numpy"` raises `ValueError` — only the two values above are accepted.) Arrow is fastest for IO, lowest memory for strings, and round-trips cleanly to parquet.

Step-by-step thought process: 1. Anchor the question in pandas-2.0 history. Before 2.0 every read function returned NumPy-backed columns with `np.nan` for missing. 2.0 introduced the keyword to let you opt into the modern dtype universe at the door instead of running `convert_dtypes` after. 2. Walk the values one by one. Omitting the keyword — the legacy compatibility default (the internal `no_default` sentinel), exactly the pre-2.0 behavior with NumPy-backed columns. `"numpy_nullable"` — masked arrays under the hood, `pd.NA`, integer columns stay integer when missing values appear. `"pyarrow"` — Arrow C++ arrays under the hood, exposed via `pd.ArrowDtype`, dtype prints as `int64[pyarrow]`. Note that `"numpy"` is *not* a valid value — pass nothing if you want legacy behavior. 3. Explain why Arrow specifically matters. Strings as Arrow strings use roughly one-tenth the memory of object-dtype strings (no per-row Python-object overhead). Reads from parquet skip the deserialization step entirely. The trade-off is that some pandas operations fall back to a slow path for unfamiliar Arrow types — improving each release but not yet at full coverage. 4. Land the recommendation. New code on a new dataset, no legacy operators relying on specific NumPy dtypes: use `"pyarrow"`. New code that needs to interop with NumPy / scikit-learn / older pandas pipelines: `"numpy_nullable"`. Legacy code that you don't want to risk breaking: omit the keyword entirely (the implicit default keeps the pre-2.0 NumPy behavior) and consider opting in slowly.

Edge cases to mention: - The keyword exists on `read_csv`, `read_parquet`, `read_orc`, `read_feather`, `read_json`, `read_excel` — not on `read_html` or `read_xml` (yet). Check the docs for the specific reader. - You can mix backends: read with `"pyarrow"` for the IO speed, then `astype` specific columns to `numpy_nullable` if you need a pandas-native dtype for a particular operation. Conversions are cheap. - `pd.ArrowDtype("string")` is one way to declare an Arrow-

backed string column. `pd.StringDtype("pyarrow")` is the masked-string variant that stores Arrow strings but uses pandas' masked-array interface. Subtle distinction, mostly visible in error messages.

What NOT to say: - "`dtype_backend` only affects strings" — wrong; it affects every column's dtype.
 - "pyarrow is always faster" — for some compute kernels it is, for others NumPy is. Faster for IO and string memory, neutral-to-slower for some numeric ops. Benchmark your workload.

Common follow-up: "*How does the Arrow backend handle missing values?*" — Arrow stores nulls as a separate validity bitmap, exposed in pandas as `pd.NA`. Behavior matches `numpy_nullable` (Kleene logic, `pd.NA` propagation), the difference is the underlying buffer layout.

```
import pandas as pd, io

csv = "id,name,score\n1,A,9.5\n2,B,\n3,,7.0\n"

# Legacy — object strings, float scores (NaN for missing int)
pd.read_csv(io.StringIO(csv)).dtypes
# id int64, name object, score float64

# Masked nullable
pd.read_csv(io.StringIO(csv), dtype_backend="numpy_nullable").dtypes
# id Int64, name string, score Float64

# Arrow-backed
pd.read_csv(io.StringIO(csv), dtype_backend="pyarrow").dtypes
# id int64[pyarrow], name string[pyarrow], score double[pyarrow]
```

Q10 · `pd.ArrowDtype` — when you'd reach for it explicitly

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Microsoft India, Goldman, Walmart Labs · Topic: dtypes, Arrow

The question (interviewer's verbatim phrasing):

I've seen `pd.ArrowDtype("timestamp[ns]")` in some code. What is it and when would I use it instead of pandas' regular `datetime64[ns]` ?

The 30-second answer: `pd.ArrowDtype` is the pandas wrapper around any Apache Arrow type — `pd.ArrowDtype("string")`, `pd.ArrowDtype("int64")`, `pd.ArrowDtype("timestamp[ns, Asia/Kolkata]")`, `pd.ArrowDtype("list<int64>")`. You use it when you want Arrow's storage and compute (faster IO, lower-memory strings, native nested types like list and struct, true null support) and you're OK with some pandas ops falling back to a slower path. Pick it for parquet-heavy workflows, string-heavy frames, and any pipeline that already lives in the Arrow ecosystem (PySpark Arrow, DuckDB, Polars interop).

Step-by-step thought process: 1. Name what Arrow is so the framing is clear. Apache Arrow is a cross-language in-memory columnar format. pandas 2.x can hold Arrow buffers directly via

`pd.ArrowDtype`, which makes it a thin shell over Arrow rather than copying into NumPy on each read. 2. Walk the three biggest practical wins. (a) Strings: Arrow strings are roughly 10× less memory than `object`-dtype strings on real frames — no per-row Python-object header. (b) IO: reading a parquet into Arrow dtypes is zero-copy; reading into NumPy dtypes involves an extra deserialization pass. (c) Nested types: `list<int64>` and `struct<x:int64, y:string>` are first-class in Arrow but unrepresentable cleanly in NumPy dtypes. 3. Cover the trade-offs honestly. Some pandas operations don't have native Arrow kernels yet — they fall back through NumPy, which can be slower or warn. The coverage gap shrinks each release. For pure numeric arithmetic on dense columns, NumPy backends are still slightly faster in many cases. 4. Land the typical pattern. Read with `dtype_backend="pyarrow"`. Operate on the frame. Convert specific columns to NumPy dtypes (`s.astype("int64")`) if you hit a slow path. Write back to parquet or pass to PyArrow / Polars for downstream consumers.

Edge cases to mention: - The type string syntax follows Arrow's conventions, not NumPy's:

`"timestamp[ns, UTC]"`, not `"datetime64[ns, UTC]"`. Easy mix-up. - `pd.ArrowDtype("list<int64>")` lets you store an actual variable-length list per cell — the object-dtype-of-lists workaround becomes a real typed column with vectorised list operations through the `.list` accessor. - Decimal types — `pd.ArrowDtype("decimal128(precision, scale)")` — give you exact decimal arithmetic, useful for money columns where `Float64` is wrong and Python `Decimal` in `object` columns is slow.

What NOT to say: - "Arrow dtypes are always faster than NumPy" — situation-dependent. Faster for strings and IO, neutral-to-slower for some numeric ops. - "I can just use Polars instead" — Polars is great, but the question is about pandas; `pd.ArrowDtype` gives you Arrow inside pandas without rewriting the codebase.

Common follow-up: "What does the `.list` accessor on an Arrow `list<int>` column give me?"

— Methods like `.list.len()`, `.list.get(i)`, `.list.contains(v)`, `.list.slice(...)`
 — vectorised list operations that the old `object`-dtype-of-lists pattern made you write as `apply(len)` loops.

```
import pandas as pd, pyarrow as pa

s = pd.Series(["Mumbai", "Delhi"], dtype=pd.ArrowDtype("string"))
ts = pd.Series(
    ["2026-05-30 10:00", "2026-05-30 11:00"],
    dtype=pd.ArrowDtype("timestamp[ns, Asia/Kolkata]"),
)
lst = pd.Series([[1, 2, 3], [4, 5]], dtype=pd.ArrowDtype(pa.list_(pa.int64())))

s.memory_usage(deep=True)      # tiny vs object dtype
lst.list.len()                 # [3, 2] – vectorised list ops
```

Section C — Datetime, the `.dt` accessor & Timezones

Datetime questions are where Indian Data interviewers separate "I learnt pandas from a tutorial" from "I have actually wrangled a real CSV." Five questions on `to_datetime`, the `.dt` accessor, timezone localisation vs conversion, DateOffset arithmetic, and `date_range`. The single most common production bug in this area is conflating `tz_localize` and `tz_convert` — one stamps a timezone onto a naive timestamp, the other moves the wall-clock to a new timezone. If you have ever joined a payments table from Mumbai to a logs table from a Frankfurt GCC and seen everything off by 4.5 hours, you have already met this bug.

Q11 · `pd.to_datetime` — `format=`, `errors=`, and the parse speed cliff

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, JP Morgan · Topic: datetime parsing

The question (interviewer's verbatim phrasing):

I'm reading a CSV where the date column is "30-05-2026" in dd-mm-yyyy format. How do I parse it correctly, and what do I do about rows that are malformed?

The 30-second answer: `pd.to_datetime(s, format="%d-%m-%Y")` — passing `format=` explicitly is the difference between sub-second parse on a million rows and a multi-minute slog where pandas guesses per cell. For malformed rows, `errors="coerce"` turns them into `pd.NaT` (Not-a-Time) instead of raising; `errors="raise"` (default) blows up the whole call; `errors="ignore"` returns the original input untouched if anything fails.

Step-by-step thought process: 1. Lead with the format-string fact because it is the single biggest perf win. Without `format=`, pandas falls back to `dateutil`'s general-purpose parser, which inspects each value individually. With `format=`, pandas uses a fast C parser that knows the exact layout. On a million-row column, that is the difference between ~300ms and ~60s. 2. Walk the `errors` keyword. `"raise"` (default) — a single unparseable row halts the entire call. `"coerce"` — unparseable rows become `pd.NaT`, the rest parse cleanly. `"ignore"` — if anything fails, return the input unchanged (rarely what you want, but useful in conditional pipelines). 3. Cover the dayfirst trap. Without `format=`, `to_datetime("01-02-2026")` is ambiguous between Jan 2 and Feb 1. The `dayfirst=True` keyword flips the default; better still, always pass `format=` explicitly so there is no guessing. 4. Mention the cache. `to_datetime(..., cache=True)` (default in modern pandas) deduplicates repeated values before parsing — a huge win on columns where the same dates appear in many rows.

Edge cases to mention: - ISO-8601 input ("2026-05-30T10:00:00") is parsed without `format=` at full speed because pandas has a fast ISO path. The slowdown is for non-ISO formats. - `format="mixed"` (pandas 2.0+) tells the parser "rows may have different formats" and falls back to

per-row inference. Useful for messy human-entered dates; slow. - The output dtype is

`datetime64[ns]` for naive timestamps, `datetime64[ns, tz]` if you also pass `utc=True` or the input has timezone offsets. Mixing in one Series will error unless you pass `utc=True` to normalise.

What NOT to say: - "I'll just let pandas figure it out" — works on toy data, kills you on real data. -

" `errors='ignore'` is the safe default" — it is the *most surprising* default; the column silently stays a string and downstream `.dt` calls fail.

Common follow-up: "What do you do if some rows are dd-mm-yyyy and some are yyyy-mm-dd in the same column?" — Either split the rows into two passes by inspecting the string, or use

`format="mixed"` for the lazy path (slower but works in one call).

```
import pandas as pd

s = pd.Series(["30-05-2026", "01-06-2026", "garbage", "15-06-2026"])

# Fast: explicit format
pd.to_datetime(s, format="%d-%m-%Y", errors="coerce")
# 0    2026-05-30
# 1    2026-06-01
# 2         NaT
# 3    2026-06-15
# dtype: datetime64[ns]

# Slow but flexible: mixed
pd.to_datetime(s, format="mixed", dayfirst=True, errors="coerce")
```

Q12 · The `.dt` accessor — `year`, `month`, `dayofweek`, and friends

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Swiggy · Topic: datetime accessor

The question (interviewer's verbatim phrasing):

I have a `datetime64[ns]` column called `paid_at`. How do I extract the day of week, the month name, and the quarter from it?

The 30-second answer: `s.dt.dayofweek` (0=Monday, 6=Sunday), `s.dt.month_name()` ("January" ... "December"), `s.dt.quarter` (1–4). The `.dt` accessor unlocks every per-element datetime operation on a Series — year, month, day, hour, minute, second, microsecond, nanosecond, date, time, dayofweek, day_name, month_name, quarter, week (deprecated → use `isocalendar().week`), `is_month_start`, `is_month_end`, `is_quarter_start`, `is_quarter_end`, `is_year_start`, `is_year_end`. It's the equivalent of `str` accessor for strings, but for datetimes.

Step-by-step thought process: 1. Name the accessor pattern. `.dt` on a `datetime64`-typed Series gives you vectorised access to the datetime parts. Without `.dt` you'd be calling `apply(lambda x: x.month)` and paying a Python-function-per-row cost. 2. Walk the most-asked attributes. `dt.year`, `dt.month`, `dt.day`, `dt.hour`, `dt.minute`, `dt.second` — integer Series. `dt.dayofweek` / `dt.weekday` — 0 to 6, Monday-origin (catches devs used to Sunday-origin SQL). `dt.day_name()` and `dt.month_name()` — string Series, can pass `locale=` for Hindi or other languages. 3. Cover the underrated boolean attributes. `dt.is_month_end`, `dt.is_quarter_end`, `dt.is_year_end` — booleans, useful for picking out the last day of each period without a `groupby`. Pairs well with `dt.is_month_start` for boundary effects. 4. Land the version note. `dt.week` was deprecated in pandas 1.1 because ISO-week semantics differ between platforms. Use `dt.isocalendar().week` for the ISO 8601 week number — returns a DataFrame with year/week/day columns.

Edge cases to mention: - The accessor is dtype-strict: calling `.dt` on an `object` column (even if the strings look like dates) raises `AttributeError`. Cast to datetime first. - `dt.date` returns Python `date` objects in an object Series — useful for grouping by date, but pandas operations on object dates are slow. Prefer `s.dt.floor("D")` to truncate to midnight while keeping `datetime64[ns]`. - For Period-dtype Series, you use the `.dt` accessor too, but the available attributes are different (`dt.qyear` for fiscal quarters, `dt.start_time`, `dt.end_time`).

What NOT to say: - "I'll write a lambda" — `apply(lambda x: x.month)` is 100x slower than `s.dt.month` on real frames. - "`dayofweek` starts at Sunday" — wrong; pandas (like Python) starts at Monday = 0.

Common follow-up: "How do I get the ISO week number that respects year boundaries — January 1st 2026 is actually ISO week 1 of 2026, but January 1st 2027 is ISO week 53 of 2026." — `s.dt.isocalendar()` returns a DataFrame with `year`, `week`, `day` columns; `.week` alone loses the year context.

```
import pandas as pd
s = pd.to_datetime(pd.Series(["2026-05-30 10:00", "2026-06-01 14:30", "2026-12-31 23:59"]))

s.dt.year          # 2026, 2026, 2026
s.dt.dayofweek     # 5, 0, 3 (Sat, Mon, Thu)
s.dt.day_name()    # "Saturday", "Monday", "Thursday"
s.dt.month_name()  # "May", "June", "December"
s.dt.quarter       # 2, 2, 4
s.dt.is_month_end  # False, False, True
s.dt.isocalendar() # year=2026, week=22/23/53, day=...
```

Q13 · `tz_localize` vs `tz_convert` — the single biggest datetime bug

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, JP Morgan, Goldman · Topic: timezone handling

The question (interviewer's verbatim phrasing):

What's the difference between `tz_localize` and `tz_convert` ? I have a naive timestamp column from a Mumbai server and I need to compare it to a UTC log timestamp.

The 30-second answer: `tz_localize` stamps a timezone onto a *naive* timestamp — it does not change the wall-clock value, it just declares "this clock reading was in Asia/Kolkata." `tz_convert` *changes* the wall-clock to a different timezone — it requires the timestamp to be already tz-aware, and it shifts the displayed time. For the Mumbai-server case: `s.dt.tz_localize("Asia/Kolkata")` declares the column as IST; then `.dt.tz_convert("UTC")` shifts the wall-clock to UTC so you can compare to your UTC log column.

Step-by-step thought process: 1. Open with the rule that prevents 90% of bugs: `tz_localize` is for naive → aware, `tz_convert` is for aware → aware. Calling the wrong one raises a clear `TypeError`, but only if you remember to check the dtype. 2. Walk through the Mumbai example explicitly. The CSV column is `datetime64[ns]` (no timezone). You know the source server was in IST. Step 1: `s = s.dt.tz_localize("Asia/Kolkata")` — now dtype is `datetime64[ns, Asia/Kolkata]`, wall-clock unchanged. Step 2: `s = s.dt.tz_convert("UTC")` — wall-clock shifts back 5h30m, dtype is now `datetime64[ns, UTC]`. Compare to the UTC log column directly. 3. Mention the DST gotchas. `tz_localize` can be called on a timestamp that falls in a DST gap or fold — "2026-03-10 02:30 America/New_York" doesn't exist (spring forward) or exists twice (fall back). Pass `ambiguous=` and `nonexistent=` to control the behavior: `"raise"`, `"NaT"`, `"shift_forward"`, or a boolean array for ambiguous. 4. Land the always-aware recommendation. For any production timestamp column, store as tz-aware UTC and convert to the display timezone only at the very end. The bugs come from mixing naive and aware columns in one calculation — pandas raises now (used to silently compare with a guess), but the error stops your pipeline.

Edge cases to mention: - India does not observe DST, so `tz_localize("Asia/Kolkata")` never raises ambiguous/nonexistent. The cross-timezone gotcha appears the moment you compare to US or European columns, which do have DST. - `tz_localize(None)` strips the timezone off an aware column without shifting the wall-clock — opposite of `tz_localize("X")`. Sometimes useful when writing to a system that can't accept tz-aware timestamps. - Pandas uses the `zoneinfo` stdlib module on Python 3.9+, falling back to `pytz` on older. The behavior is the same for non-DST zones; for DST handling the `zoneinfo` semantics are slightly cleaner (PEP 495 fold handling).

What NOT to say: - "They are the same thing" — wrong; one preserves wall-clock, one shifts it. - "I'll just add 5.5 hours manually" — fragile; doesn't handle DST, doesn't show up in dtype, breaks once the dataset spans countries.

Common follow-up: "What happens if I try to compare a tz-aware column to a tz-naive one?" — Pandas raises `TypeError: Cannot compare tz-naive and tz-aware datetime-like objects`. Localise the naive side first (or strip the aware side with `tz_localize(None)` if you're sure the wall-clock is what you want).

```
import pandas as pd

naive = pd.Series(pd.to_datetime(["2026-05-30 10:00", "2026-05-30 11:00"]))
# dtype: datetime64[ns] - naive

# Step 1: declare it as IST (no clock change)
ist = naive.dt.tz_localize("Asia/Kolkata")
# dtype: datetime64[ns, Asia/Kolkata]

# Step 2: convert to UTC (clock shifts -5:30)
utc = ist.dt.tz_convert("UTC")
# dtype: datetime64[ns, UTC]
# 0 2026-05-30 04:30:00+00:00
# 1 2026-05-30 05:30:00+00:00
```

Q14 · `DateOffset` arithmetic — `BDay`, `MonthEnd`, `pd.Timedelta`

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs, Postman · Topic: datetime arithmetic

The question (interviewer's verbatim phrasing):

I need to add "3 business days" to a payment date, and separately I need the last day of the same month. What pandas tools do I reach for?

The 30-second answer: For business-day arithmetic: `s + pd.offsets.BDay(3)` — skips weekends. For end-of-month: `s + pd.offsets.MonthEnd(0)` — snaps to the same month's last day. For fixed durations (days, hours, minutes): `s + pd.Timedelta(days=3)` — that one ignores weekends and calendar boundaries. The `pd.offsets` family is calendar-aware (knows about months, quarters, weekends, holidays); `pd.Timedelta` is just a fixed duration.

Step-by-step thought process: 1. Anchor the offset-vs-timedelta distinction. Calendar offsets (`MonthEnd`, `MonthBegin`, `QuarterEnd`, `YearEnd`, `BDay`, `Week`, `CustomBusinessDay`) know about the calendar. Timedeltas (`pd.Timedelta(days=N)`) are pure durations — no calendar awareness, no business-day logic. 2. Walk the business-day case. `BDay(3)` advances by 3 business days, Monday–Friday. For holiday-aware arithmetic, use `pd.offsets.CustomBusinessDay(holidays=[...])` or `pd.offsets.BusinessDay(calendar=...)`. NSE-trading-day calendars usually live in a separate package like `pandas-market-calendars`. 3. Walk the end-of-month case. `s + MonthEnd(0)` snaps each timestamp to the last day of *its own* month (the `0` means "this month's"). `MonthEnd(1)` rolls to next month's end, `MonthEnd(-1)` to last month's end. Similar logic for `MonthBegin`, `QuarterEnd`, `YearBegin`. 4. Land with the gotcha. Adding `MonthEnd(0)` to a timestamp that is

already the end of the month is a no-op — it stays. But `MonthEnd(1)` always advances to the next month's end. The `0` vs `1` semantics catch every team at least once.

Edge cases to mention: - `pd.Timedelta(days=30)` is not "one month" — for actual month arithmetic use `pd.offsets.MonthEnd` or `DateOffset(months=1)`. Adding

`Timedelta(days=30)` to January 31 lands on March 2, not February 28. -

`DateOffset(months=1)` is the general-purpose calendar offset for "add one month" — preserves the day-of-month where possible (Jan 31 + 1 month = Feb 28 or 29). - Offsets are vectorised — `s + BDay(3)` works on the entire Series in C, faster than a per-row `apply`.

What NOT to say: - "`pd.Timedelta(days=30)` is a month" — wrong; months are variable-length, timedeltas are fixed. - "I'll write a function with `if weekday in (5, 6)`" — fragile; `BDay` is built for exactly this.

Common follow-up: "How would you compute the trading day before a given date, skipping NSE holidays?" — Build a `CustomBusinessDay(holidays=nse_holiday_list)` and subtract it: `s - cbd`. Or use `pandas-market-calendars` which ships NSE / NYSE / BSE calendars.

```
import pandas as pd

s = pd.to_datetime(pd.Series(["2026-05-29", "2026-05-30", "2026-05-31"])) # Fri, Sat, Sun

s + pd.offsets.BDay(3)          # next 3 business days
# 2026-06-03, 2026-06-04, 2026-06-03

s + pd.offsets.MonthEnd(0)     # snap to month-end
# 2026-05-31 across the board

s + pd.Timedelta(days=30)      # fixed 30 days — calendar-blind
# 2026-06-28, 2026-06-29, 2026-06-30
```

Q15 · `pd.date_range` — generating timestamp sequences

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, PhonePe · Topic: datetime ranges

The question (interviewer's verbatim phrasing):

I need a Series of every day in May 2026, then separately every business hour between 9am and 5pm on a given Tuesday. How do you generate those?

The 30-second answer: `pd.date_range(start="2026-05-01", end="2026-05-31", freq="D")` for every day. `pd.date_range(start="2026-05-26 09:00", end="2026-05-26 17:00", freq="H")` for hourly within a day. The `freq` keyword takes a string code or an offset

object — **D** (calendar day), **B** (business day), **H** (hour), **T** or **min** (minute), **S** (second), **W** (week, defaulting to Sunday end), **MS** / **ME** (month start / end), **Q** / **Y** (quarter, year).

Step-by-step thought process: 1. Walk the three-arg pattern. `start=` + `end=` + `freq=` produces every timestamp from start to end inclusive at the given frequency. Or use `start=` + `periods=N` + `freq=` to produce exactly N timestamps without computing the end. `end=` + `periods=` works backwards. 2. Cover the frequency string codes. **D** calendar day, **B** business day, **H** hour, **min** minute, **S** second, **ms** millisecond, **us** microsecond, **ns** nanosecond. Compound codes: **2H** (every 2 hours), **15min** (every 15 minutes), **3D** (every 3 days). Anchored: **W-MON** (every Monday), **Q-DEC** (quarter ending December). 3. Mention the `tz` keyword. `pd.date_range(start=..., end=..., freq="D", tz="Asia/Kolkata")` produces tz-aware timestamps directly. Avoids the `localize` step. 4. Land the small-but-real version note. In pandas 2.2, **"H"**, **"T"**, **"S"**, **"L"**, **"U"**, **"N"** were deprecated in favour of **"h"**, **"min"**, **"s"**, **"ms"**, **"us"**, **"ns"** (lowercase, no **T** collision with date format). Code that uses uppercase still works but warns; new code should use the lowercase forms.

Edge cases to mention: - `inclusive=` keyword controls boundary inclusion: **"both"** (default), **"left"**, **"right"**, **"neither"**. Useful when you have a half-open interval. - `date_range` returns a `DatetimeIndex`, not a `Series`. Wrap in `pd.Series(...)` if you want a `Series` with that index as values. - `pd.bdate_range` is a shortcut for `date_range(..., freq="B")` and accepts a `holidays=` keyword directly.

What NOT to say: - "I'll loop with `datetime.timedelta`" — slow, error-prone, doesn't handle business days or month ends. - "`freq='M'` means month" — ambiguous; **M** was month-end, now use **ME** (month end) or **MS** (month start) explicitly in 2.2+.

Common follow-up: "How do I generate every Wednesday in 2026 for a weekly cron simulation?" — `pd.date_range(start="2026-01-01", end="2026-12-31", freq="W-WED")` — the **W-WED** anchored frequency snaps to Wednesdays.

```
import pandas as pd

pd.date_range("2026-05-01", "2026-05-31", freq="D")
# DatetimeIndex(['2026-05-01', '2026-05-02', ..., '2026-05-31'], dtype='datetime64[ns]', freq='D')

pd.date_range("2026-05-26 09:00", "2026-05-26 17:00", freq="h")
# 9 timestamps, hourly

pd.date_range("2026-05-01", periods=5, freq="B", tz="Asia/Kolkata")
# 5 business days starting May 1, IST-aware

pd.date_range("2026-01-01", "2026-12-31", freq="W-WED")
# every Wednesday in 2026
```

Section D — Real-World Wrangling Reflexes

The final five questions cover the messy reality every Data candidate hits in the round-two case-study interview: parsing a CSV that has the wrong encoding and a semicolon separator; the "mixed dtypes in a column" warning that everyone has seen and few can explain; deduplication when "same" doesn't mean "byte-identical"; coercing a string column with currency symbols and commas to a number; and the split-by-delimiter-into-N-columns move that closes out the chapter. None of these are conceptually hard — they're all things you have probably done 50 times — but in an interview they get asked because they reveal whether you've shipped pandas or only practiced it.

Q16 · `read_csv` — encoding, separator, header row, and the obvious-but-missed keywords

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan · Topic: CSV parsing

The question (interviewer's verbatim phrasing):

I'm reading a CSV that uses semicolons as separators, has a UTF-8 BOM, and the actual header row is the third line — the first two are metadata. How do you read it cleanly in one call?

The 30-second answer: `pd.read_csv(path, sep=";", encoding="utf-8-sig", skiprows=2)` — `sep=";"` for the semicolon separator, `encoding="utf-8-sig"` to strip the BOM (UTF-8 with signature), `skiprows=2` to skip the two metadata lines so the third becomes the header. The full toolbox of `read_csv` keywords is the most-asked area of the function: `sep`, `encoding`, `skiprows`, `header`, `names`, `usecols`, `dtype`, `parse_dates`, `na_values`, `thousands`, `decimal`, `quoting`, `engine`.

Step-by-step thought process: 1. Walk the three immediate fixes for the question. `sep=";"` because the default is comma; `encoding="utf-8-sig"` because plain "utf-8" reads the BOM as a visible character in the first column; `skiprows=2` to skip metadata rows. 2. Cover the related keywords that come up next. `header=N` to say "treat row N as the header" (with `skiprows` you can usually leave header at default 0 after the skip). `names=["a", "b"]` to override the header entirely with your own column names. `usecols=["a", "c"]` to read only specific columns — major memory win on wide CSVs. 3. Mention the dtype/parse_dates pair. `dtype={"id": "Int64"}` to lock specific columns to specific dtypes (avoids the int-becomes-float problem on missing values). `parse_dates=["paid_at"]` to parse date columns at read time instead of after. Both are cheap to add and prevent post-read cleanups. 4. Land the `na_values` and `thousands` keywords. `na_values=["N/A", "-", "null", ""]` tells the reader which string tokens to treat as missing. `thousands=","` for "1,000,000" → 1000000. `decimal=","` for European-style "3,14" → 3.14. These are the cleanup steps that beginners do post-read; pros do them in the read call.

Edge cases to mention: - The `c` engine is fast but stricter; the `python` engine is slow but supports more delimiter patterns including regex. If your separator is a multi-character or regex pattern, pandas auto-falls-back to the python engine and warns. - For really big CSVs, use `chunksize=` to get a `TextFileReader` iterator and process in chunks — avoids loading everything into memory. - `encoding_errors="replace"` lets you survive a malformed-byte file by substituting `?` for the bad characters. Last resort, not best practice.

What NOT to say: - "I'll open the file in a text editor first and clean it up" — fine for a one-off, not how you describe a pipeline. - "`read_csv` defaults are usually fine" — for a clean, comma-delimited, UTF-8, header-on-row-1 CSV, sure. For anything else, you're naming keywords.

Common follow-up: "What if the file is so big it doesn't fit in RAM?" — `pd.read_csv(path, chunksize=100_000)` returns an iterator of DataFrames, each 100k rows. Process and aggregate as you go. Or read with `dtype_backend="pyarrow"` to halve the string memory.

```
import pandas as pd, io

raw = "header line 1\nheader line 2\nid;name;paid_at\n1;Anita;30-05-2026\n2;Bose;31-05-2026\n"

df = pd.read_csv(
    io.StringIO(raw),
    sep=";",
    encoding="utf-8-sig",
    skiprows=2,
    parse_dates=["paid_at"],
    dayfirst=True,
    dtype={"id": "Int64", "name": "string"},
)
df.dtypes
# id          Int64
# name        string
# paid_at     datetime64[ns]
```

Q17 · The "mixed dtypes in column N" warning — what causes it, how to fix

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Postman, Cred · Topic: dtype inference

The question (interviewer's verbatim phrasing):

I keep seeing `DtypeWarning: Columns (3) have mixed types. Specify dtype option on import or set low_memory=False.` when I read a CSV. What's actually happening and what's the right fix?

The 30-second answer: The `c` parser reads CSVs in chunks (default ~10 MB) and infers each chunk's dtype independently. If column 3 looks integer in chunk 1 but has a string `"N/A"` in chunk 5, pandas falls back to `object` for that column overall — and warns because the inference was inconsistent across chunks. Right fix: pass `dtype={"col_3": "string"}` (or whatever the column actually is) at read time. Lazy fix: `low_memory=False` reads the whole file before inferring, so the inference is consistent but you lose the memory-saving chunked read.

Step-by-step thought process: 1. Explain the mechanism. The default `c` engine reads in chunks for memory efficiency. Each chunk gets its own dtype inferred. If the inferences disagree across chunks, pandas downgrades the column to `object` to hold everything and warns. 2. Walk the two scenarios that produce the warning. Scenario A: real mixed data — `1`, `2`, `"N/A"`, `4` — the column genuinely contains both integers and strings. Scenario B: type-pure but late-appearing values — column starts with `1`, `2`, `3` for 10 million rows then has `"N/A"` on row 10,000,001 — the early chunks inferred `int64`, the late chunk had to be `object`. 3. Give the right fixes in order. (a) Pass `dtype={"col_name": "appropriate_type"}` — best, most explicit. (b) Pass `na_values=["N/A"]` so the offending tokens are parsed as missing and the column stays numeric. (c) `low_memory=False` reads the entire column before inferring — slower memory profile but consistent. 4. Land the modern alternative. `dtype_backend="pyarrow"` or `dtype_backend="numpy_nullable"` infer dtypes across the whole column (not per chunk) by design, so the warning doesn't appear. Migrating to nullable dtypes also fixes the underlying "missing values force float" problem.

Edge cases to mention: - The warning includes the column number, not name — you have to look up your column order. `usecols=` and explicit `names=` make this less painful. - Even with `low_memory=False` and `dtype=`, a genuinely mixed column (real strings and real ints) will land as `object`. That is correct — pandas can't lie about the dtype. - The warning is a `DtypeWarning`, not a regular `UserWarning`. You can filter it specifically with `warnings.filterwarnings("ignore", category=pd.errors.DtypeWarning)` if you've audited and don't want it in your logs.

What NOT to say: - "Just set `low_memory=False`" — works but hides the symptom; better to fix the column dtype explicitly. - "It's a pandas bug" — it's not; it's an intentional trade-off between memory and dtype consistency.

Common follow-up: "How do you figure out which exact rows are causing the type conflict?" — Read the column as `object`, then `s[pd.to_numeric(s, errors='coerce').isna() & s.notna()]` — that filter keeps rows that are non-null but didn't parse as numeric, which is your problem set.

```
import pandas as pd, io

csv = "id,name,score\n" + "1,A,9.5\n" * 10 + "11,B,unknown\n"

# Triggers DtypeWarning on a large enough file
df = pd.read_csv(io.StringIO(csv))

# Right fix: explicit dtype + na_values
df = pd.read_csv(
    io.StringIO(csv),
    dtype={"score": "string"},          # or treat as nullable Float64 + na_values
)

# Modern fix: nullable backend infers across the whole column
df = pd.read_csv(io.StringIO(csv), dtype_backend="numpy_nullable",
                 na_values=["unknown"])
df.dtypes # score: Float64
```

Q18 · `drop_duplicates` — `subset=`, `keep=`, and the "fuzzy duplicate" caveat

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Meesho, Walmart Labs · Topic: deduplication

The question (interviewer's verbatim phrasing):

I have a user-events table where the same event got logged twice for many users. How do I keep just the latest event per user-event-type?

The 30-second answer:

`df.sort_values("timestamp").drop_duplicates(subset=["user_id", "event_type"], keep="last")`. `subset=` defines which columns make a "duplicate" — without it, every column has to match exactly, which almost never finds the duplicates you care about. `keep=` says which copy to retain: `"first"` (default), `"last"`, or `False` (drop all duplicates entirely). The `sort_values` before is mandatory if "latest" matters — `drop_duplicates` keeps rows by row order, not by any timestamp logic on its own.

Step-by-step thought process: 1. Anchor on `subset=`. Without it, two rows must be byte-identical across every column to be flagged as duplicates. In real data that almost never holds — the rows differ in a `created_at` timestamp or an autoincrement id. You must specify which columns define duplicate. 2. Walk through `keep=`. `"first"` keeps the first occurrence in row order. `"last"` keeps the last occurrence in row order. `False` drops every row that has any duplicate (keeps only the rows that were already unique). Each has a real use case. 3. Land the sort-first pattern. `drop_duplicates` looks at row order, not at any "latest by timestamp" intelligence. If you want "keep latest by created_at," sort by

created_at first, then `keep="last"`. If you want "keep earliest," sort then `keep="first"`. Forgetting to sort is the most common bug in this area. 4. Mention `duplicated()` for the diagnostic version. `df.duplicated(subset=["user_id"], keep=False)` returns a boolean Series marking every row that has a duplicate. Useful for inspecting *what* the duplicates are before deciding which to drop.

Edge cases to mention: - `drop_duplicates` does not detect fuzzy duplicates — "Anita Sharma" and "anita sharma" are different rows. You have to normalise first (lowercase, trim, strip punctuation) and then dedupe on the normalised column. - For DataFrames, the index is preserved on the kept rows — call `.reset_index(drop=True)` afterwards if you want a clean 0..N index. - `pd.Series.unique()` is a related but different operation — returns unique values from a Series as a numpy array (or pandas array for extension dtypes), not a DataFrame.

What NOT to say: - "`drop_duplicates()` removes duplicate rows" — only true if you mean byte-identical-across-every-column, which is rarely what's asked. - "I'll write a `groupby().first()`" — works but is more verbose and slower for this specific case; `drop_duplicates` is the idiomatic answer.

Common follow-up: "What about duplicates where one row has slightly different capitalisation in the name?" — Pre-normalise: `df["name_norm"] = df["name"].str.lower().str.strip()`, then `drop_duplicates(subset=["name_norm", ...])`.

```
import pandas as pd

df = pd.DataFrame({
    "user_id": [1, 1, 2, 2, 2, 3],
    "event": ["login", "login", "buy", "buy", "buy", "logout"],
    "timestamp": pd.to_datetime([
        "2026-05-30 10:00", "2026-05-30 10:00",
        "2026-05-30 11:00", "2026-05-30 11:30", "2026-05-30 12:00",
        "2026-05-30 13:00",
    ]),
})

# Latest event per user/event_type
(df.sort_values("timestamp")
 .drop_duplicates(subset=["user_id", "event"], keep="last")
 .reset_index(drop=True))

# Diagnostic: which rows have duplicates
df.duplicated(subset=["user_id", "event"], keep=False)
```

Q19 · `pd.to_numeric` with `errors="coerce"` — string-to-number cleanup

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan · Topic: type coercion

The question (interviewer's verbatim phrasing):

I have a column with strings like "1,500", "₹2,000", "N/A", "—", and "3500". I want to turn it into numbers, with the unparseable rows as NaN. What's the call?

The 30-second answer: Clean the strings first, then `pd.to_numeric(..., errors="coerce")`. The cleanup is `s.str.replace("₹", "", regex=False).str.replace(",", "", regex=False)`. Then `pd.to_numeric(cleaned, errors="coerce")` returns a `float64` Series with `NaN` where the parse failed (the "N/A" and "—" rows). Errors options: `"raise"` (default, halts on first failure), `"coerce"` (failures become NaN), `"ignore"` (returns the input unchanged if any failure).

Step-by-step thought process: 1. Acknowledge the two-step pattern. Strings with currency symbols, thousands separators, or units do not parse directly with `to_numeric`. You strip the noise first (with `str.replace` or a regex), then call the numeric coercion. 2. Walk the cleanup string ops. `str.replace("₹", "", regex=False)` removes the rupee symbol. `str.replace(",", "", regex=False)` removes thousands separators. `str.strip()` removes leading/trailing whitespace. Order doesn't matter — string methods are chainable on the Series. 3. Land `pd.to_numeric` with the three error modes. `errors="raise"` (default) is fine when you're sure the column is clean. `errors="coerce"` is the production-safe mode — failures become NaN, the call always succeeds, you then audit how many NaNs came out. `errors="ignore"` is the silent-trap option that returns the original strings if anything fails, which downstream operations will then fail on confusingly. 4. Mention `downcast=`. `pd.to_numeric(s, downcast="integer")` will cast to the smallest integer dtype that fits (`int8` if the values are tiny, `int32` if they fit, `int64` if they don't). Memory win on big frames.

Edge cases to mention: - The Indian thousands convention "1,00,000" (lakhs) has commas in non-Western positions. Standard `to_numeric` after a plain `str.replace(",", "", regex=False)` handles it because both Western and Indian commas are just commas to strip. - `to_numeric` returns `float64` even if every value parses as integer — because NaN forces float. If you want integers, follow up with `.astype("Int64")` (capital I, nullable) to keep `pd.NA` where NaN was. - For Boolean parsing — "Yes"/"No", "True"/"False" — use `s.map({"Yes": True, "No": False})` instead of `to_numeric`. There's no `to_boolean`.

What NOT to say: - "`int(s)` would work" — wrong for Series and wrong for strings with symbols. - "`errors='ignore'` is safest" — most surprising option; the column stays string-typed and downstream `mean()` calls fail.

Common follow-up: "How do you find which rows failed the parse?" — `s[~pd.to_numeric(s_cleaned, errors='coerce').isna() & s.notna()]` — that's the rows that are non-null in the original but NaN in the parse result. Inspect those to decide whether they're junk or need a different cleanup rule.

```
import pandas as pd

s = pd.Series(["1,500", "₹2,000", "N/A", "-", "3500", "1,00,000"])

cleaned = (s.str
            .replace("₹", "", regex=False)
            .str.replace(",", "", regex=False)
            .str.strip())
nums = pd.to_numeric(cleaned, errors="coerce")
# 0    1500.0
# 1    2000.0
# 2         NaN
# 3         NaN
# 4     3500.0
# 5  100000.0
# dtype: float64

# Keep as nullable Int64 instead of floating
nums.astype("Int64")
```

Q20 · Splitting a column by delimiter into multiple columns

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, PhonePe, Walmart Labs · Topic: string ops, wrangling

The question (interviewer's verbatim phrasing):

I have a "full_name" column with values like "Anita Sharma" and a "city_state" column like "Mumbai, Maharashtra". I want to split each into two new columns. What's the cleanest pandas idiom?

The 30-second answer: `df[["first_name", "last_name"]] = df["full_name"].str.split(" ", n=1, expand=True)`. The `expand=True` keyword is the critical one — it makes `str.split` return a DataFrame (one column per split position) instead of a Series-of-lists. `n=1` caps the number of splits, so "Anita Sharma Devi" splits once into ["Anita", "Sharma Devi"], not three times. Assigning the result to a list of column names creates them in one operation.

Step-by-step thought process: 1. Walk the `expand` decision. `s.str.split(" ")` returns a Series of lists — every value is a Python `list`, dtype is `object`. Useful for further list ops, painful for two-column extraction. `s.str.split(" ", expand=True)` returns a DataFrame — each split position becomes a column. That's what you want here. 2. Cover the `n=` cap. Without `n=`, every space is a delimiter — "Anita Sharma Devi" becomes ["Anita", "Sharma", "Devi"] and you get three columns instead of two. With `n=1`, the first space is the split point and everything after is one piece —

["Anita", "Sharma Devi"]. For surnames-with-spaces (Indian and Spanish names, hyphenated names), `n=1` is the safer default. 3. Land the assignment idiom. `df[["a", "b"]] = ...` on the left side of an assignment creates both columns in one operation. Cleaner than two separate assignments. The right-hand side must have exactly two columns to match. 4. Mention `rsplit` for splitting from the right. "Anita Sharma Devi".rsplit(" ", n=1) gives ["Anita Sharma", "Devi"] — useful when the *last* token is the meaningful one (the surname in some conventions).

Edge cases to mention: - If some rows have fewer splits than expected (a row with only "Anita" and no surname), the missing positions become NaN in the expanded DataFrame. Handle with `fillna("")` or `dropna()`. - For regex delimiters, pass `regex=True`: `s.str.split(r"\s+|\s*", expand=True, regex=True)` splits on either whitespace or comma-with-optional-space. Slower than literal splitting but flexible. - `s.str.extract(r"^(?P<first>\S+)\s+(?P<middle>\S+)\s+(?P<last>\S+)$", expand=True)` is the regex-based equivalent — extracts named or positional capture groups into columns. Often cleaner than `split` when the structure is more than just a delimiter.

What NOT to say: - "I'll write an `apply(lambda x: x.split(' '))`" — works but ~50x slower than the vectorised `str.split`. - "`split` automatically creates columns" — only with `expand=True`; the default is Series-of-lists.

Common follow-up: "What if the split position varies — some rows have 'first last', some have 'first middle last'?" — Use `expand=True` with no `n=` cap, get all positions as columns, then concatenate the middle ones back with `fillna("")` and string joining. Or use `str.extract` with a regex that captures named groups: `^(?P<first>\S+)(?:\s+(?P<middle>\S+))?\s+(?P<last>\S+)$`.

```
import pandas as pd

df = pd.DataFrame({
    "full_name": ["Anita Sharma", "Bose", "Chandrasekhar Iyer Devi"],
    "city_state": ["Mumbai, Maharashtra", "Delhi, NCR", "Pune, Maharashtra"],
})

df[["first_name", "last_name"]] = df["full_name"].str.split(" ", n=1, expand=True)
df[["city", "state"]] = df["city_state"].str.split(", ", n=1, expand=True)

df
#           full_name      city_state      first_name      last_name      city
# 0      Anita Sharma  Mumbai, Maharashtra      Anita      Sharma  Mumbai
# 1              Bose      Delhi, NCR          Bose          NaN      Delhi
# 2  Chandrasekhar Iyer Devi  Pune, Maharashtra  Chandrasekhar      Iyer Devi  Pune

# Regex variant — splits on comma OR whitespace
df["city_state"].str.split(r",\s*", expand=True, regex=True)
```

End of Chapter 7

Twenty questions covering the four wrangling surfaces — five each on missing data, dtypes, datetime, and the real-world reflexes. If you have walked through these, the Data round-one "do you actually know pandas" probe should land squarely in your hitting zone. The next chapter ("Here's a CSV" scenarios) takes these primitives and chains them into the multi-step case-study problems that round-two interviewers like to use. Each scenario assumes you can already reach for `dropna(subset=...)`, `to_datetime(format=...)`, `convert_dtypes()`, and the `dtype_backend="pyarrow"` keyword without thinking — which is what this chapter was for.

Chapter 8 — "Here's a CSV" Scenarios (10 multi-step Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 06 Data Track Library

Round 2 / round 3 of every Data interview in India ends with the same line. The interviewer pastes a Google Drive link into the chat, says *"open this CSV, give me ten minutes to grab a coffee, then walk me through what you found"* — and the screen-share clock starts ticking. Flipkart's analytics panels do it. Swiggy's data-science loop does it. Razorpay's growth team has a whole rubric built around it. PhonePe, Zomato, Cred, Meesho, Postman, Walmart Labs, Target's GCC — all of them. The CSV is small enough to read into memory, dirty enough to embarrass a careless `read_csv`, and the prompt is open-ended enough that the *shape of your answer* matters more than the answer itself.

Each scenario below is a complete walk-through: the CSV / data description, the interviewer's exact prompt, a **30-second outline** (what you say while the data loads), a **step-by-step solution** that mirrors the live narration, the **working pandas 2.x code**, the **gotchas** that bite people (NaN, dtypes, timezones, edge cases), and a **stand-out line** — the sentence that signals "this candidate has actually shipped data pipelines, not just survived a course."

These ten scenarios cover the full round-2 / round-3 surface: top-N per group, cohort retention, funnels, time-series resampling, sessionization, slow-changing dimensions, anti-joins, pivot summaries, outlier detection, and multi-source reconciliation. If you can narrate any one of these without freezing, you are already past the median candidate.

Table of Scenarios

- Scenario 1 — Top-3 customers per region by total spend (groupby + nlargest, tie-breaking)
- Scenario 2 — Month-over-month cohort retention from signups + events
- Scenario 3 — Multi-step user funnel from event logs (filter + groupby + percentage)
- Scenario 4 — Daily revenue from a transaction log with missing days filled as zero

- Scenario 5 — Sessionize events by 30-minute gap (`diff` + `cumsum`)
- Scenario 6 — Slow-changing dimension: build historical price-per-product (SCD type 2)
- Scenario 7 — Anti-join: customers who haven't ordered in the last 90 days (`merge indicator=True`)
- Scenario 8 — Pivot then summarize: sales by product × month, then top contributors
- Scenario 9 — Outlier detection on transaction amounts per customer segment (IQR method)
- Scenario 10 — Reconcile order data from two systems with different schemas (key mapping)

Scenario 1 — Top-3 Customers Per Region By Total Spend (Last 90 Days)

Asked at: Flipkart, Myntra, Nykaa, Meesho · **Difficulty:** Medium · **Topic:** groupby + nlargest, tie-breaking, date filtering

The data

`orders.csv` — ~2 million rows, one row per order. Columns:

Column	Type	Notes
<code>order_id</code>	string	UUID, primary key
<code>customer_id</code>	string	foreign key to customers (not in this CSV)
<code>region</code>	string	one of <code>North</code> , <code>South</code> , <code>East</code> , <code>West</code> , <code>Central</code>
<code>product</code>	string	product name
<code>amount</code>	float	order total in INR, can be NaN for cancelled orders
<code>order_date</code>	string	ISO format <code>YYYY-MM-DD HH:MM:SS</code> , IST
<code>status</code>	string	<code>completed</code> , <code>cancelled</code> , <code>refunded</code> , <code>pending</code>

The interviewer's prompt

"Find the top 3 customers by total spend per region, but only for completed orders in the last 90 days. Return a clean DataFrame with region, customer_id, total_spend, and rank — sorted by region then rank. How do you handle ties at rank 3?"

30-second answer outline

"I'd parse `order_date` as a datetime up-front, filter to `status == 'completed'` and the last-90-day window measured from `max(order_date)` not `today()`, drop NaN amounts, then groupby `(region, customer_id)`, sum the amount, and use `groupby('region').apply` with `nlargest(3, 'total_spend')` per region. For ties at rank 3, I'd use `rank(method='dense')` so tied customers share the rank, then filter `rank <= 3` — that way three customers at the same spend don't silently drop one of them."

Step-by-step walkthrough

1. "First the parse — I'd read with `parse_dates=['order_date']` and check the dtype. CSV dates default to object dtype, which then makes every comparison a string compare. Easy first slip."
2. "Window definition — 'last 90 days' from when? Almost always the answer is `max(order_date)` in the data, not `pd.Timestamp.now()`, because the CSV was extracted at some point in the past and `now()` would silently exclude valid orders. I'd ask the interviewer but I'd state my assumption clearly."
3. "Filter: `status == 'completed'` AND `order_date >= cutoff`. The `amount.notna()` check is implicit — sum of NaN with `skipna=True` is fine but I'd be explicit."
4. "Aggregate: `groupby(['region', 'customer_id'])['amount'].sum()` gives total spend per customer per region. Note: if a customer ordered in two regions, they show up twice — usually that's correct (regional view), but I'd flag it."
5. "Rank: I'd use `groupby('region')['total_spend'].rank(method='dense', ascending=False)` so ties share the rank, then filter `rank <= 3`. Pure `nlargest(3)` would drop tied rows arbitrarily."
6. "Final sort: `sort_values(['region', 'rank'])` and `reset_index`. Hand it over with the column order they asked for."

Working pandas code

```
import pandas as pd

# Parse dates at read time – never as a second step
orders = pd.read_csv(
    "orders.csv",
    parse_dates=["order_date"],
    dtype={"amount": "float64"},
)

# Window from the data's own clock, not wall-clock
max_date = orders["order_date"].max()
cutoff = max_date - pd.Timedelta(days=90)

# Filter: completed + in-window + non-null amount
filtered = orders[
    (orders["status"] == "completed")
    & (orders["order_date"] >= cutoff)
    & orders["amount"].notna()
]

# Aggregate per (region, customer)
totals = (
    filtered.groupby(["region", "customer_id"], as_index=False)["amount"]
    .sum()
    .rename(columns={"amount": "total_spend"})
)

# Dense rank within region – ties share the rank
totals["rank"] = (
    totals.groupby("region")["total_spend"]
    .rank(method="dense", ascending=False)
    .astype("int64")
)

top_3 = (
    totals[totals["rank"] <= 3]
    .sort_values(["region", "rank", "customer_id"])
    .reset_index(drop=True)
)

print(top_3.head(20))
```

Gotchas

- **parse_dates not used:** if you call `pd.to_datetime` after `read_csv`, you've already spent a full pass over the column. Slow on 2M rows, and you risk an inferred dtype mismatch on rows that look like dates but aren't.

- **today() for the window:** a CSV is a snapshot. Anchoring on `pd.Timestamp.today()` excludes valid orders if the CSV is even a day old. Anchor on `max(order_date)` and say so out loud.
- **nlargest(3) silently breaking ties:** if rank 3 and rank 4 have the same spend, `nlargest` keeps one and drops the other. The interviewer asked specifically about ties — this is the bait.
- **NaN amounts:** cancelled orders have NaN amount. `sum()` ignores NaN by default (`skipna=True`), but if you ever switch to `mean()` or `agg('sum', min_count=1)`, the behaviour shifts. Be explicit.
- **Customer in two regions:** if a customer ordered in both North and South, they appear in both region rankings. That's almost always desired (it's a regional view), but flag it.
- **Timezone:** `order_date` in IST stored as naive datetime is the most common India pattern. If you mix naive and aware datetimes, comparisons raise.

Stand-out line

Say this and you'll separate yourself: "I'd use `rank(method='dense')` rather than `nlargest(3)` so ties at rank 3 don't silently drop a customer — and I'd anchor the 90-day window on `max(order_date)` in the data, not `today()`, because a CSV is a snapshot and `today()` would shrink the window every day the question is re-run."

Scenario 2 — Month-Over-Month Cohort Retention

Asked at: Swiggy, Zomato, Dunzo, Cred, PhonePe · **Difficulty:** Hard · **Topic:** cohort analysis, period arithmetic, pivot, retention matrix

The data

Two CSVs.

`signups.csv` — one row per user:

Column	Type	Notes
<code>user_id</code>	string	UUID
<code>signup_date</code>	string	<code>YYYY-MM-DD</code> , IST
<code>acquisition_channel</code>	string	<code>paid</code> , <code>organic</code> , <code>referral</code>

`events.csv` — one row per active session:

Column	Type	Notes
<code>user_id</code>	string	foreign key to signups
<code>event_date</code>	string	<code>YYYY-MM-DD</code> , IST
<code>event_type</code>	string	<code>app_open</code> , <code>order_placed</code> , <code>payment</code>

The interviewer's prompt

"Build a month-over-month retention cohort table. Rows are signup-month cohorts, columns are months since signup (0, 1, 2, 3, ...), cells are the percentage of the cohort that had at least one `app_open` event in that month. Then tell me which cohort is leaking fastest and what you'd investigate next."

30-second answer outline

"Two-step join then pivot. First, tag each user with their signup month from `signups.csv` . Then take `events.csv` , filter to `app_open` , compute the event's month, and merge in the signup month per user. Compute `period_diff = event_month - signup_month` in months — that's the column index. Then groupby `(signup_month, period_diff)` , count distinct users, pivot to wide, and divide each row by the cohort's month-0 size to get retention percentages. The leak-finder is just `cohort_table.diff(axis=1)` to see which month each cohort drops the steepest."

Step-by-step walkthrough

1. "Read both CSVs with `parse_dates` . I'd use `dtype={'user_id': 'string'}` because `pd.merge` with object string keys is slower than with the new pyarrow-backed string dtype."
2. "Compute the signup month as a `Period('M')` — `signup_date.dt.to_period('M')` . Period arithmetic is cleaner than mixing datetimes and timedeltas for month-grain work."
3. "Filter events to `app_open` only — the interviewer specified that. Compute event_month the same way."
4. "Merge events ← signups on `user_id` with `how='inner', validate='m:1'` . The `validate='m:1'` is a contract: many events per user, exactly one signup per user. If `signups.csv` has duplicates, this raises immediately — much better than silently producing 2x rows."

5. "`period_diff = (event_month - signup_month).apply(lambda x: x.n) - n` extracts the integer month-offset from a `Period` difference."
6. "Groupby `(signup_month, period_diff)['user_id'].nunique()` gives the active-user count per cell. Pivot to wide via `pivot_table(index='signup_month', columns='period_diff', values='user_id')`."
7. "Divide every row by column 0 (the cohort's initial size) to get retention. Multiply by 100. Round to one decimal."
8. "Leak-finder: `retention.diff(axis=1)` shows month-on-month change per cohort. The cohort with the most negative value in months 1 or 2 is the steepest leaker."

Working pandas code

```
import pandas as pd

signups = pd.read_csv("signups.csv", parse_dates=["signup_date"])
events = pd.read_csv("events.csv", parse_dates=["event_date"])

# Period('M') is the cleanest month-grain primitive
signups["signup_month"] = signups["signup_date"].dt.to_period("M")
events["event_month"] = events["event_date"].dt.to_period("M")

# Filter to the retention signal – app_open
app_opens = events[events["event_type"] == "app_open"].copy()

# Merge with contract – fails fast if signups has duplicate user_ids
joined = app_opens.merge(
    signups[["user_id", "signup_month"]],
    on="user_id",
    how="inner",
    validate="m:1",
)

# Months since signup as an integer
joined["period_diff"] = joined.apply(
    lambda r: (r["event_month"] - r["signup_month"]).n, axis=1
)

# Distinct active users per (cohort, period)
cohort_counts = (
    joined.groupby(["signup_month", "period_diff"])["user_id"]
    .nunique()
    .reset_index(name="active_users")
)

# Wide cohort matrix
cohort_table = cohort_counts.pivot(
    index="signup_month", columns="period_diff", values="active_users"
).fillna(0)

# Retention = each cell / cohort size at month 0
cohort_size = cohort_table[0]
retention = cohort_table.div(cohort_size, axis=0) * 100

print(retention.round(1).head(12))

# Leakage diagnostic
month_on_month_delta = retention.diff(axis=1)
steepest_drop = month_on_month_delta.min().idxmin()
print(f"Steepest single-month drop happens at month-offset {steepest_drop}")
```

Gotchas

- **merge without validate** : if `signups.csv` has duplicate `user_id` rows (it happens — re-imports, dedup bugs), a plain `merge` silently doubles your event count. `validate='m:1'` raises instead.
- **Mixing Timestamp and Period** : subtraction of two `Timestamp` gives a `Timedelta`, which is *not* months. `Timedelta` is days-based and a 30-day window is not a calendar month. Use `Period` for month grain.
- **nunique() is slow at scale**: on 100M-row event tables, `nunique` is $O(n)$ per group. For interview-scale CSVs it's fine; mention the scaling concern.
- **Pivot fillna**: `pivot` returns NaN for missing cells (cohort × month combinations that don't exist yet because they're in the future). `fillna(0)` makes the cohort matrix readable but also implies "no users active" — which is the right meaning for past months but wrong for future months. Don't `fillna(0)` blindly across the whole frame if cohorts extend into the future.
- **apply for the period_diff**: works but slow on large frames. The vectorized alternative is `joined['event_month'].astype('int64') - joined['signup_month'].astype('int64')` since `Period` is integer-backed. Mention the perf optimisation.
- **Cohort with fewer than N users**: small cohorts (early months, low signups) can produce noisy retention. Add a `min_cohort_size` filter before reading conclusions.

Stand-out line

Say this and you'll separate yourself: "I'd use `validate='m:1'` on the signups merge — it's a contract that catches duplicate signup rows before they double my retention numbers — and I'd compute month-offset using `Period('M')` arithmetic rather than `Timedelta` because 30 days isn't a month and that off-by-one bug ships to production more often than people admit."

Scenario 3 — Multi-Step User Funnel From Event Logs

Asked at: Razorpay, Cred, PhonePe, Paytm, Postman · **Difficulty:** Medium · **Topic:** filter + groupby + percentage, funnel analytics

The data

`events.csv` — ~10 million rows, one row per user action:

Column	Type	Notes
<code>user_id</code>	string	UUID
<code>event_name</code>	string	one of <code>landing_view</code> , <code>signup_start</code> , <code>signup_complete</code> , <code>kyc_submit</code> , <code>kyc_approve</code> , <code>first_transaction</code>
<code>event_timestamp</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , IST
<code>device</code>	string	<code>web</code> , <code>android</code> , <code>ios</code>
<code>properties_json</code>	string	nested JSON, ignore for this prompt

The interviewer's prompt

"Build the conversion funnel: `landing_view` → `signup_start` → `signup_complete` → `kyc_submit` → `kyc_approve` → `first_transaction` . Show the absolute user count and conversion percentage at each step, both step-to-step (between adjacent steps) and overall (from `landing_view`). Now break it down by device — which device has the worst `kyc_submit` drop-off?"

30-second answer outline

"The funnel is just: for each step in order, count distinct users who reached that step and every prior step. The trick is the cumulative `AND` — a user who did `kyc_submit` without ever doing `signup_complete` should not count, because that's a tracking glitch, not a funnel completion. I'd build a per-user binary matrix (one column per step), then take the cumulative `AND` across the row, then count users with each step's flag set. For the device breakdown, groupby `device` and repeat."

Step-by-step walkthrough

- "Order the funnel steps as a list — that's the source of truth, not the order they happen to appear in the CSV."
- "Per-user, per-step binary: `groupby(['user_id', 'event_name']).size().unstack(fill_value=0) > 0` gives a wide DataFrame, one row per user, one column per event, True if they ever did it."
- "Reindex columns in funnel order — `reindex(columns=funnel_steps, fill_value=False)` . Missing events become explicit `False` , not NaN."

4. "Cumulative AND across the row: `df.cummin(axis=1)` over booleans works because `min(True, False) = False`. So once a user breaks the funnel, every subsequent step shows False. This is the cumulative-AND we need."
5. " `cumulative_and.sum(axis=0)` gives the count of users still in the funnel at each step. Divide by step-0 count for the overall conversion. `pct_change()` gives step-to-step."
6. "Device breakdown: same logic but `groupby(['user_id'])` carries the user's first device along, then `groupby(device)` aggregates the per-device funnel counts."

Working pandas code

```
import pandas as pd

events = pd.read_csv("events.csv", parse_dates=["event_timestamp"])

funnel_steps = [
    "landing_view",
    "signup_start",
    "signup_complete",
    "kyc_submit",
    "kyc_approve",
    "first_transaction",
]

# A user's first device – what they started on
first_device = (
    events.sort_values("event_timestamp")
    .groupby("user_id")["device"]
    .first()
)

# Wide per-user / per-step boolean matrix
user_step = (
    events.groupby(["user_id", "event_name"]).size().unstack(fill_value=0) > 0
)
user_step = user_step.reindex(columns=funnel_steps, fill_value=False)

# Cumulative AND – once you drop out, you stay out
user_step_cumulative = user_step.cummin(axis=1)

# Overall funnel
step_counts = user_step_cumulative.sum(axis=0)
overall_pct = step_counts / step_counts.iloc[0] * 100
step_to_step_pct = step_counts / step_counts.shift(1) * 100

funnel = pd.DataFrame(
    {
        "users": step_counts,
        "overall_pct": overall_pct.round(2),
        "step_to_step_pct": step_to_step_pct.round(2),
    }
)
print(funnel)

# Device breakdown
user_step_cumulative["device"] = first_device.reindex(user_step_cumulative.index)
by_device = user_step_cumulative.groupby("device")[funnel_steps].sum()
by_device_step_to_step = by_device.div(by_device.shift(axis=1)) * 100

worst_kyc_device = by_device_step_to_step["kyc_submit"].idxmin()
print(f"Worst kyc_submit drop-off: {worst_kyc_device}")
```

Gotchas

- **Not enforcing funnel order:** if you just count distinct users per step independently, you'll see `kyc_submit` users who never did `signup_complete` (tracking lag, missing events, replayed logs) and the funnel will look broken in non-intuitive ways. The cumulative AND fixes this.
- **unstack memory cost:** on 10M rows with 6 events, the wide matrix is small. On 50 events and 100M rows, you'll OOM. At interview scale this is fine; mention the limit if probed.
- **first_device** : a user can hit `landing_view` on web and `signup_complete` on android. Which device do you attribute to them? `.first()` after sort gives the entry device, which is the standard funnel attribution. State your choice.
- **pct_change direction:** `pct_change()` is $(curr - prev) / prev$, which gives a decimal — multiply by 100 if you want a percentage. The funnel-table convention here is `curr / prev * 100`, so step-to-step is a *survival* percentage, not a *drop* percentage. Be explicit about which you're showing.
- **Empty groups in device breakdown:** a small device (`ios` in tier-2 cities) might have so few users that step-2 numbers are noisy. Apply a `min_users_at_step_0` filter before ranking devices.
- **Time component:** this funnel is order-only, not time-bound. A real product funnel is usually time-bound — "completed within 7 days of landing." If the interviewer asks for that, you'd add a `event_timestamp - first_landing_timestamp <= 7 days` constraint per user.

Stand-out line

Say this and you'll separate yourself: *"I'd enforce the cumulative AND across the funnel steps with `cummin(axis=1)` on a boolean matrix — without it, a user who shows up at `kyc_submit` without ever hitting `signup_complete` gets counted, and that's a tracking glitch, not a conversion. Stating which device's drop-off I'm calling out, and being explicit about whether the percentage is survival or drop, separates funnel analysts from people who've made a chart once."*

Scenario 4 — Daily Revenue From Transaction Log, Missing Days Filled

Asked at: Razorpay, Zerodha, Groww, Jupiter, BharatPe · **Difficulty:** Easy-to-Medium · **Topic:** time-series resampling, reindexing, fill-zero semantics

The data

`transactions.csv` — ~500K rows, one row per successful transaction:

Column	Type	Notes
<code>txn_id</code>	string	UUID
<code>amount</code>	float	INR, always non-NaN for successful txns
<code>txn_timestamp</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , IST, naive
<code>merchant_id</code>	string	FK
<code>payment_method</code>	string	<code>upi</code> , <code>card</code> , <code>netbanking</code> , <code>wallet</code>

The interviewer's prompt

"Compute daily revenue across the entire date range. Make sure every day shows up — including days with zero transactions, which should show revenue 0, not be missing. Then plot the 7-day rolling average. What does the missing-day handling change about the rolling average if you forget to fill?"

30-second answer outline

"Convert `txn_timestamp` to date, groupby date and sum amount. To get every day in the range — including zero-revenue days — I'd resample with `resample('D').sum()` after setting the date as the index, which fills in the gaps as zero. The pitfall the interviewer is fishing for: if you `groupby` without resampling, missing days are silently skipped, and a 7-day rolling average on that frame is wrong — the rolling window includes 7 **non-zero** days instead of 7 **calendar** days, so a quiet weekend silently shifts the average upward."

Step-by-step walkthrough

1. "Parse `txn_timestamp` as datetime at read time. Floor to date — `dt.normalize()` or `dt.floor('D')` — gives me the day grain."
2. "Set the date as the index so `resample` and rolling windows work cleanly."
3. "`resample('D').sum()` over the amount column gives one row per calendar day with zero for empty days. Importantly: not `groupby(date).sum()`, which would drop the zero days entirely."
4. "7-day rolling mean: `rolling(7, min_periods=1).mean()` — `min_periods=1` so the first few days don't show NaN."

5. "Mention the difference: *groupby-then-rolling* gives wrong numbers; *resample-then-rolling* gives correct calendar-aware numbers."
6. "If they ask for business-day-only, swap `'D'` for `'B'` in *resample* — but most India revenue reporting is calendar-day, not business-day."

Working pandas code

```
import pandas as pd

txns = pd.read_csv("transactions.csv", parse_dates=["txn_timestamp"])

# Day grain
txns["txn_date"] = txns["txn_timestamp"].dt.normalize()

daily = (
    txns.set_index("txn_date")["amount"]
    .resample("D")
    .sum()
    .rename("daily_revenue")
)

# Sanity: are there days with zero revenue?
zero_days = (daily == 0).sum()
print(f"Calendar days with zero revenue: {zero_days}")

# 7-day rolling – calendar-aware because resample filled the gaps
daily_with_rolling = daily.to_frame()
daily_with_rolling["rolling_7d"] = daily.rolling(7, min_periods=1).mean()

print(daily_with_rolling.tail(14))

# To show the buggy version for contrast
buggy_daily = txns.groupby("txn_date")["amount"].sum()
buggy_rolling = buggy_daily.rolling(7, min_periods=1).mean()
# buggy_rolling skips empty days – last 7 *populated* days, not last 7 calendar days
```

Gotchas

- `groupby('date')` vs `resample('D')` : *groupby* drops empty days. *Resample* fills them as zero. For *any* time-series rolling / smoothing, you want *resample*.
- `min_periods` : `rolling(7).mean()` returns NaN for the first 6 days because the window is incomplete. `min_periods=1` says "give me whatever you have," which is usually what the dashboard wants. Be explicit.
- **Time-of-day truncation:** `dt.normalize()` zeroes out the time component; `dt.floor('D')` does the same in pandas 2.x. `dt.date` gives a Python `date` object (loses the datetime dtype), which breaks downstream operations — avoid it.

- **Naive vs aware:** India transaction logs are usually stored naive in IST. If they're stored as UTC and your "daily" report is supposed to be IST days, convert with `dt.tz_localize('UTC').dt.tz_convert('Asia/Kolkata').dt.normalize()` — then resample.
- **Refunds and reversals:** the prompt said "successful transactions only," but in real logs there's often a `status` column. If a refund is logged as a negative-amount transaction, `sum()` does the right thing; if it's a separate row with a different sign convention, you have a reconciliation problem.
- **Resample frequency in pandas 2.x:** `'D'` for calendar day, `'B'` for business day, `'W-MON'` for week ending Monday. Pandas 2.2 deprecated some `T / H / L` aliases in favour of `min / h / ms` — use the new ones.

Stand-out line

Say this and you'll separate yourself: "I'd resample with `'D'` rather than groupby on the date — the difference matters only when there are zero-revenue days, but the moment you put a rolling window on top, groupby silently averages over 7 populated days instead of 7 calendar days, and your weekend dip vanishes from the chart. That's the bug you'll never catch in code review and the one finance will spot first."

Scenario 5 — Sessionize Events Into 30-Minute Sessions

Asked at: Swiggy, Zomato, Postman, BrowserStack, Walmart Labs · **Difficulty:** Medium-to-Hard · **Topic:** sessionization, `diff`, `cumsum`, window logic

The data

`events.csv` — ~5 million rows, one row per user action in a mobile app:

Column	Type	Notes
<code>user_id</code>	string	UUID
<code>event_timestamp</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , IST
<code>event_name</code>	string	<code>screen_view</code> , <code>tap</code> , <code>scroll</code> , <code>add_to_cart</code> , etc.
<code>screen</code>	string	screen name where the event fired

The interviewer's prompt

"Group these events into sessions per user. A session is a sequence of events where consecutive events from the same user are within 30 minutes of each other. If the gap is more than 30 minutes, that starts a new session. Assign every event a `session_id` (unique per user-session), then tell me the median session length in events and the median session duration in minutes."

30-second answer outline

"This is the classic `diff` + `cumsum` pattern. Sort events by `(user_id, event_timestamp)`, compute the per-user gap between consecutive events using `groupby(user_id)['event_timestamp'].diff()`. Wherever that gap exceeds 30 minutes — or is NaN, which is the first event for each user — that's the start of a new session. Cumulative sum of those `is_new_session` booleans gives a global session counter. Combine with `user_id` for the per-user `session_id`. Then group by `session_id` to compute length and duration."

Step-by-step walkthrough

1. "Sort by `(user_id, event_timestamp)` — without that, `diff` is meaningless."
2. "Per-user gap: `events.groupby('user_id')['event_timestamp'].diff()`. First event per user gives NaT, which I treat as 'new session.'"
3. "`is_new_session = (gap > pd.Timedelta(minutes=30)) | gap.isna()`. The `isna` handles the per-user first row."
4. "`session_id_global = is_new_session.cumsum()`. This is a monotonically increasing counter — every True ticks it up by 1, so each contiguous run within the 30-minute threshold shares a session id."
5. "For a per-user-readable id, combine:
`events['session_id'] = events['user_id'] + '_' + session_id_global.astype(str)`. Or keep the global counter — interviewer's choice."
6. "Length: `groupby('session_id').size()` — that's events per session. `.median()` for the answer."
7. "Duration: `groupby('session_id')['event_timestamp'].agg(lambda x: x.max() - x.min())` — that's a Timedelta per session. Convert to minutes via `.dt.total_seconds() / 60` and take median."

Working pandas code

```
import pandas as pd

events = pd.read_csv("events.csv", parse_dates=["event_timestamp"])
events = events.sort_values(["user_id", "event_timestamp"]).reset_index(drop=True)

# Per-user gap to the previous event
events["gap"] = events.groupby("user_id")["event_timestamp"].diff()

# A new session starts on the first event per user OR after a >30 min gap
events["is_new_session"] = (events["gap"] > pd.Timedelta(minutes=30)) | events["gap"].isna()

# Global session counter, then a per-user-readable session_id
events["session_id_global"] = events["is_new_session"].cumsum()
events["session_id"] = events["user_id"] + "_" + events["session_id_global"].astype(str)

# Session length in events
session_length = events.groupby("session_id").size().rename("events_in_session")
median_length = session_length.median()
print(f"Median session length (events): {median_length}")

# Session duration in minutes
session_duration = (
    events.groupby("session_id")["event_timestamp"]
    .agg(lambda x: (x.max() - x.min()).total_seconds() / 60)
    .rename("duration_minutes")
)
median_duration = session_duration.median()
print(f"Median session duration (minutes): {median_duration:.2f}")
```

Gotchas

- **Not sorting before `diff`**: `diff` reads the previous row in DataFrame order, not in time order. If the CSV is not pre-sorted, you get nonsense gaps. Always sort first.
- **`groupby(...).diff()` vs `diff()` after `groupby`**: `groupby('user_id')['ts'].diff()` is the right pattern. Plain `events['ts'].diff()` would give a gap between events of *different users* at the boundary, which inflates the session count.
- **Single-event sessions have duration zero**: a user who fires one event and bounces has `max - min = 0`. That's correct — but if your median is heavily skewed by single-event sessions, it tells you about app stickiness, not session length. Optionally filter `events_in_session >= 2` for the duration median, but state the filter.
- **NaT propagation**: comparisons against NaT return False, not raise. `gap > pd.Timedelta(...)` is False for NaT rows, which is why we OR with `gap.isna()` to catch per-user first rows.

- **30-minute boundary is inclusive or exclusive?:** `>` vs `>=` . Convention is usually strict greater-than (a gap of *exactly* 30 minutes is borderline; most platforms call it the same session). State your choice.
- **`agg(lambda)` is slow:** on 5M events spread across thousands of sessions, the lambda-based agg is fine; on 500M events, you'd compute `(max - min)` per group natively with two aggregations and a subtract. Mention the perf path.
- **DST / timezone shifts:** if the data crosses a DST shift (irrelevant in India, but relevant for global apps), a session can span a "lost hour" — handle by working in UTC.

Stand-out line

Say this and you'll separate yourself: "The `diff` + `cumsum` pattern is the standard sessionizer — but the load-bearing detail is the `groupby('user_id')` on the diff. Without it, two users' last and first events get compared at the boundary, and you under-count sessions in the most-active users. That's the bug you only see in production, after the user-engagement metric is already in a board deck."

Scenario 6 — Slow-Changing Dimension (SCD Type 2) For Product Prices

Asked at: Flipkart, Myntra, Meesho, Walmart Labs, Target · **Difficulty:** Hard · **Topic:** SCD type 2, change detection, validity windows

The data

`product_price_snapshots.csv` — one row per (product, day) showing that day's price. ~3 million rows, but most consecutive days have the same price:

Column	Type	Notes
<code>product_id</code>	string	UUID
<code>snapshot_date</code>	string	<code>YYYY-MM-DD</code> , IST
<code>price</code>	float	INR, current price on that day
<code>currency</code>	string	always <code>INR</code> for this question

The interviewer's prompt

"Convert this snapshot table into a slow-changing dimension type 2 — one row per (product, valid_from, valid_to) where the price was constant. The current row should have `valid_to = NaT` (open-ended). Then tell me: for product X (any product, your choice), how many distinct price changes did it see, and what was the longest single price-stability window in days?"

30-second answer outline

"Per product, sort by date and identify where the price actually changes between adjacent rows. The standard pattern: `groupby('product_id')['price'].shift()` to get the previous day's price, then flag where current $\neq$ previous as a change-point. Cumulative sum of those change-flags within product gives a group id for each price-stability period. Group by `(product_id, group_id)` and take `min(date)` as `valid_from`, `max(date)` as the last day of validity. Forward-shift `valid_from` to get `valid_to`. The most recent period's `valid_to` is `NaT`."

Step-by-step walkthrough

1. "Sort by `(product_id, snapshot_date)`. Required for any shift-based logic."
2. "Previous-day price per product: `prev_price = df.groupby('product_id')['price'].shift(1)`. `NaN` for the first row per product."
3. "Change-point: `is_change = (df['price'] != prev_price) | prev_price.isna()`. The OR with `isna` captures the first row of every product as the start of period 1."
4. "Group id within product: `df['period_id'] = is_change.groupby(df['product_id']).cumsum()`. Each (product, period_id) is one stability window."
5. "Aggregate: `groupby(['product_id', 'period_id']).agg(valid_from=('snapshot_date', 'min'), valid_to_inclusive=('snapshot_date', 'max'), price=('price', 'first'))`. The price is constant within the group so `first` is correct."
6. "Convert inclusive end-date to exclusive `valid_to` if SCD type 2 convention requires it: usually `valid_to = next_period.valid_from`, with the most recent period showing `NaT`."
7. "Count price changes for product X: `(scd_table['product_id'] == 'X').sum() - 1` — the number of distinct stability windows minus one is the number of changes."
8. "Longest stability window: `(scd['valid_to_inclusive'] - scd['valid_from']).dt.days + 1`. Take the max for product X."

Working pandas code

```
import pandas as pd

snaps = pd.read_csv("product_price_snapshots.csv", parse_dates=["snapshot_date"])
snaps = snaps.sort_values(["product_id", "snapshot_date"]).reset_index(drop=True)

# Detect a price change vs the previous day, per product
prev_price = snaps.groupby("product_id")["price"].shift(1)
is_change = (snaps["price"] != prev_price) | prev_price.isna()

# Period id within each product
snaps["period_id"] = is_change.groupby(snaps["product_id"]).cumsum()

# Collapse to one row per (product, period)
scd = (
    snaps.groupby(["product_id", "period_id"], as_index=False)
    .agg(
        valid_from=("snapshot_date", "min"),
        valid_to_inclusive=("snapshot_date", "max"),
        price=("price", "first"),
    )
    .sort_values(["product_id", "valid_from"])
    .reset_index(drop=True)
)

# SCD type 2 exclusive valid_to: next row's valid_from per product, NaT for current row
scd["valid_to"] = scd.groupby("product_id")["valid_from"].shift(-1)

# Analytics on product X
product_x = scd[scd["product_id"] == "PRODUCT_X_ID"].copy()
num_changes = len(product_x) - 1
print(f"Product X had {num_changes} distinct price changes")

product_x["stability_days"] = (
    product_x["valid_to_inclusive"] - product_x["valid_from"]
).dt.days + 1
longest = product_x["stability_days"].max()
print(f"Longest stability window: {longest} days")

print(scd.head(20))
```

Gotchas

- **Float comparison for price change:** `price != prev_price` works on `Decimal` or rupee-grain integers, but on raw floats you can get spurious change-points from floating-point noise (e.g. `99.99999 != 100.0`). For currency, store as integer paise or as `Decimal`, or compare with `not np.isclose(...)` if forced to use float.

- **Missing days in the snapshot:** if a product has no snapshot for some days (CSV gaps), the SCD will mark the next available day as a new period even if the price didn't actually change. Decide: do you forward-fill the gap, or trust gaps as "unknown"?
- **Currency change mid-history:** the prompt said all INR, but real catalogs have currency changes. SCD needs to track `(price, currency)` jointly for change detection, not just `price`.
- **`shift(1)` doesn't respect groups by default:** you must use `groupby(...).shift(1)`, not `.shift(1)` on the whole frame. The latter compares the first row of product B to the last row of product A — a silent disaster.
- **`valid_to` semantics:** SCD type 2 has two conventions — inclusive (`valid_to = last day of validity`) or exclusive (`valid_to = first day of next period`). State which one you're using; warehouses are inconsistent. Most modern dimensional models go exclusive.
- **NaT for current period:** the most recent stability window should have `valid_to = NaT` so queries like `valid_from <= today < valid_to` work with `NaT` treated as `+infinity`. Or pick a far-future sentinel like `9999-12-31`. State the choice.
- **First row per product gets period_id 1:** if `is_change.groupby('product_id').cumsum()` produces a 0 for the very first row (because the boolean is True and cumsum treats True as 1), check the boundary. Cast booleans explicitly to int if your pandas version surprises you.

Stand-out line

Say this and you'll separate yourself: *"For SCD type 2 with floats, I'd never trust raw price equality — I'd compare integer paise or use `Decimal`, because rounding noise creates phantom change-points and an audit team will eventually find a product showing 4 'price changes' on the same day. The `shift` + `cumsum` pattern is fine; the dtypes choice is what separates a clean SCD from a noisy one."*

Scenario 7 — Anti-Join: Customers Who Haven't Ordered In 90 Days

Asked at: Swiggy, Zomato, Dunzo, BigBasket, BlinkIt · **Difficulty:** Easy-to-Medium · **Topic:** anti-join, `merge` with `indicator=True`, "find what's not there"

The data

Two CSVs.

`customers.csv` :

Column	Type	Notes
<code>customer_id</code>	string	UUID
<code>signup_date</code>	string	<code>YYYY-MM-DD</code>
<code>city</code>	string	
<code>tier</code>	string	<code>silver</code> , <code>gold</code> , <code>platinum</code>

`orders.csv` :

Column	Type	Notes
<code>order_id</code>	string	UUID
<code>customer_id</code>	string	FK
<code>order_date</code>	string	<code>YYYY-MM-DD HH:MM:SS</code>
<code>amount</code>	float	INR

The interviewer's prompt

"Find the customers who have not placed an order in the last 90 days. Include customers who have never ordered. Return their `customer_id` , `city` , `tier` , and their last order date (if any). Now: which tier has the largest absolute count of inactive customers, and which has the highest percentage of its base inactive?"

30-second answer outline

"This is an anti-join. Two ways. The pandas-idiomatic one: `merge(customers, recent_orders, how='left', indicator=True)` then filter `_merge == 'left_only'` . Or: compute `last_order_date` per customer via `groupby` `max` on the full orders table, left-merge into customers, and filter where `last_order_date < cutoff` or `NaN`. The second is more flexible because it preserves the actual last-order-date. For the tier analysis, `groupby` tier and count vs total."

Step-by-step walkthrough

1. "Decide on the cutoff — `max(order_date) - 90 days` if it's a snapshot CSV, or `today() - 90 days` if it's live data. I'd ask but state my choice."
2. "Per-customer last order date — `orders.groupby('customer_id')['order_date'].max()` . This is a small dimension table now."
3. "Left-merge into customers: `customers.merge(last_order, how='left', on='customer_id', validate='1:1')` . Customers who never ordered get NaT."
4. "Filter: `last_order_date.isna() | (last_order_date < cutoff)` . Both conditions matter — interviewer asked for never-ordered AND haven't-ordered-recently."
5. "Project the columns they asked for. Done with the headline answer."
6. "Tier analysis: groupby tier, count inactive vs total. Compute count and percentage. Sort each for the two follow-up questions."

Working pandas code

```
import pandas as pd

customers = pd.read_csv("customers.csv", parse_dates=["signup_date"])
orders = pd.read_csv("orders.csv", parse_dates=["order_date"])

# Anchor on the data's own clock
cutoff = orders["order_date"].max() - pd.Timedelta(days=90)

# Last order per customer (NaT for customers with no orders)
last_order = (
    orders.groupby("customer_id", as_index=False)["order_date"]
    .max()
    .rename(columns={"order_date": "last_order_date"})
)

customers_with_last = customers.merge(
    last_order, on="customer_id", how="left", validate="1:1"
)

# Inactive = never ordered OR last order before cutoff
inactive_mask = (
    customers_with_last["last_order_date"].isna()
    | (customers_with_last["last_order_date"] < cutoff)
)

inactive = customers_with_last.loc[
    inactive_mask,
    ["customer_id", "city", "tier", "last_order_date"],
].reset_index(drop=True)

print(inactive.head(20))
print(f"Total inactive customers: {len(inactive)}")

# Tier breakdown
tier_summary = (
    customers_with_last.assign(is_inactive=inactive_mask)
    .groupby("tier")
    .agg(total_customers=("customer_id", "count"), inactive_count=("is_inactive", "sum"))
)
tier_summary["inactive_pct"] = (
    tier_summary["inactive_count"] / tier_summary["total_customers"] * 100
).round(2)

print(tier_summary)
print("Largest absolute inactive count:", tier_summary["inactive_count"].idxmax())
print("Highest inactive percentage:", tier_summary["inactive_pct"].idxmax())
```

Gotchas

- **Forgetting never-ordered customers:** an `inner` merge of customers with orders, then anti-join, will silently drop customers who never appear in `orders.csv`. The `left` merge plus the `isna` filter catches them.
- **Anchoring on `today()`** : same trap as Scenario 1. A CSV is a snapshot. Anchor on `max(order_date)` and say so.
- **`validate='1:1'`** : assumes `customer_id` is unique in `customers.csv`. If it isn't (duplicate signups, soft-deleted re-signups), the validate raises and you fix the upstream. Without validate, you silently get duplicate rows in the result.
- **merge with `indicator=True`** : the other classic anti-join pattern, useful when you don't need to preserve the matched value. `customers.merge(orders, how='left', on='customer_id', indicator=True).query("_merge == 'left_only'")`. Mention it as the alternative.
- **Date arithmetic mixing types:** `cutoff` must be `Timestamp`, not `datetime.date`. Mixing them in comparisons either coerces silently or fails. Stay in pandas types end-to-end.
- **Tier ranking with small denominators:** if `platinum` has 5 customers and 3 inactive, that's 60% — technically the "highest inactive percentage" — but the absolute count is tiny. Flag small-denominator winners.
- **Memory on the groupby max:** `orders.groupby('customer_id').max()` over 50M orders is fine. Over 5B, you'd want a partial aggregation.

Stand-out line

Say this and you'll separate yourself: *"I'd left-merge the last-order-date in and filter on `isna() | (last_order < cutoff)` — that catches never-ordered customers and recently-inactive customers in one expression. The `indicator=True` anti-join is the textbook pattern, but losing the actual last-order-date means I'd need a second pass to power the retention dashboard the next sprint."*

Scenario 8 — Pivot Sales Wide, Then Surface Top Contributors

Asked at: Flipkart, Myntra, Nykaa, Meesho, Reliance Retail · **Difficulty:** Medium · **Topic:** pivot, melt, top-N over a derived wide table

The data

`sales.csv` — one row per sale line, ~8 million rows:

Column	Type	Notes
<code>sale_id</code>	string	UUID
<code>product</code>	string	product name (~500 distinct)
<code>quantity</code>	int	units sold
<code>unit_price</code>	float	INR
<code>sale_date</code>	string	<code>YYYY-MM-DD</code>
<code>region</code>	string	

The interviewer's prompt

"Build a wide table: rows are products, columns are months (`2026-01` , `2026-02` , ...), cells are total revenue. Then: which 5 products contributed the most revenue overall, and for the top product, which month was its peak? Now flip: which products had no sales at all in `2026-03` ?"

30-second answer outline

"Compute revenue per line as `quantity * unit_price` , derive a month period, then `pivot_table(index='product', columns='month', values='revenue', aggfunc='sum', fill_value=0)` . For top contributors, `pivot.sum(axis=1).nlargest(5)` . For the top product's peak month, `pivot.loc[top_product].idxmax()` . For products with no March sales, filter `pivot['2026-03'] == 0` — but only meaningful if March is fully complete in the data."

Step-by-step walkthrough

- "Revenue per line: `sales['revenue'] = sales['quantity'] * sales['unit_price']` . Check both columns for NaN first — multiplying NaN by anything is NaN."
- "Month period: `sales['month'] = sales['sale_date'].dt.to_period('M')` . Cleaner than `dt.strftime('%Y-%m')` strings."
- "Pivot: `pd.pivot_table(sales, index='product', columns='month', values='revenue', aggfunc='sum', fill_value=0)` . `fill_value=0` because a product with no sales in a month is genuinely zero revenue, not unknown."

4. "Top 5 contributors: `pivot.sum(axis=1).nlargest(5)` — total revenue per product across all months, top 5."
5. "Peak month for the top product: `pivot.loc[top_product].idxmax()` . Returns the Period object for the peak month."
6. "Products with no March sales: `pivot[pd.Period('2026-03')] == 0` is a boolean Series indexed by product. Filter and list. Caveat: only meaningful if the CSV is complete for March; if data was extracted mid-March, half the month is missing."

Working pandas code

```
import pandas as pd

sales = pd.read_csv("sales.csv", parse_dates=["sale_date"])
sales["revenue"] = sales["quantity"] * sales["unit_price"]
sales["month"] = sales["sale_date"].dt.to_period("M")

# Wide table: product x month
pivot = pd.pivot_table(
    sales,
    index="product",
    columns="month",
    values="revenue",
    aggfunc="sum",
    fill_value=0,
)

# Top 5 contributors overall
total_per_product = pivot.sum(axis=1)
top_5 = total_per_product.nlargest(5)
print("Top 5 products by total revenue:")
print(top_5)

# Peak month for the top product
top_product = top_5.index[0]
peak_month = pivot.loc[top_product].idxmax()
peak_revenue = pivot.loc[top_product].max()
print(f"Top product '{top_product}' peaked in {peak_month} at INR {peak_revenue:,.2f}")

# Products with no sales in 2026-03 (assuming March is complete in the data)
march_col = pd.Period("2026-03")
if march_col in pivot.columns:
    no_march_sales = pivot.index[pivot[march_col] == 0].tolist()
    print(f"{len(no_march_sales)} products had no sales in 2026-03")
else:
    print("2026-03 not present in the data – partial month or out of range")
```

Gotchas

- **pivot vs pivot_table** : `pivot` requires the (index, columns) pair to be unique — multiple sale rows per (product, month) raise. `pivot_table` aggregates duplicates with `aggfunc` . Use `pivot_table` for any real CSV.
- **fill_value=0 semantics**: in this case, zero revenue is the right meaning. But for, say, a wide table of "average rating per product per month", `fill_value=0` would skew the column average — leave NaN there. Choose carefully.
- **Period('M') in column index**: clean, but the column name is now a `Period` object, not a string. `pivot['2026-03']` won't work — `pivot[pd.Period('2026-03')]` does. Awkward but precise.
- **NaN in quantity * unit_price** : if `unit_price` has NaN for promo items, `revenue` is NaN, and `aggfunc='sum'` silently ignores it. Decide whether NaN unit_price means zero, missing-data-fail-loudly, or "free promo" with revenue 0. State your choice.
- **Memory on 8M rows × 500 products × 24 months**: the wide table is $500 \times 24 = 12\text{k}$ cells, trivial. The intermediate `sales` frame is the concern at scale, not the pivot.
- **Top-product peak with multiple peaks**: if the top product had two months with identical max revenue, `idxmax()` returns the first one. Mention if asked.
- **"No sales in March" with incomplete data**: a freshly-extracted CSV in mid-March will look like every product is below trend. Check the date range with `sales['sale_date'].agg(['min', 'max'])` before claiming "no sales."

Stand-out line

Say this and you'll separate yourself: "`pivot_table` with `fill_value=0` is right here because zero revenue is a real value — but I'd never use `fill_value=0` on a metric like average rating, where NaN means 'no data' and zero would skew the column mean to garbage. The choice of fill is a domain question, not a pandas question."

Scenario 9 — Outlier Detection Per Segment Via IQR

Asked at: Razorpay, PhonePe, Paytm, Cred, Jupiter · **Difficulty:** Medium · **Topic:** IQR outlier method, segment-aware filtering, `groupby` + `transform`

The data

`transactions.csv` — ~2 million rows:

Column	Type	Notes
<code>txn_id</code>	string	UUID
<code>customer_segment</code>	string	<code>consumer</code> , <code>merchant_small</code> , <code>merchant_large</code> , <code>enterprise</code>
<code>amount</code>	float	INR, range from 1 to 10^7+
<code>txn_timestamp</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , IST
<code>payment_method</code>	string	

The interviewer's prompt

"Flag outlier transactions using the IQR method — anything below `Q1 - 1.5*IQR` or above `Q3 + 1.5*IQR` is an outlier. But: do it per customer segment, because a ₹50,000 transaction is normal for `enterprise` and a fraud signal for `consumer` . Return a DataFrame with the original columns plus `is_outlier` and `outlier_direction` (`low` , `high` , or `null`). Then: what percentage of `consumer` segment transactions are outliers, and is that surprisingly high?"

30-second answer outline

"Per segment, compute Q1 and Q3 via `groupby('customer_segment')['amount'].transform('quantile', 0.25)` and same for 0.75. `transform` broadcasts the per-segment value back to the original row — so each row gets its segment's quantiles inline. IQR is the difference; lower bound is `Q1 - 1.5IQR`, upper is `Q3 + 1.5IQR`. Flag outliers. For the percentage, filter to `consumer` and divide. The 'surprisingly high' is the leading question — for fraud-tilted segments like `consumer` , IQR is going to flag a lot of legitimate high-value transactions, so the percentage being 5-10% is normal but the prompt wants the candidate to flag IQR's weakness on skewed distributions."

Step-by-step walkthrough

1. "Filter NaN amounts upfront — IQR on NaN-filled data is nonsense.
`dropna(subset=['amount'])` or work on a copy."
2. "Compute Q1 and Q3 per segment using `transform` . `transform` is the right tool here because it preserves the row count: every transaction row gets its segment's Q1/Q3 attached."

3. "IQR = Q3 - Q1. Lower = Q1 - 1.5IQR, Upper = Q3 + 1.5IQR. Clamp lower at 0 if amount can't be negative — though leaving the math as-is is also defensible."
4. "`is_outlier` is `(amount < lower) | (amount > upper)`. `outlier_direction` is a `np.where` based on which side."
5. "Percentage per segment: `groupby('customer_segment')['is_outlier'].mean() * 100`."
6. "Editorial moment: IQR assumes roughly symmetric distribution. Transaction amounts are heavily right-skewed (log-normal-ish), so IQR flags a lot of legitimate large transactions as outliers. Mention this honestly. For a fraud model, log-transform first or use percentile cuts (e.g. flag top 0.1%)."

Working pandas code

```
import pandas as pd
import numpy as np

txns = pd.read_csv("transactions.csv", parse_dates=["txn_timestamp"])
txns = txns.dropna(subset=["amount"]).copy()

# Per-segment Q1 and Q3, broadcast back to each row
txns["q1"] = txns.groupby("customer_segment")["amount"].transform("quantile", 0.25)
txns["q3"] = txns.groupby("customer_segment")["amount"].transform("quantile", 0.75)
txns["iqr"] = txns["q3"] - txns["q1"]

txns["lower_bound"] = txns["q1"] - 1.5 * txns["iqr"]
txns["upper_bound"] = txns["q3"] + 1.5 * txns["iqr"]

txns["is_outlier"] = (txns["amount"] < txns["lower_bound"]) | (
    txns["amount"] > txns["upper_bound"]
)
txns["outlier_direction"] = np.where(
    txns["amount"] < txns["lower_bound"],
    "low",
    np.where(txns["amount"] > txns["upper_bound"], "high", None),
)

# Percentage of consumer outliers
consumer_outlier_pct = (
    txns.loc[txns["customer_segment"] == "consumer", "is_outlier"].mean() * 100
)
print(f"Consumer outlier percentage (IQR method): {consumer_outlier_pct:.2f}%")

# Per-segment breakdown for context
by_segment = txns.groupby("customer_segment")["is_outlier"].agg(["mean", "sum", "count"])
by_segment["outlier_pct"] = by_segment["mean"] * 100
print(by_segment[["count", "sum", "outlier_pct"]].round(2))
```

Gotchas

- **groupby.transform vs groupby.agg** : `agg` returns one row per group; `transform` returns the same shape as the input. For attaching per-group statistics back to each row, `transform` is the only correct tool.
- **IQR on skewed distributions**: transaction amounts are right-skewed. IQR's upper bound on a log-normal-ish distribution flags a huge chunk of legitimate tail. The method is *fast* and *interpretable*, not *accurate* for skewed financial data. Mention this — it's the candidate-vs-junior signal.
- **Lower bound below zero**: $Q1 - 1.5 \times IQR$ can go negative on amounts that are bounded at zero. Mathematically fine; semantically meaningless. Decide whether to clamp or leave as-is.
- **Empty segments**: a segment with fewer than 4 rows has unstable quantiles. `quantile(0.25)` on a single row returns that row. Filter out segments below a minimum size before reading the outlier percentage seriously.
- **np.where returning None** : with `np.where`, the third arg is the False branch. Using `None` puts Python `None` into a numpy array, which becomes `object` dtype. Acceptable for this column but mention it. Could use `pd.NA` for a more pandas-native missing value.
- **transform('quantile', 0.25) API**: in pandas 2.x this is supported; older idioms used `lambda x: x.quantile(0.25)` inside `transform`. The string-name form is faster because pandas dispatches to a cython kernel.
- **Stale segments**: if `customer_segment` is computed daily but the CSV mixes segment-as-of-txn vs segment-as-of-today, your IQR bounds are inconsistent. State the SCD-style assumption.

Stand-out line

Say this and you'll separate yourself: "`groupby.transform` broadcasts the per-segment quantile back to each row, which is exactly what IQR-per-segment needs — but I'd say out loud that IQR on transaction amounts overflags because the distribution is right-skewed log-normal, not symmetric. For a fraud signal we'd log-transform first or use a percentile cut. IQR is fast and explainable, not accurate."

Scenario 10 — Reconcile Order Data From Two Systems

Asked at: Razorpay, Postman, Walmart Labs, JP Morgan, Goldman Sachs · **Difficulty**: Hard · **Topic**: multi-source merge, schema mapping, reconciliation, conflict resolution

The data

Two CSVs from two upstream systems — same orders, different schemas, partial overlap.

`system_a_orders.csv` — order-management system (OMS), the merchant-facing source of truth:

Column	Type	Notes
<code>oms_order_id</code>	string	OMS-side primary key
<code>external_reference</code>	string	the shared key (UUID-ish)
<code>customer_email</code>	string	
<code>gross_amount</code>	float	INR, includes tax
<code>tax_amount</code>	float	INR
<code>created_at_oms</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , UTC
<code>status_oms</code>	string	<code>placed</code> , <code>fulfilled</code> , <code>cancelled</code> , <code>refunded</code>

`system_b_payments.csv` — payment gateway (PG), captures the money-in side:

Column	Type	Notes
<code>pg_txn_id</code>	string	PG-side primary key
<code>merchant_reference</code>	string	the shared key — should equal <code>external_reference</code>
<code>email</code>	string	customer email
<code>amount</code>	float	INR, what was actually captured
<code>captured_at</code>	string	ISO <code>YYYY-MM-DD HH:MM:SS</code> , IST
<code>status_pg</code>	string	<code>captured</code> , <code>failed</code> , <code>refunded</code> , <code>pending</code>

The interviewer's prompt

"Reconcile these two CSVs. The shared key is `oms.external_reference == pg.merchant_reference`. Produce a unified DataFrame with one row per logical order, showing both sides' status and amount, and a `discrepancy` column flagging any of: OMS-only (in OMS but no PG row), PG-only (PG row with no OMS match), amount mismatch (`gross_amount != amount`), or status mismatch (cross-table inconsistency). How would you decide which side is authoritative for amount?"

30-second answer outline

"Outer-merge on the mapped key with `indicator=True`. The indicator gives me OMS-only and PG-only flags for free. Normalize timezones before any timestamp comparison — `created_at_oms` is UTC, `captured_at` is IST. For amount mismatch, define tolerance — `abs(gross_amount - amount) > 0.01` is the right check on float currency, not strict inequality. Status mismatch needs a small mapping table because the vocabularies differ. Authoritative-side question: OMS for what was `agreed` (gross_amount, including tax), PG for what was `received` (captured amount, net of failures); reconciliation flags when these diverge."

Step-by-step walkthrough

1. "Rename the keys so the merge is clean: `oms_renamed = oms.rename(columns={'external_reference': 'shared_key'})`, same on PG side."
2. "Outer merge on `shared_key` with `how='outer', indicator=True`. The `_merge` column is now `'left_only'` (OMS-only), `'right_only'` (PG-only), or `'both'`."
3. "Timezone normalize before any timestamp logic: OMS UTC → IST via `tz_localize('UTC').tz_convert('Asia/Kolkata')`; PG already IST, mark naive as IST via `tz_localize('Asia/Kolkata')`. Now both are timezone-aware and comparable."
4. "Amount discrepancy: `(merged['_merge'] == 'both') & (np.abs(merged['gross_amount'] - merged['amount']) > 0.01)`. Float-safe comparison."
5. "Status discrepancy: map both vocabularies to a canonical set. `{oms_status: canonical}` and `{pg_status: canonical}` dicts. Compare the canonical values."
6. "Combine into one `discrepancy` column with a categorical: `oms_only`, `pg_only`, `amount_mismatch`, `status_mismatch`, or `clean`. Use `np.select` for the multi-condition assignment."
7. "On authoritative side: discuss the meaning — OMS is authoritative for agreed-amount-to-be-paid, PG is authoritative for amount-actually-received. The reconciliation surfaces the gap; the business decision is which one drives revenue recognition."

Working pandas code

```

import pandas as pd
import numpy as np

oms = pd.read_csv("system_a_orders.csv", parse_dates=["created_at_oms"])
pg = pd.read_csv("system_b_payments.csv", parse_dates=["captured_at"])

# Timezone normalization – both sides become IST-aware
oms["created_at_ist"] = (
    oms["created_at_oms"].dt.tz_localize("UTC").dt.tz_convert("Asia/Kolkata")
)
pg["captured_at_ist"] = pg["captured_at"].dt.tz_localize("Asia/Kolkata")

# Rename keys to a common name
oms = oms.rename(columns={"external_reference": "shared_key"})
pg = pg.rename(columns={"merchant_reference": "shared_key"})

# Outer merge with indicator
merged = oms.merge(
    pg, on="shared_key", how="outer", indicator=True, suffixes=("_oms", "_pg")
)

# Canonical status mapping
oms_status_map = {
    "placed": "open",
    "fulfilled": "complete",
    "cancelled": "cancelled",
    "refunded": "refunded",
}
pg_status_map = {
    "captured": "complete",
    "failed": "failed",
    "refunded": "refunded",
    "pending": "open",
}
merged["status_oms_canonical"] = merged["status_oms"].map(oms_status_map)
merged["status_pg_canonical"] = merged["status_pg"].map(pg_status_map)

# Float-safe amount comparison
both = merged["_merge"] == "both"
amount_mismatch = both & (
    np.abs(merged["gross_amount"] - merged["amount"]) > 0.01
)

# Status mismatch – both sides present and canonical statuses differ
status_mismatch = (
    both
    & merged["status_oms_canonical"].notna()
    & merged["status_pg_canonical"].notna()
    & (merged["status_oms_canonical"] != merged["status_pg_canonical"])
)

# Build the discrepancy column with priorities
merged["discrepancy"] = np.select(

```

```

    [
        merged["_merge"] == "left_only",
        merged["_merge"] == "right_only",
        amount_mismatch,
        status_mismatch,
    ],
    ["oms_only", "pg_only", "amount_mismatch", "status_mismatch"],
    default="clean",
)

print(merged["discrepancy"].value_counts())
print(merged.head(10)[
    [
        "shared_key",
        "gross_amount",
        "amount",
        "status_oms",
        "status_pg",
        "discrepancy",
    ]
])

```

Gotchas

- **Timezone mismatch silently miscomputing date windows:** UTC vs IST is 5h30m. An order placed at 23:00 IST in OMS shows up as 17:30 UTC the same day, but a naive comparison to a PG timestamp at 23:30 IST will put them in different days. Localize and convert *before* any date-aware logic.
- **Schema mapping is part of the answer, not a setup step:** `external_reference` and `merchant_reference` are the same logical thing with different names. The mapping is the first observation the interviewer wants you to articulate.
- **Float equality on currency:** `gross_amount == amount` is almost guaranteed to be False on legitimately-equal amounts because of float representation. Always use `abs(diff) > tolerance` with a domain-meaningful tolerance (₹0.01 or 1 paisa).
- **Status vocabulary mismatch:** OMS says `fulfilled`, PG says `captured`. These mean the same business state but the strings don't match. A canonical-status mapping is unavoidable; don't pretend the raw strings can be compared.
- **`how='outer'` with `indicator=True` is the anti-join + match in one go:** `'left_only'` = OMS-only, `'right_only'` = PG-only, `'both'` = matched. Cleaner than two separate anti-joins.
- **Duplicate shared_key on either side:** if OMS or PG has multiple rows per `shared_key` (retries, partial captures, multi-line orders), the outer merge cross-products. `validate='1:1'` raises if violated. If duplicates are legitimate (e.g. partial payments), you'd aggregate to one row per `shared_key` per side *before* the merge.
- **Authoritative-amount question:** the interviewer wants to hear that you understand the business — OMS is the agreement (what should have been paid, gross of tax), PG is the actuality (what was

captured). Reconciliation flags the gap; revenue recognition uses one or the other based on accounting policy.

- **`np.select` order matters**: the conditions are checked in order, and the first match wins. Put exclusivity conditions before subset conditions — e.g. `oms_only` and `pg_only` before the mismatch checks, since mismatch checks reference matched rows.

Stand-out line

Say this and you'll separate yourself: "I'd validate the merge with `validate='1:1'` before reporting anything — silently doubling rows on a duplicate `shared_key` is exactly how reconciliation reports lie to finance. And float-equality on currency is wrong by construction; the comparison is `abs(diff) > paisa_tolerance`. The authoritative-side question doesn't have a pandas answer — it's whether the business reports revenue on agreement or on capture, and the reconciliation table is the artefact that surfaces the gap either way."

Chapter summary

Ten CSV scenarios, ten failure modes that separate the candidate who's seen real production data from the one who's only done textbook problems. The common threads:

1. **Anchor on the data's clock, not wall-clock.** A CSV is a snapshot. Use `max(timestamp)` for windows, not `today()`.
2. **`validate=` on every merge.** `'1:1'`, `'m:1'`, `'1:m'`, `'m:m'` — pick one and let pandas enforce it. Silent row-doubling is the bug you don't catch in code review.
3. **Float-equality on currency is wrong.** Use `abs(diff) > tolerance` or move to integer paise.
4. **Timezone normalization before any date-aware logic.** UTC vs IST is 5h30m. Two timestamps in the same logical second can fall on different days.
5. **`groupby.transform` for per-group statistics broadcast back to rows.** Outliers, normalisation, z-scores — `transform` is the right tool, not `agg`.
6. **`resample('D')` over `groupby(date)` for time-series.** Resample fills gaps; groupby drops them. Rolling windows on dropped-gap data lie quietly.
7. **`diff` + `cumsum` is the sessionization / change-detection pattern.** Sort first, group on the entity (user, product) before diffing.
8. **Cumulative AND for funnel correctness.** `cummin(axis=1)` over a boolean matrix enforces step-order without writing nested logic.
9. **Period arithmetic for month-grain work.** `Timedelta` is days-based; months are calendar, not 30-day windows. Use `Period('M')`.

10. **The stand-out lines all share a structure:** the candidate names the failure mode the naive approach would produce, then names the precise pandas-2.x primitive that prevents it. That's the signal panels are listening for.

Ship this and you'll have walked through the round-2/round-3 surface for any Data role at any Indian product company. Combine with Chapter 2 (`groupby + agg`) and Chapter 3 (`merge / join / concat`) for the prep that turns the panel from screening into a conversation.